



P4₁₆ Language Specification

The P4 Language Consortium

Version v1.2.5, 2026-01-30

Contents

1. Scope	14
2. Terms, definitions, and symbols	14
3. Overview	15
3.1. Benefits of P4	17
3.2. P4 language evolution: comparison to previous versions (P4 v1.0/v1.1)	18
4. Architecture Model	19
4.1. Standard architectures	20
4.2. Data plane interfaces	20
4.3. Extern objects and functions	20
5. Example: A very simple switch	21
5.1. Very Simple Switch Architecture	22
5.2. Very Simple Switch Architecture Description	24
5.2.1. Arbiter block.	24
5.2.2. Parser runtime block	24
5.2.3. Demux block	25
5.2.4. Available extern blocks.	25
5.3. A complete Very Simple Switch program	25
6. P4 language definition	29
6.1. Syntax and semantics	30
6.1.1. Grammar	30
6.1.2. Semantics and the P4 abstract machines	30
6.1.2.1. Pseudo-code snippets	30
6.1.2.2. Prose algorithms	30
6.1.2.2.1. Prose algorithm shorthands	31
6.2. Lexical constructs.	31
6.2.1. Identifiers	32
6.2.2. Literal constants	32
6.2.2.1. Boolean literals.	32
6.2.2.2. Integer literals.	33
6.2.2.3. String literals	33
6.2.3. Comments	34
6.2.4. Optional trailing commas	34
6.3. Scoping	35
6.3.1. Name resolution	35
6.3.2. Name visibility	36
6.4. Stateful elements.	36
6.5. Call convention: copy-in/copy-out.	37
6.5.1. Justification.	39
6.5.2. Optional parameters	40
6.6. Overloading.	41
6.7. Naming conventions	41
7. The P4 Abstract Machine	42
7.1. Preprocessing.	43
7.2. Type Checking.	43
7.2.1. P4 Intermediate Representation	43
7.2.2. Typing a P4 program	44
7.2.3. Typing context	45
7.3. Instantiation	46

7.3.1. Control-plane names	48
7.3.1.1. Computing control-plane names.....	48
7.3.1.1.1. Value sets	48
7.3.1.1.2. Tables	49
7.3.1.1.3. Keys	49
7.3.1.1.4. Actions.....	50
7.3.1.1.5. Instances.....	50
7.3.1.2. Annotations controlling naming	51
7.3.1.3. Recommendations.....	51
7.3.2. Instantiating a P4 program	52
7.3.3. Instantiation context	53
7.4. Runtime evaluation	54
7.4.1. Evaluating a P4 program	54
7.4.2. Evaluation context	55
7.4.3. The parser abstract machine	55
7.4.4. The match-action pipeline abstract machine	55
7.4.5. Concurrency model	56
7.5. Compile-time known and local compile-time known values.....	57
8. P4 types and values	58
8.1. Semantics of types.....	59
8.1.1. Type checking surface types	59
8.1.2. Well-formedness of types	59
8.2. Base types and values.....	60
8.2.1. Well-formedness.....	61
8.2.2. The void type	61
8.2.2.1. Type checking	61
8.2.3. Booleans.....	61
8.2.3.1. Operations.....	61
8.2.3.2. Type checking	62
8.2.4. Errors	62
8.2.4.1. Operations.....	63
8.2.4.2. Type checking	63
8.2.5. Match kinds	63
8.2.5.1. Operations.....	64
8.2.5.2. Type checking	64
8.2.6. Strings.....	64
8.2.6.1. Operations.....	64
8.2.6.2. Type checking	65
8.2.7. Integers (signed and unsigned).....	65
8.2.7.1. Portability	65
8.2.7.2. Integer literals.....	66
8.2.7.3. Illegal arithmetic expressions.....	66
8.2.7.4. Arbitrary-precision integers	67
8.2.7.4.1. Operations	67
8.2.7.4.2. Type checking.....	68
8.2.7.5. Fixed-width unsigned integers (bit-strings)	68
8.2.7.5.1. Operations	69
8.2.7.5.2. Type checking.....	70
8.2.7.6. Fixed-width signed integers	71
8.2.7.6.1. Operations	71

8.2.7.6.2. Type checking.	73
8.2.7.7. Dynamically-sized bit-strings	73
8.2.7.7.1. Operations	73
8.2.7.7.2. Type checking.	74
8.3. Named types.	74
8.3.1. Type definitions (type constructors)	76
8.3.2. Type resolution	77
8.3.3. Type specialization	77
8.3.4. Prefixed type names.	78
8.3.4.1. Operations.	78
8.3.4.2. Type checking	78
8.3.5. Specialized types.	79
8.3.5.1. Type checking	79
8.4. Data types and values.	79
8.4.1. Lists	80
8.4.1.1. Operations.	80
8.4.1.2. Type checking	81
8.4.1.3. Well-formedness	81
8.4.2. Tuples	81
8.4.2.1. Operations.	81
8.4.2.2. Type checking	82
8.4.2.3. Well-formedness	82
8.4.3. Header stacks	82
8.4.3.1. Operations.	83
8.4.3.2. Type checking	84
8.4.3.3. Well-formedness	85
8.4.4. Arrays	85
8.4.5. Structs.	85
8.4.5.1. Operations.	86
8.4.5.2. Type checking	86
8.4.5.3. Well-formedness	87
8.4.6. Headers	87
8.4.6.1. Operations.	89
8.4.6.2. Type checking	89
8.4.6.3. Well-formedness	89
8.4.7. Header unions.	90
8.4.7.1. Operations.	90
8.4.7.2. Type checking	93
8.4.7.3. Well-formedness	94
8.4.8. Enums.	94
8.4.8.1. Enum type declaration without an underlying type	95
8.4.8.2. Enum type declaration with an underlying type	95
8.4.8.3. Operations.	96
8.4.8.4. Type checking	98
8.4.8.5. Well-formedness	99
8.5. Object types and values	99
8.5.1. Extern object types	99
8.5.1.1. Type checking	100
8.5.1.2. Well-formedness	100
8.5.2. Parser object types.	100

8.5.2.1. Type checking	101
8.5.2.2. Well-formedness	101
8.5.3. Control object types	101
8.5.3.1. Type checking	102
8.5.3.2. Well-formedness	102
8.5.4. Package object types	102
8.5.4.1. Type checking	102
8.5.4.2. Well-formedness	103
8.5.5. Table object types	103
8.5.5.1. Well-formedness	103
8.5.6. Object reference values	103
8.5.7. typedef declaration	104
8.5.7.1. Type checking	104
8.5.7.2. Well-formedness	104
8.5.8. type declaration	105
8.5.8.1. Operations	105
8.5.8.2. Type checking	105
8.5.8.3. Well-formedness	106
8.6. Synthesized types and values	106
8.6.1. Default	106
8.6.1.1. Operations	107
8.6.1.2. Well-formedness	107
8.6.2. Invalid headers	107
8.6.2.1. Operations	108
8.6.2.2. Well-formedness	108
8.6.3. Sequences	108
8.6.3.1. Operations	108
8.6.3.2. Well-formedness	109
8.6.4. Records	109
8.6.4.1. Operations	110
8.6.4.2. Well-formedness	111
8.6.5. Sets	111
8.6.5.1. Operations	111
8.6.5.1.1. Singleton sets	112
8.6.5.1.2. The universal set	112
8.6.5.1.3. Masks	112
8.6.5.1.4. Ranges	113
8.6.5.1.5. Products	113
8.6.5.2. Well-formedness	113
8.6.6. Table metadata	114
8.6.6.1. Operations	114
8.6.6.2. Well-formedness	114
8.7. Unrolling	115
9. P4 callables	115
9.1. Parameters	117
9.1.1. Type checking	117
9.1.2. Well-formedness	118
9.2. Callable definitions (callable constructors)	119
9.2.1. Well-formedness	120
9.3. Actions	121

9.3.1. Well-formedness	122
9.4. Functions	122
9.4.1. User-defined functions	122
9.4.1.1. Well-formedness	123
9.4.2. Extern functions	124
9.4.2.1. Well-formedness	124
9.5. Methods	125
9.5.1. Extern methods	125
9.5.1.1. Well-formedness	126
9.5.2. Parser apply methods	127
9.5.2.1. Well-formedness	128
9.5.3. Control apply methods	128
9.5.3.1. Well-formedness	128
9.5.4. Table apply methods	129
9.5.4.1. Well-formedness	129
9.5.5. Built-in methods	129
10. P4 objects and constructors	130
10.1. Objects	130
10.1.1. Fully-qualified names	130
10.2. Constructors	130
10.2.1. Constructor parameters	131
10.2.1.1. Well-formedness	132
10.2.2. Constructor definitions	132
10.2.2.1. Well-formedness	133
10.3. Extern objects	133
10.3.1. Well-formedness	134
10.3.2. Instantiation	134
10.4. Parser objects	136
10.4.1. Well-formedness	137
10.4.2. Instantiation	137
10.5. Control objects	138
10.5.1. Well-formedness	139
10.5.2. Instantiation	139
10.6. Table objects	141
10.6.1. Well-formedness	142
10.7. Parser value set objects	142
10.8. Package objects	143
10.8.1. Well-formedness	144
10.8.2. Instantiation	144
11. Declarations	145
11.1. Semantics of declarations	146
11.1.1. Type checking	146
11.1.2. Compile-time evaluation	147
11.1.3. Runtime evaluation	147
11.2. Variable declarations	147
11.2.1. Type checking	148
11.2.2. Runtime evaluation	149
11.3. Constant declarations	150
11.3.1. Type checking	151
11.3.2. Compile-time evaluation	152

11.3.3. Runtime evaluation	152
11.4. Instantiations	153
11.4.1. Objects with abstract methods	154
11.4.1.1. Type checking.	154
11.4.1.2. Compile-time evaluation.	156
11.4.2. Type checking	157
11.4.3. Compile-time evaluation	160
11.5. Function declarations	161
11.5.1. Type checking	161
11.5.2. Compile-time evaluation	162
11.6. Action declarations	163
11.6.1. Type checking	163
11.6.2. Compile-time evaluation	164
11.7. Error declarations	165
11.7.1. Type checking	165
11.7.2. Compile-time evaluation	166
11.8. Match kind declarations	166
11.8.1. Type checking	167
11.8.2. Compile-time evaluation	167
11.9. Extern function declarations	168
11.9.1. Type checking	168
11.9.2. Compile-time evaluation	168
11.10. Extern object declarations	169
11.10.1. Extern methods	169
11.10.1.1. Type checking.	169
11.10.2. Extern constructors	171
11.10.2.1. Type checking.	171
11.10.3. Type checking	172
11.10.4. Compile-time evaluation	173
11.11. Parser declarations	174
11.11.1. Type checking	175
11.11.2. Compile-time evaluation	176
11.12. Control declarations	176
11.12.1. Type checking	177
11.12.2. Compile-time evaluation	178
11.13. Enum type declaration	179
11.13.1. Enum type declarations without an underlying type	179
11.13.1.1. Type checking.	179
11.13.1.2. Compile-time evaluation	180
11.13.2. Enum type declarations with an underlying type	180
11.13.2.1. Type checking.	180
11.13.2.2. Compile-time evaluation	182
11.14. Struct type declaration	182
11.14.1. Type checking	183
11.14.2. Compile-time evaluation	184
11.15. Header type declaration.	184
11.15.1. Type checking	184
11.15.2. Compile-time evaluation	185
11.16. Header union type declaration	186
11.16.1. Type checking	186

11.16.2. Compile-time evaluation	187
11.17. Typedef declaration	187
11.17.1. <code>typedef</code> declaration	188
11.17.1.1. Type checking.	188
11.17.1.2. Compile-time evaluation	189
11.17.2. New type declaration.	190
11.17.2.1. Type checking.	190
11.17.2.2. Compile-time evaluation	190
11.18. Parser type declaration	190
11.18.1. Type checking	191
11.18.2. Compile-time evaluation	191
11.19. Control type declaration.	192
11.19.1. Type checking	192
11.19.2. Compile-time evaluation	192
11.20. Package type declaration	193
11.20.1. Type checking	193
11.20.2. Compile-time evaluation	194
12. L-values	194
12.1. Semantics of l-values	195
12.1.1. Type checking	195
12.1.2. Runtime evaluation	195
12.1.2.1. Creating storage references.	195
12.1.2.2. Reading from storage references	196
12.1.2.3. Writing to storage references	196
12.2. Identifiers	196
12.2.1. Type checking	196
12.2.2. Creating a storage reference	197
12.2.3. Reading from a storage reference	197
12.2.4. Writing to a storage reference	197
12.3. Member accesses	197
12.3.1. Type checking	197
12.3.2. Creating a storage reference	198
12.3.3. Reading from a storage reference	199
12.3.4. Writing to a storage reference	199
12.4. Index accesses	201
12.4.1. Type checking	201
12.4.2. Creating a storage reference	202
12.4.3. Reading from a storage reference	202
12.4.4. Writing to a storage reference	203
12.5. Bit-slice accesses	203
12.5.1. Type checking	203
12.5.2. Creating a storage reference	204
12.5.3. Reading from a storage reference	205
12.5.4. Writing to a storage reference	205
12.6. Parenthesized l-values	206
12.6.1. Type checking	206
12.6.2. Creating a storage reference	206
13. Statements	206
13.1. Semantics of statements	206
13.1.1. Type checking	206

13.1.2. Compile-time evaluation	207
13.1.3. Runtime evaluation	207
13.2. Empty statements	208
13.2.1. Type checking	208
13.2.2. Compile-time evaluation	209
13.2.3. Runtime evaluation	209
13.3. Assignment statements	209
13.3.1. Type checking	210
13.3.2. Compile-time evaluation	211
13.3.3. Runtime evaluation	211
13.4. Call statements	212
13.4.1. Type checking	212
13.4.2. Compile-time evaluation	214
13.4.3. Runtime evaluation	214
13.5. Direct type invocation	215
13.5.1. Type checking	216
13.5.2. Compile-time evaluation	217
13.6. Return statements	218
13.6.1. Type checking	218
13.6.2. Compile-time evaluation	219
13.6.3. Runtime evaluation	219
13.7. Exit statements	219
13.7.1. Type checking	220
13.7.2. Compile-time evaluation	221
13.7.3. Runtime evaluation	221
13.8. Block statements	221
13.8.1. Type checking	221
13.8.2. Compile-time evaluation	223
13.8.3. Runtime evaluation	224
13.9. Conditional statements	226
13.9.1. Type checking	226
13.9.2. Compile-time evaluation	227
13.9.3. Runtime evaluation	228
13.10. For statements	229
13.10.1. 3-clause for statements	230
13.10.1.1. Type checking	230
13.10.1.2. Compile-time evaluation	233
13.10.1.3. Runtime evaluation	233
13.10.2. for-in statements	237
13.10.2.1. Type checking	237
13.10.2.2. Compile-time evaluation	239
13.10.2.3. Runtime evaluation	240
13.11. Break statements	242
13.11.1. Type checking	242
13.11.2. Compile-time evaluation	242
13.11.3. Runtime evaluation	243
13.12. Continue statements	243
13.12.1. Type checking	243
13.12.2. Compile-time evaluation	243
13.12.3. Runtime evaluation	243

13.13. Switch statements	244
13.13.1. Notes common to all switch statements	244
13.13.2. Switch statement on match-action table result.	245
13.13.2.1. Type checking.	245
13.13.2.2. Compile-time evaluation	247
13.13.2.3. Runtime evaluation	248
13.13.3. Switch statement on numeric or enumerated type	249
13.13.3.1. Type checking.	250
13.13.3.2. Compile-time evaluation	252
13.13.3.3. Runtime evaluation	252
14. Expressions	254
14.1. Semantics of expressions.	255
14.1.1. Type checking	255
14.1.2. Local compile-time evaluation	256
14.1.3. Compile-time evaluation	256
14.1.4. Runtime evaluation	256
14.2. Literal expressions	257
14.2.1. Type checking	257
14.2.2. Local compile-time evaluation	258
14.2.3. Compile-time evaluation	258
14.2.4. Runtime evaluation	259
14.3. Reference expressions	259
14.3.1. Type checking	259
14.3.2. Local compile-time evaluation	261
14.3.3. Compile-time evaluation	261
14.3.4. Runtime evaluation	262
14.4. Default expressions.	262
14.4.1. Type checking	263
14.4.2. Local compile-time evaluation	263
14.4.3. Compile-time evaluation	263
14.4.4. Runtime evaluation	263
14.5. Unary expressions.	263
14.5.1. Type checking	264
14.5.1.1. Negation	264
14.5.1.2. Bitwise complement	264
14.5.1.3. Plus and minus.	265
14.5.2. Local compile-time evaluation	265
14.5.3. Compile-time evaluation	265
14.5.4. Runtime evaluation	265
14.6. Binary expressions	266
14.6.1. Type checking	266
14.6.1.1. Plus, minus, and multiplication	266
14.6.1.2. Saturating plus and minus	267
14.6.1.3. Division and modulo	267
14.6.1.4. Shift left and right	268
14.6.1.5. Equality and inequality	269
14.6.1.6. Comparison	269
14.6.1.7. Bitwise and, xor, and or	270
14.6.1.8. Concatenation	270
14.6.1.9. Logical and, and or	271

14.6.2. Local compile-time evaluation	271
14.6.3. Compile-time evaluation	272
14.6.4. Runtime evaluation	273
14.7. Ternary expressions	274
14.7.1. Type checking	275
14.7.2. Local compile-time evaluation	275
14.7.3. Compile-time evaluation	276
14.7.4. Runtime evaluation	276
14.8. Cast expressions	277
14.8.1. Type checking	277
14.8.2. Local compile-time evaluation	277
14.8.3. Compile-time evaluation	277
14.8.4. Runtime evaluation	278
14.9. Invalid header expressions	278
14.9.1. Type checking	278
14.9.2. Local compile-time evaluation	279
14.9.3. Compile-time evaluation	279
14.9.4. Runtime evaluation	279
14.10. Sequence expressions	279
14.10.1. Type checking	279
14.10.2. Local compile-time evaluation	281
14.10.3. Compile-time evaluation	281
14.10.4. Runtime evaluation	282
14.11. Record expressions	282
14.11.1. Type checking	283
14.11.2. Local compile-time evaluation	285
14.11.3. Compile-time evaluation	285
14.11.4. Runtime evaluation	286
14.12. Error access expression	286
14.12.1. Type checking	287
14.12.2. Local compile-time evaluation	287
14.12.3. Compile-time evaluation	287
14.12.4. Runtime evaluation	287
14.13. Member access expressions	288
14.13.1. Type checking	288
14.13.2. Local compile-time evaluation	290
14.13.3. Compile-time evaluation	291
14.13.4. Runtime evaluation	292
14.14. Index access expressions	294
14.14.1. Type checking	294
14.14.2. Local compile-time evaluation	296
14.14.3. Compile-time evaluation	296
14.14.4. Runtime evaluation	297
14.15. Bitslice access expressions	298
14.15.1. Type checking	298
14.15.2. Local compile-time evaluation	299
14.15.3. Compile-time evaluation	299
14.15.4. Runtime evaluation	300
14.16. Call expressions	300
14.16.1. Type checking	301

14.16.2. Local compile-time evaluation	304
14.16.3. Compile-time evaluation	305
14.16.4. Runtime evaluation	307
14.17. Parenthesized expressions	307
14.17.1. Type checking	307
14.17.2. Local compile-time evaluation	308
14.17.3. Compile-time evaluation	308
14.17.4. Runtime evaluation	308
15. Casting	308
15.1. Explicit casts	309
15.2. Implicit casts	309
15.3. Performing casts	309
15.3.1. From a Boolean value	310
15.3.2. From an integer value	311
15.3.3. From an error value	313
15.3.4. From a list value	313
15.3.5. From a header stack value	314
15.3.6. From a struct value	314
15.3.7. From a header value	314
15.3.8. From an enum value	314
15.3.9. From a default value	315
15.3.10. From an invalid header value	315
15.3.11. From a sequence value	315
15.3.12. From a record value	316
15.3.13. From a set value	317
16. Abstract Parser Machine	318
16.1. Parser local declarations	318
16.1.1. Constant declarations	318
16.1.2. Instantiations	318
16.1.3. Variable declarations	318
16.1.4. Parser value set declarations	318
16.2. Parser states	319
16.3. Parser statements	319
16.3.1. Constant declaration	319
16.3.2. Variable declaration	319
16.3.3. Empty statement	320
16.3.4. Assignment statement	320
16.3.5. Call statement	320
16.3.6. Direct application statement	320
16.3.7. Block statement	320
16.3.8. Conditional statement	321
16.4. Parser transition statements	321
16.4.1. Parser select expression	321
17. Abstract Control Machine	322
17.1. Control local declarations	322
17.1.1. Constant declarations	322
17.1.2. Instantiations	322
17.1.3. Variable declarations	322
17.1.4. Action declarations	323
17.1.5. Table declarations	323

17.2. Match-action tables	323
17.2.1. Table keys	323
17.2.2. Table actions	324
17.2.3. Table default actions	324
17.2.4. Table entries	324
17.2.5. Table custom properties	324
18. Calls	324
18.1. Call targets	325
18.2. Overload resolution	328
18.2.1. Positional arguments	329
18.2.2. Named arguments	330
18.3. Callee resolution	330
18.3.1. Resolving constructors.	331
18.3.2. Resolving callables	332
18.3.2.1. Actions.	333
18.3.2.2. Functions.	334
18.3.2.3. Built-in methods.	335
18.3.2.4. Extern methods	338
18.3.2.5. Parser apply methods.	340
18.3.2.6. Control apply methods.	341
18.3.2.7. Table apply methods.	343
18.4. Call convention.	344
18.5. Runtime evaluation of calls	344
19. Operations	344
19.1. Unary operations.	344
19.1.1. Negation.	345
19.1.2. Bitwise complement.	345
19.1.3. Plus	345
19.1.4. Minus	346
19.2. Binary operations	346
19.2.1. Addition	347
19.2.2. Saturated addition	348
19.2.3. Subtraction	349
19.2.4. Saturated subtraction	350
19.2.5. Multiplication.	351
19.2.6. Division.	352
19.2.7. Modulus	352
19.2.8. Shift left	353
19.2.9. Shift right	354
19.2.10. Equality and inequality	356
19.2.11. Comparisons	361
19.2.12. Bitwise	364
19.3. Default values.	365
20. Annotations	368
Appendix A: Revision History	370
A.1. Summary of changes made in unreleased version	370
A.2. Summary of changes made in version 1.2.5, released October 11, 2024	370
A.3. Summary of changes made in version 1.2.4, released May 15, 2023	370
A.4. Summary of changes made in version 1.2.3, released July 11, 2022.	371
A.5. Summary of changes made in version 1.2.2, released May 17, 2021	372

A.6. Summary of changes made in version 1.2.1, released June 11, 2020.....	372
A.7. Summary of changes made in version 1.2.0, released October 14, 2019	373
A.8. Summary of changes made in version 1.1.0, released November 26, 2017.....	373
A.9. Initial version 1.0.0, released May 17, 2017	374
Appendix B: P4 reserved keywords	374
Appendix C: P4 reserved annotations	374
Appendix D: P4 core library	375
Appendix E: P4 grammar	376

Abstract

P4 is a language for programming the data plane of network devices. This document provides a precise definition of the P4₁₆ language, which is the 2016 revision of the P4 language (<http://p4.org>). The target audience for this document includes developers who want to write compilers, simulators, IDEs, and debuggers for P4 programs. This document may also be of interest to P4 programmers who are interested in understanding the syntax and semantics of the language at a deeper level.

1. Scope

This specification document defines the structure and interpretation of programs in the P4₁₆ language. It defines the syntax, semantic rules, and requirements for conformant implementations of the language.

It does not define:

- Mechanisms by which P4 programs are compiled, loaded, and executed on packet-processing systems,
- Mechanisms by which data are received by one packet-processing system and delivered to another system,
- Mechanisms by which the control plane manages the match-action tables and other stateful objects defined by P4 programs,
- The size or complexity of P4 programs,
- The minimal requirements of packet-processing systems that are capable of providing a conformant implementation.

It is understood that some implementations may be unable to implement the behavior defined here in all cases, or may provide options to eliminate some safety guarantees in exchange for better performance or handling larger programs. They should document where they deviate from this specification.

2. Terms, definitions, and symbols

Throughout this document, the following terms will be used:

- **Architecture:** A set of P4-programmable components and the data plane interfaces between them.
- **Control plane:** A class of algorithms and the corresponding input and output data that are concerned with the provisioning and configuration of the data plane.
- **Data plane:** A class of algorithms that describe transformations on packets by packet-processing systems.
- **Metadata:** Intermediate data generated during execution of a P4 program.
- **Packet:** A network packet is a formatted unit of data carried by a packet-switched network.
- **Packet header:** Formatted data at the beginning of a packet. A given packet may contain a sequence of packet headers representing different network protocols.
- **Packet payload:** Packet data that follows the packet headers.
- **Packet-processing system:** A data-processing system designed for processing network packets. In general, packet-processing systems implement control plane and data plane algorithms.
- **Target:** A packet-processing system capable of executing a P4 program.

All terms defined explicitly in this document should not be understood to refer implicitly to similar terms defined elsewhere. Conversely, any terms not defined explicitly in this document should be interpreted

according to generally recognizable sources---e.g., IETF RFCs.

3. Overview

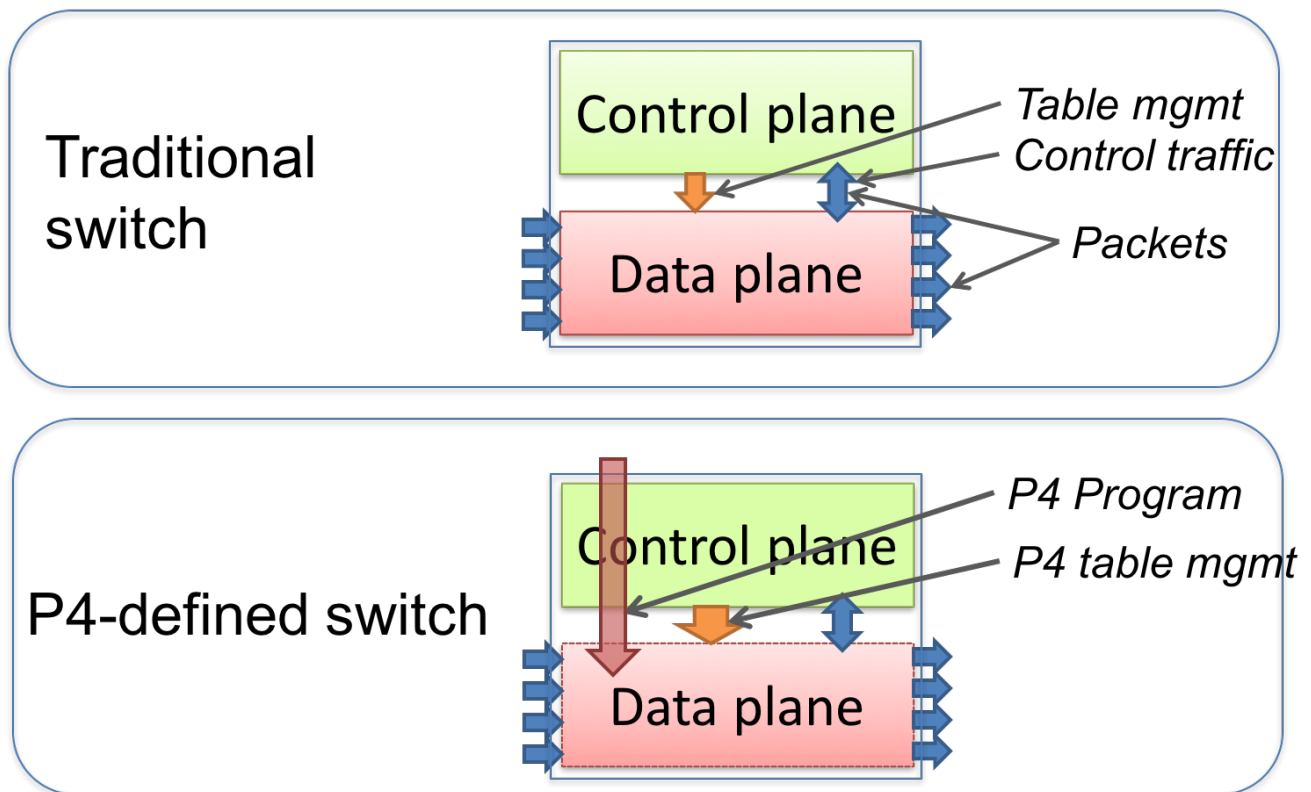


Figure 1. Traditional switches vs. programmable switches.

P4 is a language for expressing how packets are processed by the data plane of a programmable forwarding element such as a hardware or software switch, network interface card, router, or network appliance. The name P4 comes from the original paper that introduced the language, "Programming Protocol-independent Packet Processors," <https://arxiv.org/pdf/1312.1719.pdf>. While P4 was initially designed for programming switches, its scope has been broadened to cover a large variety of devices. In the rest of this document we use the generic term *target* for all such devices.

Many targets implement both a control plane and a data plane. P4 is designed to specify only the data plane functionality of the target. P4 programs also partially define the interface by which the control plane and the data-plane communicate, but P4 cannot be used to describe the control-plane functionality of the target. In the rest of this document, when we talk about P4 as "programming a target", we mean "programming the data plane of a target".

As a concrete example of a target, Figure 1 illustrates the difference between a traditional fixed-function switch and a P4-programmable switch. In a traditional switch the manufacturer defines the data-plane functionality. The control-plane controls the data plane by managing entries in tables (e.g. routing tables), configuring specialized objects (e.g. meters), and by processing control-packets (e.g. routing protocol packets) or asynchronous events, such as link state changes or learning notifications.

A P4-programmable switch differs from a traditional switch in two essential ways:

- The data plane functionality is not fixed in advance but is defined by a P4 program. The data plane is configured at initialization time to implement the functionality described by the P4 program (shown by the long red arrow) and has no built-in knowledge of existing network protocols.
- The control plane communicates with the data plane using the same channels as in a fixed-function device, but the set of tables and other objects in the data plane are no longer fixed, since they are defined by a P4 program. The P4 compiler generates the API that the control plane uses to

communicate with the data plane.

Hence, P4 can be said to be protocol independent, but it enables programmers to express a rich set of protocols and other data plane behaviors.

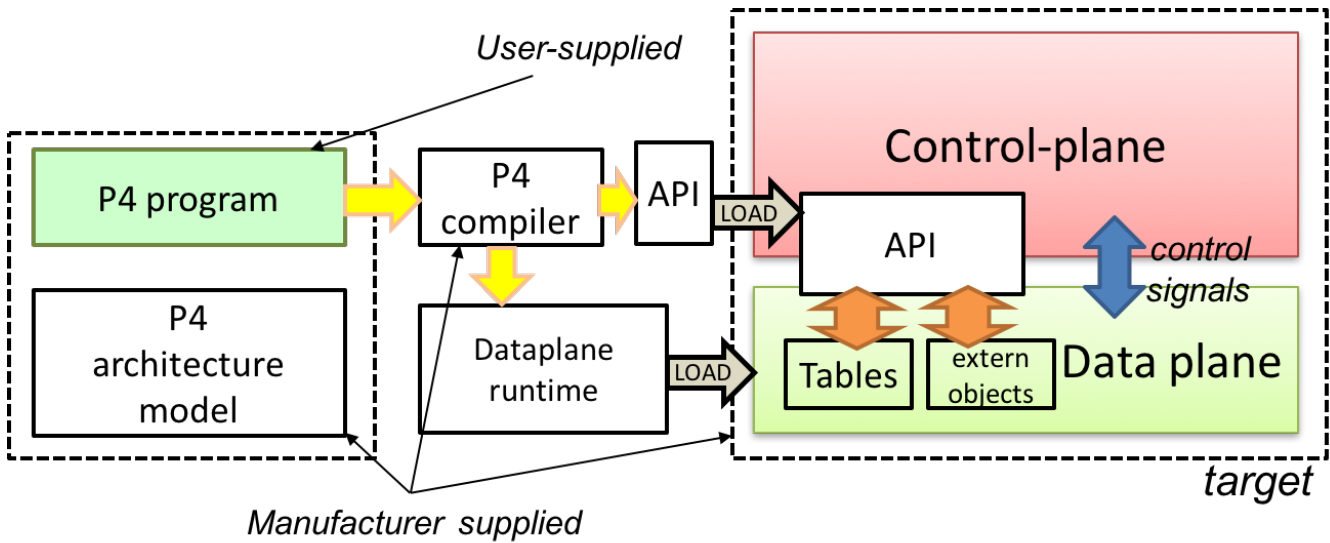


Figure 2. Programming a target with P4.

The core abstractions provided by the P4 language are:

- **Header types** describe the format (the set of fields and their sizes) of each header within a packet.
- **Parsers** describe the permitted sequences of headers within received packets, how to identify those header sequences, and the headers and fields to extract from packets.
- **Tables** associate user-defined keys with actions. P4 tables generalize traditional switch tables; they can be used to implement routing tables, flow lookup tables, access-control lists, and other user-defined table types, including complex multi-variable decisions.
- **Actions** are code fragments that describe how packet header fields and metadata are manipulated. Actions can include data, which is supplied by the control-plane at runtime.
- **Match-action units** perform the following sequence of operations:
 - Construct lookup keys from packet fields or computed metadata,
 - Perform table lookup using the constructed key, choosing an action (including the associated data) to execute, and
 - Finally, execute the selected action.
- **Control flow** expresses an imperative program that describes packet-processing on a target, including the data-dependent sequence of match-action unit invocations. Deparsing (packet reassembly) can also be performed using a control flow.
- **Extern objects** are architecture-specific constructs that can be manipulated by P4 programs through well-defined APIs, but whose internal behavior is hard-wired (e.g., checksum units) and hence not programmable using P4.
- **User-defined metadata**: user-defined data structures associated with each packet.
- **Intrinsic metadata**: metadata provided by the architecture associated with each packet—e.g., the input port where a packet has been received.

Figure 2 shows a typical tool workflow when programming a target using P4.

Target manufacturers provide the hardware or software implementation framework, an architecture definition, and a P4 compiler for that target. P4 programmers write programs for a specific architecture, which defines a set of P4-programmable components on the target as well as their external data plane

interfaces.

Compiling a set of P4 programs produces two artifacts:

- a data plane configuration that implements the forwarding logic described in the input program and
- an API for managing the state of the data plane objects from the control plane

P4 is a domain-specific language that is designed to be implementable on a large variety of targets including programmable network interface cards, FPGAs, software switches, and hardware ASICs. As such, the language is restricted to constructs that can be efficiently implemented on all of these platforms.

Assuming a fixed cost for table lookup operations and interactions with extern objects, all P4 programs (i.e., parsers and controls) execute a constant number of operations for each byte of an input packet received and analyzed. Although parsers may contain loops, provided some header is extracted on each cycle, the packet itself provides a bound on the total execution of the parser. In other words, under these assumptions, the computational complexity of a P4 program is linear in the total size of all headers, and never depends on the size of the state accumulated while processing data (e.g., the number of flows, or the total number of packets processed). These guarantees are necessary (but not sufficient) for enabling fast packet processing across a variety of targets.

P4 conformance of a target is defined as follows: if a specific target T supports only a subset of the P4 programming language, say $P4^T$, programs written in $P4^T$ executed on the target should provide the exact same behavior as is described in this document. Note that P4 conformant targets can provide arbitrary P4 language extensions and `extern` elements.

3.1. Benefits of P4

Compared to state-of-the-art packet-processing systems (e.g., based on writing microcode on top of custom hardware), P4 provides a number of significant advantages:

- **Flexibility:** P4 makes many packet-forwarding policies expressible as programs, in contrast to traditional switches, which expose fixed-function forwarding engines to their users.
- **Expressiveness:** P4 can express sophisticated, hardware-independent packet processing algorithms using solely general-purpose operations and table look-ups. Such programs are portable across hardware targets that implement the same architectures (assuming sufficient resources are available).
- **Resource mapping and management:** P4 programs describe storage resources abstractly (e.g., IPv4 source address); compilers map such user-defined fields to available hardware resources and manage low-level details such as allocation and scheduling.
- **Software engineering:** P4 programs provide important benefits such as type checking, information hiding, and software reuse.
- **Component libraries:** Component libraries supplied by manufacturers can be used to wrap hardware-specific functions into portable high-level P4 constructs.
- **Decoupling hardware and software evolution:** Target manufacturers may use abstract architectures to further decouple the evolution of low-level architectural details from high-level processing.
- **Debugging:** Manufacturers can provide software models of an architecture to aid in the development and debugging of P4 programs.

3.2. P4 language evolution: comparison to previous versions (P4 v1.0/v1.1)

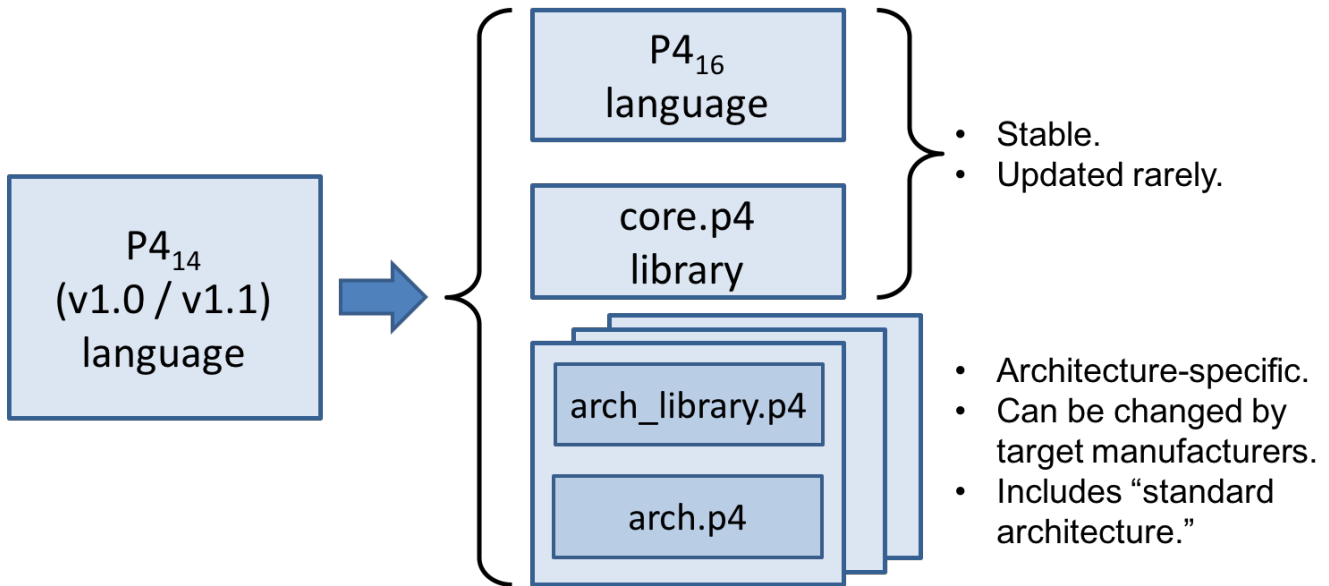


Figure 3. Evolution of the language between versions $P4_{14}$ (versions 1.0 and 1.1) and $P4_{16}$.

Compared to $P4_{14}$, the earlier version of the language, $P4_{16}$ makes a number of significant, backwards-incompatible changes to the syntax and semantics of the language. The evolution from the previous version ($P4_{14}$) to the current one ($P4_{16}$) is depicted in Figure 3. In particular, a large number of language features have been eliminated from the language and moved into libraries including counters, checksum units, meters, etc.

Hence, the language has been transformed from a complex language (more than 70 keywords) into a relatively small core language (less than 40 keywords, shown in Appendix B) accompanied by a library of fundamental constructs that are needed for writing most P4.

The v1.1 version of P4 introduced a language construct called `extern` that can be used to describe library elements. Many constructs defined in the v1.1 language specification will thus be transformed into such library elements (including constructs that have been eliminated from the language, such as counters and meters). Some of these `extern` objects are expected to be standardized, and they will be in the scope of a future document describing a standard library of P4 elements. In this document we provide several examples of `extern` constructs. $P4_{16}$ also introduces and repurposes some v1.1 language constructs for describing the programmable parts of an architecture. These language constructs are: `parser`, `state`, `control`, and `package`.

One important goal of the $P4_{16}$ language revision is to provide a **stable** language definition. In other words, we strive to ensure that all programs written in $P4_{16}$ will remain syntactically correct and behave identically when treated as programs for future versions of the language. Moreover, if some future version of the language requires breaking backwards compatibility, we will seek to provide an easy path for migrating $P4_{16}$ programs to the new version.

4. Architecture Model

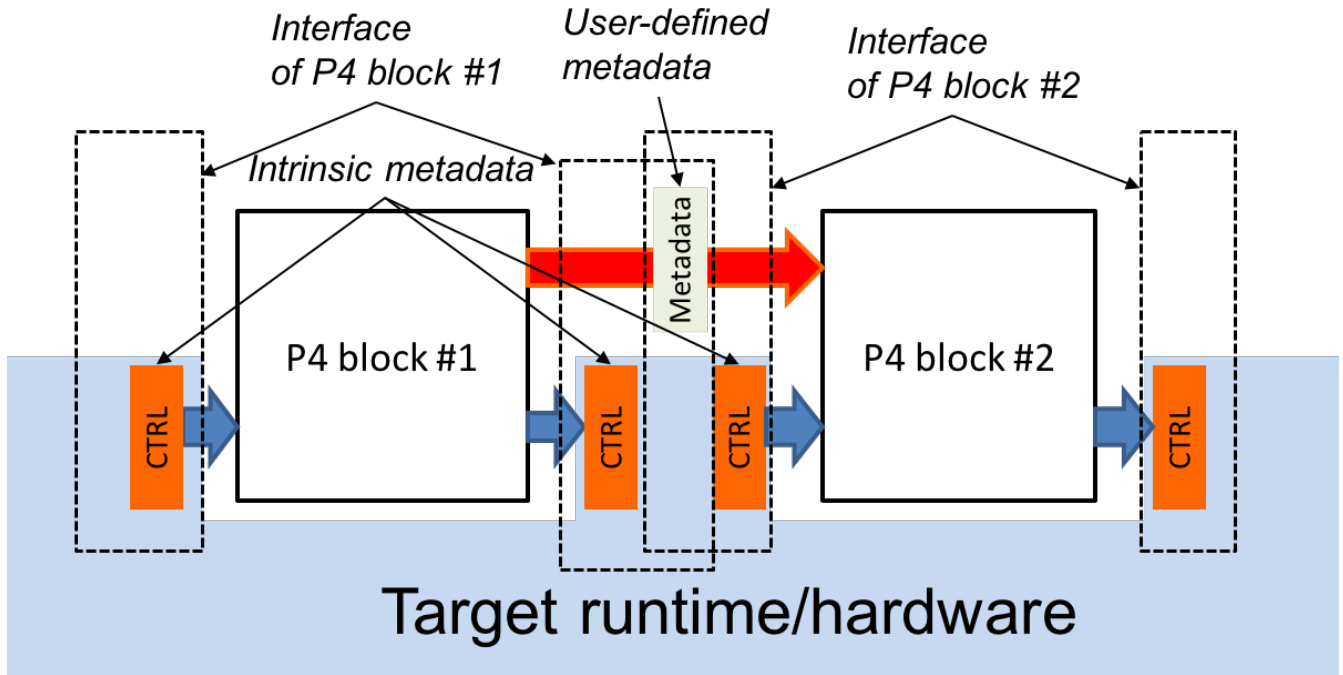


Figure 4. P4 program interfaces.

The *P4 architecture* identifies the P4-programmable blocks (e.g., parser, ingress control flow, egress control flow, deparser, etc.) and their data plane interfaces.

The P4 architecture can be thought of as a contract between the program and the target. Each manufacturer must therefore provide both a P4 compiler as well as an accompanying architecture definition for their target. (We expect that P4 compilers can share a common front-end that handles all architectures). The architecture definition does not have to expose the entire programmable surface of the data plane—a manufacturer may even choose to provide multiple definitions for the same hardware device, each with different capabilities (e.g., with or without multicast support).

Figure 4 illustrates the data plane interfaces between P4-programmable blocks. It shows a target that has two programmable blocks (#1 and #2). Each block is programmed through a separate fragment of P4 code. The target interfaces with the P4 program through a set of control registers or signals. Input controls provide information to P4 programs (e.g., the input port that a packet was received from), while output controls can be written to by P4 programs to influence the target behavior (e.g., the output port where a packet has to be directed). Control registers/signals are represented in P4 as *intrinsic metadata*. P4 programs can also store and manipulate data pertaining to each packet as *user-defined metadata*.

The behavior of a P4 program can be fully described in terms of transformations that map vectors of bits to vectors of bits. To actually process a packet, the architecture model interprets the bits that the P4 program writes to intrinsic metadata. For example, to cause a packet to be forwarded on a specific output port, a P4 program may need to write the index of an output port into a dedicated control register. Similarly, to cause a packet to be dropped, a P4 program may need to set a "drop" bit into another dedicated control register. Note that the details of how intrinsic metadata are interpreted is architecture-specific.

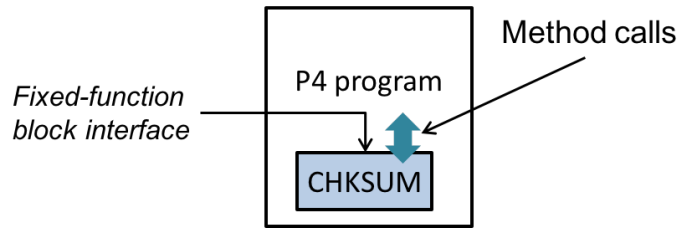


Figure 5. P4 program invoking the services of a fixed-function object.

P4 programs can invoke services implemented by extern objects and functions provided by the architecture. Figure 5 depicts a P4 program invoking the services of a built-in checksum computation unit on a target. The implementation of the checksum unit is not specified in P4, but its interface is. In general, the interface for an extern object describes each operation it provides, as well as their parameter and return types.

In general, P4 programs are not expected to be portable across different architectures. For example, executing a P4 program that broadcasts packets by writing into a custom control register will not function correctly on a target that does not have the control register. However, P4 programs written for a given architecture should be portable across all targets that faithfully implement the corresponding model, provided there are sufficient resources.

4.1. Standard architectures

We expect that the P4 community will evolve a small set of standard architecture models pertaining to specific verticals. Wide adoption of such standard architectures will promote portability of P4 programs across different targets. However, defining these standard architectures is outside of the scope of this document.

4.2. Data plane interfaces

To describe a functional block that can be programmed in P4, the architecture includes a type declaration that specifies the interfaces between the block and the other components in the architecture. For example, the architecture might contain a declaration such as the following:

```
control MatchActionPipe<H>(in bit<4> inputPort,
                           inout H parsedHeaders,
                           out bit<4> outputPort);
```

This type declaration describes a block named `MatchActionPipe` that can be programmed using a data-dependent sequence of match-action unit invocations and other imperative constructs (indicated by the `control` keyword). The interface between the `MatchActionPipe` block and the other components of the architecture can be read off from this declaration:

- The first parameter is a 4-bit value named `inputPort`. The direction `in` indicates that this parameter is an input that cannot be modified.
- The second parameter is an object of type `H` named `parsedHeaders`, where `H` is a type variable representing the headers that will be defined later by the P4 programmer. The direction `inout` indicates that this parameter is both an input and an output.
- The third parameter is a 4-bit value named `outputPort`. The direction `out` indicates that this parameter is an output whose value is undefined initially but can be modified.

4.3. Extern objects and functions

P4 programs can also interact with objects and functions provided by the architecture. Such objects are

described using the `extern` construct, which describes the interfaces that such objects expose to the data-plane.

An `extern` object describes a set of methods that are implemented by an object, but not the implementation of these methods (i.e., it is similar to an abstract class in an object-oriented language). For example, the following construct could be used to describe the operations offered by an incremental checksum unit:

```
extern Checksum16 {  
    Checksum16();           // constructor  
    void clear();           // prepare unit for computation  
    void update<T>(in T data); // add data to checksum  
    void remove<T>(in T data); // remove data from existing checksum  
    bit<16> get();         // get the checksum for the data added since last clear  
}
```

5. Example: A very simple switch

As an example to illustrate the features of architectures, consider implementing a very simple switch in P4. We will first describe the architecture of the switch and then write a complete P4 program that specifies the data plane behavior of the switch. This example demonstrates many important features of the P4 programming language.

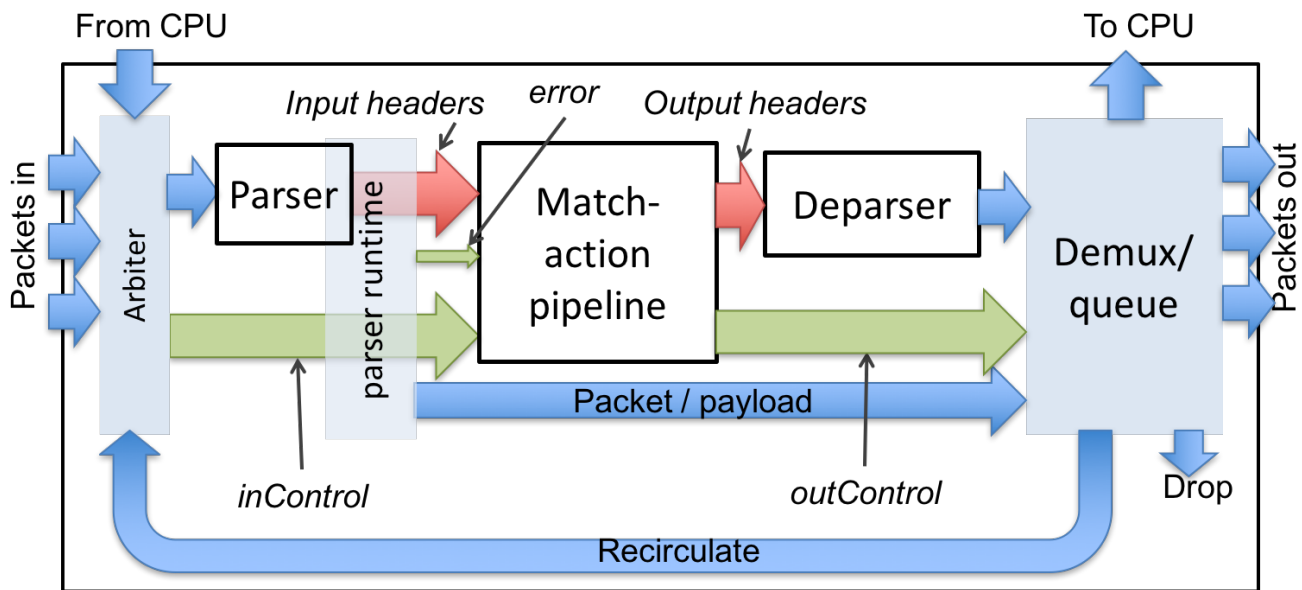


Figure 6. The Very Simple Switch (VSS) architecture.

We call our architecture the "Very Simple Switch" (VSS). [Figure 6](#) is a diagram of this architecture. There is nothing inherently special about VSS---it is just a pedagogical example that illustrates how programmable switches can be described and programmed in P4. VSS has a number of fixed-function blocks (shown in cyan in our example), whose behavior is described in [Section 5.2](#). The white blocks are programmable using P4.

VSS receives packets through one of 8 input Ethernet ports, through a recirculation channel, or from a port connected directly to the CPU. VSS has one single parser, feeding into a single match-action pipeline, which feeds into a single deparser. After exiting the deparser, packets are emitted through one of 8 output Ethernet ports or one of 3 "special" ports:

- Packets sent to the "CPU port" are sent to the control plane
- Packets sent to the "Drop port" are discarded

- Packets sent to the "Recirculate port" are re-injected in the switch through a special input port

The white blocks in the figure are programmable, and the user must provide a corresponding P4 program to specify the behavior of each such block. The red arrows indicate the flow of user-defined data. The cyan blocks are fixed-function components. The green arrows are data plane interfaces used to convey information between the fixed-function blocks and the programmable blocks—exposed in the P4 program as intrinsic metadata.

5.1. Very Simple Switch Architecture

The following P4 program provides a declaration of VSS in P4, as it would be provided by the VSS manufacturer. The declaration contains several type declarations, constants, and finally declarations for the three programmable blocks; the code uses syntax highlighting. The programmable blocks are described by their types; the implementation of these blocks has to be provided by the switch programmer.

```
// File "very_simple_switch_model.p4"
// Very Simple Switch P4 declaration
// core library needed for packet_in and packet_out definitions
# include <core.p4>
/* Various constants and structure declarations */
/* ports are represented using 4-bit values */
typedef bit<4> PortId;
/* only 8 ports are "real" */
const PortId REAL_PORT_COUNT = 4w8; // 4w8 is the number 8 in 4 bits
/* metadata accompanying an input packet */
struct InControl {
    PortId inputPort;
}
/* special input port values */
const PortId RECIRCULATE_IN_PORT = 0xD;
const PortId CPU_IN_PORT = 0xE;
/* metadata that must be computed for outgoing packets */
struct OutControl {
    PortId outputPort;
}
/* special output port values for outgoing packet */
const PortId DROP_PORT = 0xF;
const PortId CPU_OUT_PORT = 0xE;
const PortId RECIRCULATE_OUT_PORT = 0xD;
/* Prototypes for all programmable blocks */
/**
 * Programmable parser.
 * @param <H> type of headers; defined by user
 * @param b input packet
 * @param parsedHeaders headers constructed by parser
 */
parser Parser<H>(packet_in b,
                 out H parsedHeaders);
/**
 * Match-action pipeline
 * @param <H> type of input and output headers
 * @param headers headers received from the parser and sent to the deparser
 * @param parseError error that may have surfaced during parsing
 * @param inCtrl information from architecture, accompanying input packet
 * @param outCtrl information for architecture, accompanying output packet
 */
control Pipe<H>(inout H headers,
                in error parseError, // parser error
                in InControl inCtrl, // input port
                out OutControl outCtrl); // output port
/**
 * VSS deparser.
 * @param <H> type of headers; defined by user
```

```

* @param b output packet
* @param outputHeaders headers for output packet
*/
control Deparser<H>(inout H outputHeaders,
                    packet_out b);
/**
* Top-level package declaration - must be instantiated by user.
* The arguments to the package indicate blocks that
* must be instantiated by the user.
* @param <H> user-defined type of the headers processed.
*/
package VSS<H>(Parser<H> p,
               Pipe<H> map,
               Deparser<H> d);
// Architecture-specific objects that can be instantiated
// Checksum unit
extern Checksum16 {
    Checksum16(); // constructor
    void clear(); // prepare unit for computation
    void update<T>(in T data); // add data to checksum
    void remove<T>(in T data); // remove data from existing checksum
    bit<16> get(); // get the checksum for the data added since last clear
}

```

Let us describe some of these elements:

- The included file `core.p4` is described in more detail in [Appendix D](#). It defines some standard data-types and error codes.
- `bit<4>` is the type of bit-strings with 4 bits.
- The syntax `4w0xF` indicates the value 15 represented using 4 bits. An alternative notation is `4w15`. In many circumstances the width modifier can be omitted, writing just `15`.
- `error` is a built-in P4 type for holding error codes
- Next follows the declaration of a parser:

```

parser Parser<H>(packet_in b, out H parsedHeaders);

```

This declaration describes the interface for a parser, but not yet its implementation, which will be provided by the programmer. The parser reads its input from a `packet_in`, which is a pre-defined P4 extern object that represents an incoming packet, declared in the `core.p4` library. The parser writes its output (the `out` keyword) into the `parsedHeaders` argument. The type of this argument is `H`, yet unknown---it will also be provided by the programmer.

- The declaration

```

control Pipe<H>(inout H headers,
                in error parseError,
                in InControl inCtrl,
                out OutControl outCtrl);

```

describes the interface of a Match-Action pipeline named `Pipe`.

The pipeline receives three inputs: the headers `headers`, a parser error `parseError`, and the `inCtrl` control data. [Figure 6](#) indicates the different sources of these pieces of information. The pipeline writes its outputs into `outCtrl`, and it must update in place the headers to be consumed by the deparser.

- The top-level package is called `VSS`; in order to program a VSS, the user will have to instantiate a package of this type (shown in the next section). The top-level package declaration also depends on a

type variable `H`:

```
package VSS<H>
```

A type variable indicates a type yet unknown that must be provided by the user at a later time. In this case `H` is the type of the set of headers that the user program will be processing; the parser will produce the parsed representation of these headers, and the match-action pipeline will update the input headers in place to produce the output headers.

- The `package VSS` declaration has three complex parameters, of types `Parser`, `Pipe`, and `Deparser` respectively; which are exactly the declarations we have just described. In order to program the target one has to supply values for these parameters.
- In this program the `inCtrl` and `outCtrl` structures represent control registers. The content of the headers structure is stored in general-purpose registers.
- The `extern Checksum16` declaration describes an extern object whose services can be invoked to compute checksums.

5.2. Very Simple Switch Architecture Description

In order to fully understand VSS's behavior and write meaningful P4 programs for it, and for implementing a control plane, we also need a full behavioral description of the fixed-function blocks. This section can be seen as a simple example illustrating all the details that have to be handled when writing an architecture description. The P4 language is not intended to cover the description of all such functional blocks---the language can only describe the interfaces between programmable blocks and the architecture. For the current program, this interface is given by the `Parser`, `Pipe`, and `Deparser` declarations. In practice we expect that the complete description of the architecture will be provided as an executable program and/or diagrams and text; in this document we will provide informal descriptions in English.

5.2.1. Arbiter block

The input arbiter block performs the following functions:

- It receives packets from one of the physical input Ethernet ports, from the control plane, or from the input recirculation port.
- For packets received from Ethernet ports, the block computes the Ethernet trailer checksum and verifies it. If the checksum does not match, the packet is discarded. If the checksum does match, it is removed from the packet payload.
- Receiving a packet involves running an arbitration algorithm if multiple packets are available.
- If the arbiter block is busy processing a previous packet and no queue space is available, input ports may drop arriving packets, without indicating the fact that the packets were dropped in any way.
- After receiving a packet, the arbiter block sets the `inCtrl.inputPort` value that is an input to the match-action pipeline with the identity of the input port where the packet originated. Physical Ethernet ports are numbered 0 to 7, while the input recirculation port has a number 13 and the CPU port has the number 14.

5.2.2. Parser runtime block

The parser runtime block works in concert with the parser. It provides an error code to the match-action pipeline, based on the parser actions, and it provides information about the packet payload (e.g., the size of the remaining payload data) to the demux block. As soon as a packet's processing is completed by the parser, the match-action pipeline is invoked with the associated metadata as inputs (packet headers and user-defined metadata).

5.2.3. Demux block

The core functionality of the demux block is to receive the headers for the outgoing packet from the deparser and the packet payload from the parser, to assemble them into a new packet and to send the result to the correct output port. The output port is specified by the value of `outCtrl.outputPort`, which is set by the match-action pipeline.

- Sending the packet to the drop port causes the packet to disappear.
- Sending the packet to an output Ethernet port numbered between 0 and 7 causes it to be emitted on the corresponding physical interface. The packet may be placed in a queue if the output interface is already busy emitting another packet. When the packet is emitted, the physical interface computes a correct Ethernet checksum trailer and appends it to the packet.
- Sending a packet to the output CPU port causes the packet to be transferred to the control plane. In this case, the packet that is sent to the CPU is the **original input packet**, and not the packet received from the deparser---the latter packet is discarded.
- Sending the packet to the output recirculation port causes it to appear at the input recirculation port. Recirculation is useful when packet processing cannot be completed in a single pass.
- If the `outputPort` has an illegal value (e.g., 9), the packet is dropped.
- Finally, if the demux unit is busy processing a previous packet and there is no capacity to queue the packet coming from the deparser, **the demux unit may drop the packet**, irrespective of the output port indicated.

Please note that some of the behaviors of the demux block may be unexpected—we have highlighted them in bold. We are not specifying here several important behaviors related to queue size, arbitration, and timing, which also influence the packet processing.

The arrow shown from the parser runtime to the demux block represents an additional information flow from the parser to the demux: the packet being processed as well as the offset within the packet where parsing ended (i.e., the start of the packet payload).

5.2.4. Available extern blocks

The VSS architecture provides an incremental checksum extern block, called `Checksum16`. The checksum unit has a constructor and four methods:

- `clear()`: prepares the unit for a new computation
- `update<T>(in T data)`: add some data to be checksummed. The data must be either a bit-string, a header-typed value, or a `struct` containing such values. The fields in the header/struct are concatenated in the order they appear in the type declaration.
- `get()`: returns the 16-bit one's complement checksum. When this function is invoked the checksum must have received an integral number of bytes of data.
- `remove<T>(in T data)`: assuming that `data` was used for computing the current checksum, `data` is removed from the checksum.

5.3. A complete Very Simple Switch program

Here we provide a complete P4 program that implements basic forwarding for IPv4 packets on the VSS architecture. This program does not utilize all of the features provided by the architecture---e.g., recirculation---but it does use preprocessor `#include` directives (see [\[sec-preprocessor\]](#)).

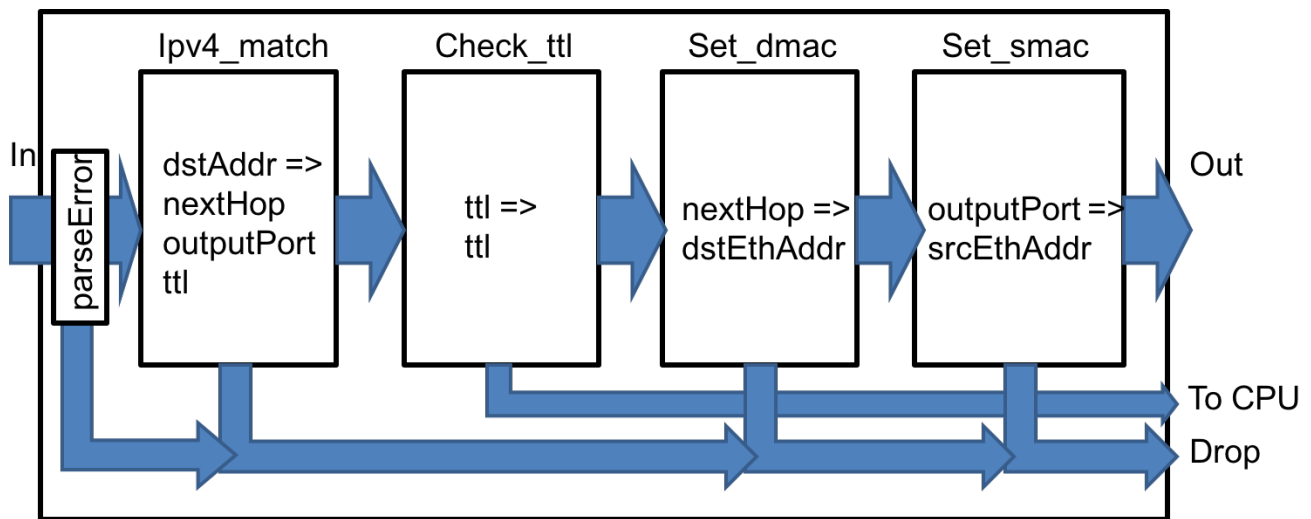


Figure 7. Diagram of the match-action pipeline expressed by the VSS P4 program.

The parser attempts to recognize an Ethernet header followed by an IPv4 header. If either of these headers are missing, parsing terminates with an error. Otherwise it extracts the information from these headers into a `Parsed_packet` structure. The match-action pipeline is shown in Figure 7; it comprises four match-action units (represented by the P4 `table` keyword):

- If any parser error has occurred, the packet is dropped (i.e., by assigning
- `outputPort` to `DROP_PORT`)
- The first table uses the IPv4 destination address to determine the
- `outputPort` and the IPv4 address of the next hop. If this lookup fails, the packet is dropped. The table also decrements the IPv4 `ttl` value.
- The second table checks the `ttl` value: if the `ttl` becomes 0, the packet is sent to the control plane through the CPU port.
- The third table uses the IPv4 address of the next hop (which was computed by the first table) to determine the Ethernet address of the next hop.
- Finally, the last table uses the `outputPort` to identify the source Ethernet address of the current switch, which is set in the outgoing packet.

The deparser constructs the outgoing packet by reassembling the Ethernet and IPv4 headers as computed by the pipeline.

```

// Include P4 core library
# include <core.p4>

// Include very simple switch architecture declarations
# include "very_simple_switch_model.p4"

// This program processes packets comprising an Ethernet and an IPv4
// header, and it forwards packets using the destination IP address

typedef bit<48>  EthernetAddress;
typedef bit<32>  IPv4Address;

// Standard Ethernet header
header Ethernet_h {
    EthernetAddress dstAddr;
    EthernetAddress srcAddr;
    bit<16>         etherType;
}

// IPv4 header (without options)
header IPv4_h {

```



```

    bit<4>      version;
    bit<4>      ihl;
    bit<8>      diffserv;
    bit<16>     totalLen;
    bit<16>     identification;
    bit<3>      flags;
    bit<13>     fragOffset;
    bit<8>      ttl;
    bit<8>      protocol;
    bit<16>     hdrChecksum;
    IPv4Address srcAddr;
    IPv4Address dstAddr;
}

// Structure of parsed headers
struct Parsed_packet {
    Ethernet_h ethernet;
    IPv4_h      ip;
}

// Parser section

// User-defined errors that may be signaled during parsing
error {
    IPv4OptionsNotSupported,
    IPv4IncorrectVersion,
    IPv4ChecksumError
}

parser TopParser(packet_in b, out Parsed_packet p) {
    Checksum16() ck; // instantiate checksum unit

    state start {
        b.extract(p.ethernet);
        transition select(p.ethernet.etherType) {
            0x0800: parse_ipv4;
            // no default rule: all other packets rejected
        }
    }

    state parse_ipv4 {
        b.extract(p.ip);
        verify(p.ip.version == 4w4, error.IPv4IncorrectVersion);
        verify(p.ip.ihl == 4w5, error.IPv4OptionsNotSupported);
        ck.clear();
        ck.update(p.ip);
        // Verify that packet checksum is zero
        verify(ck.get() == 16w0, error.IPv4ChecksumError);
        transition accept;
    }
}

// Match-action pipeline section

control TopPipe(inout Parsed_packet headers,
                in error parseError, // parser error
                in InControl inCtrl, // input port
                out OutControl outCtrl) {
    IPv4Address nextHop; // local variable

    /**
     * Indicates that a packet is dropped by setting the
     * output port to the DROP_PORT
     */
    action Drop_action() {
        outCtrl.outputPort = DROP_PORT;
    }

    /**

```



```

* Set the next hop and the output port.
* Decrements ipv4 ttl field.
* @param ipv4_dest ipv4 address of next hop
* @param port output port
*/
action Set_nhop(IPv4Address ipv4_dest, PortId port) {
    nextHop = ipv4_dest;
    headers.ip.ttl = headers.ip.ttl - 1;
    outCtrl.outputPort = port;
}

/**
* Computes address of next IPv4 hop and output port
* based on the IPv4 destination of the current packet.
* Decrements packet IPv4 TTL.
* @param nextHop IPv4 address of next hop
*/
table ipv4_match {
    key = { headers.ip.dstAddr: lpm; } // longest-prefix match
    actions = {
        Drop_action;
        Set_nhop;
    }
    size = 1024;
    default_action = Drop_action;
}

/**
* Send the packet to the CPU port
*/
action Send_to_cpu() {
    outCtrl.outputPort = CPU_OUT_PORT;
}

/**
* Check packet TTL and send to CPU if expired.
*/
table check_ttl {
    key = { headers.ip.ttl: exact; }
    actions = { Send_to_cpu; NoAction; }
    const default_action = NoAction; // defined in core.p4
}

/**
* Set the destination MAC address of the packet
* @param dmac destination MAC address.
*/
action Set_dmac(EthernetAddress dmac) {
    headers.ethernet.dstAddr = dmac;
}

/**
* Set the destination Ethernet address of the packet
* based on the next hop IP address.
* @param nextHop IPv4 address of next hop.
*/
table dmac {
    key = { nextHop: exact; }
    actions = {
        Drop_action;
        Set_dmac;
    }
    size = 1024;
    default_action = Drop_action;
}

/**
* Set the source MAC address.
* @param smac: source MAC address to use

```

```

    */
    action Set_smac(EthernetAddress smac) {
        headers.ethernet.srcAddr = smac;
    }

/**
 * Set the source mac address based on the output port.
 */
table smac {
    key = { outCtrl.outputPort: exact; }
    actions = {
        Drop_action;
        Set_smac;
    }
    size = 16;
    default_action = Drop_action;
}

apply {
    if (parseError != error.NoError) {
        Drop_action(); // invoke drop directly
        return;
    }

    ipv4_match.apply(); // Match result will go into nextHop
    if (outCtrl.outputPort == DROP_PORT) return;

    check_ttl.apply();
    if (outCtrl.outputPort == CPU_OUT_PORT) return;

    dmac.apply();
    if (outCtrl.outputPort == DROP_PORT) return;

    smac.apply();
}

}

// deparser section
control TopDeparser(inout Parsed_packet p, packet_out b) {
    Checksum16() ck;
    apply {
        b.emit(p.ethernet);
        if (p.ip.isValid()) {
            ck.clear(); // prepare checksum unit
            p.ip.hdrChecksum = 16w0; // clear checksum
            ck.update(p.ip); // compute new checksum.
            p.ip.hdrChecksum = ck.get();
        }
        b.emit(p.ip);
    }
}

// Instantiate the top-level VSS package
VSS(TopParser(),
    TopPipe(),
    TopDeparser()) main;

```

6. P4 language definition

This section provides a high-level overview of the P4 programming language, focusing on its syntax, semantics, lexical constructs, scoping rules, calling conventions, and naming conventions.

[Chapter 7](#) introduces the P4 abstract machine, which serves as the model for describing the behavior of P4 programs. [Chapter 8](#) through [Chapter 17](#) define the P4 syntax and semantics in detail.

6.1. Syntax and semantics

6.1.1. Grammar

The complete grammar of P4₁₆ is given in [Appendix E](#), using Yacc/Bison grammar description language. This text is based on the same grammar. We adopt several standard conventions when we provide excerpts from the grammar:

- UPPERCASE symbols denote terminals in the grammar.
- Excerpts from the grammar are given in BNF notation as follows:

```
p4program
: /* empty */
| p4program declaration
| p4program ;
;
```

6.1.2. Semantics and the P4 abstract machines

We describe the semantics of P4 in terms of abstract machines executing traditional imperative code.

P4 compilers are free to reorganize the code they generate in any way as long as the externally visible behaviors of the P4 programs are preserved as described by this specification where externally visible behavior is defined as:

- The input/output behavior of all P4 blocks, and
- The state maintained by extern blocks.

Throughout this specification, the P4 semantics is described using a combination of:

- P4 code snippets
- Pseudo-code snippets
- Prose algorithms

6.1.2.1. Pseudo-code snippets

Pseudo-code (mostly used for describing the semantics of various P4 constructs) are shown with fixed-size fonts as in the following example:

```
ParserModel.verify(bool condition, error err) {
    if (condition == false) {
        ParserModel.parserError = err;
        goto reject;
    }
}
```

6.1.2.2. Prose algorithms

The semantics of P4 constructs are also described using prose algorithms. For example, below is the signature of an algorithm used to describe type checking of P4 expressions.

Expr_ok:

- Typing `expression`, under cursor `cursor` and typing context `typingContext`:
- Results in `typedExpressionIR`.

Detailed discussion of what `cursor`, `typingContext`, and `typedExpressionIR` mean is given in [Section 7.2](#). For now, it suffices to know that `Expr_ok` takes a `cursor`, a `typingContext`, and a `typedExpressionIR` as inputs and results in a `typedExpressionIR`.

An algorithm is defined in terms of case-analysis, where each case corresponds to a variant of a P4 construct. For example, the following algorithm defines how to type check boolean literal expressions.

And the following algorithm defines type checking of integer literal expressions.

And so on for other variants of P4 expressions. These prose algorithms are presented throughout this specification.

6.1.2.2.1. Prose algorithm shorthands

To make the prose algorithms concise, we often use the shorthand `!`. Some algorithms result in an optional result to indicate possible failure. The following example algorithm computes modulo of two natural numbers (including zero).

$n_a \bmod n_b$

1. If n_b is not equal to 0:
 - a. Return $n_a \setminus n_b$.
2. If n_b is equal to 0:
 - a. Return `.`

Because the divisor may be zero, the result is optional. Now consider the case when the algorithm is called.

$n \bmod 42$

1. Let n_{result} be `!  $n \bmod 42$` .
2. Return n_{result} .

Here, `!` indicates unwrapping of the optional result. This shortened form is equivalent to first getting the optional result, checking that it is not `None`, and then unwrapping it.

6.2. Lexical constructs

All P4 keywords use only ASCII characters. All P4 identifiers must use only ASCII characters. P4 compilers

should handle correctly strings containing 8-bit characters in comments and string literals. P4 is case-sensitive. Whitespace characters, including newlines are treated as token separators. Indentation is free-form; however, P4 has C-like block constructs, and all our examples use C-style indentation. Tab characters are treated as spaces.

The lexer recognizes the following kinds of terminals:

- **IDENTIFIER**: start with a letter or underscore, and contain letters, digits and underscores
- **TYPE_IDENTIFIER**: identifier that denotes a type name
- **INTEGER**: integer literals
- **DONTCARE**: a single underscore
- Keywords such as **RETURN**. By convention, each keyword terminal corresponds to a language keyword with the same spelling but using lowercase. For example, the **RETURN** terminal corresponds to the `return` keyword.

6.2.1. Identifiers

P4 identifiers may contain only letters, numbers, and the underscore character `_`, and must start with a letter or underscore. The special identifier consisting of a single underscore `_` is reserved to indicate a "don't care" value; its type may vary depending on the context. Certain keywords (e.g., `apply`) can be used as identifiers if the context makes it unambiguous.

```
identifier
: `ID text
;

nonTypeName
: identifier
| APPLY
| KEY
| ACTIONS
| STATE
| ENTRIES
| TYPE
| PRIORITY
;

typeIdentifier
: `TID text
;

typeName = typeIdentifier

name
: nonTypeName
| typeName
| LIST
;
```

6.2.2. Literal constants

6.2.2.1. Boolean literals

```
booleanLiteral
: TRUE
| FALSE
;
```

There are two Boolean literal constants: `true` and `false`.

6.2.2.2. Integer literals

```
integerLiteral
  : D int
  | nat W int
  | nat S int
  ;
```

Integer literals are non-negative arbitrary-precision integers. By default, literals are represented in base 10. The following prefixes must be employed to specify the base explicitly:

- `0x` or `0X` indicates base 16 (hexadecimal)
- `0o` or `0O` indicates base 8 (octal)
- `0d` or `0D` indicates base 10 (decimal)
- `0b` or `0B` indicates base 2

The width of a numeric literal in bits can be specified by an unsigned number prefix consisting of a number of bits and a signedness indicator:

- `w` indicates unsigned numbers
- `s` indicates signed numbers

Note that a leading zero by itself does not indicate an octal (base 8) constant. The underscore character is considered a digit within number literals but is ignored when computing the value of the parsed number. This allows long constant numbers to be more easily read by grouping digits together. The underscore cannot be used in the width specification or as the first character of an integer literal. No comments or whitespaces are allowed within a literal. Here are some examples of numeric literals:

```
32w255      // a 32-bit unsigned number with value 255
32w0d255    // same value as above
32w0xFF     // same value as above
32s0xFF     // a 32-bit signed number with value 255
8w0b10101010 // an 8-bit unsigned number with value 0xAA
8w0b_1010_1010 // same value as above
8w170      // same value as above
8s0b1010_1010 // an 8-bit signed number with value -86
16w0377    // 16-bit unsigned number with value 377 (not 255!)
16w0o377    // 16-bit unsigned number with value 255 (base 8)
```

6.2.2.3. String literals

```
stringLiteral
  : " text "
  ;
```

String literals are specified as an arbitrary sequence of 8-bit characters, enclosed within double quote characters `"` (ASCII code 34). Strings start with a double quote character and extend to the first double quote sign which is not immediately preceded by an odd number of backslash characters (ASCII code 92). P4 does not make any validity checks on strings (i.e., it does not check that strings represent legal UTF-8 encodings).

Since P4 does not allow strings to exist at runtime, string literals are generally passed unchanged through the P4 compiler to other third-party tools or compiler-backends. The compiler can, however, perform

compile-time concatenation (constant-folding) of concatenation expressions into single literal. When such concatenation is performed, the binary representation of the string literals (excluding the quotes) is concatenated in the order they appear in the source code. There are no escape sequences that would be treated specially when strings are concatenated.

The backends and other tools can define their own handling of escape sequences (e.g., how to specify Unicode characters, or handle unprintable ASCII characters).

Here are 3 examples of string literals:

```
"simple string"
"string \" with \" embedded \" quotes"
"string with embedded
line terminator"
```

Here is an example of concatenation expression and an equivalent string literal:

```
"one string \" with a quote inside;" ++ (" " ++ "another string")
// can be constant folded to
"one string \" with a quote inside; another string"
```

6.2.3. Comments

P4 supports several kinds of comments:

- Single-line comments, introduced by `//` and spanning to the end of line,
- Multi-line comments, enclosed between `/*` and `*/`
- Nested multi-line comments are not supported.
- Javadoc-style comments, starting with `/**` and ending with `*/`

Use of Javadoc-style comments is strongly encouraged for the tables and actions that are used to synthesize the interface with the control-plane.

P4 treats comments as token separators and no comments are allowed within a token---e.g. `bi/**/t` is parsed as two tokens, `bi` and `t`, and not as a single token `bit`.

6.2.4. Optional trailing commas

The P4 grammar allows several kinds of comma-separated lists to end in an optional comma.

```
trailingCommaOpt
: /* empty */
| ,
;
```

For example, the following declarations are both legal, and have the same meaning:

```
enum E {
    a, b, c
}

enum E {
    a, b, c,
```

```
}
```

This is particularly useful in combination with preprocessor directives:

```
enum E {  
#if SUPPORT_A  
    a,  
#endif  
    b,  
    c,  
}
```

6.3. Scoping

Scopes are used to organize P4 programs into nested namespaces. There are three kinds of scopes in P4, from outermost to innermost: the top-level scope (global namespace), the scope within a **parser**, **control**, or **extern** declaration (block namespace), and the scope within a block statement (local namespace). Local namespaces can be nested within other local namespaces.

```
// top-level (global) scope  
const bit<8> GLOBAL = 255;  
  
control c() {  
    // block scope within a control  
    bit<8> block = GLOBAL;  
  
    apply {  
        // local scope  
        bit<8> local = block;  
    }  
}  
  
bit<8> f() {  
    // local scope  
    bit<8> local = GLOBAL;  
    {  
        // nested local scope  
        bit<8> local_nested = local;  
    }  
    return local;  
}
```

Cursors are used to track scopes while defining the semantic rules of P4.

```
cursor  
: GLOBAL  
| BLOCK  
| LOCAL  
;
```

NOTE

The division of global/block/local scopes and **cursor** is newly introduced. Maybe add more explanation here later. Also introduce a metavariable for **cursor**.

6.3.1. Name resolution

P4 namespaces are organized in a hierarchical fashion.


```

prefixedNonTypeName
: nonTypeName
| `ID . nonTypeName
;

prefixedTypeName
: typeName
| `TID . typeName
;

```

Identifiers prefixed with a dot are always resolved in the top-level namespace.

```

const bit<32> x = 2;
control c() {
    int<32> x = 0;
    apply {
        x = x + (int<32>).x; // x is the int<32> local variable,
                           // .x is the top-level bit<32> variable
    }
}

```

References to resolve an identifier are attempted inside-out, starting with the current scope and proceeding to all lexically enclosing scopes. The compiler may provide a warning if multiple resolutions are possible for the same name (name shadowing).

```

const bit<4> x = 1;
control c() {
    const bit<8> x = 8; // x declaration shadows global x
    const bit<4> y = .x; // reference to top-level x
    const bit<8> z = x; // reference to c's local x
    apply {}
}

```

The order of declarations is important; with the exception of parser states, all uses of a symbol must follow the symbol's declaration. (This is a departure from P4₁₄, which allows declarations in any order. This requirement significantly simplifies the implementation of compilers for P4, allowing compilers to use additional information about declared identifiers to resolve ambiguities.)

See [Section 14.3](#) for how variable references are resolved with respect to scopes.

6.3.2. Name visibility

Identifiers defined in the top-level namespace are globally visible. Declarations within a `parser` or `control` are private and cannot be referred to from outside of the enclosing `parser` or `control`.

6.4. Stateful elements

Most P4 constructs are stateless: given some inputs they produce a result that solely depends on these inputs. There are only two stateful constructs that may retain information across packets:

- `tables`: Tables are read-only for the data plane, but their entries can be modified by the control-plane,
- `extern` objects: many objects have state that can be read and written by the control plane and data plane. All constructs from the P4₁₄ language version that encapsulate state (e.g., counters, meters, registers) are represented using `extern` objects in P4₁₆.

In P4 all stateful elements must be explicitly allocated at compilation-time through the process called

"instantiation".

In addition, `parsers`, `control` blocks, and `packages` may contain stateful element instantiations. Thus, they are also treated as stateful elements, even if they appear to contain no state, and must be instantiated before they can be used. However, although they are stateful, `tables` do not need to be instantiated explicitly—declaring a `table` also creates an instance of it. This convention is designed to support the common case, since most tables are used just once. To have finer-grained control over when a `table` is instantiated, a programmer can declare it within a `control`.

Recall the example in [Section 5.3](#): `TopParser`, `TopPipe`, `TopDeparser`, `Checksum16`, and `Switch` are types. There are two instances of `Checksum16`, one in `TopParser` and one in `TopDeparser`, both called `ck`. The `TopParser`, `TopDeparser`, `TopPipe`, and `Switch` are instantiated at the end of the program, in the declaration of the `main` instance object, which is an instance of the `Switch` type (a `package`).

6.5. Call convention: copy-in/copy-out

P4 provides multiple constructs for writing modular programs: extern methods, parsers, controls, actions. All these constructs behave similarly to procedures in standard general-purpose programming languages:

- They have named and typed parameters.
- They introduce a new local scope for parameters and local variables.
- They allow arguments to be passed by binding them to their parameters.

```
parameterList
: /* empty */
| nonEmptyParameterList
;

nonEmptyParameterList
: parameter
| nonEmptyParameterList , parameter
;

parameter
: annotationList direction type name initializerOpt
;

direction
: /* empty */
| IN
| OUT
| INOUT
;
```

Invocations are executed using copy-in/copy-out semantics.

Each parameter may be labeled with a direction:

- `in` parameters are read-only. It is an error to use an `in` parameter on the left-hand side of an assignment or to pass it to a callee as a non-`in` argument. `in` parameters are initialized by copying the value of the corresponding argument when the invocation is executed.
- `out` parameters are, with a few exceptions listed below, uninitialized and are treated as l-values (See [\[sec-lvalues\]](#)) within the body of the method or function. An argument passed as an `out` parameter must be an l-value; after the execution of the call, the value of the parameter is copied to the corresponding storage location for that l-value.
- `inout` parameters behave like a combination of `in` and `out` parameters simultaneously: On entry the

value of the arguments is copied to the parameters. On return the value of the parameters is copied back to the arguments. In consequence, an argument passed as an **inout** parameter must be an l-value.

- The meaning of parameters with no direction depends upon the kind of entity the parameter is for:
 - For anything other than an action, e.g. a control, parser, or function, a directionless parameter means that the value supplied as an argument in a call must be a compile-time known value (see [Section 7.5](#)).
 - For an action, a directionless parameter indicates that it is "action data". See [\[sec-actions\]](#) for the meaning of action data, but its meaning includes the following possibilities:
 - The parameter's value is provided in the P4 program. In this case, the parameter behaves as if the direction were **in**. Such an argument expression need not be a compile-time known value.
 - The parameter's value is provided by the control plane software when an entry is added to a table that uses that action. See [\[sec-actions\]](#).
 - A directionless parameter of an extern object type is passed by reference.

Direction **out** parameters are always initialized at the beginning of execution of the portion of the program that has the **out** parameters, e.g. **control**, **parser**, **action**, function, etc. This initialization is not performed for parameters with any direction that is not **out**.

- If a direction **out** parameter is of type **header** or **header_union**, it is set to "invalid".
- If a direction **out** parameter is of type header stack, all elements of the header stack are set to "invalid", and its **nextIndex** field is initialized to 0 (see [\[sec-expr-hs\]](#)).
- If a direction **out** parameter is a compound type, e.g. a struct or tuple, other than one of the types listed above, then apply these rules recursively to its members.
- If a direction **out** parameter has any other type, e.g. **bit<W>**, an implementation need not initialize it to any predictable value.

For example, if a direction **out** parameter has type **s2_t** named **p**:

```
header h1_t {
    bit<8> f1;
    bit<8> f2;
}
struct s1_t {
    h1_t h1a;
    bit<3> a;
    bit<7> b;
}
struct s2_t {
    h1_t h1b;
    s1_t s1;
    bit<5> c;
}
```

then at the beginning of execution of the part of the program that has the **out** parameter **p**, it must be initialized so that **p.h1b** and **p.s1.h1a** are invalid. No other parts of **p** are required to be initialized.

Arguments are evaluated from left to right prior to the invocation of the function itself. The order of evaluation is important when the expression supplied for an argument can have side-effects. Consider the following example:

```
extern void f(inout bit x, in bit y);
extern bit g(inout bit z);
bit a;
```

```
f(a, g(a));
```

Note that the evaluation of `g` may mutate its argument `a`, so the compiler has to ensure that the value passed to `f` for its first parameter is not changed by the evaluation of the second argument. The semantics for evaluating a function call is given by the following algorithm (implementations can be different as long as they provide the same result):

1. Arguments are evaluated from left to right as they appear in the function call expression.
2. If a parameter has a default value and no corresponding argument is supplied, the default value is used as an argument.
3. For each `out` and `inout` argument the corresponding l-value is saved (so it cannot be changed by the evaluation of the following arguments). This is important if the argument contains indexing operations into a header stack.
4. The value of each argument is saved into a temporary.
5. The function is invoked with the temporaries as arguments. We are guaranteed that the temporaries that are passed as arguments are never aliased to each other, so this "generated" function call can be implemented using call-by-reference if supported by the architecture.
6. On function return, the temporaries that correspond to `out` or `inout` arguments are copied in order from left to right into the l-values saved in Step 3.

According to this algorithm, the previous function call is equivalent to the following sequence of statements:

```
bit tmp1 = a;      // evaluate a; save result
bit tmp2 = g(a);   // evaluate g(a); save result; modifies a
f(tmp1, tmp2);     // evaluate f; modifies tmp1
a = tmp1;         // copy inout result back into a
```

To see why Step 3 in the above algorithm is important, consider the following example:

```
header H { bit z; }
H[2] s;
f(s[a].z, g(a));
```

The evaluation of this call is equivalent to the following sequence of statements:

```
bit tmp1 = a;      // save the value of a
bit tmp2 = s[tmp1].z; // evaluate first argument
bit tmp3 = g(a);   // evaluate second argument; modifies a
f(tmp2, tmp3);     // evaluate f; modifies tmp2
s[tmp1].z = tmp2;  // copy inout result back; dest is not s[a].z
```

When used as arguments, `extern` objects can only be passed as directionless parameters—e.g., see the packet argument in the very simple switch example.

NOTE Put a forward reference to [Section 18.4](#).

6.5.1. Justification

The main reason for using copy-in/copy-out semantics (instead of the more common call-by-reference semantics) is for controlling the side-effects of `extern` functions and methods. `extern` methods and functions are the main mechanism by which a P4 program communicates with its environment. With

copy-in/copy-out semantics `extern` functions cannot hold references to P4 program objects; this enables the compiler to limit the side-effects that `extern` functions may have on the P4 program both in space (they can only affect `out` parameters) and in time (side-effects can only occur at function call time).

In general, `extern` functions are arbitrarily powerful: they can store information in global storage, spawn separate threads, "collude" with each other to share information—but they cannot access any variable in a P4 program. With copy-in/copy-out semantics the compiler can still reason about P4 programs that invoke `extern` functions.

There are additional benefits of using copy-in copy-out semantics:

- It enables P4 to be compiled for architectures that do not support references (e.g., where all data is allocated to named registers. Such architectures may require indices into header stacks that appear in a program to be compile-time known values.)
- It simplifies some compiler analyses, since function parameters can never alias to each other within the function body.

Following is a summary of the constraints imposed by the parameter directions:

- When used as arguments, extern objects can only be passed as directionless parameters.
- All constructor parameters are evaluated at compilation-time, and in consequence they must all be directionless (they cannot be `in`, `out`, or `inout`); this applies to `package`, `control`, `parser`, and `extern` objects. Expressions for these parameters must be supplied at compile-time, and they must evaluate to compile-time known values. See [\[sec-parameterization\]](#) for further details.
- For actions all directionless parameters must be at the end of the parameter list. When an action appears in a `table`'s `actions` list, only the parameters with a direction must be bound. See [\[sec-actions\]](#) for further details.
- Actions can also be explicitly invoked using function call syntax, either from a control block or from another action. In this case, values for all action parameters must be supplied explicitly, including values for the directionless parameters. The directionless parameters in this case behave like `in` parameters. See [\[sec-invoke-actions\]](#) for further details.
- Default expressions are only allowed for `in` or directionless parameters, and the expressions supplied as defaults must be compile-time known values.
- If parameters with default values do not appear at the end of the list of parameters, invocations that use the default values must use named arguments, as in the following example:

```
extern void f(in bit a, in bit<3> b = 2, in bit<5> c);

void g() {
  f(a = 1, b = 2, c = 3); // ok
  f(a = 1, c = 3);      // ok, equivalent to the previous call, b uses default value
  f(1, 2, 3);           // ok, equivalent to the previous call
  // f(1, 3); // illegal, since the parameter b is not the last in the list
}
```

6.5.2. Optional parameters

A parameter that is annotated with the `@optional` annotation is optional: the user may omit the value for that parameter in an invocation. Optional parameters can only appear for arguments of: packages, parser types, control types, extern functions, extern methods, and extern object constructors. Optional parameters cannot have default values. If a procedure-like construct has both optional parameters and default values then it can only be called using named arguments. It is recommended, but not mandatory, for all optional parameters to be at the end of a parameter list.

The implementation of such objects is not expressed in P4, so the meaning and implementation of

optional parameters should be specified by the target architecture. For example, we can imagine a two-stage switch architecture where the second stage is optional. This could be declared as a package with an optional parameter:

```
package pipeline(/* parameters omitted */);
package switch(pipeline first, @optional pipeline second);

pipeline(/* arguments omitted */) ingress;
switch(ingress) main; // a switch with a single-stage pipeline
```

Here the target architecture could implement the elided optional argument using an empty pipeline.

The following example shows optional parameters and parameters with default values.

```
extern void h(in bit<32> a, in bool b = true); // default value

// function calls
h(10); // same as h(10, true);
h(a = 10); // same as h(10, true);
h(a = 10, b = true);

struct Empty {}
control nothing(inout Empty h, inout Empty m) {
    apply {}
}

parser parserProto<H, M>(packet_in p, out H h, inout M m);
control controlProto<H, M>(inout H h, inout M m);

package pack<HP, MP, HC, MC>(
    @optional parserProto<HP, MP> _parser, // optional parameter
    controlProto<HC, MC> _control = nothing()); // default
parameter value

pack() main; // No value for _parser, _control is an instance of nothing()
```

6.6. Overloading

Functions, methods, and constructors are the only P4 constructs that support overloading: there can exist multiple methods with the same name in the same scope. When overloading is used, the compiler must be able to disambiguate at compile-time which method, function, or constructor is being called, either by the number of arguments or by the names of the arguments, when calls are specifying argument names. Argument type information is not used in disambiguating calls.

Notice that overloading of abstract methods, parsers, controls, or packages is not allowed:

```
parser p(packet_in p, out bit<32> value) {
    ...
}

// The following will cause an error about a duplicate declaration
//parser p(packet_in p, out Headers headers) {
//    ...
//}
```

See [Section 18.2](#) for details on how overload is resolved.

6.7. Naming conventions

P4 provides a rich assortment of types. Base types include bit-strings, numbers, and errors. There are also built-in types for representing constructs such as parsers, pipelines, actions, and tables. Users can construct new types based on these: structures, enumerations, headers, header stacks, header unions, etc.

In this document we adopt the following conventions:

- Built-in types are written with lowercase characters—e.g., `int<20>`,
- User-defined types are capitalized—e.g., `IPv4Address`,
- Type variables are always uppercase—e.g., `parser P<H, IH>()`,
- Variables are uncapitalized-- e.g., `ipv4header`,
- Constants are written with uppercase characters—e.g., `CPU_PORT`, and
- Errors and enumerations are written in camel-case-- e.g. `PacketTooShort`.

7. The P4 Abstract Machine

A $P4_{16}$ program is a list of declarations:

```
p4program
: /* empty */
| p4program declaration
| p4program ;
;
```

An empty declarations is indicated with a single semicolon. (Allowing empty declarations accommodates the habits of C/C++ and Java programmers—e.g., certain constructs, like `struct`, do not require a terminating semicolon).

The evaluation of a P4 program is done in four stages:

- **preprocessing:** at compile time, the P4 program is preprocessed to handle directives such as `#include` and `#define`.
- **type checking:** at compile time, the P4 program is type checked according to the P4 type system.
- **instantiation:** at compile time, all stateful blocks in the P4 program are instantiated.
- **runtime evaluation:** at runtime each P4 functional block is executed to completion, in isolation, when it receives control from the architecture

The preprocessing stage is described in [Section 7.1](#).

The type system is described in [Section 7.2](#). In addition to checking the types, the type system also converts an input P4 program into an intermediate representation ($P4_{IR}$). $P4_{IR}$ is an extension of the surface syntax of P4, e.g., the types of all expressions are made explicit, implicit type casts are inserted, and omitted type arguments are inferred. $P4_{IR}$ is described in [Section 7.2.1](#).

The instantiation stage is described in [Section 7.3](#). It traverses the type checked $P4_{IR}$ program, and recursively instantiates all stateful blocks. The result of the instantiation stage is a global store that holds all instantiated stateful blocks.

The runtime evaluation stage is described in [Section 7.4](#). It describes how each P4 block, represented as stateful objects within the global store, is executed to completion. These blocks are orchestrated by the architecture to compose the overall packet processing pipeline.

NOTE

Added an overview of the P4 abstract machine. The contents are either added or gathered from various sections of the P4 Language Specification.

7.1. Preprocessing

To aid the composition of programs from multiple source files, P4 compilers should support the following subset of the C preprocessor functionality:

- `#define` for defining macros (without arguments)
- `#undef`
- `#if #else #endif #ifdef #ifndef #elif`
- `#include`

The preprocessor should also remove the sequence backslash newline (ASCII codes 92, 10) to facilitate splitting content across multiple lines when convenient for formatting.

Additional C preprocessor capabilities may be supported, but are not guaranteed---e.g., macros with arguments. Similar to C, `#include` can specify a file name either within double quotes or within `<>`.

```
# include <system_file>
# include "user_file"
```

The difference between the two forms is the order in which the preprocessor searches for header files when the path is incompletely specified.

P4 compilers should correctly handle `#line` directives that may be generated during preprocessing. This functionality allows P4 programs to be built from multiple source files, potentially produced by different programmers at different times:

- the P4 core library, defined in this document,
- the architecture, defining data plane interfaces and extern blocks,
- user-defined libraries of useful components (e.g. standard protocol header definitions), and
- the P4 programs that specify the behavior of each programmable block.

The P4 language specification defines a core library that includes several common programming constructs. A description of the core library is provided in [Appendix D](#). All P4 programs must include the core library. Including the core library is done with

```
# include <core.p4>
```

7.2. Type Checking

P4₁₆ is a statically-typed language. Programs that do not pass the type checker are considered invalid and rejected by the compiler. P4 provides a number of base types as well as type operators that construct derived types.

7.2.1. P4 Intermediate Representation

P4_{IR} is an intermediate representation for P4 programs. P4_{IR} extends the surface syntax of P4₁₆ with additional information obtained during type checking. They include:

- All expressions are annotated with their types.
- Implicit casts are made explicit.
- Omitted type arguments are inferred and made explicit.

$P4_{IR}$ is designed to stay as close as possible to the surface syntax of $P4_{16}$. This makes it easy to relate $P4_{IR}$ programs to their source $P4_{16}$ programs.

Take expressions as an example.

```
expression
: literalExpression
| referenceExpression
| defaultExpression
| unaryExpression
| binaryExpression
| ternaryExpression
| castExpression
| dataExpression
| accessExpression
| callExpression
| parenthesizedExpression
;

expressionIR
: literalExpressionIR
| referenceExpressionIR
| defaultExpressionIR
| unaryExpressionIR
| binaryExpressionIR
| ternaryExpressionIR
| castExpressionIR
| dataExpressionIR
| accessExpressionIR
| callExpressionIR
| parenthesizedExpressionIR
;

expressionNoteIR
: `( typeIR ctk )
;

typedExpressionIR
: expressionIR # expressionNoteIR
;
```

$typedExpressionIR$ is an intermediate representation of $expression$ that is annotated with type information, $expressionNoteIR$. Nevertheless, the structure of the grammar mostly stays the same.

```
binaryExpression
: expression binop expression
;

binaryExpressionIR
: typedExpressionIR binop typedExpressionIR
;
```

$binaryExpressionIR$ is an intermediate representation of $binaryExpression$. The only difference is that the operands are now of type $typedExpressionIR$.

NOTE | Added explanation for $P4_{IR}$. More explanation needed?

7.2.2. Typing a P4 program

For a well-typed $P4_{16}$ program, $p4program$, a type checker produces a $P4_{IR}$ program, $p4programIR$:

```
p4programIR
  : declarationIR* ;
  ;
```

$Program_ok$ type checks a $p4program$ and produces a $p4programIR$.

$Program_ok$:

- Typing $p4program$:
- Results in the typed P4 program $p4programIR$.

Below is a prose algorithm that implements $Program_ok$. Details of what each meta-variable means are provided in the subsequent sections.

Typing $p4program$:

1. Let $declaration^*$ be the list of declarations in $p4program$.
2. Let TC_0 be an empty typing context.
3. Let the updated typing context TC_1 and $declarationIR^*$ be
 - the result of typing $declaration^*$ under typing context TC_0 .
4. Result in the typed P4 program $declarationIR^*$.

7.2.3. Typing context

```
globalTypingLayer = {CONSTRUCTOR constructorTypeDefEnv, TYPE typeDefEnv, CALLABLE
callableTypeDefEnv, FRAME typeFrame}

blockTypingLayer = {KIND blockKind, TYPE typeDefEnv, CALLABLE callableTypeDefEnv, FRAME
typeFrame}

localTypingLayer = {KIND localKind, TYPE typeDefEnv, FRAMES typeFrame*}

typingContext = {GLOBAL globalTypingLayer, BLOCK blockTypingLayer, LOCAL
localTypingLayer}
```

A typing context is used to keep track of the types of all named declarations in scope at a given program point. It consists of three layers:

- A global typing layer, which contains all declarations at the top level of a $P4_{16}$ program.
- A block typing layer, which contains all declarations in the current block scope.
- A local typing layer, which contains all declarations in the current local scope.

Each layer contains mappings from names to types.

```
varTypeIR
```

```

    : direction typeIR ctk value?
    ;

typeFrame = map<id, varTypeIR>

```

`typeFrame` is a map from variable names to their types, `varTypeIR`. In addition to the type of the variable, `varTypeIR` also contains the direction of the variable (`direction`), whether it is a compile-time constant (`ctk`), and whether it is associated with a compile-time known value (`value?`).

The compile-time known-ness is discussed in [Section 7.5](#).

Notice that `localTypingLayer` contains multiple `typeFrames`. This is because a local scope can be nested with block statements.

```

typeDefIR
  : nameTypeDefIR
  | aliasTypeDefIR
  | dataTypeDefIR
  | objectTypeDefIR
  ;

typeDefEnv = map<typeId, typeDefIR>

```

`typeDefEnv` is a map from type names to their type definitions, `typeDefIR`. `typeDefIR` is a type constructor, derived from user-defined type declarations.

```

callableTypeDefIR
  : actionTypeDefIR
  | functionTypeDefIR
  | methodTypeDefIR
  ;

callableTypeDefEnv = map<callableId, callableTypeDefIR>

```

Functions, methods, and actions can be defined in $P_{4_{16}}$. These are collectively called callables. `callableTypeDefEnv` is a map from callable names to their type definitions, `callableTypeDefIR`.

```

constructorTypeDefIR
  : CONSTRUCTOR `< typeParameterIR* , typeParameterIR* > `( constructorParameterIR* ) :
  typeIR
  ;

constructorTypeDefEnv = map<constructorId, constructorTypeDefIR>

```

Stateful objects can be defined in $P_{4_{16}}$, which are instantiated using constructors. `constructorTypeDefEnv` is a map from constructor names to their type definitions, `constructorTypeDefIR`.

NOTE | Added explanation for the typing context. Is this the right place?

7.3. Instantiation

Instantiation of a program proceeds in order of declarations, starting in the top-level namespace:

- All declarations (e.g., `parsers`, `controls`, types, constants) evaluate to themselves.
- Each `table` evaluates to a table instance.

- Constructor invocations evaluate to stateful objects of the corresponding type. For this purpose, all constructor arguments are evaluated recursively and bound to the constructor parameters. Constructor arguments must be compile-time known values. The order of evaluation of the constructor arguments should be unimportant --- all evaluation orders should produce the same results.
- Instantiations evaluate to named stateful objects.
- The instantiation of a `parser` or `control` block recursively evaluates all stateful instantiations declared in the block.
- The result of the program's evaluation is the value of the top-level `main` variable.

Note that all stateful values are instantiated at compilation time.

As an example, consider the following program fragment:

```
// architecture declaration
parser P(/* parameters omitted */);
control C(/* parameters omitted */);
control D(/* parameters omitted */);

package Switch(P prs, C ctrl, D dep);

extern Checksum16 { /* body omitted */}

// user code
Checksum16() ck16; // checksum unit instance

parser TopParser(/* parameters omitted */)(Checksum16 unit) { /* body omitted */}
control Pipe(/* parameters omitted */) { /* body omitted */}
control TopDeparser(/* parameters omitted */)(Checksum16 unit) { /* body omitted */}

Switch(TopParser(ck16),
      Pipe(),
      TopDeparser(ck16)) main;
```

The evaluation of this program proceeds as follows:

1. The declarations of `P`, `C`, `D`, `Switch`, and `Checksum16` all evaluate to themselves.
2. The `Checksum16() ck16` instantiation is evaluated and it produces an object named `ck16` with type `Checksum16`.
3. The declarations for `TopParser`, `Pipe`, and `TopDeparser` evaluate as themselves.
4. The `main` variable instantiation is evaluated:
 - a. The arguments to the constructor are evaluated recursively.
 - b. `TopParser(ck16)` is a constructor invocation.
 - i. Its argument is evaluated recursively; it evaluates to the `ck16` object.
 - c. The constructor itself is evaluated, leading to the instantiation of an object of type `TopParser`.
 - d. Similarly, `Pipe()` and `TopDeparser(ck16)` are evaluated as constructor calls.
 - e. All the arguments of the `Switch` package constructor have been evaluated (they are an instance of `TopParser`, an instance of `Pipe`, and an instance of `TopDeparser`). Their signatures are matched with the `Switch` declaration.
 - f. Finally, the `Switch` constructor can be evaluated. The result is an instance of the `Switch` package (that contains a `TopParser` named `prs` the first parameter of the `Switch`; a `Pipe` named `ctrl`; and a `TopDeparser` named `dep`).
5. The result of the program evaluation is the value of the `main` variable, which is the above instance of

the `Switch` package.

Figure 8 shows the result of the evaluation in a graphical form. The result is always a graph of instances. There is only one instance of `Checksum16`, called `ck16`, shared between the `TopParser` and `TopDeparser`. Whether this is possible is architecture-dependent. Specific target compilers may require distinct checksum units to be used in distinct blocks.

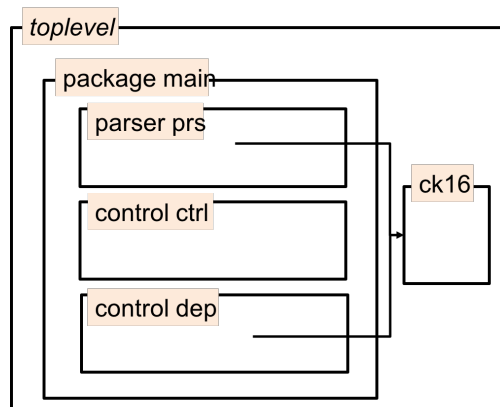


Figure 8. Evaluation result.

7.3.1. Control-plane names

Every controllable entity exposed in a P4 program must be assigned a unique, fully-qualified name, which the control plane may use to interact with that entity. The following entities are controllable.

- value sets
- tables
- keys
- actions
- extern instances

A fully qualified name consists of the local name of a controllable entity prepended with the fully qualified name of its enclosing namespace. Hence, the following program constructs, which enclose controllable entities, must themselves have unique, fully-qualified names.

- control instances
- parser instances

Evaluation may create multiple instances from one type, each of which must have a unique, fully-qualified name.

7.3.1.1. Computing control-plane names

The fully-qualified name of a construct is derived by concatenating the fully-qualified name of its enclosing construct with its local name. Constructs with no enclosing namespace, i.e. those defined at the global scope, have the same local and fully-qualified names. The local names of controllable entities and enclosing constructs are derived from the syntax of a P4 program as follows.

7.3.1.1.1. Value sets

For each `value_set` construct, its syntactic name becomes the local name of the value set. For example:

```
struct vsk_t {  
    @match(ternary)
```

```

    bit<16> port;
}
value_set<vsk_t>(4) pvs;

```

This `value_set`'s local name is `pvs`.

7.3.1.1.2. Tables

For each `table` construct, its syntactic name becomes the local name of the table. For example:

```

control c(/* parameters omitted */)() {
    table t { /* body omitted */ }
}

```

This table's local name is `t`.

7.3.1.1.3. Keys

Syntactically, table keys are expressions. For simple expressions, the local key name can be generated from the expression itself; the algorithm by which a compiler derives control-plane names for complex key expressions is target-dependent.

The spec suggests, but does not mandate, the following algorithm for generating names for some kinds of key expressions:

Kind	Example	Name
The <code>isValid()</code> method.	<code>h.isValid()</code>	<code>"h.isValid()"</code>
Array accesses.	<code>header_stack[1]</code>	<code>"header_stack[1]"</code>
Constants.	<code>1</code>	<code>"1"</code>
Field projections.	<code>data.f1</code>	<code>"data.f1"</code>
Slices.	<code>f1[3:0]</code>	<code>"f1[3:0]"</code>
Masks.	<code>h.src & 0xFFFF</code>	<code>"h.src & 0xFFFF"</code>

In the following example, the previous algorithm would derive for table `t` two keys with names `data.f1` and `hdrs[3].f2`.

```

table t {
    keys = {
        data.f1 : exact;
        hdrs[3].f2 : exact;
    }
    actions = { /* body omitted */ }
}

```

If a compiler cannot generate a name for a key it **requires** the key expression to be annotated with a `@name` annotation ([\[sec-control-plane-api-annotations\]](#)), as in the following example:

```

table t {
    keys = {
        data.f1 + 1 : exact @name("f1_mask");
    }
    actions = { /* body omitted */ }
}

```

Here, the `@name("f1_mask")` annotation assigns the local name "f1_mask" to this key.

7.3.1.1.4. Actions

For each `action` construct, its syntactic name is the local name of the action. For example:

```
control c(/* parameters omitted */()) {  
    action a(...) { /* body omitted */ }  
}
```

This action's local name is `a`.

7.3.1.1.5. Instances

The local names of `extern`, `parser`, and `control` instances are derived based on how the instance is used. If the instance is bound to a name, that name becomes its local control plane name. For example, if `control C` is declared as,

```
control C(/* parameters omitted */()) { /* body omitted */ }
```

and instantiated as,

```
C() c_inst;
```

then the local name of the instance is `c_inst`.

Alternatively, if the instance is created as an actual argument, then its local name is the name of the formal parameter to which it will be bound. For example, if `extern E` and `control C` are declared as,

```
extern E { /* body omitted */ }  
control C(/* parameters omitted */)(E e_in) { /* body omitted */ }
```

and instantiated as,

```
C(E()) c_inst;
```

then the local name of the extern instance is `e_in`.

If the construct being instantiated is passed as an argument to a package, the instance name is derived from the user-supplied type definition when possible. In the following example, the local name of the instance of `MyC` is `c`, and the local name of the `extern` is `e2`, not `e1`.

```
extern E { /* body omitted */ }  
control ArchC(E e1);  
package Arch(ArchC c);  
  
control MyC(E e2)() { /* body omitted */ }  
Arch(MyC()) main;
```

Note that in this example, the architecture will supply an instance of the extern when it applies the instance of `MyC` passed to the `Arch` package. The fully-qualified name of that instance is `main.c.e2`.

Next, consider a larger example that demonstrates name generation when there are multiple instances.

```
control Callee() {  
  table t { /* body omitted */ }  
  apply { t.apply(); }  
}  
control Caller() {  
  Callee() c1;  
  Callee() c2;  
  apply {  
    c1.apply();  
    c2.apply();  
  }  
}  
control Simple();  
package Top(Simple s);  
Top(Caller()) main;
```

The compile-time evaluation of this program produces the structure in [Figure 9](#). Notice that there are two instances of the `table t`. These instances must both be exposed to the control plane. To name an object in this hierarchy, one uses a path composed of the names of containing instances. In this case, the two tables have names `s.c1.t` and `s.c2.t`, where `s` is the name of the argument to the package instantiation, which is derived from the name of its corresponding formal parameter.

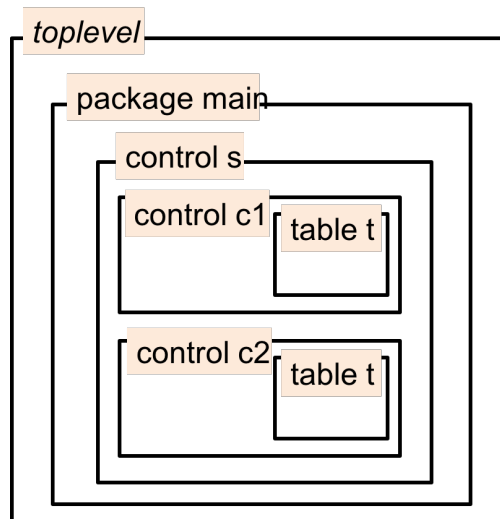


Figure 9. Evaluating a program that has several instantiations of the same component.

7.3.1.2. Annotations controlling naming

Control plane-related annotations ([\[sec-control-plane-api-annotations\]](#)) can alter the names exposed to the control plane in the following ways.

- The `@hidden` annotation hides a controllable entity from the control plane. This is the only case in which a controllable entity is not required to have a unique, fully-qualified name.
- The `@name` annotation may be used to change the local name of a controllable entity.

Programs that yield the same fully-qualified name for two different controllable entities are invalid.

NOTE | `@hidden` is not yet supported in P4-SpecTec.

7.3.1.3. Recommendations

The control plane may refer to a controllable entity by a postfix of its fully qualified name when it is

unambiguous in the context in which it is used. Consider the following example.

```
control c(/* parameters omitted */)() {
  action a (/* parameters omitted */) { /* body omitted */ }
  table t {
    keys = { /* body omitted */ }
    actions = { a; } }
}
c() c_inst;
```

Control plane software may refer to action `c_inst.a` as `a` when inserting rules into table `c_inst.t`, because it is clear from the definition of the table which action `a` refers to.

Not all unambiguous postfix shortcuts are recommended. For instance, consider the last example in [Section 7.3.1.1.5](#). One might be tempted to refer to `s.c1` simply as `c1`, as no other instance named `c1` appears in the program. However, this leads to a brittle program since future modifications can never introduce an instance named `c1`, or include libraries of P4 code that contain instances with that name.

7.3.2. Instantiating a P4 program

```
objectId = nameIR*

object
: externObject
| parserObject
| controlObject
| packageObject
| tableObject
| valueSetObject
;
```

The stateful elements of a P4 program are represented as `objects`. These objects are identified by their fully-qualified names (`objectId`).

```
store = map<objectId, object>
```

A $P4_{16}$ program is first type checked to yield a $P4_{IR}$ program, and is then instantiated to create a `store`, which maps `objectIds` to `objects`.

Program_inst:

- Loading `p4program`:
- Results in the global instantiation layer `globalInstLayer` and the store `store`.

Loading `p4program`:

1. Let the typed P4 program `declarationIR*`; be
 - the result of `typing p4program`.
2. Let `IC0` be an empty instantiation context.
3. Let `STO0` be an empty store.
4. Let the updated instantiation context `IC1` and the store `STO1` be

- the result of **loading** `declarationIR*`, **under instantiation context** `IC0`, and **store** `STO0`.
5. Result in the global instantiation layer `IC1.GLOBAL` and the store `STO1`.

7.3.3. Instantiation context

```
globalInstLayer = {CONSTRUCTOR constructorDefEnv, TYPE typeDefEnv, CALLABLE
callableDefEnv, FRAME frame}

blockInstLayer = {TYPE theta, CALLABLE callableDefEnv, STATE stateEnv, FRAME frame}

localInstLayer = {TYPE theta, FRAMES frame*}

instContext = {PATH objectId, GLOBAL globalInstLayer, BLOCK blockInstLayer, LOCAL
localInstLayer}
```

An instantiation context is used to keep track of the types and compile-time known values of all named declarations in scope at a given program point.

- A global instantiation layer, which contains all declarations at the top level of a P4 program.
- A block instantiation layer, which contains all declarations in the current block scope.
- A local instantiation layer, which contains all declarations in the current local scope.

```
value
: baseValue
| dataValue
| objectReferenceValue
| synthesizedValue
;

frame = map<id, value>
```

`frame` is a map from variable names to their compile-time known values, `value`. The compile-time knownness is discussed in [Section 7.5](#).

```
typeDefIR
: nameTypeDefIR
| aliasTypeDefIR
| dataTypeDefIR
| objectTypeDefIR
;

typeDefEnv = map<typeId, typeDefIR>
```

`typeDefEnv` is a map from type names to their type definitions, `typeDefIR`. `typeDefIR` is a type constructor, derived from user-defined type declarations.

```
callableDef
: actionDef
| functionDef
| methodDef
;

callableDefEnv = map<callableId, callableDef>
```

`callableDefEnv` is a map from callable names to their definitions, `callableDef`.

```

constructorDef
: externObjectConstructorDef
| parserObjectConstructorDef
| controlObjectConstructorDef
| packageObjectConstructorDef
;

constructorDefEnv = map<callableId, constructorDef>

```

Stateful objects are instantiated using constructors. `constructorDefEnv` is a map from constructor names to their definitions, `constructorDef`.

NOTE | Added explanation for the instantiation context. Is this the right place?

7.4. Runtime evaluation

The runtime evaluation of a P4 program is orchestrated by the architecture model. Each architecture model needs to specify the order and the conditions under which the various P4 component programs are dynamically executed. For example, in the Simple Switch example from [Section 5.1](#) the execution flow goes `Parser` `Pipe` `Deparser`.

Once a P4 execution block is invoked its execution proceeds until termination according to the semantics defined in this document. P4 execution blocks are represented as objects in the global store, which is statically allocated during instantiation.

7.4.1. Evaluating a P4 program

P4 program evaluation proceeds by invoking the objects in the global store. A call to a programmable block invokes the corresponding abstract machine:

Call_eval:

- Calling `callee < typeArgumentListIR > ( argumentListIR )`, under cursor `cursor`, evaluation context `evalContext`, and store `store`:
- Results in the updated evaluation context `evalContext`, store `store`, and call result `callResult`.

A call to an extern function invokes the corresponding extern function:

ExternFunctionCall_eval:

- Calling an extern function `nameIR ( nameIR* )`, under caller's evaluation context `evalContext`, evaluation context `evalContext`, and store `store`:
- Results in the updated evaluation context `evalContext`, store `store`, and call result `callResult`.

A call to an extern method invokes the corresponding extern method:

ExternMethodCall_eval:

- Calling an extern method `nameIR ( nameIR* )` on object `objectId`, under evaluation context `evalContext`, and store `store`:

- Results in the updated evaluation context `evalContext`, store `store`, and call result `callResult`.

7.4.2. Evaluation context

```
globalEvalLayer = globalInstLayer
blockEvalLayer = blockInstLayer
localEvalLayer = localInstLayer
evalContext = {GLOBAL globalEvalLayer, BLOCK blockEvalLayer, LOCAL localEvalLayer}
```

An evaluation context is used to keep track of the types and compile-time known values of all named declarations in scope at a given program point.

- A global evaluation layer, which contains all declarations at the top level of a P4 program.
- A block evaluation layer, which contains all declarations in the current block scope.
- A local evaluation layer, which contains all declarations in the current local scope.

The layers are the same as in the instantiation context, except that the `frame` is a map from variable names to their current runtime values.

The following sections describe the abstract machines that define the dynamic semantics of the programmable blocks in a P4 target.

NOTE | Added explanation for the evaluation context. Is this the right place?

7.4.3. The parser abstract machine

The semantics of a P4 parser can be formulated in terms of an abstract machine that manipulates a `ParserModel` data structure. This section describes this abstract machine in pseudo-code.

A parser starts execution in the `start` state and ends execution when one of the `reject` or `accept` states has been reached.

```
ParserModel {
    error parseError;
    onPacketArrival(packet p) {
        ParserModel.parseError = error.NoError;
        goto start;
    }
}
```

An architecture must specify the behavior when the `accept` and `reject` states are reached. For example, an architecture may specify that all packets reaching the `reject` state are dropped without further processing. Alternatively, it may specify that such packets are passed to the next block after the parser, with intrinsic metadata indicating that the parser reached the `reject` state, along with the error recorded.

7.4.4. The match-action pipeline abstract machine

We can describe the computational model of a match-action pipeline, embodied by a control block: the body of the control block is executed, similarly to the execution of a traditional imperative program:

- At runtime, statements within a block are executed in the order they appear in the control block.

- Execution of the `return` statement causes the immediate termination of the execution of the current `control` block and a return to the caller.
- Execution of the `exit` statement causes the immediate termination of the execution of the current `control` block and of all the enclosing caller `control` blocks.
- Applying a `table` executes the corresponding match-action unit.

7.4.5. Concurrency model

A typical packet processing system needs to execute multiple simultaneous logical "threads." At the very least there is a thread executing the control plane, which can modify the contents of the tables. Architecture specifications should describe in detail the interactions between the control-plane and the data-plane. The data plane can exchange information with the control plane through `extern` function and method calls. Moreover, high-throughput packet-processing systems may be processing multiple packets simultaneously, e.g., in a pipelined fashion, or concurrently parsing a first packet while performing match-action operations on a second packet. This section specifies the semantics of P4 programs with respect to such concurrent executions.

Each top-level `parser` or `control` block is executed as a separate thread when invoked by the architecture. All the parameters of the block and all local variables are thread-local—i.e., each thread has a private copy of these resources. This applies to the `packet_in` and `packet_out` parameters of parsers and deparsers.

As long as a P4 block uses only thread-local storage (e.g., metadata, packet headers, local variables), its behavior in the presence of concurrency is identical with the behavior in isolation, since any interleaving of statements from different threads must produce the same output.

In contrast, `extern` blocks instantiated by a P4 program are global, shared across all threads. If `extern` blocks mediate access to state (e.g., counters, registers)—i.e., the methods of the `extern` block read and write state, these stateful operations are subject to data races. P4 mandates that execution of a method call on an `extern` instance is atomic.

To allow users to express atomic execution of larger code blocks, P4 provides an `@atomic` annotation, which can be applied to block statements, parser states, control blocks, or whole parsers.

Consider the following example:

```
extern Register { /* body omitted */ }
control Ingress() {
  Register() r;
  table flowlet { /* read state of r in an action */ }
  table new_flowlet { /* write state of r in an action */ }
  apply {
    @atomic {
      flowlet.apply();
      if (ingress_metadata.flow_ipg > FLOWLET_INACTIVE_TIMEOUT)
        new_flowlet.apply();
    }
  }
}
```

This program accesses an extern object `r` of type `Register` in actions invoked from tables `flowlet` (reading) and `new_flowlet` (writing). Without the `@atomic` annotation these two operations would not execute atomically: a second packet may read the state of `r` before the first packet had a chance to update it.

Note that even within an `action` definition, if the action does something like reading a register, modifying it, and writing it back, in a way that only the modified value should be visible to the next packet, then, to guarantee correct execution in all cases, that portion of the action definition should be enclosed within a block annotated with `@atomic`.

A compiler backend must reject a program containing `@atomic` blocks if it cannot implement the atomic

execution of the instruction sequence. In such cases, the compiler should provide reasonable diagnostics.

The P4 semantics described throughout this document assumes a sequential execution model. Thus, it does not describe the interactions between multiple threads accessing shared state. The purpose of this document is to clearly define the semantics of P4 constructs in isolation. The exact concurrency model, including the interactions between multiple threads, is target-dependent and beyond the scope of this document.

NOTE | Added disclaimer about concurrency model.

7.5. Compile-time known and local compile-time known values

Certain expressions in a P4 program have the property that their value can be determined at compile time. Moreover, for some of these expressions, their value can be determined only using information in the current scope. We call these *compile-time known values* and *local compile-time known values* respectively.

The following are local compile-time known values:

- Integer literals, Boolean literals, and string literals.
- Identifiers declared as constants using the `const` keyword.
- Identifiers declared in an `error`, `enum`, or `match_kind` declaration.
- The `default` identifier.
- The `size` field of a value with type header stack.
- The `_` identifier when used as a `select` expression label.
- The expression `{#}` representing an invalid header or header union value.
- Identifiers that represent declared types, actions, functions, tables, parsers, controls, or packages.
- Sequence expression where all components are local compile-time known values.
- Structure-valued expressions, where all fields are local compile-time known values.
- Expressions evaluating to a list type, where all elements are local compile-time known values.
- Legal casts applied to local compile-time known values.
- Indexing a local compile-time known stack or tuple value with a local compile-time known index.
- Accessing a field of a local compile-time known struct, header, or header union value.
- The following expressions `(, `', `|', |-, *, /, %, !, &, |, ^, &&, ||, <<`, >>, ~, >, <, ==, !=, , >=, ++, [:, ?)` when their operands are all local compile-time known values.
- Expressions of the form `e.minSizeInBits()`, `e.minSizeInBytes()`, `e.maxSizeInBits()` and `e.maxSizeInBytes()` where the type of `e` is not generic.

NOTE | It is questionable whether instances constructed by instance declarations and constructor invocations should be considered local compile-time known values. So, the corresponding bullet is commented out in this document. A related spec issue is made here: <https://github.com/p4lang/p4-spec/issues/1309>.

The following are compile-time known values:

- All local compile-time known values.

- Constructor parameters (i.e., the declared parameters for a `parser`, `control`, etc.)
- Tuple expression where all components are compile-time known values.
- Expressions evaluating to a list type, where all elements are compile-time known values.
- Structure-valued expressions, where all fields are compile-time known values.
- Expressions evaluating to a list type, where all elements are compile-time known values.
- Legal casts applied to compile-time known values.
- Indexing a compile-time known stack or tuple value with a compile-time known index.
- Accessing a field of a compile-time known struct, header, or header union value.
- The following expressions `(, `-, `| |, |-, *, /, %, !, &, |, ^, &&, ||, <<, >>, ~, >, <, ==, !=, , >=, ++, [:, ?)` when their operands are all compile-time known values.
- Expressions of the form `e.minSizeInBits()`, `e.minSizeInBytes()`, `e.maxSizeInBits()` and `e.maxSizeInBytes()` where the type of `e` is generic.

Intuitively, the main difference between *compile-time known values* and *local compile-time known values* is that the former also contains constructor parameters. The distinction is important when it comes to defining the meaning of features like types. For example, in the type `bit<e>`, the expression `e` must be a local compile-time known value. Suppose instead that `e` were a constructor parameter—i.e., merely a compile-time known value. In this situation, it would be impossible to resolve `bit<e>` to a concrete type using purely local information—we would have to wait until the constructor was instantiated and the value of `e` known.

Local compile-time known values can be identified in the type checking phase. Compile-time known values can be identified after type checking, during the instantiation phase. Once the constructor parameters are bound to actual arguments and type parameters are bound to actual types, compile-time known values can be evaluated to concrete values.

`ctk` is a marker used in $P4_{IR}$ to indicate whether an expression is local compile-time known, compile-time known, or neither.

```
ctk
  : LCTK
  | CTK
  | DYN
  ;
```

`typedExpressionIRs` in $P4_{IR}$, produced by typing `expressions` in $P4_{16}$, annotated with their type `typeIR` and a `ctk` marker indicating the compile-time known status.

```
expressionNoteIR
  : `( typeIR ctk )
  ;

typedExpressionIR
  : expressionIR # expressionNoteIR
  ;
```

8. P4 types and values

$P4_{16}$ is a statically-typed language. Programs that do not pass the type checker are considered invalid and rejected by the compiler. P4 provides a number of base types as well as type operators that construct derived types.

The syntax for P4 types is as follows:

```
type
  : baseType
  | namedType
  | headerStackType
  | listType
  | tupleType
  ;

typeOrVoid
  : type
  | VOID
  | identifier
  ;
```

These types are represented in P4_{IR} as follows:

```
typeIR
  : baseTypeIR
  | nameTypeIR
  | aliasTypeIR
  | dataTypeIR
  | objectTypeIR
  | synthesizedTypeIR
  ;
```

The types represent the kinds of values that can be used in P4 programs. The runtime representation of P4 values is as follows:

```
value
  : baseValue
  | dataValue
  | objectReferenceValue
  | synthesizedValue
  ;
```

8.1. Semantics of types

8.1.1. Type checking surface types

`typeIR` is constructed from type checking the surface syntax of P4 types (`type`) with the following relation:

`Type_ok`:

- Typing `typeOrVoid`, under cursor `cursor` and typing context `typingContext`:
- Results in `typeIR` and a set of fresh type variables `typeid*`.

8.1.2. Well-formedness of types

The P4 type system also imposes well-formedness constraints on types, which enforce additional restrictions on the structure of types, such as type nesting.

Type_wf:

- `typeIR` is a well-formed type, with bound type variables `B`

8.2. Base types and values

P4 supports the following built-in base types:

- `bool`, which represents Boolean values
- The `error` type, which is used to convey errors in a target-independent, compiler-managed way.
- The `match_kind` type, which is used for describing the implementation of table lookups,
- The `string` type, which can be used with compile-time known values of type string.
- `int`, which represents arbitrary-sized integer values
- Fixed-width signed integers represented using two's complement `int<>`
- Bit-strings of fixed width, denoted by `bit<>`
- Bit-strings of dynamically-computed width with a fixed maximum width `varbit<>`

```
baseType
: BOOL
| ERROR
| MATCH_KIND
| STRING
| INT
| INT `< int >
| INT `< `( expression ) >
| BIT
| BIT `< int >
| BIT `< `( expression ) >
| VARBIT `< int >
| VARBIT `< `( expression ) >
;
```

In addition, P4 supports void types:

```
typeOrVoid
: type
| VOID
| identifier
;
```

- The `void` type, which has no values and can be used only in a few restricted circumstances.

In P4_{IR}, base types and values are represented as:

```
baseTypeIR
: voidTypeIR
| boolTypeIR
| errorTypeIR
| matchKindTypeIR
| stringTypeIR
| integerTypeIR
;
```

```

baseValue
  : boolValue
  | errorValue
  | matchKindValue
  | stringValue
  | integerValue
  ;

```

8.2.1. Well-formedness

The well-formedness rules for base types are as follows:

`baseTypeIR` is a well-formed type, with bound type variables `B` when:

1. The relation holds.

8.2.2. The void type

```

voidTypeIR
  : VOID
  ;

```

The void type is written `void`. It contains no values. It is not included in the production rule `baseType` as it can only appear in few restricted places in P4 programs.

There is no value of type `void`.

8.2.2.1. Type checking

Typing `VOID`, under cursor `p` and typing context `TC`:

1. Result in `VOID` and a set of fresh type variables `·`.

8.2.3. Booleans

```

boolTypeIR
  : BOOL
  ;

boolValue
  : `B bool
  ;

```

The Boolean type `bool` contains just two values, `false` and `true`. Boolean values are not integers or bit-strings.

8.2.3.1. Operations

The following operations are provided on Boolean expressions:

- And, denoted by `&&`

- Or, denoted by `||`
- Negation, denoted by `!`
- Equality and inequality tests, denoted by `==` and `!=` respectively.

The precedence of these operators is similar to C and uses short-circuited evaluation where relevant.

Additionally, the size of a boolean can be determined at compile-time ([\[sec-minsizeinbits\]](#)).

P4 does not implicitly cast from bit-strings to Booleans or vice versa. As a consequence, a program that is valid in a language like C such as,

```
if (x) /* body omitted */
```

(where `x` has an integer type) must instead be written in P4 as:

```
if (x != 0) /* body omitted */
```

See the discussion on arbitrary-precision types and implicit casts in [Section 15.2](#) for details on how the `0` in this expression is evaluated.

8.2.3.2. Type checking

Typing `BOOL`, under cursor `p` and typing context `TC`:

1. Result in `BOOL` and a set of fresh type variables `·`.

8.2.4. Errors

```
errorTypeIR
: ERROR
;

errorValue
: ERROR . nameIR
;
```

The error type contains opaque distinct values that can be used to signal errors. It is written as `error`. New elements of the error type are defined with the following syntax:

```
errorDeclaration
: ERROR `{ nameList }
;
```

All elements of the `error` type are inserted into the `error` namespace, irrespective of the place where an error is defined. `error` is similar to an enumeration (`enum`) type in other languages. A program can contain multiple `error` declarations, which the compiler will merge together. It is an error to declare the same identifier multiple times.

Error declaration is described in detail in [Section 11.7](#).

For example, the following declaration creates two elements of the `error` type (these errors are declared in

the P4 core library):

```
error { ParseError, PacketTooShort }
```

The underlying representation of errors is target-dependent.

See [Section 11.7](#) for the semantics of error declarations and [Section 14.12](#) for accessing error values.

8.2.4.1. Operations

Symbolic names declared by an `error` declaration belong to the `error` namespace. The `error` type only supports equality (`==`) and inequality (`!=`) comparisons. The result of such a comparison is a Boolean value.

For example, the following operation tests for the occurrence of an error:

```
error errorFromParser;

if (errorFromParser != error.NoError) { /* code omitted */ }
```

8.2.4.2. Type checking

Typing `ERROR`, under cursor `p` and typing context `TC`:

1. Result in `ERROR` and a set of fresh type variables `.`

8.2.5. Match kinds

```
matchKindTypeIR
: MATCH_KIND
;

matchKindValue
: MATCH_KIND . nameIR
;
```

The `match_kind` type is very similar to the `error` type and is used to declare a set of distinct names that may be used in a table's key property (described in [\[sec-table-props\]](#)). All identifiers are inserted into the top-level namespace. It is an error to declare the same `match_kind` identifier multiple times.

```
matchKindDeclaration
: MATCH_KIND `{ nameList trailingCommaOpt }
;
```

The P4 core library contains the following `match_kind` declaration:

```
match_kind {
  exact,
  ternary,
  lpm
}
```

Architectures may support additional `match_kinds`. The declaration of new `match_kinds` can only occur within model description files; P4 programmers cannot declare new match kinds.

Match kind declaration is described in detail in [Section 11.8](#).

8.2.5.1. Operations

Values of type `match_kind` are similar to `enum` values. They support only assignment and comparisons for equality and inequality.

```
match_kind { fuzzy }
const bool same = exact == fuzzy; // always 'false'
```

8.2.5.2. Type checking

Typing `MATCH_KIND`, under cursor `p` and typing context `TC`:

1. Result in `MATCH_KIND` and a set of fresh type variables `.`

8.2.6. Strings

```
stringTypeIR
: STRING
;

stringValue = stringLiteral
```

The type `string` represents strings. The values of type `string` are either string literals, or concatenations of multiple `string`-typed expressions.

One cannot declare variables with a `string` type. Parameters with type `string` can be only directionless (see [Section 18.4](#)). P4 does not support string manipulation in the dataplane; the `string` type is only allowed for describing compile-time known values (i.e., string literals, as discussed in [Section 6.2.2.3](#)). Even so, the string type is useful, for example, in giving the type signature for extern functions such as the following:

```
extern void log(string message);
```

As another example, the following annotation indicates that the specified name should be used for a given table in the generated control-plane API:

```
@name("acl") table t1 { /* body omitted */ }
```

8.2.6.1. Operations

The only operation allowed on strings is concatenation, denoted by `++`. For string concatenation, both operands must be strings and the result is also a string. String concatenation can only be performed at compile time.

```
extern void log(string message);
```

```
void foo(int<8> v) {
  // ...
  log("my log message " ++
      "continuation of the log message");
}
```

8.2.6.2. Type checking

Typing *STRING*, under cursor *p* and typing context *TC*:

1. Result in *STRING* and a set of fresh type variables $\cdot$.

8.2.7. Integers (signed and unsigned)

```
integerTypeIR
  : intTypeIR
  | fixedIntTypeIR
  | fixedBitTypeIR
  | varBitTypeIR
  ;

integerLiteral
  : D int
  | nat W int
  | nat S int
  ;

integerValue
  : integerLiteral
  | nat . nat V int
  ;
```

P4 supports arbitrary-size integer values. The typing rules for the integer types are chosen according to the following principles:

- **Inspired by C:** Typing of integers is modeled after the well-defined parts of C, expanded to cope with arbitrary fixed-width integers. In particular, the type of the result of an expression only depends on the expression operands, and not on how the result of the expression is consumed.
- **No undefined behaviors:** P4 attempts to avoid many of C's behaviors, which include the size of an integer (int), the results produced on overflow, and the results produced for some input combinations (e.g., shifts with negative amounts, overflows on signed numbers, etc.). P4 computations on integer types have no undefined behaviors.
- **Least surprise:** The P4 typing rules are chosen to behave as closely as possible to traditional well-behaved C programs.
- **Forbid rather than surprise:** Rather than provide surprising or undefined results (e.g., in C comparisons between signed and unsigned integers), we have chosen to forbid expressions with ambiguous interpretations. For example, P4 does not allow binary operations that combine signed and unsigned integers.

The priority of arithmetic operations is identical to C—e.g., multiplication binds tighter than addition.

8.2.7.1. Portability

No P4 target can support all possible types and operations. For example, the type `bit<23132312>` is legal in P4, but it is highly unlikely to be supported on any target in practice. Hence, each target can impose

restrictions on the types it can support. Such restrictions may include:

- The maximum width supported
- Alignment and padding constraints (e.g., arithmetic may only be supported on widths which are an integral number of bytes).
- Constraints on some operands (e.g., some architectures may only support multiplications by small values, or shifts with small values).

The documentation supplied with a target should clearly specify restrictions, and target-specific compilers should provide clear error messages when such restrictions are encountered. An architecture may reject a well-typed P4 program and still be conformant to the P4 spec. However, if an architecture accepts a P4 program as valid, the runtime program behavior should match this specification.

NOTE P4-SpecTec hangs on large integers.

8.2.7.2. Integer literals

The types of integer literals are as follows:

- An integer with no type prefix has type `int`.
- A non-negative integer prefixed with an integer width `W` and the character `w` has type `bit<W>`.
- An integer prefixed with an integer width `W` and the character `s` has type `int<W>`.

The table below shows several examples of integer literals and their types. For additional examples of literals see [Section 6.2.2](#).

Literal	Interpretation
<code>10</code>	Type is <code>int</code> , value is 10
<code>8w10</code>	Type is <code>bit<8></code> , value is 10
<code>8s10</code>	Type is <code>int<8></code> , value is 10
<code>2s3</code>	Type is <code>int<2></code> , value is -1 (last 2 bits), overflow warning
<code>1w10</code>	Type is <code>bit<1></code> , value is 0 (last bit), overflow warning
<code>1s1</code>	Type is <code>int<1></code> , value is -1, overflow warning

8.2.7.3. Illegal arithmetic expressions

Many arithmetic expressions that would be allowed in other languages are illegal in P4. To illustrate, consider the following declarations:

```
bit<8> x;  
bit<16> y;  
int<8> z;
```

The table below shows several expressions which are illegal because they do not obey the P4 typing rules. For each expression we provide several ways that the expression could be manually rewritten into a legal expression. Note that for some expression there are several legal alternatives, which may produce different results! The compiler cannot guess the user intent, so P4 requires the user to disambiguate.

Expression	Why it is illegal	Alternatives
<code>x + y</code>	Different widths	<code>(bit<16>)x + y</code>

Expression	Why it is illegal	Alternatives
		$x + (\text{bit}<8>)y$
$x + z$	Different signedness	$(\text{int}<8>)x + z$
		$x + (\text{bit}<8>)z$
$(\text{int}<8>)y$	Cannot change both sign and width	$(\text{int}<8>)(\text{bit}<8>)y$
		$(\text{int}<8>)(\text{int}<16>)y$
$y + z$	Different widths and signs	$(\text{int}<8>)(\text{bit}<8>)y + z$
		$y + (\text{bit}<16>)(\text{bit}<8>)z$
		$(\text{bit}<8>)y + (\text{bit}<8>)z$
		$(\text{int}<16>)y + (\text{int}<16>)z$
$x \ll z$	RHS of shift cannot be signed	$x \ll (\text{bit}<8>)z$
$x < z$	Different signs	$x < (\text{bit}<8>)z$
		$(\text{int}<8>)x < z$
$1 \ll x$	Either LHS should have a fixed width (bit shift),	$32w1 \ll x$
	Or RHS must be compile-time known (int shift)	None
~ 1	Bitwise operation on int	$\sim 32w1$
$5 \& -3$	Bitwise operation on int	$32w5 \& -3$

8.2.7.4. Arbitrary-precision integers

The arbitrary-precision data type describes integers with an unlimited precision. This type is written as `int`.

This type is reserved for integer literals and expressions that involve only literals. No P4 runtime value can have an `int` type; at compile time the compiler will convert all `int` values that have a runtime component to fixed-width types, according to the rules described below.

The following example shows three constant definitions whose values are arbitrary-precision integers.

```
const int a = 5;
const int b = 2 * a;
const int c = b - a + 3;
```

Parameters with type `int` are not supported for actions. Parameters with type `int` for other callable entities of a program, e.g. controls, parsers, or functions, must be directionless, indicating that all calls must provide a compile-time known value as an argument for such a parameter. See [Section 18.4](#) for more details on directionless parameters.

8.2.7.4.1. Operations

The type `int` denotes arbitrary-precision integers. In P4, all expressions of type `int` must be compile-time known values. The type `int` supports the following operations:

- Negation, denoted by unary `-`
- Unary plus, denoted by `+`. This operation behaves like a no-op.
- Addition, denoted by `+`.

- Subtraction, denoted by `-`.
- Comparison for equality and inequality, denoted by `==` and `!=` respectively. These operations produce a Boolean result.
- Numeric comparisons `<`, `<=`, `>`, and `>=`. These operations produce a Boolean result.
- Multiplication, denoted by `*`.
- Truncating integer division between positive values, denoted by `/`.
- Modulo between positive values, denoted by `%`.
- Arithmetic shift left and right denoted by `<<` and `>>`. These operations produce an `int` result. The right operand must be either an unsigned value of type `bit<S>` or a compile-time known value that is a non-negative integer.
- Bit slices, denoted by `[H:L]`, where `H` and `L` must be expressions that evaluate to non-negative, local compile-time known values, and `H >= L` must be true. The types of `H` and `L` (which do not need to be identical) must be one of the following:
 - `int` - an arbitrary-precision integer ([[sec-arbitrary-precision-integers](#)])
 - `bit<W>` - a `W`-bit unsigned integer where `W >= 0` ([[sec-unsigned-integers](#)])
 - `int<W>` - a `W`-bit signed integer where `W >= 1` ([[sec-signed-integers](#)])
 - a serializable `enum` with an underlying type that is `bit<W>` or `int<W>` ([[sec-enum-types](#)]).

The result is an unsigned bit-string of width `H - L + 1`, including the bits numbered from `L` (which becomes the least significant bit of the result) to `H` (the most significant bit of the result) from the source operand. The conditions `0 <= L <= H` are checked statically. If necessary, the source integer value that is sliced is automatically extended to have a width with `H` bits. Note that both endpoints of the extraction are inclusive. The bounds are required to be values that are known at compile time so that the width of the result can be computed at compile time. A slice of a negative or positive value is always a positive value.

Each operand that participates in any of these operation must have type `int` (except shifts). Binary operations cannot be used to combine values of type `int` with values of a fixed-width type (except shifts). However, the compiler automatically inserts casts from `int` to fixed-width types in certain situations—see [Chapter 15](#).

All computations on `int` values are carried out without loss of information. For example, multiplying two 1024-bit values may produce a 2048-bit value (note that concrete representation of `int` values is not specified). `int` values can be cast to `bit<w>` and `int<w>` values. Casting an `int` value to a fixed-width type will preserve the least-significant bits. If truncation causes significant bits to be lost, the compiler should emit a warning.

Note: bitwise-operations (`|`, `&`, `^`, `~`) are not defined on expressions of type `int`. In addition, it is illegal to apply division and modulo to negative values. Saturating arithmetic is not supported for arbitrary-precision integers.

8.2.7.4.2. Type checking

Typing `INT`, under cursor `p` and typing context `TC`:

1. Result in `INT` and a set of fresh type variables `.`

8.2.7.5. Fixed-width unsigned integers (bit-strings)

An unsigned integer (which we also call a "bit-string") has an arbitrary width, expressed in bits. A bit-

string of width W is declared as: `bit<W>`. W must be an expression that evaluates to a local compile-time known value (see [Section 7.5](#)) that is a non-negative integer. When using an expression for the size, the expression must be parenthesized. Bitstrings with width 0 are allowed; they have no actual bits, and can only have the value 0. See [\[sec-uninitialized-values-and-writing-invalid-headers\]](#) for additional details. Note that `bit<W>` type refers to both cases of `bit<W>` and `bit<(expression)>` where the width is a local compile-time known value.

```
const bit<32> x = 10;    // 32-bit constant with value 10.
const bit<(x + 2)> y = 15; // 12-bit constant with value 15.
                        // expression for width must use ()
```

Bits within a bit-string are numbered from 0 to $W-1$. Bit 0 is the least significant, and bit $W-1$ is the most significant.

For example, the type `bit<128>` denotes the type of bit-string values with 128 bits numbered from 0 to 127, where bit 127 is the most significant.

The type `bit` is a shorthand for `bit<1>`.

P4 architectures may impose additional constraints on bit types: for example, they may limit the maximum size, or they may only support some arithmetic operations on certain sizes (e.g., 16-, 32-, and 64-bit values).

8.2.7.5.1. Operations

This section discusses all operations that can be performed on expressions of type `bit<W>` for some width W , also known as bit-strings.

Arithmetic operations "wrap around", similar to C operations on unsigned values (i.e., representing a large value on W bits will only keep the least-significant W bits of the value). In particular, P4 does not have arithmetic exceptions—the result of an arithmetic operation is defined for all possible inputs.

P4 target architectures may optionally support saturating arithmetic. All saturating operations are limited to a fixed range between a minimum and maximum value. Saturating arithmetic has advantages, in particular when used as counters. The result of a saturating counter max-ing out is much closer to the real result than a counter that overflows and wraps around. According to Wikipedia [Saturating Arithmetic](#) is as numerically close to the true answer as possible; for 8-bit binary signed arithmetic, when the correct answer is 130, it is considerably less surprising to get an answer of 127 from saturating arithmetic than to get an answer of -126 from modular arithmetic. Likewise, for 8-bit binary unsigned arithmetic, when the correct answer is 258, it is less surprising to get an answer of 255 from saturating arithmetic than to get an answer of 2 from modular arithmetic. At this time, P4 defines saturating operations only for addition and subtraction. For an unsigned integer with bit-width of W , the minimum value is 0 and the maximum value is 2^W-1 . The precedence of saturating addition and subtraction operations is the same as for modular arithmetic addition and subtraction.

All binary operations except shifts and concatenation require both operands to have the same exact type and width; supplying operands with different widths produces an error at compile time. No implicit casts are inserted by the compiler to equalize the widths. There are no other binary operations that accept signed and unsigned values simultaneously besides shifts and concatenation. The following operations are provided on bit-string expressions:

- Test for equality between bit-strings of the same width, designated by `==`. The result is a Boolean value.
- Test for inequality between bit-strings of the same width, designated by `!=`. The result is a Boolean value.
- Unsigned comparisons `<`, `>`, `<=`, `>=`. Both operands must have the same width and the result is a Boolean value.

value.

Each of the following operations produces a bit-string result when applied to bit-strings of the same width:

- Negation, denoted by unary `-`. The result is computed by subtracting the value from 2^w . The result is unsigned and has the same width as the input. The semantics is the same as the C negation of unsigned numbers.
- Unary plus, denoted by `+`. This operation behaves like a no-op.
- Addition, denoted by `+`. This operation is associative. The result is computed by truncating the result of the addition to the width of the output (similar to C).
- Subtraction, denoted by `-`. The result is unsigned, and has the same type as the operands. It is computed by adding the negation of the second operand (similar to C).
- Multiplication, denoted by `*`. The result has the same width as the operands and is computed by truncating the result to the output's width. P4 architectures may impose additional restrictions—e.g., they may only allow multiplication by a non-negative integer power of two.
- Bitwise "and" between two bit-strings of the same width, denoted by `&`.
- Bitwise "or" between two bit-strings of the same width, denoted by `|`.
- Bitwise "complement" of a single bit-string, denoted by `~`.
- Bitwise "xor" of two bit-strings of the same width, denoted by `^`.
- Saturating addition, denoted by `|+|`.
- Saturating subtraction, denoted by `|-|`.

Bit-strings also support the following operations:

- Logical shift left and right by a non-negative integer value (which need not be a compile-time known value), denoted by `<<` and `>>` respectively. In a shift, the left operand is unsigned, and right operand must be either an expression of type `bit<S>` or a non-negative integer value that is known at compile time. The result has the same type as the left operand. Shifting by an amount greater than or equal to the width of the input produces a result where all bits are zero.
- Extraction of a set of contiguous bits, also known as a slice, denoted by `[H:L]`, where `H` and `L` must be expressions that evaluate to non-negative, local compile-time known values, and `H >= L`. The types of `H` and `L` (which do not need to be identical) must be numeric (`[sec-numeric-values]`). The result is a bit-string of width `H - L + 1`, including the bits numbered from `L` (which becomes the least significant bit of the result) to `H` (the most significant bit of the result) from the source operand. The conditions `0 <= L <= H < W` are checked statically (where `W` is the length of the source bit-string). Note that both endpoints of the extraction are inclusive. The bounds are required to be local compile-time known values so that the width of the result can be computed at compile time. Slices are also l-values, which means that P4 supports assigning to a slice: `e[H:L] = x`. The effect of this statement is to set bits `H` through `L` (inclusive of both) of `e` to the bit-pattern represented by `x`, and leaves all other bits of `e` unchanged. A slice of an unsigned integer is an unsigned integer.
- Concatenation of bit-strings and/or fixed-width signed integers, denoted by `++`. The two operands must be either `bit<W>` or `int<W>`, and they can be of different signedness and width. The result has the same signedness as the left operand and the width equal to the sum of the two operands' width. In concatenation, the left operand is placed as the most significant bits.

Additionally, the size of a bit-string can be determined at compile-time (`[sec-minsizeinbits]`).

8.2.7.5.2. Type checking

Typing `baseType`, under cursor `p` and typing context `TC`:

1. If `baseType` is `BIT`:
 - a. Result in `BIT < 1 >` and a set of fresh type variables $\cdot$.
2. Else if let `BIT < int >` be `baseType`:
 - a. Check that `int` has type `nat`.
 - b. Let `n` be `int`.
 - c. Result in `BIT < n >` and a set of fresh type variables $\cdot$.
3. Else if let `BIT < ( expression ) >` be `baseType`:
 - a. Let `typedExpressionIR` be
 - the result of typing `expression`, under cursor `p` and typing context `TC`.
 - b. Check that the compile-time known-ness of `typedExpressionIR` is `LCTK`.
 - c. Let `value` be
 - the result of local compile-time evaluation of `typedExpressionIR`, under cursor `p` and typing context `TC`.
 - d. Check that `value` has type `integerValue`.
 - e. Let `integerValue` be `value`.
 - f. Let `n` be ! the natural number representation of `integerValue`.
 - g. Result in `BIT < n >` and a set of fresh type variables $\cdot$.

8.2.7.6. Fixed-width signed integers

Signed integers are represented using two's complement. An integer with `W` bits is declared as: `int<W>`. `W` must be an expression that evaluates to a local compile-time known (see Section 7.5) value that is a non-negative integer. Note that `int<W>` type refers to both cases of `int<W>` and `int<(expression)>` where the width is a local compile-time known value.

Bits within an integer are numbered from 0 to `W-1`. Bit 0 is the least significant, and bit `W-1` is the sign bit.

For example, the type `int<64>` describes the type of integers represented using exactly 64 bits with bits numbered from 0 to 63, where bit 63 is the most significant (sign) bit.

P4 architectures may impose additional constraints on signed types: for example, they may limit the maximum size, or they may only support some arithmetic operations on certain sizes (e.g., 16-, 32-, and 64-bit values).

A signed integer with width 1 can only have two legal values: 0 and -1.

8.2.7.6.1. Operations

This section discusses all operations that can be performed on expressions of type `int<W>` for some `W`. Recall that the `int<W>` denotes signed `W`-bit integers, represented using two's complement.

In general, P4 arithmetic operations do not detect "underflow" or "overflow": operations simply "wrap around", similar to C operations on unsigned values. Hence, attempting to represent large values using `W` bits will only keep the least-significant `W` bits of the value.

P4 supports saturating arithmetic (addition and subtraction) for signed integers. Targets may optionally reject programs using saturating arithmetic. For a signed integer with bit-width of `W`, the minimum value is $-2^{(W-1)}$ and the maximum value is $2^{(W-1)}-1$.

P4 also does not support arithmetic exceptions. The runtime result of an arithmetic operation is defined for all combinations of input arguments.

All binary operations except shifts and concatenation require both operands to have the same exact type (signedness) and width and supplying operands with different widths or signedness produces a compile-time error. No implicit casts are inserted by the compiler to equalize the types. Except for shifts and concatenation, P4 does not have any binary operations that operate simultaneously on signed and unsigned values.

Note that bitwise operations on signed integers are well-defined, since the representation is mandated to be two's complement.

The `int<W>` datatype supports the following operations; all binary operations require both operands to have the exact same type. The result always has the same width as the left operand.

- Negation, denoted by unary `-`.
- Unary plus, denoted by `+`. This operation behaves like a no-op.
- Addition, denoted by `+`.
- Subtraction, denoted by `-`.
- Comparison for equality and inequality, denoted `==` and `!=` respectively. These operations produce a Boolean result.
- Numeric comparisons, denoted by `<`, `<=`, `>`, and `>=`. These operations produce a Boolean result.
- Multiplication, denoted by `*`. Result has the same width as the operands. P4 architectures may impose additional restrictions—e.g., they may only allow multiplication by a power of two.
- Bitwise "and" between two bit-strings of the same width, denoted by `&`.
- Bitwise "or" between two bit-strings of the same width, denoted by `|`.
- Bitwise "complement" of a single bit-string, denoted by `~`.
- Bitwise "xor" of two bit-strings of the same width, denoted by `^`.
- Saturating addition, denoted by `|+|`.
- Saturating subtraction, denoted by `|-|`.

The `int<W>` datatype also support the following operations:

- Arithmetic shift left and right denoted by `<<` and `>>`. The left operand is signed and the right operand must be either an unsigned number of type `bit<S>` or a compile-time known value that is a non-negative integer. The result has the same type as the left operand. Shifting left produces the exact same bit pattern as a shift left of an unsigned value. Shift left can thus overflow, when it leads to a change of the sign bit. Shifting by an amount greater than the width of the input produces a "correct" result:
 - all result bits are zero when shifting left
 - all result bits are zero when shifting a non-negative value right
 - all result bits are one when shifting a negative value right
- Extraction of a set of contiguous bits, also known as a slice, denoted by `[H:L]`, where `H` and `L` must be expressions that evaluate to non-negative, local compile-time known values, and `H >= L` must be true. The types of `H` and `L` (which do not need to be identical) must be numeric ([\[sec-numeric-values\]](#)). The result is an unsigned bit-string of width `H - L + 1`, including the bits numbered from `L` (which becomes the least significant bit of the result) to `H` (the most significant bit of the result) from the source operand. The conditions `0 <= L <= H < W` are checked statically (where `W` is the length of the source bit-string). Note that both endpoints of the extraction are inclusive. The bounds are required to be values that are known at compile time so that the width of the result can be computed at compile time.

Slices are also l-values, which means that P4 supports assigning to a slice: $e[H:L] = x$. The effect of this statement is to set bits H through L of e to the bit-pattern represented by x , and leaves all other bits of e unchanged. A slice of a signed integer is treated as an unsigned integer.

- Concatenation of bit-strings and/or fixed-width signed integers, denoted by $++$. The two operands must be either $\text{bit}\langle W \rangle$ or $\text{int}\langle W \rangle$, and they can be of different signedness and width. The result has the same signedness as the left operand and the width equal to the sum of the two operands' width. In concatenation, the left operand is placed as the most significant bits.

Additionally, the size of a fixed-width signed integer can be determined at compile-time (`[sec-minsizeinbits]`).

8.2.7.6.2. Type checking

Typing baseType , under cursor p and typing context TC :

1. If let $\text{INT} < \text{int} >$ be baseType :
 - a. Check that int has type nat .
 - b. Let n be int .
 - c. Result in $\text{INT} < n >$ and a set of fresh type variables $\cdot$.
2. Else if let $\text{INT} < (\text{expression}) >$ be baseType :
 - a. Let typedExpressionIR be
 - the result of typing expression , under cursor p and typing context TC .
 - b. Check that the compile-time known-ness of typedExpressionIR is LCTK .
 - c. Let value be
 - the result of local compile-time evaluation of typedExpressionIR , under cursor p and typing context TC .
 - d. Check that value has type integerValue .
 - e. Let integerValue be value .
 - f. Let n be ! the natural number representation of integerValue .
 - g. Result in $\text{INT} < n >$ and a set of fresh type variables $\cdot$.

8.2.7.7. Dynamically-sized bit-strings

Some network protocols use fields whose size is only known at runtime (e.g., IPv4 options). To support restricted manipulations of such values, P4 provides a special bit-string type whose size is set at runtime, called a **varbit**.

The type $\text{varbit}\langle W \rangle$ denotes a bit-string with a width of at most W bits, where W is a local compile-time known value (see Section 7.5) that is a non-negative integer. For example, the type $\text{varbit}\langle 120 \rangle$ denotes the type of bit-string values that may have between 0 and 120 bits. Most operations that are applicable to fixed-size bit-strings (unsigned numbers) **cannot** be performed on dynamically sized bit-strings. Note that $\text{varbit}\langle W \rangle$ type refers to both cases of $\text{varbit}\langle W \rangle$ and $\text{varbit}\langle \text{expression} \rangle$ where the width is a local compile-time known value.

P4 architectures may impose additional constraints on **varbit** types: for example, they may limit the maximum size, or they may require **varbit** values to always contain an integer number of bytes at runtime.

8.2.7.7.1. Operations

To support parsing headers with variable-length fields, P4 offers a type `varbit`. Each occurrence of the type `varbit` has a statically-declared maximum width, as well as a dynamic width, which must not exceed the static bound. Prior to initialization a variable-size bit-string has an unknown dynamic width.

Variable-length bit-strings support a limited set of operations:

- Assignment to another variable-sized bit-string. The target of the assignment must have the same static width as the source. When executed, the assignment sets the dynamic width of the target to the dynamic width of the source.
- Comparison for equality or inequality with another `varbit` field. Two `varbit` fields can be compared only if they have the same type. Two varbits are equal if they have the same dynamic width and all the bits up to the dynamic width are the same.

The following operations are *not* supported directly on a value of type `varbit`, but instead on any type for which `extract` and `emit` operations are supported (e.g. a value with type header) that may contain a field of type `varbit`. They are mentioned here only to ease finding this information in a section dedicated to type `varbit`.

- Parser extraction into a header containing a variable-sized bit-string using the two-argument `extract` method of a `packet_in` extern object (see [\[sec-packet-extract-two\]](#)). This operation sets the dynamic width of the field.
- The `emit` method of a `packet_out` extern object can be performed on a header and a few other types (see [\[sec-deparse\]](#)) that contain a field with type `varbit`. Such an `emit` method call inserts a variable-sized bit-string with a known dynamic width into the packet being constructed.

Additionally, the maximum size of a variable-length bit-string can be determined at compile-time ([\[sec-minsizeinbits\]](#)).

8.2.7.7.2. Type checking

Typing `baseType`, under cursor `p` and typing context `TC`:

1. If `let VARBIT < int > be baseType`:
 - a. Check that `int` has type `nat`.
 - b. Let `n` be `int`.
 - c. Result in `VARBIT < n >` and a set of fresh type variables `·`.
2. Else if `let VARBIT < ( expression ) > be baseType`:
 - a. Let `typedExpressionIR` be
 - the result of typing `expression`, under cursor `p` and typing context `TC`.
 - b. Check that the compile-time known-ness of `typedExpressionIR` is `LCTK`.
 - c. Let `value` be
 - the result of local compile-time evaluation of `typedExpressionIR`, under cursor `p` and typing context `TC`.
 - d. Check that `value` has type `integerValue`.
 - e. Let `integerValue` be `value`.
 - f. Let `n` be ! the natural number representation of `integerValue`.
 - g. Result in `VARBIT < n >` and a set of fresh type variables `·`.

8.3. Named types

```
prefixedTypeName
: typeName
| `TID . typeName
;

specializedType
: prefixedTypeName `< typeArgumentList >
;

namedType
: prefixedTypeName
| specializedType
;
```

Named types are types that reference another type by name. User-defined types can be introduced with their names. For example, the following declares a new struct type `S`:

```
struct S {
    bit<32> x;
    bit<32> y;
}
```

This type can then be referenced by its name, as can be seen in the parameter type of the following example:

```
void f(S s) {}
```

Here, `S` is a named type (`typeName`) that refers to the struct type defined earlier.

Types such as struct, header, and header union types can be generic. In order to use such a generic type it must be specialized with appropriate type arguments. For example,

```
// generic structure type
struct S<T> {
    T field;
    bool valid;
}

struct G<T> {
    S<T> s;
}

// specialize S by replacing 'T' with 'bit<32>'
const S<bit<32>> s = { field = 32w0, valid = false };
// Specialize G by replacing 'T' with 'bit<32>'
const G<bit<32>> g = { s = { field = 0, valid = false } };

// generic header type
header H<T> {
    T field;
}

// Specialize H by replacing 'T' with 'bit<8>'
const H<bit<8>> h = { field = 1 };
// Header stack produced from a specialization of a generic header type
H<bit<8>>[10] stack;

// Generic header union
```



```
header_union HU<T> {
    H<bit<32>> h32;
    H<bit<8>> h8;
    H<T> ht;
}

// Header union with a type obtained by specializing a generic header union type
HU<bit> hu;
```

8.3.1. Type definitions (type constructors)

```
typeDeclaration
: derivedTypeDeclaration
| typedefDeclaration
| parserTypeDeclaration
| controlTypeDeclaration
| packageTypeDeclaration
;
```

`typeDeclaration` introduces a user-defined type definition. These can be referenced by name. Generic type definitions can be specialized with type arguments to produce a type instance.

The internal representation of type definitions is as follows:

```
nameTypeDefIR = nameTypeIR

aliasTypeDefIR = aliasTypeIR

dataTypeDefIR
: structTypeDefIR
| headerTypeDefIR
| headerUnionTypeDefIR
| enumTypeDefIR
;

objectTypeDefIR
: externObjectTypeDefIR
| parserObjectTypeDefIR
| controlObjectTypeDefIR
| packageObjectTypeDefIR
| tableObjectTypeDefIR
;

typeDefIR
: nameTypeDefIR
| aliasTypeDefIR
| dataTypeDefIR
| objectTypeDefIR
;
```

For example, `structTypeDeclaration` introduces a struct type definition, `structTypeDefIR`:

```
structTypeDefIR
: STRUCT typeId `< typeParameterIR* > `{ fieldTypeIR* }
;

structTypeDeclaration
: annotationList STRUCT name typeParameterListOpt `{ typeFieldList }
;
```

Some type definitions like struct and header type definitions can be polymorphic, meaning they can have type variables as parameters. On the other hand, type definitions such as enum and type alias definitions can only be monomorphic. Below table summarizes which kinds of type definitions can be polymorphic:

NOTE	Add a table for <code>\$is_monomorphic_typeDefIR</code> here.
NOTE	Add explainer on explicit and implicit type parameters for type definitions here.

8.3.2. Type resolution

Notice that P4 has struct types, but the surface syntax does not have an explicit construct for struct types. Instead, struct types are always referenced by name. To easily handle named types in $P4_{IR}$, the internal representation of types contain representations for the referred types.

```
structTypeIR
: STRUCT typeId `< typeArgumentIR* > `{ fieldTypeIR* }
;
```

For example, a struct type is represented as `structTypeIR` in $P4_{IR}$. Thus, after type checking, a named type in $P4_{16}$ is resolved to its referred type in $P4_{IR}$. For named types referring to monomorphic type definitions, the resolved type is simply the type defined by the type definition.

```
enum E_t { A, B, C; }
void f(E_t e) {}
```

In the above example, the named type `E_t` is resolved to the enum type `enumTypeIR`.

```
struct S<T> { T t; }
void f(S<bit<32>> s) {}
```

In the above example, the named type `S<bit<32>>` is resolved to a `structTypeIR`, where it specializes a `structTypeDefIR` with the type argument `bit<32>`. Type specialization is discussed in detail in the next section.

8.3.3. Type specialization

`typeDefIRbase` specialized by `typeArgumentIR*`

1. Let (`typeParameterIRexpl*`, `typeParameterIRimpl*`) be the type parameters of `typeDefIRbase*`.
2. Let `typeParameterIR*` be `typeParameterIRexpl*` concatenated with `typeParameterIRimpl*`.
3. Check that the length of `typeParameterIR*` is equal to the length of `typeArgumentIR*`.
4. Let `theta` be { (`typeParameterIR` : `typeArgumentIR`) }.
5. Let `typeIRbase` be the underlying type of `typeDefIRbase*`.
6. Let `typeIRspec` be `typeIRbase` substituted by `theta`.
7. Return `typeIRspec*`.

8.3.4. Prefixed type names

```
prefixedTypeName
: typeName
| `TID . typeName
;
```

A `prefixedTypeName` may refer to either a monomorphic type definition or a polymorphic type definition without type arguments. The names can be prefixed by a dot (.) to refer to the top-level namespace.

On type checking, the referenced type definition is looked up by name. If the referenced type definition is monomorphic, then the resolved type is simply the type defined by the type definition. If the referenced type definition is polymorphic (but without type arguments), then the resolved type is a specialized type with no type arguments.

In some cases, the reference may be a type parameter. In other words, the type is indeed a variable type, not a concrete type until it is bound to a type argument. These are internally represented as:

```
nameTypeDefIR = nameTypeIR
```

8.3.4.1. Operations

Because functions and methods can be generic, they offer the possibility of declaring values with types that are type variables:

```
void f<T>() {
  T x; // the type of x is T, a type variable
}
```

The type of such objects is not known until the function is specialized with specific type arguments.

Currently the only operations that are available for such values are assignment (explicit through `=`, or implicit, through argument passing). This behavior is similar to languages such as Java, and different from languages such as C++.

A future version of P4 may introduce a notion of *type constraints* which would enable more operations on such values. Because of this limitation, such values are currently of limited utility.

8.3.4.2. Type checking

Typing `prefixedTypeName`, under cursor `p` and typing context `TC`:

1. Let `prefixedNameIR` be the prefixed name of `prefixedTypeName`.
2. Let `typeDefIR'` be ! the type definition of `prefixedNameIR` from the `p` layer of `TC`.
3. If `typeDefIR'` is monomorphic:
 - a. Let `typeIR` be the underlying type of `typeDefIR'`.
 - b. Result in `typeIR` and a set of fresh type variables `·`.
4. Else:
 - a. Let `typeIR` be `typeDefIR'` specialized by `·`.
 - b. Result in `typeIR` and a set of fresh type variables `·`.

8.3.5. Specialized types

A generic type may be specialized by specifying arguments for its type variables. In cases where the compiler can infer type arguments, type specialization is not necessary. When a type is specialized, all its type variables must be bound.

NOTE

P4-SpecTec currently does **not** infer type arguments for type specialization, where it only performs inference for function and method calls.

```
specializedType
: prefixedTypeName `< typeArgumentList >
;
```

For example, the following extern declaration describes a generic block of registers, where the type of the elements stored in each register is an arbitrary `T`.

```
extern Register<T> {
  Register(bit<32> size);
  T read(bit<32> index);
  void write(bit<32> index, T value);
}
```

The type `T` has to be specified when instantiating a set of registers, by specializing the `Register` type:

```
Register<bit<32>>(128) registerBank;
```

The instantiation of `registerBank` is made using the `Register` type specialized with the `bit<32>` bound to the `T` type argument.

8.3.5.1. Type checking

Typing `prefixedTypeName < typeArgumentList >`, under cursor `p` and typing context `TC`:

1. Let `prefixedNameIR` be the prefixed name of `prefixedTypeName`.
2. Let `typeDefIR'` be ! the type definition of `prefixedNameIR` from the `p` layer of `TC`.
3. Check that `typeDefIR'` is polymorphic.
4. Let `typeIRarg*` and a set of fresh type variables `typeIdfresh*` be
 - the result of typing `typeArgumentList`, under cursor `p` and typing context `TC`.
5. Let `typeIR` be `typeDefIR'` specialized by `typeIRarg*`.
6. Result in `typeIR` and a set of fresh type variables `typeIdfresh*`.

8.4. Data types and values

```
dataTypeIR
: listTypeIR
| tupleTypeIR
| headerStackTypeIR
| structTypeIR
| headerTypeIR
```

```

| headerUnionTypeIR
| enumTypeIR
;

dataValue
: listValue
| tupleValue
| headerStackValue
| structValue
| headerValue
| headerUnionValue
| enumValue
;

```

Data types represent the types that aggregate other types, including lists, tuples, header stacks, structs, headers, header unions, and enums.

8.4.1. Lists

A list holds zero or more values, where every element must have the same type. The type of a list where all elements have type T is written as

```
list<T>
```

```

listType
: LIST `< typeArgument >
;

```

Internally, list types and values are represented as follows:

```

listTypeIR
: LIST `< typeIR >
;

listValue
: LIST `[ value* ]
;

```

8.4.1.1. Operations

The value of a list is written using curly braces, with each element separated by a comma. The left curly brace is preceded by a `(list<T>)` where T is the list element type. Such a value can be passed as an argument, e.g. to extern constructor functions.

```

struct pair_t {
    bit<16> a;
    bit<32> b;
}

extern E {
    E(list<pair_t> data);
    void run();
}

control c() {
    E((list<pair_t>) {{2, 3}, {4, 5}}) e;
    apply {
        e.run();
    }
}

```

```
}  
}
```

Additionally, the size of a list can be determined at compile-time ([`sec-minsizeinbits`]).

8.4.1.2. Type checking

Typing `LIST < typeArgument >`, under cursor `p` and typing context `TC`:

1. Let `typeIRarg` and a set of fresh type variables `typeIDfresh*` be
 - the result of typing `typeArgument`, under cursor `p` and typing context `TC`.
2. Result in `LIST < typeIRarg >` and a set of fresh type variables `typeIDfresh*`.

8.4.1.3. Well-formedness

`LIST < typeIR' >` is a well-formed type, with bound type variables `B` when:

1. Check that `typeIR'` can be nested in a list type.
2. Check that `typeIR'` is a well-formed type, with bound type variables `B`.
3. The relation holds.

8.4.2. Tuples

A tuple is similar to a `struct`, in that it holds multiple values. The type of tuples with `n` component types `T1`, ..., `Tn` is written as

```
tuple<T1, /* more fields omitted */ , Tn>
```

The type `tuple<>` is a tuple type with no components.

```
tupleType  
: TUPLE `< typeArgumentList >  
;
```

Internally, tuple types and values are represented as follows:

```
tupleTypeIR  
: TUPLE `< typeIR* >  
;  
  
tupleValue  
: TUPLE `( value* )  
;
```

8.4.2.1. Operations

The fields of a tuple can be accessed using array index syntax `x[0]`, `x[1]`. The indexes **must** be local compile-time known values, to enable the type-checker to identify the field types statically.

Tuples can be compared for equality using `==` and `!=`; two tuples are equal if and only if all their fields are respectively equal.

Currently tuple fields are not left-values, even if the tuple itself is. (i.e., a tuple can only be assigned monolithically, and the field values cannot be changed individually.) This restriction may be lifted in a future version of the language.

8.4.2.2. Type checking

Typing `TUPLE < typeArgumentList >`, under cursor `p` and typing context `TC`:

1. Let `typeIRarg*` and a set of fresh type variables `typeIdfresh*` be
 - the result of **typing** `typeArgumentList`, under cursor `p` and typing context `TC`.
2. Result in `TUPLE < typeIRarg* >` and a set of fresh type variables `typeIdfresh*`.

8.4.2.3. Well-formedness

`TUPLE < typeIR* >` is a well-formed type, with bound type variables `B` when:

1. Check that `typeIR'` can be nested in a tuple type, for all `typeIR'` in `typeIR*`.
2. Check that `typeIR'` is a well-formed type, with bound type variables `B`, for all `typeIR'` in `typeIR*`.
3. The relation holds.

8.4.3. Header stacks

A header stack represents an array of headers or header unions. A header stack type is defined as:

```
headerStackType
  : namedType `[ expression ]
  ;
```

where `namedType` refers to a header or header union type. For a header stack `hs[n]`, the term `n` is the maximum defined size, and must be a local compile-time known value that is a positive integer. Nested header stacks are not supported.

Internally, header stack types and values are represented as follows:

```
headerStackTypeIR
  : typeIR `[ nat ]
  ;

headerStackValue
  : HEADER_STACK `[ value* `( nat ; nat ) ]
  ;
```

At runtime a stack contains `n` values with type `namedType`, only some of which may be valid. The valid elements of a header stack need not be contiguous.

For example, the following declarations,

```
header Mpls_h {
    bit<20> label;
    bit<3> tc;
    bit bos;
    bit<8> ttl;
}
Mpls_h[10] mpls;
```

introduce a header stack called `mpls` containing ten entries, each of type `Mpls_h`.

8.4.3.1. Operations

P4 provides a set of computations for manipulating header stacks. A header stack `hs` of type `h[n]` can be understood in terms of the following pseudocode:

```
// type declaration
struct hs_t {
    bit<32> nextIndex;
    bit<32> size;
    h[n] data; // Ordinary array
}

// instance declaration and initialization
hs_t hs;
hs.nextIndex = 0;
hs.size = n;
```

Intuitively, a header stack can be thought of as a struct containing an ordinary array of headers `hs` and a counter `nextIndex` that can be used to simplify the construction of parsers for header stacks, as discussed below. The `nextIndex` counter is initialized to 0.

Given a header stack value `hs` of size `n`, the following expressions are legal:

- `hs[index]`: produces a reference to the header at the specified position within the stack; if `hs` is an l-value, the result is also an l-value. The header may be invalid. Some implementations may impose the constraint that the index expression must be a compile-time known value. A P4 compiler must give an error if an index that is a compile-time known value is out of range.

Accessing a header stack `hs` with an index less than 0 or greater than or equal to `hs.size` results in an undefined value. See [\[sec-uninitialized-values-and-writing-invalid-headers\]](#) for more details.

The `index` is an expression that must be of numeric types ([\[sec-numeric-values\]](#)).

- `hs.size`: produces a 32-bit unsigned integer that returns the size of the header stack (a local compile-time known value).
- assignment from a header stack `hs` into another stack requires the stacks to have the same types and sizes. All components of `hs` are copied, including its elements and their validity bits, as well as `nextIndex`.

To help programmers write parsers for header stacks, P4 also offers computations that automatically advance through the stack as elements are parsed:

- `hs.next`: produces a reference to the element with index `hs.nextIndex` in the stack. May only be used in a parser. If the stack's `nextIndex` counter is greater than or equal to `size`, then evaluating this expression results in a transition to `reject` and sets the error to `error.StackOutOfBounds`. If `hs` is an l-value, then `hs.next` is also an l-value.
- `hs.last`: produces a reference to the element with index `hs.nextIndex - 1` in the stack, if such an element

exists. May only be used in a `parser`. If the `nextIndex` counter is less than 1, or greater than `size`, then evaluating this expression results in a transition to `reject` and sets the error to `error.StackOutOfBounds`. Unlike `hs.next`, the resulting reference is never an l-value.

- `hs.lastIndex`: produces a 32-bit unsigned integer that encodes the index `hs.nextIndex - 1`. May only be used in a `parser`. If the `nextIndex` counter is 0, then evaluating this expression produces an undefined value.

Finally, P4 offers the following computations that can be used to manipulate the elements at the front and back of the stack:

- `hs.push_front(int count)`: shifts `hs` "right" by `count`. The first `count` elements become invalid. The last `count` elements in the stack are discarded. The `hs.nextIndex` counter is incremented by `count`. The `count` argument must be a compile-time known value that is a positive integer. The return type is `void`.
- `hs.pop_front(int count)`: shifts `hs` "left" by `count` (i.e., element with index `count` is copied in stack at index 0). The last `count` elements become invalid. The `hs.nextIndex` counter is decremented by `count`. The `count` argument must be a compile-time known value that is a positive integer. The return type is `void`.

The following pseudocode defines the behavior of `push_front` and `pop_front`:

```
void push_front(int count) {
    for (int i = this.size-1; i >= 0; i -= 1) {
        if (i >= count) {
            this[i] = this[i-count];
        } else {
            this[i].setInvalid();
        }
    }
    this.nextIndex = this.nextIndex + count;
    if (this.nextIndex > this.size) this.nextIndex = this.size;
    // Note: this.last, this.next, and this.lastIndex adjust with this.nextIndex
}

void pop_front(int count) {
    for (int i = 0; i < this.size; i++) {
        if (i+count < this.size) {
            this[i] = this[i+count];
        } else {
            this[i].setInvalid();
        }
    }
    if (this.nextIndex >= count) {
        this.nextIndex = this.nextIndex - count;
    } else {
        this.nextIndex = 0;
    }
    // Note: this.last, this.next, and this.lastIndex adjust with this.nextIndex
}
```

Similar to structs and headers, the size of a header stack is a compile-time known value (Section [\[seminsizeinbits\]](#)).

Two header stacks can be compared for equality (`==`) or inequality (`!=`) only if they have the same element type and the same length. Two stacks are equal if and only if all their corresponding elements are equal. Note that the `nextIndex` value is not used in the equality comparison.

8.4.3.2. Type checking

Typing `namedType [ expressionsize ], under cursor p and typing context TC:`

1. Let `typeIdRbase` and a set of fresh type variables `typeIdfresh*` be

- the result of **typing** `namedType`, under cursor `p` and typing context `TC`.
2. Let `typedExpressionIRsize` be
 - the result of **typing** `expressionsize`, under cursor `p` and typing context `TC`.
 3. Check that **the compile-time known-ness of** `typedExpressionIRsize` **is** `LCTK`.
 4. Let `value` be
 - the result of **local compile-time evaluation of** `typedExpressionIRsize`, under cursor `p` and typing context `TC`.
 5. Check that `value` has type `integerValue`.
 6. Let `integerValuesize` be `value`.
 7. Let `nsize` be **! the natural number representation of** `integerValuesize`.
 8. Check that `nsize` is greater than 0.
 9. Result in `typeIRbase [ nsize ]` and a set of fresh type variables `typedfresh*`.

8.4.3.3. Well-formedness

`typeIR' [ nat ]` is a well-formed type, with bound type variables `B` when:

1. Check that `typeIR'` can be nested in a header stack type.
2. Check that `typeIR'` is a well-formed type, with bound type variables `B`.
3. The relation holds.

8.4.4. Arrays

NOTE | This feature will be added to P4 v1.2.5.6. It is not modeled in P4-SpecTec yet.

8.4.5. Structs

P4 `struct` types are defined with the following syntax:

```
typeField
: annotationList type name ;
;

structTypeDeclaration
: annotationList STRUCT name typeParameterListOpt `{ typeFieldList }
;
;
```

This declaration introduces a new struct type definition with the specified name in the current scope. Field names have to be distinct. An empty struct (with no fields) is legal.

```
structTypeDefIR
: STRUCT typeId `< typeParameterIR* > `{ fieldTypeIR* }
;
;
```

For example, the structure `Parsed_headers` below contains the headers recognized by a simple parser:

```

header Tcp_h { /* fields omitted */ }
header Udp_h { /* fields omitted */ }
struct Parsed_headers {
    Ethernet_h ethernet;
    Ip_h       ip;
    Tcp_h      tcp;
    Udp_h      udp;
}

```

Struct types can be used by referring to their names. Generic struct types can be used by specializing their type parameters with type arguments. Internally, struct types and values are represented as follows:

```

fieldTypeIR
: annotationList typeIR nameIR ;
;

structTypeIR
: STRUCT typeId `< typeArgumentIR* > `{ fieldTypeIR* }
;

fieldValue
: value nameIR ;
;

structValue
: STRUCT typeId `{ fieldValue* }
;

```

8.4.5.1. Operations

The only operation defined on expressions whose type is a `struct` is field access, written using dot (".") notation—e.g., `s.field`. If `s` is an l-value, then `s.field` is also an l-value. P4 also allows copying `structs` using assignment when the source and target of the assignment have the same type. Finally, `structs` can be initialized with a sequence expression, as discussed in [Section 8.6.3](#), or with a record expression, as described in [Section 8.6.4](#). Both of these cases must initialize all fields of the structure. The size of a struct can be determined at compile-time ([\[sec-minsizeinbits\]](#)).

Two structs can be compared for equality (==) or inequality (!=) only if they have the same type and all of their fields can be recursively compared for equality. Two structures are equal if and only if all their corresponding fields are equal.

The following example shows a structure initialized in several different ways:

```

struct S {
    bit<32> a;
    bit<32> b;
}
const S x = { 10, 20 };           // sequence expression
const S x = { a = 10, b = 20 };   // record expression
const S x = (S) { a = 10, b = 20 }; // record expression

```

See [\[sec-uninitialized-values-and-writing-invalid-headers\]](#) for a description of the behavior if `struct` fields are read without being initialized.

8.4.5.2. Type checking

Struct types are introduced by `struct` declarations. Its typing rule is explained in [Section 11.14](#). Struct types are then used by referencing their names. Thus struct types can be obtained from named types, as

illustrated in [Section 8.3](#).

8.4.5.3. Well-formedness

`STRUCT typeId < typeArgumentIR* > { ( annotationList typeIRfield nameIRfield ; )* }` is a well-formed type, with bound type variables `B` when:

1. Check that the elements of `nameIRfield*` are distinct.
2. Check that `typeIRfield` can be nested in a struct type, for all `typeIRfield` in `typeIRfield*`.
3. Check that `typeIRfield` is a well-formed type, with bound type variables `B`, for all `typeIRfield` in `typeIRfield*`.
4. The relation holds.

8.4.6. Headers

The declaration of a `header` type is given by the following syntax:

```
typeField
: annotationList type name ;
;

headerTypeDeclaration
: annotationList HEADER name typeParameterListOpt `{ typeFieldList }
;
```

where each `type` is restricted to a bit-string type (fixed or variable), a fixed-width signed integer type, a boolean type, or a struct that itself contains other struct fields, nested arbitrarily, as long as all of the "leaf" types are `bit<W>`, `int<W>`, a serializable `enum`, or a `bool`. If a `bool` is used inside a P4 header, all implementations encode the `bool` as a one bit long field, with the value `1` representing `true` and `0` representing `false`. Field names have to be distinct.

A header declaration introduces a new identifier in the current scope; the type definition can be referred to using this identifier.

```
headerTypeDefIR
: HEADER typeId `{ typeParameterIR* > `{ fieldTypeIR* }
;
```

A header is similar to a `struct` in C, containing all the specified fields. However, in addition, a header also contains a hidden Boolean "validity" field. When the "validity" bit is `true` we say that the "header is valid". When a local variable with a header type is declared, its "validity" bit is automatically set to `false`. The "validity" bit can be manipulated by using the header methods `isValid()`, `setValid()`, and `setInvalid()`.

Header types may be empty:

```
header Empty_h { }
```

Note that an empty header still contains a validity bit.

When a struct is inside of a header, the order of the fields for the purposes of `extract` and `emit` calls is the order of the fields as defined in the source code. An example of a header including a struct is included below.

```

struct ipv6_addr {
    bit<32> Addr0;
    bit<32> Addr1;
    bit<32> Addr2;
    bit<32> Addr3;
}

header ipv6_t {
    bit<4>    version;
    bit<8>    trafficClass;
    bit<20>   flowLabel;
    bit<16>   payloadLen;
    bit<8>    nextHdr;
    bit<8>    hopLimit;
    ipv6_addr src;
    ipv6_addr dst;
}

```

Headers that do not contain any **varbit** field are *fixed size*. Headers containing **varbit** fields have *variable size*. The size (in bits) of a fixed-size header is simply the sum of the sizes of all component fields (without counting the validity bit). There is no padding or alignment of the header fields. Targets may impose additional constraints on header types—e.g., restricting headers to sizes that are an integer number of bytes.

For example, the following declaration describes a typical Ethernet header:

```

header Ethernet_h {
    bit<48> dstAddr;
    bit<48> srcAddr;
    bit<16> etherType;
}

```

The following variable declaration uses the newly introduced type **Ethernet_h**:

```

Ethernet_h ethernetHeader;

```

P4's parser language provides an **extract** method that can be used to "fill in" the fields of a header from a network packet, as described in [\[sec-packet-data-extraction\]](#). The successful execution of an **extract** operation also sets the validity bit of the extracted header to **true**.

Here is an example of an IPv4 header with variable-sized options:

```

header IPv4_h {
    bit<4>    version;
    bit<4>    ihl;
    bit<8>    diffserv;
    bit<16>   totalLen;
    bit<16>   identification;
    bit<3>    flags;
    bit<13>   fragOffset;
    bit<8>    ttl;
    bit<8>    protocol;
    bit<16>   hdrChecksum;
    bit<32>   srcAddr;
    bit<32>   dstAddr;
    varbit<320> options;
}

```

As demonstrated by a code example in [\[sec-packet-extract-two\]](#), another way to support headers that

contain variable-length fields is to define two headers—one fixed length, one containing a `varbit` field—and extract each part in separate parsing steps.

Notice that the names `isValid`, `setValid`, `minSizeInBits`, etc. are all valid header field names.

Internally, header types and values are represented as follows:

```
headerTypeIR
: HEADER typeId `< typeArgumentIR* > `{ fieldTypeIR* }
;

headerValue
: HEADER typeId `{ bool ; fieldValue* }
;
```

8.4.6.1. Operations

Headers provide the same operations as `structs`. Assignment between headers also copies the "validity" header bit.

In addition, headers support the following methods:

- The method `isValid()` returns the value of the "validity" bit of the header.
- The method `setValid()` sets the header's validity bit to "true". It can only be applied to an l-value.
- The method `setInvalid()` sets the header's validity bit to "false". It can only be applied to an l-value.

Similar to a `struct`, a header object can be initialized with a sequence expression (see [Section 8.6.3](#))—the sequence elements are assigned to the header fields in the order they appear—or with a record expression (see [Section 8.6.4](#)). When initialized the header automatically becomes valid:

```
header H { bit<32> x; bit<32> y; }
H h;
h = { 10, 12 }; // This also makes the header h valid
h = { y = 12, x = 10 }; // Same effect as above
```

Two headers can be compared for equality (`==`) or inequality (`!=`) only if they have the same type. Two headers are equal if and only if they are both invalid, or they are both valid and all their corresponding fields are equal. Furthermore, the size of a header can be determined at compile-time ([\[sec-minsizeinbits\]](#)).

See [\[sec-uninitialized-values-and-writing-invalid-headers\]](#) for a description of the behavior if header fields are read without being initialized, or header fields are written to a currently invalid header.

8.4.6.2. Type checking

Header types are introduced by `header` declarations. Its typing rule is explained in [Section 11.15](#). Header types are then used by referencing their names. Thus header types can be obtained from named types, as illustrated in [Section 8.3](#).

8.4.6.3. Well-formedness

`HEADER typeId < typeArgumentIR* > { ( annotationList typeIRfield nameIRfield ; )* }` is a well-formed type, with bound type variables `B` when:

1. Check that the elements of $\text{nameIR}_{\text{field}}^*$ are distinct.
2. Check that $\text{typeIR}_{\text{field}}$ can be used in a header type, for all $\text{typeIR}_{\text{field}}$ in $\text{typeIR}_{\text{field}}^*$.
3. Check that $\text{typeIR}_{\text{field}}$ is a well-formed type, with bound type variables B , for all $\text{typeIR}_{\text{field}}$ in $\text{typeIR}_{\text{field}}^*$.
4. The relation holds.

8.4.7. Header unions

A header union represents an alternative containing at most one of several different headers. Header unions can be used to represent "options" in protocols like TCP and IP. They also provide hints to P4 compilers that only one alternative will be present, allowing them to conserve storage resources.

A header union is defined as:

```
typeField
: annotationList type name ;
;

headerUnionTypeDeclaration
: annotationList HEADER_UNION name typeParameterListOpt `{ typeFieldList }
;
;
```

This declaration introduces a new type definition with the specified name in the current scope.

```
headerUnionTypeDefIR
: HEADER_UNION typeId `< typeParameterIR* > `{ fieldTypeIR* }
;
;
```

Each element of the list of fields used to declare a header union must be of header type. An empty list of fields is legal. Field names have to be distinct.

As an example, the type `ip_h` below represents the union of an IPv4 and IPv6 headers:

```
header_union IP_h {
  IPv4_h v4;
  IPv6_h v6;
}
```

A header union is not considered a type with fixed length.

Internally, header union types and values are represented as follows:

```
headerUnionTypeIR
: HEADER_UNION typeId `< typeArgumentIR* > `{ fieldTypeIR* }
;
;

headerUnionValue
: HEADER_UNION typeId `{ fieldValue* }
;
;
```

8.4.7.1. Operations

A variable declared with a union type is initially invalid. For example:

```

header H1 {
    bit<8> f;
}
header H2 {
    bit<16> g;
}
header_union U {
    H1 h1;
    H2 h2;
}

U u; // u invalid

```

This also implies that each of the headers `h1` through `hn` contained in a header union are also initially invalid. Unlike headers, a union cannot be initialized. However, the validity of a header union can be updated by assigning a valid header to one of its elements:

```

U u;
H1 my_h1 = { 8w0 }; // my_h1 is valid
u.h1 = my_h1;       // u and u.h1 are both valid

```

We can also assign a sequence to an element of a header union,

```

U u;
u.h2 = { 16w1 };    // u and u.h2 are both valid

```

or set their validity bits directly.

```

U u;
u.h1.setValid();    // u and u.h1 are both valid
H1 my_h1 = u.h1;    // my_h1 is now valid, but contains an undefined value

```

Note that reading an uninitialized header produces an undefined value, even if the header is itself valid.

If `u` is an expression whose type is a header union `U` with fields ranged over by `hi`, then the expression `u.hi` evaluates to a header, and thus it can be used wherever a header expression is allowed. If `u` is a left-value, then `u.hi` is also a left-value.

The following operations:

- `u.hi.setValid()`: sets the valid bit for header `hi` to `true` and sets the valid bit for all other headers to `false`, which implies that it is unspecified what value reading any member header of `u` will return.
- `u.hi.setInvalid()`: if the valid bit for any member header of `u` is `true` then sets it to `false`, which implies that it is unspecified what value reading any member header of `u` will return.

The assignment to a union field:

```
u.hi = e;
```

has the following meaning:

- if `e` is valid, then it is equivalent to:

```
u.hi.setValid();
```



```
u.hi = e;
```

- if `e` is invalid, then it is equivalent to:

```
u.hi.setInvalid();
```

Assignments between variables of the same type of header union are permitted. The assignment `u1 = u2` copies the full state of header union `u2` to `u1`. If `u2` is valid, then there is some header `u2.hi` that is valid. The assignment behaves the same as `u1.hi = u2.hi`. If `u2` is not valid, then `u1` becomes invalid (i.e. if any header of `u1` was valid, it becomes invalid).

`u.isValid()` returns true if any member of the header union `u` is valid, otherwise it returns false. `setValid()` and `setInvalid()` methods are not defined for header unions.

Supplying an expression with a union type to `emit` simply emits the single header that is valid, if any.

The following example shows how we can use header unions to represent IPv4 and IPv6 headers uniformly:

```
header_union IP {
    IPv4 ipv4;
    IPv6 ipv6;
}

struct Parsed_packet {
    Ethernet ethernet;
    IP ip;
}

parser top(packet_in b, out Parsed_packet p) {
    state start {
        b.extract(p.ethernet);
        transition select(p.ethernet.etherType) {
            16w0x0800 : parse_ipv4;
            16w0x86DD : parse_ipv6;
        }
    }
    state parse_ipv4 {
        b.extract(p.ip.ipv4);
        transition accept;
    }
    state parse_ipv6 {
        b.extract(p.ip.ipv6);
        transition accept;
    }
}
```

As another example, we can also use unions to parse (selected) TCP options:

```
header Tcp_option_end_h {
    bit<8> kind;
}
header Tcp_option_nop_h {
    bit<8> kind;
}
header Tcp_option_ss_h {
    bit<8> kind;
    bit<32> maxSegmentSize;
}
header Tcp_option_s_h {
    bit<8> kind;
```

```

    bit<24> scale;
}
header Tcp_option_sack_h {
    bit<8>      kind;
    bit<8>      length;
    varbit<256> sack;
}
header_union Tcp_option_h {
    Tcp_option_end_h  end;
    Tcp_option_nop_h  nop;
    Tcp_option_ss_h   ss;
    Tcp_option_s_h    s;
    Tcp_option_sack_h sack;
}

typedef Tcp_option_h[10] Tcp_option_stack;

struct Tcp_option_sack_top {
    bit<8> kind;
    bit<8> length;
}

parser Tcp_option_parser(packet_in b, out Tcp_option_stack vec) {
    state start {
        transition select(b.lookahead<bit<8>>()) {
            8w0x0 : parse_tcp_option_end;
            8w0x1 : parse_tcp_option_nop;
            8w0x2 : parse_tcp_option_ss;
            8w0x3 : parse_tcp_option_s;
            8w0x5 : parse_tcp_option_sack;
        }
    }
    state parse_tcp_option_end {
        b.extract(vec.next.end);
        transition accept;
    }
    state parse_tcp_option_nop {
        b.extract(vec.next.nop);
        transition start;
    }
    state parse_tcp_option_ss {
        b.extract(vec.next.ss);
        transition start;
    }
    state parse_tcp_option_s {
        b.extract(vec.next.s);
        transition start;
    }
    state parse_tcp_option_sack {
        bit<8> n = b.lookahead<Tcp_option_sack_top>().length;
        // n is the total length of the TCP SACK option in bytes.
        // The length of the varbit field 'sack' of the
        // Tcp_option_sack_h header is thus n-2 bytes.
        b.extract(vec.next.sack, (bit<32>) (8 * n - 16));
        transition start;
    }
}
}

```

Similar to headers, the size of a header union is a local compile-time known value (Section [\[seminsizeinbits\]](#)).

Two header unions can be compared for equality (==) or inequality (!=) if they have the same type. The unions are equal if and only if all their corresponding fields are equal (i.e., either all fields are invalid in both unions, or in both unions the same field is valid, and the values of the valid fields are equal as headers).

8.4.7.2. Type checking

Header union types are introduced by `header_union` declarations. Its typing rule is explained in [Section 11.16](#). Header union types are then used by referencing their names. Thus header union types can be obtained from named types, as illustrated in [Section 8.3](#).

8.4.7.3. Well-formedness

`HEADER_UNION typeId < typeArgumentIR* > { ( annotationList typeIRfield nameIRfield ; )* }` is a well-formed type, with bound type variables `B` when:

1. Check that the elements of `nameIRfield*` are distinct.
2. Check that `typeIRfield` can be used in a header union type, for all `typeIRfield` in `typeIRfield*`.
3. Check that `typeIRfield` is a well-formed type, with bound type variables `B`, for all `typeIRfield` in `typeIRfield*`.
4. The relation holds.

8.4.8. Enums

An enumeration type is defined using the following syntax:

```
namedExpression
  : name = expression
  ;

namedExpressionList
  : namedExpression
  | namedExpressionList , namedExpression
  ;

enumTypeDeclaration
  : annotationList ENUM name `{ nameList trailingCommaOpt }
  | annotationList ENUM type name `{ namedExpressionList trailingCommaOpt }
  ;
```

An `enum` declaration introduces a new enum type definition:

```
enumTypeDefIR = enumTypeIR
```

Enums cannot be parameterized over type parameters, thus is always monomorphic.

Enums can be declared in two different ways, depending on whether or not an underlying representation type is provided. The two forms are described below.

Internally, enum types and values are represented as follows:

```
valueFieldIR
  : nameIR = value ;
  ;

enumTypeIR
  : ENUM typeId `{ nameIR* }
  | ENUM typeId `< typeIR > `{ valueFieldIR* }
  ;

enumValue
```

```
: typeId . nameIR
| typeId . nameIR . value
;
```

8.4.8.1. Enum type declaration without an underlying type

For example, the declaration

```
enum Suits { Clubs, Diamonds, Hearths, Spades }
```

introduces a new enumeration type, which contains four elements—e.g., `Suits.Clubs`. An `enum` declaration introduces a new identifier in the current scope for naming the created type along with its distinct elements. The underlying representation of the `Suits` enum is not specified, so their "size" in bits is not specified (it is target-specific).

8.4.8.2. Enum type declaration with an underlying type

It is also possible to specify an `enum` with an underlying representation. These are sometimes called serializable `enums`, because headers are allowed to have fields with such `enum` types. This requires the programmer provide both the fixed-width unsigned (or signed) integer type and an associated integer value for each symbolic entry in the enumeration.

For example, the declaration

```
enum bit<16> EtherType {
    VLAN      = 0x8100,
    QINQ      = 0x9100,
    MPLS      = 0x8847,
    IPV4      = 0x0800,
    IPV6      = 0x86dd
}
```

introduces a new enumeration type, which contains five elements—e.g., `EtherType.IPV4`. This `enum` declaration specifies the fixed-width unsigned integer representation for each entry in the `enum` and provides an underlying type: `bit<16>`. This kind of `enum` declaration can be thought of as declaring a new `bit<16>` type, where variables or fields of this type are expected to be unsigned 16-bit integer values, and the mapping of symbolic to numeric values defined by the `enum` are also defined as a part of this declaration. In this way, an `enum` with an underlying type can be thought of as being a type derived from the underlying type carrying equality, assignment, and casts to/from the underlying type.

Compiler implementations are expected to raise an error if the fixed-width integer representation for an enumeration entry falls outside the representation range of the underlying type.

For example, the declaration

```
enum bit<8> FailingExample {
    first      = 1,
    second     = 2,
    third      = 3,
    unrepresentable = 300
}
```

would raise an error because `300`, the value associated with `FailingExample.unrepresentable` cannot be represented as a `bit<8>` value.

The initializer `expression` must be a local compile-time known value.

Annotations, represented by the non-terminal `annotationList`, are described in [\[sec-annotations\]](#).

8.4.8.3. Operations

Symbolic names declared by an `enum` belong to the namespace introduced by the `enum` declaration rather than the top-level namespace.

```
enum X { v1, v2, v3 }
X.v1  // reference to v1
v1    // error - v1 is not in the top-level namespace
```

Similar to errors, `enum` expressions without a specified underlying type only support equality (`==`) and inequality (`!=`) comparisons. Expressions whose type is an `enum` without a specified underlying type cannot be cast to or from any other type.

An `enum` may also specify an underlying type, such as the following:

```
enum bit<8> E {
  e1 = 0,
  e2 = 1,
  e3 = 2
}
```

More than one symbolic value in an `enum` may map to the same fixed-width integer value.

```
enum bit<8> NonUnique {
  b1 = 0,
  b2 = 1,  // Note, both b2 and b3 map to the same value.
  b3 = 1,
  b4 = 2
}
```

An `enum` with an underlying type also supports explicit casts to and from the underlying type. For instance, the following code:

```
bit<8> x;
E a = E.e2;
E b;

x = (bit<8>) a; // sets x to 1
b = (E) x;      // sets b to E.e2
```

casts `a` (which was initialized with `E.e2`) to a `bit<8>`, using the specified fixed-width unsigned integer representation for `E.e2`, i.e. `1`. The variable `b` is then set to the symbolic value `E.e2`, which corresponds to the fixed-width unsigned integer value `1`.

Because it is always safe to cast from an `enum` to its underlying fixed-width integer type, implicit casting from an `enum` to its fixed-width (signed or unsigned) integer type is also supported (see [Section 15.2](#)):

```
bit<8> x = E.e2; // sets x to 1 (E.e2 is automatically casted to bit<8>)

E a = E.e2
bit<8> y = a << 3; // sets y to 8 (a is automatically casted to bit<8> and then shifted)
```

Implicit casting from an underlying fixed-width type to an `enum` is **not** supported.

```

enum bit<8> E1 {
    e1 = 0, e2 = 1, e3 = 2
}

enum bit<8> E2 {
    e1 = 10, e2 = 11, e3 = 12
}
E1 a = E1.e1;
E2 b = E2.e2;

a = b;          // Error: b is automatically casted to bit<8>,
                // but bit<8> cannot be automatically casted to E1

a = (E1) b; // OK

a = E1.e1 + 1; // Error: E.e1 is automatically casted to bit<8>,
                // and the right-hand expression has
                // the type bit<8>, which cannot be casted to E automatically.

a = (E1)(E1.e1 + 1); // Final explicit casting makes the assignment legal

a = E1.e1 + E1.e2; // Error: both arguments to the addition are automatically
                // casted to bit<8>. Thus the addition itself is legal, but
                // the assignment is not

a = (E1)(E2.e1 + E2.e2); // Final explicit casting makes the assignment legal

```

A reasonable compiler might generate a warning in cases that involve multiple automatic casts.

```

E1    a = E1.e1;
E2    b = E2.e2;
bit<8> c;

if (a > b) { // Potential warning: two automatic and different casts to bit<8>.
    // code omitted
}

c = a + b; // Legal, but a warning would be reasonable

```

Note that while it is always safe to cast from an `enum` to its fixed-width unsigned integer type, and vice versa, there may be cases where casting a fixed-width unsigned integer value to its related `enum` type produces an unnamed value.

```

bit<8> x = 5;
E e = (E) x; // sets e to an unnamed value

```

sets `e` to an unnamed value, since there is no symbol corresponding to the fixed-width unsigned integer value 5.

For example, in the following code, the `else` clause of the `if/else if/else` block can be reached even though the matches on `x` are complete with respect to the symbols defined in `MyPartialEnum_t`:

```

enum bit<2> MyPartialEnum_t {
    VALUE_A = 2w0,
    VALUE_B = 2w1,
    VALUE_C = 2w2
}

bit<2> y = < some value >;
MyPartialEnum_t x = (MyPartialEnum_t)y;

```

```

if (x == MyPartialEnum_t.VALUE_A) {
    // some code here
} else if (x == MyPartialEnum_t.VALUE_B) {
    // some code here
} else if (x == MyPartialEnum_t.VALUE_C) {
    // some code here
} else {
    // A P4 compiler MUST ASSUME that this branch can be executed
    // some code here
}

```

Additionally, if an enumeration is used as a field of a header, we would expect the `transition select` to match `default` when the parsed integer value does not match one of the symbolic values of `EtherType` in the following example:

```

enum bit<16> EtherType {
    VLAN      = 0x8100,
    IPV4      = 0x0800,
    IPV6      = 0x86dd
}

header ethernet {
    // Some fields omitted
    EtherType etherType;
}

parser my_parser(/* parameters omitted */) {
    state parse_ethernet {
        packet.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType) {
            EtherType.VLAN : parse_vlan;
            EtherType.IPV4 : parse_ipv4;
            EtherType.IPV6 : parse_ipv6;
            default: reject;
        }
    }
}

```

Any variable with an `enum` type that contains an unnamed value (whether as the result of a cast to an `enum` with an underlying type, parse into the field of an `enum` with an underlying type, or simply the declaration of any `enum` without a specified initial value) will not be equal to any of the values defined for that type. Such an unnamed value should still lead to predictable behavior in cases where any legal value would match, e.g. it should match in any of these situations:

- If used in a `select` expression, it should match `default` or `_` in a key set expression.
- If used as a key with `match_kind ternary` in a table, it should match a table entry where the field has all bit positions "don't care".
- If used as a key with `match_kind lpm` in a table, it should match a table entry where the field has a prefix length of 0.

Note that if an `enum` value lacking an underlying type appears in the control-plane API, the compiler must select a suitable serialization data type and representation. For `enum` values with an underlying type and representations, the compiler should use the specified underlying type as the serialization data type and representation.

Additionally, the size of a serializable enum can be determined at compile-time. However, the size of an enum without an underlying type cannot be determined at compile-time ([\[sec-minsizeinbits\]](#)).

8.4.8.4. Type checking

Enum types are introduced by `enum` declarations. Its typing rule is explained in [Section 11.13](#). Enum types

are then used by referencing their names. Thus enum types can be obtained from named types, as illustrated in [Section 8.3](#).

8.4.8.5. Well-formedness

`enumTypeIR` is a well-formed type, with bound type variables `B` when:

1. If let `ENUM _ { nameIRfield }*` be `enumTypeIR`:
 - a. Check that the elements of `nameIRfield*` are distinct.
 - b. The relation holds.
2. Else:
 - a. Let `ENUM _ < typeIR' > { ( nameIRfield = _ ; )* }` be `enumTypeIR`.
 - b. Check that the elements of `nameIRfield*` are distinct.
 - c. Check that `typeIR'` can be used in a serializable enum type.
 - d. Check that `typeIR'` is a well-formed type, with bound type variables `B`.
 - e. The relation holds.

8.5. Object types and values

```
objectTypeIR
: externObjectTypeIR
| parserObjectTypeIR
| controlObjectTypeIR
| packageObjectTypeIR
| tableObjectTypeIR
;

objectReferenceValue
: REF objectId
;
```

Stateful objects are represented using `objectTypeIR`. At runtime, these objects are passed along directionless parameters as references (`objectReferenceValue`).

The types `parser`, `control`, and `package` cannot be used as types of arguments for actions, functions, methods, parsers, controls, or tables. They *can* be used as types for the arguments passed to constructors.

8.5.1. Extern object types

An extern object declaration declares an object and all methods that can be invoked to perform computations and to alter the state of the object. Extern declarations may only appear as allowed by the architecture model and may be specific to a target.

```
externConstructorPrototype
: annotationList typeIdentifier `( parameterList ) ;
;

externMethodPrototype
: annotationList functionPrototype ;
| annotationList ABSTRACT functionPrototype ;
;
```



```

externConstructorOrMethodPrototype
  : externConstructorPrototype
  | externMethodPrototype
  ;

externObjectDeclaration
  : annotationList EXTERN nonTypeName typeParameterListOpt `{
externConstructorOrMethodPrototypeList }
  ;

```

For example, the P4 core library introduces two extern objects `packet_in` and `packet_out` used for manipulating packets (see [\[sec-packet-data-extraction\]](#) and [\[sec-deparse\]](#)). Here is an example showing how the methods of these objects can be invoked on a packet:

```

extern packet_out {
  void emit<T>(in T hdr);
}
control d(packet_out b, in Hdr h) {
  apply {
    b.emit(h.ipv4);          // write ipv4 header into output packet
  }                          // by calling emit method
}

```

An extern object declaration introduces a new extern object type definition.

```

externObjectTypeDefIR
  : EXTERN typeId `< typeParameterIR* , typeParameterIR* > externMethodTypeDefEnv
  ;

```

Extern object types can be referred to using their name. Internally, extern object types are represented as:

```

externObjectTypeIR
  : EXTERN typeId `< typeArgumentIR* > externMethodTypeDefEnv
  ;

```

8.5.1.1. Type checking

Extern object types are introduced by `extern` declarations. Its typing rule is explained in [\[sem-extern-object-declaration\]](#). Extern object types are then used by referencing their names. Thus extern object types can be obtained from named types, as illustrated in [Section 8.3](#).

8.5.1.2. Well-formedness

`EXTERN typeId < typeArgumentIR* > { ( callableId : externMethodTypeDefIR )* }` is a well-formed type, with bound type variables `B` when:

1. Check that `externMethodTypeDefIR` is a well-formed callable type definition, with bound type variables `B`, for all `externMethodTypeDefIR` in `externMethodTypeDefIR*`.
2. The relation holds.

8.5.2. Parser object types

A parser type declaration describes the signature of a parser. A parser should have at least one argument of type `packet_in`, representing the received packet that is processed.

```

parserTypeDeclaration
  : annotationList PARSE name typeParameterListOpt `( parameterList ) ;
  ;

```

For example, the following is a type declaration of a parser type named **P** that is parameterized on a type variable **H**. That parser receives as input a `packet_in` value **b** and produces two values:

- A value with a user-defined type **H**
- A value with a predefined type `Counters`

```

struct Counters { /* Fields omitted */ }
parser P<H>(packet_in b,
            out H packetHeaders,
            out Counters counters);

```

A parser type declaration introduces a new parser object type definition.

```

parserObjectTypeDefIR
  : PARSE typeId `< typeParameterIR* , typeParameterIR* > `( parameterIR* )
  ;

```

Parser types can be referred to using their name and type arguments. Internally, parser types are represented as:

```

parserObjectTypeIR
  : PARSE typeId `< typeArgumentIR* > `( parameterIR* )
  ;

```

8.5.2.1. Type checking

Parser object types are introduced by parser type declarations. Its typing rule is explained in [Section 11.18](#). Parser object types are then used by referencing their names. Thus parser object types can be obtained from named types, as illustrated in [Section 8.3](#).

8.5.2.2. Well-formedness

`PARSE typeId < typeArgumentIR* > ( parameterIR* )` is a well-formed type, with bound type variables **B** when:

1. Check that `parameterIR` is a well-formed parameter type, with bound type variables **B**, for all `parameterIR` in `parameterIR*`.
2. The relation holds.

8.5.3. Control object types

A control type declaration describes the signature of a control block.

```

controlTypeDeclaration
  : annotationList CONTROL name typeParameterListOpt `( parameterList ) ;
  ;

```

Control type declarations are similar to parser type declarations.

A control type declaration introduces a new control object type definition.

```
controlObjectTypeDefIR
: CONTROL typeId `< typeParameterIR* , typeParameterIR* > `( parameterIR* )
;
```

Control types can be referred to using their name and type arguments. Internally, control types are represented as:

```
controlObjectTypeIR
: CONTROL typeId `< typeArgumentIR* > `( parameterIR* )
;
```

8.5.3.1. Type checking

Control object types are introduced by control type declarations. Its typing rule is explained in [Section 11.19](#). Control object types are then used by referencing their names. Thus control object types can be obtained from named types, as illustrated in [Section 8.3](#).

8.5.3.2. Well-formedness

CONTROL typeId < typeArgumentIR* > (parameterIR*) is a well-formed type, with bound type variables **B** when:

1. Check that **parameterIR** is a well-formed parameter type, with bound type variables **B**, for all **parameterIR** in **parameterIR***.
2. The relation holds.

8.5.4. Package object types

A package type describes the signature of a package.

```
packageTypeDeclaration
: annotationList PACKAGE name typeParameterListOpt `( parameterList ) ;
;
```

This introduces a new package object type definition.

```
packageObjectTypeDefIR
: PACKAGE typeId `< typeParameterIR* , typeParameterIR* > `{ typeIR* }
;
```

Package types can be referred to using their name and type arguments. Internally, package types are represented as:

```
packageObjectTypeIR
: PACKAGE typeId `< typeArgumentIR* > `{ typeIR* }
;
```

8.5.4.1. Type checking

Package object types are introduced by `package` declarations. Its typing rule is explained in [Section 11.20](#). Package object types are then used by referencing their names. Thus package object types can be obtained from named types, as illustrated in [Section 8.3](#).

8.5.4.2. Well-formedness

`PACKAGE typeId < typeArgumentIR* > { typeIR* }` is a well-formed type, with bound type variables `B` when:

1. Check that `typeIR'` is a well-formed type, with bound type variables `B`, for all `typeIR'` in `typeIR*`.
2. The relation holds.

8.5.5. Table object types

A `table` declaration introduces a table instance and a table type definition.

```
tableDeclaration
  : annotationList TABLE name `{ tablePropertyList }
  ;

tableObjectTypeDefIR = tableObjectTypeIR
```

Internally, table types are represented as:

```
tableObjectTypeIR
  : TABLE typeId `{ tableMetadataStructTypeIR }
  ;
```

Table metadata types are introduced in [\[sec-table-metadata-type-and-value\]](#).

8.5.5.1. Well-formedness

`TABLE typeId { tableMetadataStructTypeIR }` is a well-formed type, with bound type variables `B` when:

1. Check that `tableMetadataStructTypeIR` is a well-formed type, with bound type variables `B`.
2. The relation holds.

8.5.6. Object reference values

P4 objects are stateful, thus their values are references to the actual objects allocated in the global store.

```
objectId = nameIR*

objectReferenceValue
  : REF objectId
  ;
```

Objects are referred to by their fully-qualified names from the top-level scope. === Alias types

A **typedef** or **type** declaration can be used to give an alias to a type.

```
typedef
  : type
  | derivedTypeDeclaration
  ;

typedefDeclaration
  : annotationList TYPEDEF typedef name ;
  | annotationList TYPE type name ;
  ;
```

Internally, the alias is represented as:

```
aliasTypeIR
  : typedefTypeIR
  | newTypeIR
  ;
```

8.5.7. **typedef** declaration

```
typedef bit<32> u32;
typedef struct Point { int<32> x; int<32> y; } Pt;
typedef Empty_h[32] HeaderStack;
```

The two types are treated as synonyms, and all operations that can be executed using the original type can be also executed using the newly created type.

If **typedef** is used with a generic type the type must be specialized with the suitable number of type arguments:

```
struct S<T> {
  T field;
}

// typedef S X; -- illegal: S does not have type arguments
typedef S<bit<32>> X; // -- legal
```

Internally, the alias is represented as:

```
typedefTypeIR
  : TYPEDEF typeId typeIR
  ;
```

8.5.7.1. Type checking

typedefs are introduced by **typedef** declarations. Its typing rule is explained in [Section 11.17.1](#). **typedefs** are then used by referencing their names. Thus **typedef** types can be obtained from named types, as illustrated in [Section 8.3](#).

8.5.7.2. Well-formedness

TYPEDEF typeId typeIR' is a well-formed type, with bound type variables B when:

1. Check that `typeIR` can be used in a `typedef`.
2. Check that `typeIR` is a well-formed type, with bound type variables `B`.
3. The relation holds.

8.5.8. `type` declaration

Similarly to `typedef`, the keyword `type` can be used to introduce a new type.

```
type bit<32> U32;  
U32 x = (U32)0;
```

While similar to `typedef`, the `type` keyword introduces a new type which is not a synonym with the original type: values of the original type and the newly introduced type cannot be mixed in expressions.

Currently the types that can be created by the `type` keyword are restricted to one of: `bit<>`, `int<>`, `bool`, or types defined using `type` from such types.

One important use of such types is in describing P4 values that need to be exchanged with the control plane through communication channels (e.g., through the control-plane API or through network packets sent to the control plane). For example, a P4 architecture may define a type for the switch ports:

```
type bit<9> PortId_t;
```

This declaration will prevent `PortId_t` values from being used in arithmetic expressions without casts. Moreover, this declaration may enable special manipulation of such values by software that lies outside of the datapath (e.g., a target-specific toolchain could include software that automatically converts values of type `PortId_t` to a different representation when exchanged with the control-plane software).

Internally, the alias is represented as:

```
newTypeIR  
  : TYPE typeId typeIR  
  ;
```

8.5.8.1. Operations

Values with a type introduced by the `type` keyword provide only a few operations:

- assignment to left-values of the same type
- comparisons for equality and inequality if the original type supported such comparisons
- casts to and from the original type

```
type bit<32> U32;  
U32 x = (U32)0; // cast needed  
U32 y = (U32)((bit<32>)x + 1); // casts needed for arithmetic  
bit<32> z = 1;  
bool b0 = x == (U32)z; // cast needed  
bool b1 = (bit<32>)x == z; // cast needed  
bool b2 = x == y; // no cast needed
```

8.5.8.2. Type checking

types are introduced by **type** declarations. Its typing rule is explained in [Section 11.17.2](#). types are then used by referencing their names. Thus **type** types can be obtained from named types, as illustrated in [Section 8.3](#).

8.5.8.3. Well-formedness

TYPE **typeId** **typeIR'** is a well-formed type, with bound type variables **B** when:

1. Check that **typeIR'** can be used in a new type.
2. Check that **typeIR'** is a well-formed type, with bound type variables **B**.
3. The relation holds.

8.6. Synthesized types and values

For the purposes of type-checking the P4 compiler can synthesize some type representations which cannot be directly expressed by users.

These types and values are represented as:

```
synthesizedTypeIR
: defaultTypeIR
| invalidHeaderTypeIR
| sequenceTypeIR
| recordTypeIR
| setTypeIR
| tableMetadataTypeIR
;

synthesizedValue
: defaultValue
| invalidHeaderValue
| sequenceValue
| recordValue
| setValue
| tableMetadataValue
;
```

8.6.1. Default

A left-value can be initialized automatically with a default value of the suitable type using:

```
defaultExpression
: ...
;
```

Internally, the type and value of **...** itself are represented as:

```
defaultTypeIR
: DEFAULT
;

defaultValue
: DEFAULT
;
```

8.6.1.1. Operations

For example:

```
struct S {
    bit<32> b32;
    bool b;
}

enum int<8> NO {
    one = 1,
    zero = 0,
    two = 2
}

enum N1 {
    A, B, C, F
}

S s0 = ...; // initialize s0 with the default value { 0, false }
NO n0 = ...; // initialize n0 with the default value 0
N1 n1 = ...; // initialize n1 with the default value N1.A
```

See [Section 14.4](#) for semantics of default initialization.

NOTE

The default type and value is new. Because every expressions must have a type and value, we need to represent those of

8.6.1.2. Well-formedness

DEFAULT is a well-formed type, with bound type variables **B** when:

1. The relation holds.

8.6.2. Invalid headers

The expression `{#}` represents an invalid header of some type, but it can be any header or header union type. A P4 compiler may require an explicit cast on this expression in cases where it cannot determine the particular header or header union type from the context.

```
invalidHeaderExpression
: {#}
;
```

Note that the `#` character cannot be misinterpreted as a preprocessor directive, since it cannot be the first character on a line when it occurs in the single lexical token `{#}`, which may not have whitespace or any other characters between those shown.

Internally, the type and value of `{#}` itself are represented as:

```
invalidHeaderTypeIR
: HEADER_INVALID
;

invalidHeaderValue
: {#}
```



```
;
```

8.6.2.1. Operations

For example:

```
header H { bit<32> x; bit<32> y; }
H h;
h = {#}; // This makes the header h become invalid
```

NOTE

The invalid header type and value is new. Because every expressions must have a type and value, we need to represent those of `{#}`.

8.6.2.2. Well-formedness

`HEADER_INVALID` is a well-formed type, with bound type variables `B` when:

1. The relation holds.

8.6.3. Sequences

A sequence expression is written using curly braces, with each element separated by a comma:

```
sequenceElementExpression = expressionList

sequenceOrRecordElementExpression
: sequenceElementExpression
| recordElementExpression
;

sequenceOrRecordExpression
: `{ sequenceOrRecordElementExpression trailingCommaOpt }
;
```

The type of a sequence expression is a sequence type. Its value is a sequence value.

```
sequenceTypeIR
: SEQ `< typeIR* >
| SEQ `< typeIR* , ... >
;

sequenceValue
: SEQ `( value* )
| SEQ `( value* , ... )
;
```

8.6.3.1. Operations

Sequence expressions can be assigned to expressions of type `tuple`, `struct` or `header`, and can also be passed as arguments to methods. Sequences may be nested.

For instance, sequences can be assigned to other tuples with the same type:

```
tuple<bit<32>, bool> x = { 10, false };
```

The following program fragment uses a sequence expression to pass several header fields simultaneously to a learning provider:

```
extern LearningProvider<T> {  
    LearningProvider();  
    void learn(in T data);  
}  
  
LearningProvider<tuple<bit<48>, bit<32>>>() lp;  
  
lp.learn( { hdr.ethernet.srcAddr, hdr.ipv4.src } );
```

A sequence may be used to initialize a structure if the tuple has the same number of elements as fields in the structure. The effect of such an initializer is to assign the n^{th} element of the tuple to the n^{th} field in the structure:

```
struct S {  
    bit<32> a;  
    bit<32> b;  
}  
const S x = { 10, 20 }; // a = 10, b = 20
```

Sequence expressions that have ... as their last element are allowed to give values to only a subset of the fields of the struct or header type to which it evaluates. Any field names not given a value explicitly will be given their default value (see [\[sec-initializing-with-default-values\]](#)).

NOTE

The sequence type is newly introduced. In the current P4 language specification, sequence expressions are referred to as tuple expressions, and (somewhat confusingly and implicitly) has tuple type. However, this is problematic since tuples have limited casting rules that cannot be applied to several initialization scenarios. Thus, we introduce sequence types to represent the expression sequence within curly braces, apart from tuple types.

8.6.3.2. Well-formedness

`sequenceTypeIR` is a well-formed type, with bound type variables `B` when:

1. If let `SEQ < typeIR* >` be `sequenceTypeIR`:
 - a. Check that `typeIR'` is a well-formed type, with bound type variables `B`, for all `typeIR'` in `typeIR*`.
 - b. The relation holds.
2. Else:
 - a. Let `SEQ < typeIR* , ... >` be `sequenceTypeIR`.
 - b. Check that `typeIR'` is a well-formed type, with bound type variables `B`, for all `typeIR'` in `typeIR*`.
 - c. The relation holds.

8.6.4. Records

One can write record expressions that evaluate to a structure or header. The syntax of these expressions is

given by:

```
recordElementExpression
: name = expression
| name = expression , ...
| name = expression , namedExpressionList
| name = expression , namedExpressionList , ...
;

sequenceOrRecordElementExpression
: sequenceElementExpression
| recordElementExpression
;

sequenceOrRecordExpression
: `{ sequenceOrRecordElementExpression trailingCommaOpt }
;
```

Internally, the type and value of a record expression are represented as:

```
recordTypeIR
: RECORD `{ fieldTypeIR* }
| RECORD `{ fieldTypeIR* , ... }
;

recordValue
: RECORD `{ fieldValue* }
| RECORD `{ fieldValue* , ... }
;
```

8.6.4.1. Operations

The following example shows a structure-valued expression used in an equality comparison expression:

```
struct S {
    bit<32> a;
    bit<32> b;
}

void f() {
    S s;

    // Compare s with a record expression
    bool b = s == (S) { a = 1, b = 2 };
}
```

Record expressions can be used in the right-hand side of assignments, in comparisons, in field selection expressions, and as arguments to functions, method or actions. Record expressions are not left values.

Record expressions that do not have ... as their last element must provide a value for every member of the struct or header type to which it evaluates, by mentioning each field name exactly once.

Record expressions that have ... as their last element are allowed to give values to only a subset of the fields of the struct or header type to which it evaluates. Any field names not given a value explicitly will be given their default values (see [\[sec-initializing-with-default-values\]](#)).

The order of the fields of the `struct` or `header` type does not need to match the order of the values of the structure-valued expression.

It is a compile-time error if a field name appears more than once in the same structure-valued

expression.

NOTE

The record type and value is new. Because every expression must have a type and value, we need to represent those of `{ recordElementExpression optTrailingComma }`.

8.6.4.2. Well-formedness

`recordTypeIR` is a well-formed type, with bound type variables `B` when:

1. If let `RECORD { ( _ typeIRfield nameIRfield ; )* }` be `recordTypeIR`:
 - a. Check that the elements of `nameIRfield*` are distinct.
 - b. Check that `typeIRfield` is a well-formed type, with bound type variables `B`, for all `typeIRfield` in `typeIRfield*`.
 - c. The relation holds.
2. Else:
 - a. Let `RECORD { ( _ typeIRfield nameIRfield ; )* , ... }` be `recordTypeIR`.
 - b. Check that the elements of `nameIRfield*` are distinct.
 - c. Check that `typeIRfield` is a well-formed type, with bound type variables `B`, for all `typeIRfield` in `typeIRfield*`.
 - d. The relation holds.

8.6.5. Sets

```
setTypeIR
: SET `< typeIR >
;
```

The set type describes *sets* of values of some type `T`. Set types can only appear in restricted contexts in P4 programs. For example, the range expression `8w5 .. 8w8` describes a set containing the 8-bit numbers 5, 6, 7, and 8, so its type is `set<bit<8>>`. This expression can be used as a label in a `select` expression (see [\[sec-select\]](#)), matching any value in this range. Set types cannot be named or declared by P4 programmers, they are only synthesized by the compiler internally and used for type-checking.

Set values are represented as:

```
setValue
: SET `{ value }
| SET `{ value* `( nat ) }
| SET `{ value &&& value }
| SET `{ value .. value }
| SET `{ ... }
;
```

8.6.5.1. Operations

Some P4 expressions denote sets of values. These expressions can appear only in a few contexts—parsers and table entries. For example, the `select` expression ([\[sec-select\]](#)) has the following structure:

```
select (expression) {
```

```

set1: state1;
set2: state2;
// More labels omitted
}

```

Here the expressions `set1`, `set2`, etc. evaluate to sets of values and the `select` expression tests whether `expression` belongs to the sets used as labels.

```

keysetExpression
: simpleKeysetExpression
| tupleKeysetExpression
;

tupleKeysetExpression
: `( expression &&& expression )
| `( expression .. expression )
| `( DEFAULT )
| `( _ )
| `( simpleKeysetExpression , simpleKeysetExpressionList )
;

simpleKeysetExpressionList
: simpleKeysetExpression
| simpleKeysetExpressionList , simpleKeysetExpression
;

simpleKeysetExpression
: expression
| expression &&& expression
| expression .. expression
| DEFAULT
| _
;

```

The mask (`&&&`) and range (`..`) operators have the same precedence; the just above the `?:` operator.

8.6.5.1.1. Singleton sets

In a set context, expressions denote singleton sets. For example, in the following program fragment,

```

select (hdr.ipv4.version) {
  4: continue;
}

```

The label `4` denotes the singleton set containing the `int` value `4`.

8.6.5.1.2. The universal set

In a set context, the expressions `default` or `_` denote the universal set, which contains all possible values of a given type:

```

select (hdr.ipv4.version) {
  4: continue;
  _: reject;
}

```

8.6.5.1.3. Masks

The infix operator `&&&` takes two arguments of the same numeric type ([\[sec-numeric-values\]](#)), and

creates a value of the same type. The right value is used as a "mask", where each bit set to 0 in the mask indicates a "don't care" bit. More formally, the set denoted by `a &&& b` is defined as follows:

```
a &&& b = { c where a & b = c & b }
```

For example:

```
8w0x0A &&& 8w0x0F
```

denotes a set that contains 16 different `bit<8>` values, whose bit-pattern is `XXXX1010`, where the value of an `X` can be any bit. Note that there may be multiple ways to express a keyset using a mask operator—e.g., `8w0xFA &&& 8w0x0F` denotes the same keyset as in the example above.

Similar to other binary operations, the mask operator allows the compiler to automatically insert casts to unify the argument types in certain situations ([Section 15.2](#)).

P4 architectures may impose additional restrictions on the expressions on the left and right-hand side of a mask operator: for example, they may require that either or both sub-expressions be compile-time known values.

8.6.5.1.4. Ranges

The infix operator `..` takes two arguments of the same numeric type `T` ([\[sec-numeric-values\]](#)), and creates a value of the type `set<T>`. The set contains all values numerically between the first and the second, inclusively. For example:

```
4s5 .. 4s8
```

denotes a set of 4 consecutive `int<4>` values `4s5`, `4s6`, `4s7`, and `4s8`.

Similar to other binary operations, the range operator allows the compiler to automatically insert casts to unify the argument types in certain situations ([\[sec-implicit-casts\]](#)).

A range where the second value is smaller than the first one represents an empty set.

8.6.5.1.5. Products

Multiple sets can be combined using Cartesian product:

```
select(hdr.ipv4.ihl, hdr.ipv4.protocol) {  
  (4w0x5, 8w0x1): parse_icmp;  
  (4w0x5, 8w0x6): parse_tcp;  
  (4w0x5, 8w0x11): parse_udp;  
  (_, _): accept; }
```

The type of a product of sets is a set of sequences or tuples.

8.6.5.2. Well-formedness

`SET < typeIR' >` is a well-formed type, with bound type variables `B` when:

1. Check that `typeIR'` can be used in a set type.

2. Check that `typeIR` is a well-formed type, with bound type variables `B`.
3. The relation holds.

8.6.6. Table metadata

A `table` can be invoked by calling its `apply` method. Calling an `apply` method on a table instance returns a value with a table metadata `struct` type with three fields. This structure is synthesized by the compiler automatically. For each `table T`, the compiler synthesizes a table metadata `enum` and a `struct`, shown in pseudo-P4:

```
enum action_list(T) {
    // one field for each action in the actions list of table T
}
struct apply_result(T) {
    bool hit;
    bool miss;
    action_list(T) action_run;
}
```

8.6.6.1. Operations

The evaluation of the `apply` method sets the `hit` field to `true` and the field `miss` to `false` if a match is found in the lookup-table; if a match is not found `hit` is set to `false` and `miss` to `true`. These bits can be used to drive the execution of the control-flow in the control block that invoked the table:

```
if (ipv4_match.apply().hit) {
    // there was a hit
} else {
    // there was a miss
}

if (ipv4_host.apply().miss) {
    ipv4_lpm.apply(); // Look up the route only if host table missed
}
```

The `action_run` field indicates which kind of action was executed (irrespective of whether it was a hit or a miss). It can be used in a switch statement:

```
switch (dmac.apply().action_run) {
    Drop_action: { return; }
}
```

Internally, the type and value of table metadata are represented as:

```
tableMetadataTypeIR
: tableMetadataEnumTypeIR
| tableMetadataStructTypeIR
;

tableMetadataValue
: tableMetadataEnumValue
| tableMetadataStructValue
;
```

8.6.6.2. Well-formedness

`typeIR` is a well-formed type, with bound type variables `B` when:

1. If let `TABLE_ENUM _ { nameIR* } be typeIR`:
 - a. Check that the elements of `nameIR*` are distinct.
 - b. The relation holds.
2. Else if let `TABLE_STRUCT _ { HIT _; MISS _; ACTION_RUN tableMetadataEnumTypeIR; } be typeIR`:
 - a. Check that `tableMetadataEnumTypeIR` is a well-formed type, with bound type variables `B`.
 - b. The relation holds.

8.7. Unrolling

`typedefs`, described in [Section 8.5.7](#), introduce indirection in types.

```
struct S {  
    bit<8> t;  
    int<8> u;  
}  
  
typedef S A;
```

The type `A` is an alias of the struct type `S`. Nevertheless, the underlying type of `A` is a `structTypeIR` with two fields, `t` and `u`, of types `int<8>` and `bit<8>` respectively. It is useful to be able to work with the underlying type of `A`, when dealing with type checking, type equivalence, and other type-based analyses.

Unrolling is the process of recursively expanding all `typedefs` until a type is fully expanded to its underlying type.

`typeIR` with `typedefs` unrolled

1. If let `TYPEDDEF _ typeIR'` be `typeIR`:
 - a. Return `typeIR'` with `typedefs` unrolled.
2. Otherwise:
 - a. Return `typeIR`.

9. P4 callables

`P416` includes four entities that can be invoked: actions, functions, methods, and object constructors. Object constructors are explained in [\[sec-def-object\]](#). The other three callable entities are collectively referred to as *callables*.

Callable types are created by the P4 compiler internally to represent the types of actions, functions, and methods during type-checking. We also call the type of a callable its signature. Libraries can contain callable declarations.

For example, consider the following declarations:

```
extern void random(in bit<5> logRange, out bit<32> value);
```



```

bit<32> add(in bit<32> left, in bit<32> right) {
  return left + right;
}

```

These declarations describe two callables:

- `random`, which has an extern function type, representing the following information:
 - the result type is `void`
 - the function has two inputs
 - the first formal parameter has direction `in`, type `bit<5>`, and name `logRange`
 - the second formal parameter has direction `out`, type `bit<32>`, and name `value`
- `add`, also has a function type, representing the following information:
 - the result type is `bit<32>`
 - the function has two inputs
 - both inputs have direction `in` and type `bit<32>`

Specifically, during type checking, callable declarations introduce a type constructor for the callable type:

```

callableTypeDefIR
: actionTypeDefIR
| functionTypeDefIR
| methodTypeDefIR
;

```

These callable type definitions are used to construct callable types via specialization:

```

callableTypeIR
: actionTypeIR
| functionTypeIR
| methodTypeIR
;

```

The well-formedness rules for callable types are checked by:

`CallableType_wf`:

- `callableTypeIR` is a well-formed callable type, with bound type variables `B`

The runtime representation of a callable is given by:

```

callableDef
: actionDef
| functionDef
| methodDef
;

```

9.1. Parameters

```
direction
: /* empty */
| IN
| OUT
| INOUT
;

parameter
: annotationList direction type name initializerOpt
;
```

Callables can have parameters. Each parameter has a direction, a type, a name, and an optional default value.

A parameter is internally represented as:

```
parameterIR
: annotationList direction typeIR id parameterInitializerOptIR
;
```

The high-level overview of parameters with respect to the copy in/copy out calling convention is described in [Section 6.5](#). Below are the formal typing and well-formedness rules for parameters that implement the restrictions described there.

9.1.1. Type checking

[Parameter_ok](#):

- Typing [parameter](#), under cursor [cursor](#) and typing context [typingContext](#):
- Results in the updated typing context [typingContext](#),
 - [parameterIR](#), and a set of fresh type variables [typeId*](#).

Typing a parameter produces a [parameterIR](#) if it is well-typed. A parameter is well-typed when its type is well-formed and its default value (if any) has the same type as the parameter type.

[Typing](#) [annotationList direction type name initializerOpt](#), under cursor [p](#) and typing context [TC₀](#):

1. If [initializerOpt](#) is `EMPTY`:
 - a. Let [typeIR](#) and a set of fresh type variables [typeId_{fresh}*](#) be
 - the result of [typing type](#), under cursor [p](#) and typing context [TC₀](#).
 - b. Let [bound](#) be [bound type variables from the p layer of TC₀](#).
 - c. Let [B](#) be [the union of the sets bound and { typeId_{fresh}* }](#).
 - d. Check that [typeIR](#) is a well-formed type, with bound type variables [B](#).
 - e. Let [nameIR](#) be [the name of name](#).
 - f. Let [parameterIR](#) be [annotationList direction typeIR nameIR](#).
 - g. Let [TC₁](#) be [parameterIR added to the p layer of TC₀](#).

h. Result in the updated typing context TC_1 ,

- $parameterIR$, and a set of fresh type variables $typeId_{fresh}^*$.

2. Else:

a. Let $= expression_{init}$ be $initializerOpt$.

b. Let $typeIR$ and a set of fresh type variables $typeId_{fresh}^*$ be

- the result of typing $type$, under cursor p and typing context TC_0 .

c. Let $bound$ be bound type variables from the p layer of TC_0 .

d. Let B be the union of the sets $bound$ and $\{typeId_{fresh}^*\}$.

e. Check that $typeIR$ is a well-formed type, with bound type variables B .

f. Let $typedExpressionIR_{init}$ be

- the result of typing $expression_{init}$, under cursor p and typing context TC_0 .

g. Let $typeIR_{init}$ be the type of $typedExpressionIR_{init}$.

h. If the compile-time known-ness of $typedExpressionIR_{init}$ is CTK:

i. Let $typedExpressionIR_{init_cast}$ be ! $typedExpressionIR_{init}$ implicitly cast to $typeIR$.

ii. Let $nameIR$ be the name of $name$.

iii. Let $parameterInitializerIR$ be $= typedExpressionIR_{init_cast}$.

iv. Let $parameterIR$ be $annotationList$ direction $typeIR$ $nameIR$ $parameterInitializerIR$.

v. Let TC_1 be $parameterIR$ added to the p layer of TC_0 .

vi. Result in the updated typing context TC_1 ,

- $parameterIR$, and a set of fresh type variables $typeId_{fresh}^*$.

i. Else if the compile-time known-ness of $typedExpressionIR_{init}$ is LTK:

i. Let $typedExpressionIR_{init_cast}$ be ! $typedExpressionIR_{init}$ implicitly cast to $typeIR$.

ii. Let $nameIR$ be the name of $name$.

iii. Let $value_{init}$ be

- the result of local compile-time evaluation of $typedExpressionIR_{init_cast}$, under cursor p and typing context TC_0 .

iv. Let $parameterInitializerIR$ be $= \text{VALUE } value_{init}$.

v. Let $parameterIR$ be $annotationList$ direction $typeIR$ $nameIR$ $parameterInitializerIR$.

vi. Let TC_1 be $parameterIR$ added to the p layer of TC_0 .

vii. Result in the updated typing context TC_1 ,

- $parameterIR$, and a set of fresh type variables $typeId_{fresh}^*$.

9.1.2. Well-formedness

ParameterType_{wf}:

- $annotationList$ direction $typeIR$ id $parameterInitializerIR^?$ is a well-formed parameter type, with bound type variables B

A parameter type is well-formed when its type is well-formed. Also, the type should not be a synthesized type. Types that can only be local compile-time known or compile-time known (e.g., `int` type) must be directionless. A default value can be specified for `in` or directionless parameters only.

`annotationList direction typeIR id parameterInitializerIR2` is a well-formed parameter type, with bound type variables `B` when:

1. If `parameterInitializerIR2` is none:
 - a. Check that `typeIR` is a well-formed type, with bound type variables `B`.
 - b. Let `typeIRunroll` be `typeIR` with `typedefs` unrolled.
 - c. Check that `typeIRunroll` does not have type `synthesizedTypeIR`.
 - d. Check that if `typeIRunroll` is equal to `INT` or `typeIRunroll` has type `stringTypeIR` or `typeIRunroll` has type `objectTypeIR`, then `direction` is equal to ``EMPTY``.
 - e. The relation holds.
2. Else:
 - a. Check that `typeIR` is a well-formed type, with bound type variables `B`.
 - b. Let `typeIRunroll` be `typeIR` with `typedefs` unrolled.
 - c. Check that `typeIRunroll` does not have type `synthesizedTypeIR`.
 - d. Check that if `typeIRunroll` is equal to `INT` or `typeIRunroll` has type `stringTypeIR` or `typeIRunroll` has type `objectTypeIR`, then `direction` is equal to ``EMPTY``.
 - e. Check that `direction` is equal to `IN` or `direction` is equal to ``EMPTY``.
 - f. Check that `~$optional_annotation_of_parameterIR'(annotationList)`.
 - g. The relation holds.

9.2. Callable definitions (callable constructors)

Similar to how type declarations introduce type definitions (see [Section 8.3.1](#)), callable declarations introduce callable definitions. These are constructors for callable types, that are parameterized by type parameters (if any).

```
actionTypeDefIR = actionTypeIR

functionTypeDefIR
: definedFunctionTypeDefIR
| externFunctionTypeDefIR
;

methodTypeDefIR
: externMethodTypeDefIR
| parserApplyMethodTypeDefIR
| controlApplyMethodTypeDefIR
| tableApplyMethodTypeDefIR
;

callableTypeDefIR
: actionTypeDefIR
| functionTypeDefIR
| methodTypeDefIR
;
```

When specialized with concrete type arguments, callable type definitions yield callable types:

```
actionTypeIR
: ACTION nameIR `( parameterIR* )
;

functionTypeIR
: definedFunctionTypeIR
| externFunctionTypeIR
;

methodTypeIR
: builtinMethodTypeIR
| externMethodTypeIR
| parserApplyMethodTypeIR
| controlApplyMethodTypeIR
| tableApplyMethodTypeIR
;

callableTypeIR
: actionTypeIR
| functionTypeIR
| methodTypeIR
;
```

For example, a function declaration introduces a function type definition:

```
T identity<T>(in T t) { return t; }
```

`identity` is a generic function type definition that falls into:

```
definedFunctionTypeDefIR
: FUNCTION nameIR `< typeParameterIR* , typeParameterIR* > `( parameterIR* ) : typeIR
;
```

Notice that `definedFunctionTypeDefIR` does *not* include the function body. Because callable type definitions are used during type checking, the body of the callable is not part of the type definition. After specialization with proper type argument(s), it yields a function type:

```
definedFunctionTypeIR
: FUNCTION nameIR `( parameterIR* ) : typeIR
;
```

9.2.1. Well-formedness

`#{relation: CallableTypeDef_wf}`

`#{rulegroup: CallableTypeDef_wf} #{ruleprose: CallableTypeDef_wf}`

9.3. Actions

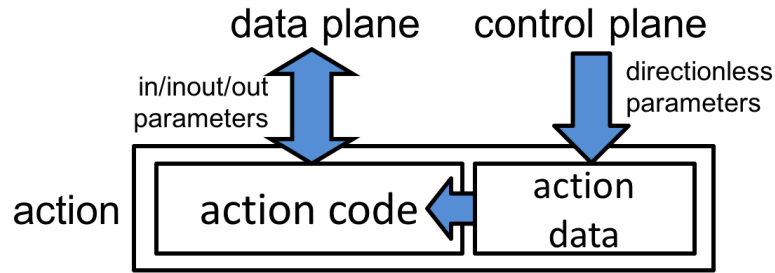


Figure 10. Actions contain code and data. The code is in the P4 program, while the data is provided in the table entries, typically populated by the control plane. Other parameters are bound by the data plane.

Actions are code fragments that can read and write the data being processed. Actions may contain data values that can be written by the control plane and read by the data plane. Actions are the main construct by which the control plane can dynamically influence the behavior of the data plane. Figure 10 shows the abstract model of an `action`.

```
actionDeclaration
: annotationList ACTION name ` ( parameterList ) blockStatement
;
```

Syntactically actions resemble functions with no return value. Actions may be declared within a control block; in this case they can only be used within instances of that control block.

The following example shows an action declaration:

```
action Forward_a(out bit<9> outputPort, bit<9> port) {
    outputPort = port;
}
```

Action parameters may not have `extern` types. Action parameters that have no direction (e.g., `port` in the previous example) indicate "action data." All such parameters must appear at the end of the parameter list. When used in a match-action table (see [\[sec-table-action-list\]](#)), these parameters will be provided by the table entries (e.g., as specified by the control plane, the `default_action` table property, or the `entries` table property).

The body of an action consists of a sequence of statements and declarations. No `table`, `control`, or `parser` applications can appear within actions.

Some targets may impose additional restrictions on action bodies—e.g., only allowing straight-line code, with no conditional statements or expressions.

Actions can be executed in two ways:

- Implicitly: by tables during match-action processing.
- Explicitly: either from a `control` block or from another `action`. In either case, the values for all action parameters must be supplied explicitly, including values for the directionless parameters. In this case, the directionless parameters behave like `in` parameters.

Actions are introduced by `action` declarations. Its typing and instantiation rules are explained in [Section 11.6](#). It introduces an action type definition:

```
actionTypeDefIR = actionTypeIR
```

Because actions cannot be generic, the action type is the same as the action type definition:

```
actionTypeIR
: ACTION nameIR `( parameterIR* )
;
```

The runtime representation of an action is as follows:

```
actionDef
: ACTION nameIR `( parameterListIR ) blockStatementIR
;
```

9.3.1. Well-formedness

ACTION nameIR (parameterIR*) is a well-formed callable type, with bound type variables **B** when:

1. Check that **parameterIR** is a well-formed parameter type, with bound type variables **B**, for all **parameterIR** in **parameterIR***.
2. Check that $\sim \$is_optional_parameterIR(parameterIR)$, for all **parameterIR** in **parameterIR***.
3. Let **direction*** be the list of **direction** and **typeIR*** be the list of **typeIR**, obtained by repeating:
 - Let **direction** and **typeIR** be the direction and the type of **parameterIR**.for each **parameterIR** in **parameterIR***
4. Check that **directionless parameters in direction*** only appear at the end.
5. Check that **typeIR can be used in an action type**, for all **typeIR** in **typeIR***.
6. The relation holds.

9.4. Functions

Two kinds of functions are supported in P4: user-defined functions and extern functions. The former are functions whose implementation is provided in P4 source code, while the latter are functions whose implementation is provided externally by the target architecture.

The internal representation of functions is as follows:

```
functionTypeDefIR
: definedFunctionTypeDefIR
| externFunctionTypeDefIR
;

functionTypeIR
: definedFunctionTypeIR
| externFunctionTypeIR
;

functionDef
: definedFunctionDef
| externFunctionDef
;
```

9.4.1. User-defined functions

Functions can only be declared at the top level and all parameters must have a direction. P4 functions are modeled after functions as found in most other programming languages, but the language does not permit recursive functions.

```
functionPrototype
: typeOrVoid name typeParameterListOpt `( parameterList )
;

functionDeclaration
: annotationList functionPrototype blockStatement
;
```

Here is an example of a function that returns the maximum of two 32-bit values:

```
bit<32> max(in bit<32> left, in bit<32> right) {
  return (left > right) ? left : right;
}
```

A function returns a value using the `return` statement. A function with a return type of `void` can simply use the `return` statement with no arguments. A function with a non-void return type must return a value of the suitable type on all possible execution paths.

The typing and instantiation rules for `function` declarations is explained in [Section 11.5](#). This introduces a function type definition, which can be specialized to a function type.

```
definedFunctionTypeDefIR
: FUNCTION nameIR `< typeParameterIR* , typeParameterIR* > `( parameterIR* ) : typeIR
;

definedFunctionTypeIR
: FUNCTION nameIR `( parameterIR* ) : typeIR
;
```

Also, the runtime representation of a defined function is as follows:

```
definedFunctionDef
: FUNCTION nameIR `< typeParameterListIR > `( parameterListIR ) blockStatementIR
;
```

9.4.1.1. Well-formedness

`FUNCTION nameIR ( parameterIR* ) : typeIRret` is a well-formed callable type, with bound type variables `B` when:

1. Check that `parameterIR` is a well-formed parameter type, with bound type variables `B`, for all `parameterIR` in `parameterIR*`.
2. Check that `~$is_optional_parameterIR(parameterIR)`, for all `parameterIR` in `parameterIR*`.
3. Let `typeIR*` be the list of `typeIR`, obtained by repeating:

- Let `typeIR` be the type of `parameterIR`.

for each `parameterIR` in `parameterIR*`

4. Check that typeIR can be used in a defined function type, for all typeIR in typeIR^* .
5. Check that $\text{typeIR}_{\text{ret}}$ is a well-formed return type, with bound type variables B .
6. The relation holds.

9.4.2. Extern functions

An extern function declaration describes the name and type signature of the function, but not its implementation.

```
externDeclaration
  : externFunctionDeclaration
  | externObjectDeclaration
  ;
```

extern declarations introduce extern function type definitions, which can be specialized to extern function types.

```
externFunctionTypeDefIR
  : EXTERN_FUNCTION nameIR `< typeParameterIR* , typeParameterIR* > `( parameterIR* ) :
  typeIR
  ;

externFunctionTypeIR
  : EXTERN_FUNCTION nameIR `( parameterIR* ) : typeIR
  ;
```

Also, the runtime representation of an extern function is as follows:

```
externFunctionDef
  : EXTERN_FUNCTION nameIR `< typeParameterListIR > `( parameterListIR )
  ;
```

See [Section 11.9](#) for the typing and instantiation rules for **extern** function declarations.

9.4.2.1. Well-formedness

$\text{EXTERN_FUNCTION nameIR (parameterIR*) : typeIR}_{\text{ret}}$ is a well-formed callable type, with bound type variables B when:

1. Check that parameterIR is a well-formed parameter type, with bound type variables B , for all parameterIR in parameterIR^* .
2. Let typeIR^* be the list of typeIR , obtained by repeating:
 - Let typeIR be the type of parameterIR .
 for each parameterIR in parameterIR^*
3. Check that typeIR can be used in an extern function type, for all typeIR in typeIR^* .
4. Check that $\text{typeIR}_{\text{ret}}$ is a well-formed return type, with bound type variables B .
5. The relation holds.

9.5. Methods

Objects in P4 can have methods associated with them. Specifically, **extern** objects have methods declared in their definitions. **parser**, **control**, and **table** objects have built-in **apply** methods.

Further, P4 allows built-in methods for certain types, such as **isValid()** and **setValid()** methods for header types.

The internal representation of methods is defined as follows:

```
methodTypeDefIR
: externMethodTypeDefIR
| parserApplyMethodTypeDefIR
| controlApplyMethodTypeDefIR
| tableApplyMethodTypeDefIR
;

methodTypeIR
: builtinMethodTypeIR
| externMethodTypeIR
| parserApplyMethodTypeIR
| controlApplyMethodTypeIR
| tableApplyMethodTypeIR
;

methodDef
: externMethodDef
| parserApplyMethodDef
| controlApplyMethodDef
| tableApplyMethodDef
;
```

9.5.1. Extern methods

An extern object declaration declares an object and all methods that can be invoked to perform computations and to alter the state of the object.

```
externConstructorPrototype
: annotationList typeIdIdentifier `( parameterList ) ;
;

externMethodPrototype
: annotationList functionPrototype ;
| annotationList ABSTRACT functionPrototype ;
;

externConstructorOrMethodPrototype
: externConstructorPrototype
| externMethodPrototype
;

externObjectDeclaration
: annotationList EXTERN nonTypeName typeParameterListOpt `{
externConstructorOrMethodPrototypeList }
;
```

For example, the P4 core library introduces two extern objects **packet_in** and **packet_out** used for manipulating packets (see [\[sec-packet-data-extraction\]](#) and [\[sec-deparse\]](#)). Here is an example showing how the methods of these objects can be invoked on a packet:

```

extern packet_out {
    void emit<T>(in T hdr);
}
control d(packet_out b, in Hdr h) {
    apply {
        b.emit(h.ipv4);          // write ipv4 header into output packet
    }                            // by calling emit method
}

```

Internally, extern methods are represented as follows:

```

externMethodTypeDefIR
: EXTERN_METHOD nameIR `< typeParameterIR* , typeParameterIR* > `( parameterIR* ) :
typeIR
| EXTERN_METHOD ABSTRACT nameIR `< typeParameterIR* , typeParameterIR* > `(
parameterIR* ) : typeIR
;

externMethodTypeIR
: EXTERN_METHOD nameIR `( parameterIR* ) : typeIR
| EXTERN_METHOD ABSTRACT nameIR `( parameterIR* ) : typeIR
;

externMethodDef
: EXTERN_METHOD nameIR `< typeParameterListIR > `( parameterListIR ) blockStatementIR?
| EXTERN_METHOD ABSTRACT nameIR `< typeParameterListIR > `( parameterListIR )
;

```

Typical extern object methods are implemented by the target architecture. P4 programmers can only call such methods.

Some types of extern objects may provide methods that can be implemented by the P4 programmers. Such methods are described with the **abstract** keyword prior to the method definition. Here is an example:

```

extern Balancer {
    Balancer();
    // get the number of active flows
    bit<32> getFlowCount();
    // return port index used for load-balancing
    // @param address: IPv4 source address of flow
    abstract bit<4> on_new_flow(in bit<32> address);
}

```

When such an object is instantiated the user has to supply an implementation of all the **abstract** methods (see [\[sec-instantion-abstract-method\]](#)).

9.5.1.1. Well-formedness

`externMethodTypeIR` is a well-formed callable type, with bound type variables **B** when:

1. If let `EXTERN_METHOD _ ( parameterIR* ) : typeIRret` be `externMethodTypeIR`:
 - a. Check that `parameterIR` is a well-formed parameter type, with bound type variables **B**, for all `parameterIR` in `parameterIR*`.
 - b. Let `typeIR*` be the list of `typeIR`, obtained by repeating:
 - Let `typeIR` be the type of `parameterIR`.

for each parameterIR in parameterIR^*

- c. Check that typeIR can be used in an extern method type, for all typeIR in typeIR^* .
- d. Check that $\text{typeIR}_{\text{ret}}$ is a well-formed return type, with bound type variables B .
- e. The relation holds.

2. Else:

- a. Let $\text{EXTERN_METHOD ABSTRACT_}(\text{parameterIR}^*) : \text{typeIR}_{\text{ret}}$ be $\text{externMethodTypeIR}$.
- b. Check that parameterIR is a well-formed parameter type, with bound type variables B , for all parameterIR in parameterIR^* .
- c. Check that $\sim \text{\$is_optional_parameterIR}(\text{parameterIR})$, for all parameterIR in parameterIR^* .
- d. Let typeIR^* be the list of typeIR , obtained by repeating:

- Let typeIR be the type of parameterIR .

for each parameterIR in parameterIR^*

- e. Check that typeIR can be used in an extern method type, for all typeIR in typeIR^* .
- f. Check that $\text{typeIR}_{\text{ret}}$ is a well-formed return type, with bound type variables B .
- g. The relation holds.

9.5.2. Parser apply methods

P4 allows parsers to invoke the services of other parsers, similar to subroutines. To invoke the services of another parser, the sub-parser must be first instantiated; the services of an instance are invoked by calling it using its `apply` method.

The following example shows a sub-parser invocation:

```
parser callee(packet_in packet, out IPv4 ipv4) { /* body omitted */ }
parser caller(packet_in packet, out Headers h) {
  callee() subparser; // instance of callee
  state subroutine {
    subparser.apply(packet, h.ipv4); // invoke sub-parser
    transition accept; // accept if sub-parser ends in accept state
  }
}
```

A parser apply method type definition is implicitly introduced by a `parser` declaration. Its semantics is explained in [Section 11.11](#). A parser apply method is represented internally as follows:

```
parserApplyMethodTypeDefIR = parserApplyMethodTypeIR

parserApplyMethodTypeIR
: PARSE_APPLY `( parameterIR* )
;

parserApplyMethodDef
: PARSE_APPLY `( parameterListIR ) `{ parserLocalDeclarationListIR ; stateEnv }
;
```

9.5.2.1. Well-formedness

`PARSER_APPLY ( parameterIR* )` is a well-formed callable type, with bound type variables `B` when:

1. Check that `parameterIR` is a well-formed parameter type, with bound type variables `B`, for all `parameterIR` in `parameterIR*`.
2. Check that `~$is_optional_parameterIR(parameterIR)`, for all `parameterIR` in `parameterIR*`.
3. Let `typeIR*` be the list of `typeIR`, obtained by repeating:
 - Let `typeIR` be the type of `parameterIR`.for each `parameterIR` in `parameterIR*`
4. Check that `typeIR` can be used in a parser apply method type, for all `typeIR` in `typeIR*`.
5. The relation holds.

9.5.3. Control apply methods

P4 allows controls to invoke the services of other controls, similar to subroutines. To invoke the services of another control, it must be first instantiated; the services of an instance are invoked by calling it using its `apply` method.

The following example shows a control invocation:

```
control Callee(inout IPv4 ipv4) { /* body omitted */ }
control Caller(inout Headers h) {
  Callee() instance; // instance of callee
  apply {
    instance.apply(h.ipv4); // invoke control
  }
}
```

A control apply method type definition is implicitly introduced by a `control` declaration. Its semantics is explained in [Section 11.12](#). A control apply method is represented internally as follows:

```
controlApplyMethodTypeDefIR = controlApplyMethodTypeIR

controlApplyMethodTypeIR
: CONTROL_APPLY `( parameterIR* )
;

controlApplyMethodDef
: CONTROL_APPLY `( parameterListIR ) `{ controlLocalDeclarationListIR ; actionDefEnv ;
controlBodyIR }
;
```

9.5.3.1. Well-formedness

`CONTROL_APPLY ( parameterIR* )` is a well-formed callable type, with bound type variables `B` when:

1. Check that `parameterIR` is a well-formed parameter type, with bound type variables `B`, for all `parameterIR` in `parameterIR*`.

2. Check that $\sim \$is_optional_parameterIR(parameterIR)$, for all $parameterIR$ in $parameterIR^*$.
3. Let $typeIR^*$ be the list of $typeIR$, obtained by repeating:
 - Let $typeIR$ be the type of $parameterIR$.
 for each $parameterIR$ in $parameterIR^*$
4. Check that $typeIR$ can be used in a control apply method type, for all $typeIR$ in $typeIR^*$.
5. The relation holds.

9.5.4. Table apply methods

A **table** can be invoked by calling its **apply** method. Calling an apply method on a table instance returns a synthesized table metadata structure, explained in [Section 8.6.6](#).

A table apply method definition is introduced by a **table** declaration. Its semantics rule is explained in [Section 17.1.5](#). A table declaration may not be generic. Thus, the table apply method type definition does not have type parameters, and is same as the table apply method type. A table apply method is represented internally as follows:

```
tableApplyMethodTypeDefIR = tableApplyMethodTypeIR

tableApplyMethodTypeIR
  : TABLE_APPLY : tableMetadataStructTypeIR
  ;

tableApplyMethodDef
  : TABLE_APPLY `{ tablePropertyListIR }
  ;
```

9.5.4.1. Well-formedness

TABLE_APPLY : tableMetadataStructTypeIR is a well-formed callable type, with bound type variables **B** when:

1. The relation holds.

9.5.5. Built-in methods

Built-in methods are methods associated with certain types defined by the P4 language. Thus, they do not have method type definitions, and only have method types. The internal representations are:

```
builtinMethodTypeIR
  : BUILTIN_METHOD nameIR `( parameterIR* ) : typeIR
  ;
```

NOTE | Refine builtin methods to concretely show which builtins are offered.

10. P4 objects and constructors

10.1. Objects

`tables`, `value_sets`, and `extern` objects are stateful entities in P4 programs. These objects should be instantiated statically prior to program execution. `parsers`, `controls`, and `packages` may contain stateful objects, thus they must also be instantiated.

The runtime representation of these objects is defined as follows:

```
object
: externObject
| parserObject
| controlObject
| packageObject
| tableObject
| valueSetObject
;
```

The instantiation phase (described in [Section 7.3](#)) creates objects in the source P4 program, and allocates them in the global store. At runtime (described in [Section 7.4](#)), these objects interact, as defined by the target architecture, to process packets.

10.1.1. Fully-qualified names

Instantiation may create multiple instances of a type, each of which must have a unique, fully-qualified name. Thus, objects are identified by their full-qualified names (`objectId`). [Section 7.3.1](#) describes how a fully-qualified name is constructed.

```
objectId = nameIR*
```

10.2. Constructors

Objects can be instantiated in three ways:

- Using instantiations, described in [Section 11.4](#).
- Using constructor call expressions, described in [\[sec-constructor-call-expression\]](#).
- Using direct type invocation statements, described in [Section 13.5](#).

Instantiations and constructor call expressions invoke constructors, which creates an object as a result. Direct application statements are syntactic sugar for instantiations of `parser` and `control` objects.

Constructor invocation is similar to callable invocation; constructors can also be overloaded, and called using named arguments.

The following example shows a constructor invocation for setting the target-dependent implementation property of a table:

```
extern ActionProfile {
    ActionProfile(bit<32> size); // constructor
}
table tbl {
    actions = { /* body omitted */ }
```

```
implementation = ActionProfile(1024); // constructor invocation
}
```

10.2.1. Constructor parameters

In order to support libraries of useful P4 components, `externs`, `parsers`, `controls`, and `packages` can additionally be parameterized through the use of constructor parameters.

Consider the parser declaration syntax:

```
parserDeclaration
: annotationList PARSE name typeParameterListOpt `( parameterList )
  constructorParameterListOpt `{ parserLocalDeclarationList parserStateList }
;

constructorParameterListOpt
: /* empty */
| `( parameterList )
;
```

From this grammar fragment we infer that a `parser` declaration may have two sets of parameters:

- The runtime parser parameters (`parameterList`)
- Optional parser constructor parameters (`constructorParameterListOpt`)

Because constructors are evaluated entirely at compilation time, all constructor arguments must be expressions that can be evaluated at compilation time. In consequence, constructor parameters must be directionless (i.e., they cannot be `in`, `out`, or `inout`).

Consider the following example:

```
parser GenericParser(packet_in b, out Packet_header p)
    (bool udpSupport) { // constructor parameters
    state start {
        b.extract(p.ethernet);
        transition select(p.ethernet.etherType) {
            16w0x0800: ipv4;
        }
    }
    state ipv4 {
        b.extract(p.ipv4);
        transition select(p.ipv4.protocol) {
            6: tcp;
            17: tryudp;
        }
    }
    state tryudp {
        transition select(udpSupport) {
            false: accept;
            true : udp;
        }
    }
    state udp {
        // body omitted
    }
}
```

When instantiating the `GenericParser` it is necessary to supply a value for the `udpSupport` parameter, as in the following example:


```
// topParser is a GenericParser where udpSupport = false
GenericParser(false) topParser;
```

Internally, constructor parameters are represented as:

```
constructorParameterIR = parameterIR

parameterIR
  : annotationList direction typeIR id parameterInitializerOptIR
  ;
```

10.2.1.1. Well-formedness

ConstructorParameterType_wf:

- **constructorParameterIR** is a well-formed constructor parameter type, with bound type variables **B**

constructorParameterIR is a well-formed constructor parameter type, with bound type variables **B** when:

1. Let **direction** be the direction of **constructorParameterIR**.
2. Check that **direction** is `EMPTY`.
3. Check that **constructorParameterIR** is a well-formed parameter type, with bound type variables **B**.
4. The relation holds.

10.2.2. Constructor definitions

Similar to how type declarations introduce type definitions and callable declarations introduce callable definitions (see [Section 9.2](#)), constructor declarations introduce constructor definitions. These are parameterized by type parameters, and produces a constructor type when specialized with specific type arguments.

The runtime representation of constructors is defined as follows:

```
constructorDef
  : externObjectConstructorDef
  | parserObjectConstructorDef
  | controlObjectConstructorDef
  | packageObjectConstructorDef
  ;
```

When type arguments and constructor arguments are supplied to a constructor definition, an object of the constructed type is created.

Similar to how callable types are internally used during type checking to represent callables, constructor types internally represent the types of constructors.

```
constructorTypeIR
  : CONSTRUCTOR `( constructorParameterIR* ) : typeIR
  ;
```

```

constructorTypeDefIR
  : CONSTRUCTOR `< typeParameterIR* , typeParameterIR* > `( constructorParameterIR* ) :
typeIR
;

```

10.2.2.1. Well-formedness

ConstructorTypeDef_wf:

- $\text{CONSTRUCTOR } < \text{typeParameterIR}_{\text{expl}}^* , \text{typeParameterIR}_{\text{impl}}^* > (\text{parameterIR}^*) : \text{typeIR}_{\text{object}}$ is a well-formed constructor type definition, with bound type variables B

$\text{CONSTRUCTOR } < \text{typeParameterIR}_{\text{expl}}^* , \text{typeParameterIR}_{\text{impl}}^* > (\text{parameterIR}^*) : \text{typeIR}_{\text{object}}$ is a well-formed constructor type definition, with bound type variables B when:

1. Check that $\text{typeIR}_{\text{object}}$ can be constructed.
2. Let typeParameterIR^* be $\text{typeParameterIR}_{\text{expl}}^*$ concatenated with $\text{typeParameterIR}_{\text{impl}}^*$.
3. Check that the elements of typeParameterIR^* are distinct.
4. Let B_{inner} be the union of the sets B and $\{ \text{typeParameterIR}^* \}$.
5. Check that $\text{CONSTRUCTOR } (\text{parameterIR}^*) : \text{typeIR}_{\text{object}}$ is a well-formed constructor type, with bound type variables B_{inner} .
6. The relation holds.

10.3. Extern objects

An **extern** object declaration introduces an extern object type and its associated methods. Extern object declarations can also optionally declare constructor methods; these must have the same name as the enclosing **extern** type, no type parameters, and no return type.

```

externConstructorPrototype
  : annotationList typeIdentifier `( parameterList ) ;
;

externMethodPrototype
  : annotationList functionPrototype ;
  | annotationList ABSTRACT functionPrototype ;
;

externObjectDeclaration
  : annotationList EXTERN nonTypeName typeParameterListOpt `{
externConstructorOrMethodPrototypeList }
;

```

Internally, an extern object constructor is represented as:

```

externObjectConstructorDef
  : EXTERN nameIR `< typeParameterListIR > `( constructorParameterListIR ) `{
externMethodPrototypeListIR }
;

externMethodPrototypeIR

```

```

: annotationList functionPrototypeIR ;
| annotationList ABSTRACT functionPrototypeIR ;
;

```

Extern objects are then instantiated by constructor invocations to yield:

```

externObject
: EXTERN typeId `< theta > `{ externState frame externMethodDefEnv }
;

```

The only operation on **extern** objects is invoking a method of an extern object instance using a method call expression.

Controls, parsers, packages, and extern constructors can have parameters of type **extern** only if they are directionless parameters. Extern object instances can thus be used as arguments in the construction of such objects.

No other operations are available on extern objects, e.g., equality comparison is not defined.

10.3.1. Well-formedness

CONSTRUCTOR (parameterIR^{*}) : typeIR_{object} is a well-formed constructor type, with bound type variables **B** when:

1. Check that **parameterIR** is a well-formed constructor parameter type, with bound type variables **B**, for all **parameterIR** in **parameterIR^{*}**.
2. Check that **typeIR_{object}** is a well-formed type, with bound type variables **B**.
3. Let **typeIR** be **typeIR_{object}** with typedefs unrolled.
4. Check that **typeIR** has type **externObjectTypeIR**.
5. Let **typeIR^{*}** be the list of **typeIR'**, obtained by repeating:
 - Let **typeIR'** be the type of **parameterIR**.
for each **parameterIR** in **parameterIR^{*}**
6. Check that **typeIR'** can be used in an extern constructor type, for all **typeIR'** in **typeIR^{*}**.
7. The relation holds.

10.3.2. Instantiation

Loading **externObjectConstructorDef** < typeArgumentIR^{*} > (argumentIR^{*}) with defaults **id_{default}^{*}** and optionals **id_{optional}^{*}**, under cursor **p**, instantiation context **IC**, and store **STO₀**:

1. Let **EXTERN** **nameIR** < typeParameterIR^{*} > (constructorParameterIR^{*}) { externMethodPrototypeIR^{*} } be **externObjectConstructorDef**.
2. Let **p_{callee}** be **BLOCK**.
3. Let **IC_{callee_0}** be an instantiation context same as **IC** up to the **GLOBAL** layer.
4. Let **theta** be { (typeParameterIR : typeArgumentIR^{*}) }.

5. Let IC_{callee_1} be IC_{callee_0} with $BLOCK.TYPE$ set to θ .
6. Let $GIVEN$ constructorParameterIR_{aligned}^{*} DEFAULT constructorParameterIR_{default}^{*} be aligning constructorParameterIR^{*} with argumentIR^{*} where defaults are id_{default}^{*} and optionals are id_{optional}^{*}.
7. Let id_{aligned}^{*} be the list of id_{aligned}, obtained by repeating:
 - Let id_{aligned} be the name of constructorParameterIR_{aligned}.
 for each constructorParameterIR_{aligned} in constructorParameterIR_{aligned}^{*}
8. Let the updated store STO_1 and instantiation context IC_{callee_2} be
 - the result of copy-in from p layer of IC with argumentIR^{*} to p_{callee} layer of IC_{callee_1} with id_{aligned}^{*}, under store STO_0 .
9. Let the updated store STO_2 and instantiation context IC_{callee_3} be
 - the result of copy-in defaults constructorParameterIR_{default}^{*} from p layer of IC to p_{callee} layer of IC_{callee_2} , under store STO_0 .
10. Let the updated instantiation context IC_{callee_4} be
 - the result of compile-time evaluation of externMethodPrototypeIR^{*}, under instantiation context IC_{callee_3} .
11. Let $\{ (callableId : callableDef)^* \}$ be $IC_{callee_4}.BLOCK.CALLABLE$.
12. Check that callableDef has type externMethodDef, for all callableDef in callableDef^{*}.
13. Let externMethodDef^{*} be the list of externMethodDef, obtained by repeating:
 - Let externMethodDef be callableDef.
 for each callableDef in callableDef^{*}
14. Let id_{cparam}^{*} be the list of id_{cparam}, obtained by repeating:
 - Let id_{cparam} be the name of constructorParameterIR.
 for each constructorParameterIR in constructorParameterIR^{*}
15. Let value[?]* be the list of value[?], obtained by repeating:
 - Let value[?] be id_{cparam} in map $IC_{callee_4}.BLOCK.FRAME$.
 for each id_{cparam} in id_{cparam}^{*}
16. Check that value[?] is defined, for all value[?] in value[?]*.
17. Let value_{cparam}^{*} be the list of value_{cparam}, obtained by repeating:
 - Let value_{cparam} be value[?].
 for each value[?] in value[?]*
18. Let externState be \$init_externState(nameIR, typeArgumentIR^{*}, value_{cparam}^{*}).
19. Let object_{extern} be EXTERN nameIR < $IC_{callee_4}.BLOCK.TYPE$ > { externState $IC_{callee_4}.BLOCK.FRAME$ { (callableId : externMethodDef)^{*} } }.
20. Result in
 - the updated store STO_2 ,

- and the constructed `objectextern`.

10.4. Parser objects

A parser declaration comprises a name, a list of parameters, an optional list of constructor parameters, local elements, and parser states (as well as optional annotations).

```
parserDeclaration
  : annotationList PARSE name typeParameterListOpt `( parameterList )
  constructorParameterListOpt `{ parserLocalDeclarationList parserStateList }
  ;

parserLocalDeclaration
  : constantDeclaration
  | instantiation
  | variableDeclaration
  | valueSetDeclaration
  ;

parserState
  : annotationList STATE name `{ parserStatementList transitionStatement }
  ;
```

At least one state, named `start`, must be present in any `parser`. A parser may not define two states with the same name. It is also illegal for a parser to give explicit definitions for the `accept` and `reject` states—those states are logically distinct from the states defined by the programmer.

Preceding the parser states, a `parser` may also contain a list of local elements. These can be constants, variables, or instantiations of objects that may be used within the parser. Such objects may be instantiations of `extern` objects, or other `parsers` that may be invoked as subroutines. However, it is illegal to instantiate a `control` block within a `parser`.

The states and local elements are all in the same namespace, thus the following example will produce an error:

```
// erroneous example
parser p() {
  bit<4> t;
  state start {
    t = 1;
    transition t;
  }
  state t { // error: name t is duplicated
    transition accept;
  }
}
```

For an example containing a complete declaration of a parser see [Section 5.3](#).

A `parser` declaration introduces a parser object constructor, represented as:

```
parserObjectConstructorDef
  : PARSE `< typeParameterListIR > `( parameterListIR ) `( constructorParameterListIR )
  `{ parserLocalDeclarationListIR parserStateListIR }
  ;
```

Parser objects are then instantiated by constructor invocations to yield:

```

parserObject
  : PARSE < theta > `( parameterListIR ) `{ frame parserLocalDeclarationListIR
stateEnv }
;

```

10.4.1. Well-formedness

$\text{CONSTRUCTOR} (\text{parameterIR}^*) : \text{typeIR}_{\text{object}}$ is a well-formed constructor type, with bound type variables B when:

1. Check that parameterIR is a well-formed constructor parameter type, with bound type variables B , for all parameterIR in parameterIR^* .
2. Check that $\sim \$\text{is_optional_parameterIR}(\text{parameterIR})$, for all parameterIR in parameterIR^* .
3. Check that $\text{typeIR}_{\text{object}}$ is a well-formed type, with bound type variables B .
4. Let typeIR be $\text{typeIR}_{\text{object}}$ with typedefs unrolled.
5. Check that typeIR has type $\text{parserObjectTypeIR}$.
6. Let typeIR^* be the list of typeIR' , obtained by repeating:
 - Let typeIR' be the type of parameterIR .
 for each parameterIR in parameterIR^*
7. Check that typeIR' can be used in a parser constructor type, for all typeIR' in typeIR^* .
8. The relation holds.

10.4.2. Instantiation

Loading $\text{parserObjectConstructorDef} < \text{typeArgumentIR}^* > (\text{argumentIR}^*)$ with defaults $\text{id}_{\text{default}}^*$ and optionals $\text{id}_{\text{optional}}^*$, under cursor p , instantiation context IC , and store STO_0 :

1. Let $\text{PARSER} < \text{typeParameterIR}^* > (\text{parameterIR}^*) (\text{constructorParameterIR}^*) \{ \text{parserLocalDeclarationIR}^* \text{ parserStateIR}^* \}$ be $\text{parserObjectConstructorDef}$.
2. Let p_{callee} be BLOCK.
3. Let $\text{IC}_{\text{callee}_0}$ be an instantiation context same as IC up to the GLOBAL layer.
4. Let theta be $\{ (\text{typeParameterIR} : \text{typeArgumentIR})^* \}$.
5. Let $\text{IC}_{\text{callee}_1}$ be $\text{IC}_{\text{callee}_0}$ with BLOCK.TYPE set to theta .
6. Let $\text{GIVEN} \quad \text{constructorParameterIR}_{\text{aligned}}^* \quad \text{DEFAULT} \quad \text{constructorParameterIR}_{\text{default}}^* \quad \text{be} \quad \text{aligning} \quad \text{constructorParameterIR}^* \text{ with } \text{argumentIR}^* \text{ where defaults are } \text{id}_{\text{default}}^* \text{ and optionals are } \text{id}_{\text{optional}}^*.$
7. Let $\text{id}_{\text{aligned}}^*$ be the list of $\text{id}_{\text{aligned}}$, obtained by repeating:
 - Let $\text{id}_{\text{aligned}}$ be the name of $\text{constructorParameterIR}_{\text{aligned}}$.
 for each $\text{constructorParameterIR}_{\text{aligned}}$ in $\text{constructorParameterIR}_{\text{aligned}}^*$
8. Let the updated store STO_1 and instantiation context $\text{IC}_{\text{callee}_2}$ be
 - the result of copy-in from p layer of IC with argumentIR^* to p_{callee} layer of $\text{IC}_{\text{callee}_1}$ with $\text{id}_{\text{aligned}}^*$, under store STO_0 .

9. Let the updated store STO_2 and instantiation context IC_{callee_3} be
 - the result of **copy-in defaults** $constructorParameterIR_{default}^*$ from p layer of IC to p_{callee} layer of IC_{callee_2} , under store STO_0 .
10. Let the updated instantiation context IC_{local} , the store STO_3 , and $parserLocalDeclarationIR_{inst}^*$ be
 - the result of **compile-time evaluation** of $parserLocalDeclarationIR^*$, under instantiation context IC_{callee_3} and store STO_2 .
11. Let the updated instantiation context IC_{state} and the store STO_4 be
 - the result of **compile-time evaluation** of $parserStateIR^*$, under instantiation context IC_{local} and store STO_3 .
12. Let $object_{parser}$ be $PARSER < theta > (parameterIR^*) \{ IC_{callee_3}.BLOCK.FRAME \ parserLocalDeclarationIR_{inst}^* \ IC_{state}.BLOCK.STATE \}$.
13. Result in
 - the updated store STO_4 ,
 - and the constructed $object_{parser}$.

10.5. Control objects

P4 parsers are responsible for extracting bits from a packet into headers. These headers (and other metadata) can be manipulated and transformed within **control** blocks. The body of a control block resembles a traditional imperative program. Within the body of a control block, match-action units can be invoked to perform data transformations. Match-action units are represented in P4 by constructs called **tables**.

Syntactically, a **control** block is declared with a name, parameters, optional type parameters, and a sequence of declarations of constants, variables, **actions**, **tables**, and other instantiations:

```
controlDeclaration
: annotationList CONTROL name typeParameterListOpt `( parameterList )
  constructorParameterListOpt `{ controlLocalDeclarationList APPLY controlBody }
;

controlLocalDeclaration
: constantDeclaration
| instantiation
| variableDeclaration
| actionDeclaration
| tableDeclaration
;

controlBody = blockStatement
```

It is illegal to instantiate a **parser** within a **control** block.

P4 does not support exceptional control-flow within a **control** block. The only statement which has a non-local effect on control flow is **exit**, which causes execution of the enclosing control block to immediately terminate. That is, there is no equivalent of the **verify** statement or the **reject** state from parsers. Hence, all error handling must be performed explicitly by the programmer.

A **control** declaration introduces a control object constructor, represented as:

```
controlObjectConstructorDef
: CONTROL `< typeParameterListIR > `( parameterListIR ) `( constructorParameterListIR
```

```
) `{ controlLocalDeclarationListIR APPLY controlBodyIR }
;
```

Control objects are then instantiated by constructor invocations to yield:

```
controlObject
: CONTROL `` ( parameterListIR ) `{ frame controlLocalDeclarationListIR
actionDefEnv controlBodyIR }
;
```

10.5.1. Well-formedness

CONSTRUCTOR (parameterIR^{*}) : typeIR_{object} is a well-formed constructor type, with bound type variables **B** when:

1. Check that **parameterIR** is a well-formed constructor parameter type, with bound type variables **B**, for all **parameterIR** in **parameterIR^{*}**.
2. Check that $\sim \$is_optional_parameterIR(parameterIR)$, for all **parameterIR** in **parameterIR^{*}**.
3. Check that **typeIR_{object}** is a well-formed type, with bound type variables **B**.
4. Let **typeIR** be **typeIR_{object}** with typedefs unrolled.
5. Check that **typeIR** has type **controlObjectTypeIR**.
6. Let **typeIR^{*}** be the list of **typeIR'**, obtained by repeating:
 - Let **typeIR'** be the type of **parameterIR**.
for each **parameterIR** in **parameterIR^{*}**
7. Check that **typeIR'** can be used in a control constructor type, for all **typeIR'** in **typeIR^{*}**.
8. The relation holds.

10.5.2. Instantiation

Loading **controlObjectConstructorDef** < typeArgumentIR^{*} > (argumentIR^{*}) with defaults **id_{default}^{*}** and optionals **id_{optional}^{*}**, under cursor **p**, instantiation context **IC**, and store **STO₀**:

1. Let **CONTROL** < typeParameterIR^{*} > (parameterIR^{*}) { controlLocalDeclarationIR^{*} APPLY controlBodyIR } be **controlObjectConstructorDef**.
2. Let **p_{callee}** be **BLOCK**.
3. Let **IC_{callee_0}** be an instantiation context same as **IC** up to the **GLOBAL** layer.
4. Let **theta** be { (typeParameterIR : typeArgumentIR^{*})^{*} }.
5. Let **IC_{callee_1}** be **IC_{callee_0}** with **BLOCK.TYPE** set to **theta**.
6. Let **GIVEN** **constructorParameterIR_{aligned}^{*}** **DEFAULT** **constructorParameterIR_{default}^{*}** be aligning **constructorParameterIR^{*}** with **argumentIR^{*}** where defaults are **id_{default}^{*}** and optionals are **id_{optional}^{*}**.
7. Let **id_{aligned}^{*}** be the list of **id_{aligned}**, obtained by repeating:
 - Let **id_{aligned}** be the name of **constructorParameterIR_{aligned}**.

for each $\text{constructorParameterIR}_{\text{aligned}}$ in $\text{constructorParameterIR}_{\text{aligned}}^*$

8. Let the updated store STO_1 and instantiation context $\text{IC}_{\text{callee}_2}$ be
 - the result of **copy-in from p layer of IC with argumentIR^{*} to p_{callee} layer of IC_{callee_1} with id_{aligned}^{*}, under store STO_0 .**
9. Let the updated store STO_2 and instantiation context $\text{IC}_{\text{callee}_3}$ be
 - the result of **copy-in defaults constructorParameterIR_{default}^{*} from p layer of IC to p_{callee} layer of IC_{callee_2}, under store STO_0 .**
10. Let the updated instantiation context $\text{IC}_{\text{local}_0}$, the store STO_3 , and $\text{controlLocalDeclarationIR}^*$ be
 - the result of **compile-time evaluation of controlLocalDeclarationIR^{*}, under instantiation context $\text{IC}_{\text{callee}_3}$ and store STO_2 .**
11. Let the updated instantiation context $\text{IC}_{\text{local}_1}$, store STO_4 , and $\text{controlBodyIR}'$ be
 - the result of **compile-time evaluation of controlBodyIR, under instantiation context $\text{IC}_{\text{local}_0}$, and store STO_3 .**
12. Let $\{ (\text{callableId} : \text{callableDef})^* \}$ be $\text{IC}_{\text{local}_1}.\text{BLOCK.CALLABLE}$.
13. Check that callableDef has type actionDef , for all callableDef in callableDef^* .
14. Let actionDef^* be the list of actionDef , obtained by repeating:
 - Let actionDef be callableDef .

for each callableDef in callableDef^*
15. Let $\text{object}_{\text{control}}$ be $\text{CONTROL} < \text{theta} > (\text{parameterIR}^*) \{ \text{IC}_{\text{callee}_3}.\text{BLOCK.FRAME } \text{controlLocalDeclarationIR}^* \{ (\text{callableId} : \text{actionDef})^* \} \text{controlBodyIR}' \}$.
16. Result in
 - the updated store STO_4 ,
 - and the constructed $\text{object}_{\text{control}}$.

10.6. Table objects

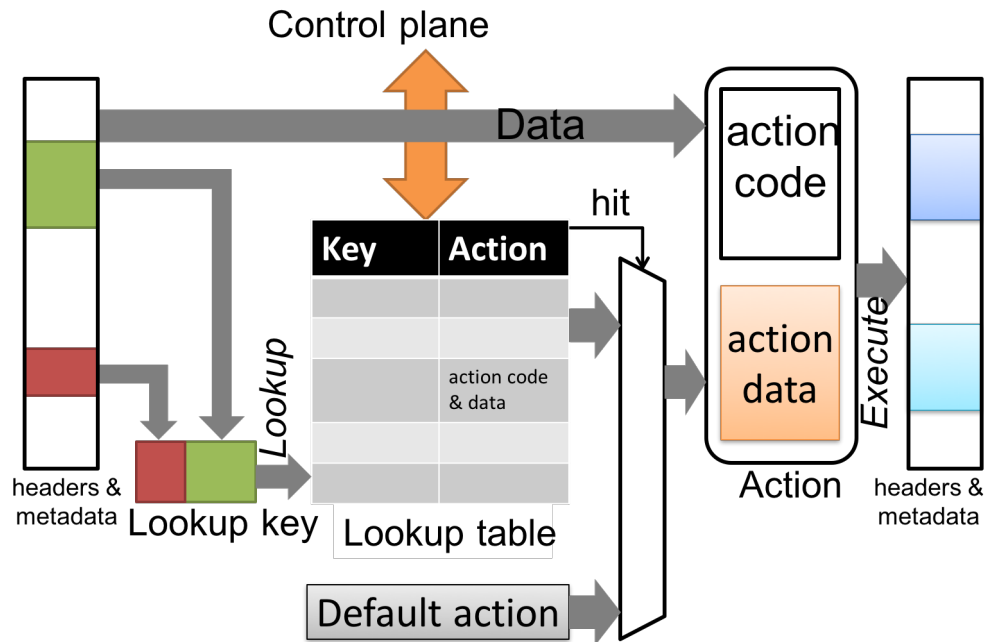


Figure 11. Match-Action Unit Dataflow.

A **table** describes a match-action unit. The structure of a match-action unit is shown in Figure 11. Processing a packet using a match-action table executes the following steps:

- Key construction.
- Key lookup in a lookup table (the "match" step). The result of key lookup is an "action".
- Action execution (the "action step") over the input data, resulting in mutations of the data.

A **table** declaration introduces a table instance. To obtain multiple instances of a table, it must be declared within a control block that is itself instantiated multiple times.

The look-up table is a finite map whose contents are manipulated asynchronously (read/write) by the target control plane, through a separate control-plane API (see Figure 11). Note that the term "table" is overloaded: it can refer to the P4 **table** objects that appear in P4 programs, as well as the internal look-up tables used in targets. We will use the term "match-action unit" when necessary to disambiguate.

Syntactically, a table is defined in terms of a set of key-value properties. Some of these properties are "standard" properties, but the set of properties can be extended by target-specific compilers as needed. Note that duplicated properties are invalid and the compiler should reject them.

```
tableDeclaration
  : annotationList TABLE name `{ tablePropertyList }
  ;

tableProperty
  : KEY = `{ tableKeyList }
  | ACTIONS = `{ tableActionList }
  | annotationList constOpt ENTRIES = `{ tableEntryList }
  | annotationList constOpt tableCustomName initializer ;
  ;
```

The standard table properties include:

- **key**: An expression that describes how the key used for look-up is computed.

- **actions**: A list of all actions that may be found in the table.

In addition, the tables may optionally define the following properties,

- **default_action**: an action to execute when the lookup in the lookup table fails to find a match for the key used.
- **size**: an integer specifying the desired size of the table.
- **entries**: entries that are initially added to a table when the P4 program is loaded, some or all of which may be unchangeable by the control plane software.
- **largest_priority_wins** - Only useful for some tables with the **entries** property. See [\[sec-entries\]](#) for details.
- **priority_delta** - Only useful for some tables with the **entries** property. See [\[sec-entries\]](#) for details.

The compiler must set the **default_action** to **NoAction** (and also insert it into the list of **actions**) for tables that do not define the **default_action** property. Hence, all tables can be thought of as having a **default_action** property, either implicitly or explicitly.

In addition, tables may contain architecture-specific properties (see [\[sec-additional-table-properties\]](#)).

A property marked as **const** cannot be changed dynamically by the control plane. The **key**, **actions**, and **size** properties cannot be modified so the **const** keyword is not needed for these.

A **table** declaration creates a table object, thus table constructors are not defined in P4_{IR}. Table objects are defined as:

```
tableObject
: TABLE typeId `{ frame tableObjectProperty }
;

tablePropertyIR
: tableKeysPropertyIR
| tableActionsPropertyIR
| tableDefaultActionPropertyIR
| tableEntriesPropertyIR
| tableCustomPropertyIR
;

tableObjectProperty = {KEYS tableKeysPropertyIR, ACTIONS tableActionsPropertyIR,
DEFAULT_ACTION tableDefaultActionPropertyIR, ENTRIES tableEntriesPropertyIR, CUSTOMS
tableCustomPropertyIR*}
```

10.6.1. Well-formedness

CONSTRUCTOR (·) : typeIR_{object} is a well-formed constructor type, with bound type variables **B** when:

1. Check that **typeIR_{object}** is a well-formed type, with bound type variables **B**.
2. Let **typeIR** be **typeIR_{object}** with typedefs unrolled.
3. Check that **typeIR** has type **tableObjectTypeIR**.
4. The relation holds.

10.7. Parser value set objects

In some cases, the values that determine the transition from one parser state to another need to be

determined at runtime. MPLS is one example where the value of the MPLS label field is used to determine what headers follow the MPLS tag and this mapping may change dynamically at runtime. To support this functionality, P4 supports the notion of a Parser Value Set. This is a named set of values with a runtime API to add and remove values from the set.

Value sets are declared locally within a parser. They should be declared before being referenced in parser `keysetExpression` and can be used as a label in a select expression.

The syntax for declaring value sets is:

```
valueSetDeclaration
: annotationList VALUE_SET `< valueSetType > `( expression ) name ;
;

valueSetType
: baseType
| tupleType
| prefixedTypeName
;
```

Parser Value Sets support a `size` argument to provide hints to the compiler to reserve hardware resources to implement the value set. Thus, the `size` argument must be a compile-time known value. For example, this parser value set:

```
value_set<bit<16>>(4) pvs;
```

creates a `value_set` of size 4 with entries of type `bit<16>`.

The semantics of the `size` argument is similar to the `size` property of a table. If a value set has a `size` argument with value `N`, it is recommended that a compiler should choose a data plane implementation that is capable of storing `N` value set entries. See "Size property of P4 tables and parser value sets" ^[1] for further discussion on the implementation of parser value set size.

The value set is populated by the control plane by methods specified in the P4Runtime specification. ^[2]

Similar to `tables`, a `value_set` declaration creates a new parser value set object. Thus, no constructor is defined for them. The object is represented as:

```
valueSetObject
: VALUE_SET `{ value* `( nat ) }
;
```

10.8. Package objects

A package type declaration introduces a package object constructor.

```
packageTypeDeclaration
: annotationList PACKAGE name typeParameterListOpt `( parameterList ) ;
;
```

All parameters of a package are evaluated at compilation time, and in consequence they must all be directionless (they cannot be `in`, `out`, or `inout`). Otherwise package types are very similar to parser type declarations. Packages can only be instantiated; there are no runtime behaviors associated with them.

Package object constructors are defined as follows:

```
packageObjectConstructorDef
  : PACKAGE `< typeParameterListIR > `( constructorParameterListIR )
  ;
```

Package objects are then instantiated to yield:

```
packageObject
  : PACKAGE `< theta > `{ frame }
  ;
```

10.8.1. Well-formedness

CONSTRUCTOR (parameterIR^{*}) : typeIR_{object} is a well-formed constructor type, with bound type variables **B** when:

1. Check that **parameterIR** is a well-formed constructor parameter type, with bound type variables **B**, for all **parameterIR** in **parameterIR^{*}**.
2. Check that **typeIR_{object}** is a well-formed type, with bound type variables **B**.
3. Let **typeIR** be **typeIR_{object}** with typedefs unrolled.
4. Check that **typeIR** has type **packageObjectTypeIR**.
5. Let **typeIR^{*}** be the list of **typeIR'**, obtained by repeating:
 - Let **typeIR'** be the type of **parameterIR**.
 for each **parameterIR** in **parameterIR^{*}**
6. Check that **typeIR'** can be used in a package constructor type, for all **typeIR'** in **typeIR^{*}**.
7. The relation holds.

10.8.2. Instantiation

Loading **packageObjectConstructorDef** < typeArgumentIR^{*} > (argumentIR^{*}) with defaults **id_{default}^{*}** and optionals **id_{optional}^{*}**, under cursor **p**, instantiation context **IC**, and store **STO₀**:

1. Let **PACKAGE** < typeParameterIR^{*} > (constructorParameterIR^{*}) be **packageObjectConstructorDef**.
2. Let **p_{callee}** be **BLOCK**.
3. Let **IC_{callee_0}** be an instantiation context same as **IC** up to the **GLOBAL** layer.
4. Let **IC_{callee_1}** be **IC_{callee_0}** with **BLOCK.TYPE** set to { (typeParameterIR : typeArgumentIR^{*}) }.
5. Let **GIVEN** constructorParameterIR_{aligned}^{*} **DEFAULT** constructorParameterIR_{default}^{*} be **aligning** constructorParameterIR^{*} with argumentIR^{*} where defaults are **id_{default}^{*}** and optionals are **id_{optional}^{*}**.
6. Let **id_{aligned}^{*}** be the list of **id_{aligned}**, obtained by repeating:
 - Let **id_{aligned}** be the name of **constructorParameterIR_{aligned}**.
 for each **constructorParameterIR_{aligned}** in **constructorParameterIR_{aligned}^{*}**
7. Let the updated store **STO₁** and instantiation context **IC_{callee_2}** be

- the result of **copy-in from** p **layer of** IC **with** $argumentIR^*$ **to** p_{callee} **layer of** IC_{callee_1} **with** $id_{aligned}^*$, **under store** STO_0 .
- Let the updated store STO_2 and instantiation context IC_{callee_3} be
 - the result of **copy-in defaults** $constructorParameterIR_{default}^*$ **from** p **layer of** IC **to** p_{callee} **layer of** IC_{callee_2} , **under store** STO_1 .
 - Let $object_{package}$ be $PACKAGE < IC_{callee_3}.BLOCK.TYPE > \{ IC_{callee_3}.BLOCK.FRAME \}$.
 - Result in
 - the updated store STO_2 ,
 - and the constructed $object_{package}$.

[1] <https://github.com/p4lang/p4-spec/blob/master/p4-16/spec/docs/p4-table-and-parser-value-set-sizes.md>

[2] The P4Runtime API is defined as a Google Protocol Buffer **.proto** file and an accompanying English specification document here: <https://github.com/p4lang/p4runtime>

11. Declarations

The syntax of top-level declarations is defined as follows:

```
declaration
: constantDeclaration
| instantiation
| functionDeclaration
| actionDeclaration
| errorDeclaration
| matchKindDeclaration
| externDeclaration
| parserDeclaration
| controlDeclaration
| typeDeclaration
;

externDeclaration
: externFunctionDeclaration
| externObjectDeclaration
;

typeDeclaration
: derivedTypeDeclaration
| typedefDeclaration
| parserTypeDeclaration
| controlTypeDeclaration
| packageTypeDeclaration
;
```

As described in [Chapter 7](#), declarations are type checked, instantiated at compile time, and evaluated at runtime.

11.1. Semantics of declarations

11.1.1. Type checking

Decl_ok:

- Typing **declaration** under typing context **typingContext**:
- Results in the updated typing context **typingContext** and **declarationIR**.

After type checking, declarations are represented in $P4_{IR}$ as follows:

```
declarationIR
: constantDeclarationIR
| instantiationIR
| functionDeclarationIR
| actionDeclarationIR
| errorDeclarationIR
| matchKindDeclarationIR
| externDeclarationIR
| parserDeclarationIR
| controlDeclarationIR
| typeDeclarationIR
;
```

```

externDeclarationIR
: externFunctionDeclarationIR
| externObjectDeclarationIR
;

typeDeclarationIR
: derivedTypeDeclarationIR
| typedefDeclarationIR
| parserTypeDeclarationIR
| controlTypeDeclarationIR
| packageTypeDeclarationIR
;

```

11.1.2. Compile-time evaluation

Decl_inst:

- Loading `declarationIR`, under instantiation context `instContext`, and store `store`:
- Results in the updated instantiation context `instContext` and the store `store`.

Instantiation statically allocates instantiated objects in the global `store`. [Section 11.4](#) describes how objects are instantiated from a declaration.

11.1.3. Runtime evaluation

All declarations except constant and variable declarations are evaluated at compile-time. See [Section 11.2](#) for the runtime evaluation of variable declarations and [Section 11.3](#) for the runtime evaluation of constant declarations.

The subsequent sections describe each kind of declaration in detail.

11.2. Variable declarations

Local variables are declared with a type, a name, and an optional initializer (as well as an optional annotation):

```

variableDeclaration
: annotationList type name initializerOpt ;
;

initializerOpt
: /* empty */
| initializer
;

initializer
: = expression
;

```

Variable declarations without an initializer are uninitialized (except for headers and other header-related types, which are initialized to invalid in the same way as described for direction `out` parameters in [Section 18.4](#)). The language places few restrictions on the types of variables: most P4 types that can be written explicitly can be used (e.g., base types, `struct`, `header`, header stack, `tuple`). However, it is impossible to declare variables with type `int`, or with types that are only synthesized by the compiler (e.g., `set`). In addition, variables of type `parser`, `control`, `package`, or `extern` types must be declared using instantiations (see [Section 11.4](#)).

Reading the value of a variable that has not been initialized yields an undefined result. The compiler should attempt to detect and emit a warning in such situations.

NOTE | P4-SpecTec implements zero-initialization for local variables.

Variable declarations can appear in the following locations within a P4 program:

- In a block statement,
- In a **parser** state,
- In a **control** block's **apply** sub-block,
- In the list of local declarations in a **parser**, and
- In the list of local declarations in a **control**.

Note that variable declarations *cannot* appear at the top level of a P4 program.

Variables have local scope, and behave like stack-allocated variables in languages such as C. The value of a variable is never preserved from one invocation of its enclosing block to the next. In particular, variables cannot be used to maintain state between different network packets.

11.2.1. Type checking

Variable declarations are type checked using the following relation:

VarDecl_ok:

- Typing **variableDeclaration**, under cursor **cursor** and typing context **typingContext**:
- Results in the updated typing context **typingContext** and **variableDeclarationIR**.

After typing, variable declarations are represented in P4_{IR} as:

```
variableDeclarationIR
: annotationList typeIR nameIR initializerOptIR ;
;

initializerIR
: = typedExpressionIR
;
```

These are produced by the following rules:

Typing **annotationList type name initializerOpt ;**, under cursor **p** and typing context **TC₀**:

1. If **initializerOpt** is **EMPTY**:
 - a. Let **typeIR** and a set of fresh type variables **typeId*** be
 - the result of **typing type**, under cursor **p** and typing context **TC₀**.
 - b. Check that **typeId*** is an empty list.
 - c. Let **bound** be **bound type variables from the p layer of TC₀**.
 - d. Check that **typeIR** is a well-formed type, with bound type variables **bound**.
 - e. Check that **typeIR** supports assignment.

- f. Let `nameIR` be the name of `name`.
 - g. Let `TC1` be `TC0` where `nameIR` to `INOUT typeIR DYN` · is added to the `p` layer.
 - h. Let `variableDeclarationIR` be `annotationList typeIR nameIR` · ;.
 - i. Result in the updated typing context `TC1` and `variableDeclarationIR`.
2. Else:
- a. Let `= expressioninit` be `initializerOpt`.
 - b. Let `typeIR` and a set of fresh type variables `typeId*` be
 - the result of typing `type`, under cursor `p` and typing context `TC0`.
 - c. Check that `typeId*` is an empty list.
 - d. Let `bound` be bound type variables from the `p` layer of `TC0`.
 - e. Check that `typeIR` is a well-formed type, with bound type variables `bound`.
 - f. Check that `typeIR` supports assignment.
 - g. Let `typedExpressionIRinit` be
 - the result of typing `expressioninit`, under cursor `p` and typing context `TC0`.
 - h. Let `typedExpressionIRinit_cast` be ! `typedExpressionIRinit` implicitly cast to `typeIR`.
 - i. Let `nameIR` be the name of `name`.
 - j. Let `TC1` be `TC0` where `nameIR` to `INOUT typeIR DYN` · is added to the `p` layer.
 - k. Let `variableDeclarationIR` be `annotationList typeIR nameIR = typedExpressionIRinit_cast` ;.
 - l. Result in the updated typing context `TC1` and `variableDeclarationIR`.

11.2.2. Runtime evaluation

Variable declarations are evaluated at runtime using the following relation:

VarDecl_{eval}:

- Evaluating `variableDeclarationIR`, under cursor `cursor`, evaluation context `evalContext`, and store `store`:
- Results in
 - the updated evaluation context `evalContext`, store `store`,
 - and the result `declarationResult`.

The result of evaluating a variable declaration is:

```
continueEmptyResult
: /* empty */
;

exitResult
: EXIT
;

abortResult
: exitResult
| rejectTransitionResult
;
```

```

declarationResult
: continueEmptyResult
| abortResult
;

```

See [Section 13.7](#) for how `exitResult` may occur. The relation is defined by the following rule:

Evaluating `annotationList typeIR nameIR initializerIR? ;` under cursor `p`, evaluation context `EC0`, and store `store`:

1. If `initializerIR?` is none:
 - a. Let `typeIRsubst` be `typeIR` substituted by bound type variables in `EC0` from the BLOCK layer.
 - b. Let `valueinit` be the default value for type `typeIRsubst`.
 - c. Let `EC1` be `EC0` where `nameIR` to `valueinit` is added to the `p` layer.
 - d. Result in
 - the updated evaluation context `EC1`, store `store`,
 - and the result ``EMPTY`.
2. Else:
 - a. Let `= typedExpressionIRinit` be `initializerIR?`.
 - b. Let the updated evaluation context `EC1`, store `STO1` and `expressionResult` be
 - the result of evaluating `typedExpressionIRinit`, under cursor `p`, evaluation context `EC0`, and store `store`.
 - c. If let `abortResult` be `expressionResult`:
 - i. Result in
 - the updated evaluation context `EC1`, store `STO1`,
 - and the result `abortResult`.
 - d. Else:
 - i. Let `valueinit` be `expressionResult`.
 - ii. Let `EC2` be `EC1` where `nameIR` to `valueinit` is added to the `p` layer.
 - iii. Result in
 - the updated evaluation context `EC2`, store `STO1`,
 - and the result ``EMPTY`.

NOTE

Currently in P4-SpecTec, instantiation of a variable declaration is implemented as a no-op. But, instantiation *may* occur within the right-hand side, initializer expression. Ideally, revisit this.

11.3. Constant declarations

Constant values are defined with the syntax:

```

constantDeclaration
: annotationList CONST type name initializer ;

```

```

;
initializer
: = expression
;

```

The `initializer` expression must be a local compile-time known value. A constant declaration introduces a constant whose value has the specified type. The following are all legal constant declarations:

```

const bit<32> COUNTER = 32w0x0;
struct Version {
    bit<32> major;
    bit<32> minor;
}
const Version version = { 32w0, 32w0 };

```

11.3.1. Type checking

Constant declarations are type checked with the relation:

`ConstDecl_ok`:

- Typing `constantDeclaration`, under cursor `cursor` and typing context `typingContext`:
- Results in the updated typing context `typingContext` and `constantDeclarationIR`.

After typing, in $P4_{IR}$, constant declarations have the syntax:

```

constantDeclarationIR
: annotationList CONST typeIR nameIR constantInitializerIR ;
;

constantInitializerIR
: = `VALUE value
;

```

Notice that `constantInitializerIR` is reduced to a `value` in $P4_{IR}$, where `value` is computed from `initializer` via local compile-time evaluation. It is defined by the following rule:

Typing `annotationList CONST type name = expressionvalue ;`, under cursor `p` and typing context TC_0 :

1. Let `typeIR` and a set of fresh type variables `typedId*` be
 - the result of **typing type**, under cursor `p` and typing context TC_0 .
2. Check that `typedId*` is an empty list.
3. Let `bound` be **bound type variables from the `p` layer of TC_0** .
4. Check that `typeIR` is a **well-formed type**, with bound type variables `bound`.
5. Let `typedExpressionIRvalue` be
 - the result of **typing expression_{value}**, under cursor `p` and typing context TC_0 .
6. Check that **the compile-time known-ness of `typedExpressionIRvalue` is LCTK**.
7. Let `typedExpressionIRvalue_cast` be **! typedExpressionIR_{value} implicitly cast to typeIR**.

8. Let `value` be
 - the result of **local compile-time evaluation of** `typedExpressionIRvalue_cast`, **under cursor** `p` **and typing context** `TC0`.
9. Let `nameIR` be **the name of** `name`.
10. Let `TC1` be `TC0` **where** `nameIR` **to** ``EMPTY typeIR` `LCTK value` **is added to the** `p` **layer**.
11. Let `constantDeclarationIR` be `annotationList` `CONST typeIR` `nameIR = `VALUE value ;`.
12. Result in the updated typing context `TC1` and `constantDeclarationIR`.

When used as a top-level declaration, constant declarations are typed with:

Typing `constantDeclaration` **under typing context** `TC0`:

1. Let the updated typing context `TC1` and `constantDeclarationIR` be
 - the result of **typing** `constantDeclaration`, **under cursor** `GLOBAL` **and typing context** `TC0`.
2. Result in the updated typing context `TC1` and `constantDeclarationIR`.

11.3.2. Compile-time evaluation

Constant declarations are evaluated during instantiation phase with:

ConstDecl_inst:

- Loading `constantDeclarationIR`, under cursor `cursor`, and instantiation context `instContext`:
- Results in the updated instantiation context `instContext`.

It is defined by the following rule:

Loading `annotationList` `CONST typeIR` `nameIR = `VALUE value ;`, **under cursor** `p`, **and instantiation context** `IC0`:

1. Let `IC1` be `IC0` **where** `nameIR` **to** `value` **is added to the** `p` **layer**.
2. Result in the updated instantiation context `IC1`.

When used as a top-level declaration, constant declarations are compile-time evaluated with:

Loading `constantDeclarationIR`, **under instantiation context** `IC0`, **and store** `STO`:

1. Let the updated instantiation context `IC1` be
 - the result of **loading** `constantDeclarationIR`, **under cursor** `GLOBAL`, **and instantiation context** `IC0`.
2. Result in the updated instantiation context `IC1` and the store `STO`.

11.3.3. Runtime evaluation

Constant declarations are evaluated at runtime using the following relation:

ConstDecl_eval:

- Evaluating `constantDeclarationIR`, under cursor `cursor`, and evaluation context `evalContext`:
- Results in the updated evaluation context `evalContext`.

Evaluating `annotationList CONST typeIR nameIR = `VALUE value ;`, under cursor `p`, and evaluation context `EC`:

1. Let EC_1 be EC_0 where `nameIR` to `value` is added to the `p` layer.
2. Result in the updated evaluation context EC_1 .

11.4. Instantiations

Instantiations are similar to variable declarations, but are reserved for the types with constructors (`extern` objects, `control` blocks, `parsers`, and `packages`):

```
instantiation
: annotationList type `( argumentList ) name ;
| annotationList type `( argumentList ) name objectInitializer ;
;
```

An instantiation is written as a constructor invocation followed by a name. Instantiations are always executed at compilation time (Section [Section 7.3](#)). The effect is to allocate an object with the specified name, and to bind it to the result of the constructor invocation. Note that instantiation arguments can be specified by name.

For example, a hypothetical bank of counter objects can be instantiated as follows:

```
// from target library
enum CounterType {
    Packets,
    Bytes,
    Both
}
extern Counter {
    Counter(bit<32> size, CounterType type);
    void increment(in bit<32> index);
}
// user program
control c(/* parameters omitted */) {
    Counter(32w1024, CounterType.Both) ctr; // instantiation
    apply { /* body omitted */ }
}
```

A P4 program may not instantiate controls and parsers at the top-level. This restriction is designed to ensure that most state resides in the architecture itself, or is local to a `parser` or `control`. For example, the following program is not valid:

```
// Program
control c(/* parameters omitted */) { /* body omitted */ }
c() c1; // illegal top-level instantiation
```

because control `c1` is instantiated at the top-level. Note that top-level declarations of constants and instantiations of extern objects are permitted.

11.4.1. Objects with abstract methods

When instantiating an extern type that has `abstract` methods users have to supply implementations for all such methods. This is done using object initializers:

```
objectInitializer
: = `{ objectDeclarationList }
;

objectDeclarationList
: /* empty */
| objectDeclarationList objectDeclaration
;

objectDeclaration
: functionDeclaration
| instantiation
;
```

In $P4_{IR}$, object initializers have the syntax:

```
objectInitializerIR
: = `{ objectDeclarationListIR }
;

objectDeclarationListIR = objectDeclarationIR*

objectDeclarationIR
: functionDeclarationIR
| instantiationIR
;
```

Abstract method implementations must use the same number of parameters, and the same parameter directions as those declared in the extern object declaration. Note that overloading of abstract methods is not allowed, thus an implementation could be matched to a declaration only with the method name.

Abstract method implementations can only use the supplied arguments or refer to values that are in the same initializer block or the top-level scope. When calling another method of the same instance the `this` keyword is used to indicate the current object instance:

```
// Instantiate a balancer
Balancer() b = { // provide an implementation for the abstract methods
    bit<4> on_new_flow(in bit<32> address) {
        // uses the address and the number of flows to balance the load
        bit<32> count = this.getFlowCount(); // call method of the same instance
        return (address + count)[3:0];
    }
}
```

Abstract methods may be invoked by users explicitly, or they may be invoked by the target architecture. The architectural description has to specify when the abstract methods are invoked and what the meaning of their arguments and return values is; target architectures may impose additional constraints on abstract methods.

11.4.1.1. Type checking

An object declaration is type checked with the relation:

ObjectDecl_ok:

- Typing `objectDeclaration` under cursor `cursor` and typing context `typingContext` with `typeFrame` and `externMethodTypeDefEnv`:
- Results in `typeFrame` and `externMethodTypeDefEnv` of the object declaration and `objectDeclarationIR`.

It is defined by the following rules:

Typing `objectDeclaration` under cursor `p` and typing context `TC0` with `typeFrame` and `externMethodTypeDefEnv`:

1. If let `instantiation` be `objectDeclaration`:
 - a. Let the updated typing context `TC1` and `instantiationIR` be
 - the result of typing `instantiation`, under cursor `p` and typing context `TC0`.
 - b. Let `nameIR` be the name of `instantiationIR`.
 - c. Let `varTypeIR'` be ! the type of variable `nameIR` from the `p` layer of `TC1`.
 - d. Let `typeFrameinit` be the map `typeFrame` with the value for `nameIR` is updated to `varTypeIR'`.
 - e. Result in `typeFrameinit` and `externMethodTypeDefEnv` of the object declaration and `instantiationIR`.
2. Else:
 - a. Let `annotationList` `typeOrVoid` `name` `typeParameterListOpt` (`parameterList`) `blockStatement` be `objectDeclaration`.
 - b. Let `TC1` be `TC0` with `BLOCK.KIND` set to `EXTERN`.
 - c. Let `TC2` be `TC1` with `BLOCK.FRAME` set to `typeFrame`.
 - d. Let the updated typing context `TC3` and type parameters `typeIdexpl*` be
 - the result of typing `typeParameterListOpt`, under cursor `LOCAL` and typing context `TC2`.
 - e. Let the updated typing context `TC4`, `parameterIR*`, and a set of fresh type variables `typeIdimpl*` be
 - the result of typing `parameterList`, under cursor `LOCAL` and typing context `TC3`.
 - f. Let `typeIRret` and a set of fresh type variables `typeId*` be
 - the result of typing `typeOrVoid`, under cursor `LOCAL` and typing context `TC3`.
 - g. Check that `typeId*` is an empty list.
 - h. Let `TC5` be `TC4` with `LOCAL.KIND` set to `EXTERN_METHOD : typeIRret`.
 - i. Let the updated typing context `_`, abstract control flow `f`, and `blockStatementIR` be
 - the result of typing `blockStatement`, under typing context `TC5`, abstract control flow `CONT`, and loop context `NOLOOP`.
 - j. Check that `f` is equal to `RET` or `typeIRret` is equal to `VOID`.
 - k. Let `callableId` be the callable identifier for `name` (`parameterList`).
 - l. Let `nameIR` be the name of `name`.
 - m. Let `externMethodTypeDefIR` be `EXTERN_METHOD nameIR < typeIdexpl*, typeIdimpl* > ( parameterIR* ) : typeIRret`.

- n. Let **bound** be **bound type variables from the p layer of TC_0** .
- o. Check that **externMethodTypeDefIR** is a well-formed callable type definition, with bound type variables **bound**.
- p. Let **externMethodTypeDefEnv_{init}** be **the map externMethodTypeDefEnv with the value for callableId is updated to externMethodTypeDefIR**.
- q. Let **functionDeclarationIR** be **annotationList typeIR_{ret} nameIR < typeId_{expl}^{*}, typeId_{impl}^{*} > (parameterIR^{*}) blockStatementIR**.
- r. Result in **typeFrame** and **externMethodTypeDefEnv_{init}** of the object declaration and **functionDeclarationIR**.

A list of object declarations is type checked with the relation:

ObjectDeclList_ok:

- Typing **objectDeclarationList** under cursor **cursor** and typing context **typingContext**:
- Results in **typeFrame** and **externMethodTypeDefEnv** of the object declaration block and **objectDeclarationListIR**.

11.4.1.2. Compile-time evaluation

At compile-time, an object declaration is loaded with the relation:

ObjectDecl_inst:

- Loading **objectDeclarationIR** under instantiation context **instContext** and store **store**:
- Results in the updated instantiation context **instContext** and store **store**.

It is defined by the following rules:

Loading objectDeclarationIR under instantiation context IC_0 and store store:

1. If let **functionDeclarationIR** be **objectDeclarationIR**:
 - a. Let the updated instantiation context **IC_1** be
 - the result of **loading functionDeclarationIR, under cursor BLOCK, and instantiation context IC_0** .
 - b. Result in the updated instantiation context **IC_1** and store **store**.
2. Else:
 - a. Let **instantiationIR** be **objectDeclarationIR**.
 - b. Let the updated instantiation context **IC_1** , store **STO_1** , and the reference to the object **_** be
 - the result of **loading instantiationIR, under cursor GLOBAL, instantiation context IC_0 , and store store**.
 - c. Result in the updated instantiation context **IC_1** and store **STO_1** .

A list of object declarations is loaded with the relation:

ObjectDecls_inst:

- Loading `objectDeclarationListIR` under instantiation context `instContext` and store `store`:
- Results in the updated instantiation context `instContext` and store `store`.

11.4.2. Type checking

Instantiations are type checked with the relation:

InstDecl_ok:

- Typing `instantiation`, under cursor `cursor` and typing context `typingContext`:
- Results in the updated typing context `typingContext` and `instantiationIR`.

After type checking, in $P4_{IR}$, instantiations have the syntax:

```
instantiationIR
  : annotationList typeIR constructorTargetIR `( argumentListIR ) nameIR
  objectInitializerOptIR ;
;
```

Typing `instantiation`, under cursor `p` and typing context TC_0 :

1. If let `annotationList type ( argumentList ) name ; be instantiation`:
 - a. If let `prefixedTypeName be type`:
 - i. Let the updated typing context TC_1 and `instantiationIR` be
 - the result of typing `annotationList prefixedTypeName < `EMPTY > (argumentList) name ; under cursor p and typing context TC_0` .
 - ii. Result in the updated typing context TC_1 and `instantiationIR`.
 - b. Else if let `prefixedTypeName < typeArgumentList > be type`:
 - i. Let the updated typing context TC_1 and `instantiationIR` be
 - the result of typing `annotationList prefixedTypeName < typeArgumentList > ( argumentList ) name ; under cursor p and typing context  $TC_0$` .
 - ii. Result in the updated typing context TC_1 and `instantiationIR`.
2. Else:
 - a. Let `annotationList type ( argumentList ) name = { objectDeclarationList } ; be instantiation`.
 - b. If let `prefixedTypeName be type`:
 - i. Let the updated typing context TC_1 and `instantiationIR` be
 - the result of typing `annotationList prefixedTypeName < `EMPTY > (argumentList) name = { objectDeclarationList } ; under cursor p and typing context TC_0` .
 - ii. Result in the updated typing context TC_1 and `instantiationIR`.
 - c. Else if let `prefixedTypeName < typeArgumentList > be type`:
 - i. Let the updated typing context TC_1 and `instantiationIR` be

- the result of **typing** `annotationList prefixedTypeName < typeArgumentList > ( argumentList ) name = { objectDeclarationList } ;` under cursor `p` and typing context `TC0`.
- ii. Result in the updated typing context `TC1` and `instantiationIR`.

When used as a top-level declaration, instantiations are typed with:

Typing `instantiation` under typing context `TC0`:

1. Let the updated typing context `TC1` and `instantiationIR` be
 - the result of **typing** `instantiation`, under cursor `GLOBAL` and typing context `TC0`.
2. Result in the updated typing context `TC1` and `instantiationIR`.

There are two sub-relations, depending on whether object initializers were provided or not.

When object initializers are absent

InstDecl_non_objectInitializer_ok:

- Typing `annotationList prefixedTypeName < typeArgumentList > ( argumentList ) name ;` under cursor `cursor` and typing context `typingContext`:
- Results in the updated typing context `typingContext` and `instantiationIR`.

Typing `annotationList prefixedTypeName < typeArgumentList > ( argumentList ) name ;` under cursor `p` and typing context `TC0`:

1. Let `typeArgumentIR*` and a set of fresh type variables `typeIdimpl*` be
 - the result of **typing** `typeArgumentList`, under cursor `p` and typing context `TC0`.
2. Let `argumentIR*` be
 - the result of **typing** `argumentList`, under cursor `p` and typing context `TC0`.
3. Let `prefixedNameIR` be the **prefixed name of** `prefixedTypeName`.
4. Let `constructorTargetIR` be `prefixedNameIR < typeArgumentIR* >`.
5. Let `constructorTypeIR` with fresh type variables `typeIdinserted*`, defaults `iddefault*` and optionals `idoptional*` be
 - the result of **finding the type of constructor call** `constructorTargetIR ( argumentIR* )`, under cursor `p` and typing context `TC0`.
6. Let `typeIdinfer*` be `typeIdimpl*` concatenated with `typeIdinserted*`.
7. Let constructed type `typeIRobject` with inferred `typeArgumentIRinferred*` and cast-inserted `argumentIRcast*` be
 - the result of **typing the instantiation** `constructorTypeIR < typeArgumentIR* > ( argumentIR* )` with omitted type arguments `typeIdinfer*`, defaults `iddefault*` and optionals `idoptional*`, under cursor `p`, typing context `TC0`, and NAMED instantiation site context.
8. Let `nameIR` be the **name of** `name`.
9. Let `TC1` be `TC0` where `nameIR to `EMPTY typeIRobject CTK`` is added to the `p` layer.
10. Let `constructorTargetIRinferred` be `prefixedNameIR < typeArgumentIRinferred* >`.
11. Let `instantiationIR` be `annotationList typeIRobject constructorTargetIRinferred ( argumentIRcast* ) nameIR · ;`.

12. Result in the updated typing context TC_1 and instantiationIR.

When object initializers are present

InstDecl_objectInitializer_ok:

- Typing `annotationList prefixedTypeName < typeArgumentList > ( argumentList ) name = { objectDeclarationList }` ; under cursor `cursor` and typing context `typingContext`:
- Results in the updated typing context `typingContext` and instantiationIR.

Typing `annotationList prefixedTypeName < typeArgumentList > ( argumentList ) name = { objectDeclarationList }` ; under cursor `p` and typing context TC_0 :

1. Let $typeArgumentIR^*$ and a set of fresh type variables $typeId_{impl}^*$ be
 - the result of typing `typeArgumentList`, under cursor `p` and typing context TC_0 .
2. Let $argumentIR^*$ be
 - the result of typing `argumentList`, under cursor `p` and typing context TC_0 .
3. Let $prefixedNameIR$ be the prefixed name of `prefixedTypeName`.
4. Let $constructorTargetIR$ be $prefixedNameIR < typeArgumentIR^* >$.
5. Let $constructorTypeIR$ with fresh type variables $typeId_{inserted}^*$, defaults $id_{default}^*$ and optionals $id_{optional}^*$ be
 - the result of finding the type of constructor call `constructorTargetIR ( argumentIR^* )`, under cursor `p` and typing context TC_0 .
6. Let $typeId_{infer}^*$ be $typeId_{impl}^*$ concatenated with $typeId_{inserted}^*$.
7. Let constructed type $typeIR_{object}$ with inferred $typeArgumentIR_{inferred}^*$ and cast-inserted $argumentIR_{cast}^*$ be
 - the result of typing the instantiation `constructorTypeIR < typeArgumentIR^* > ( argumentIR^* )` with omitted type arguments $typeId_{infer}^*$, defaults $id_{default}^*$ and optionals $id_{optional}^*$, under cursor `p`, typing context TC_0 , and NAMED instantiation site context.
8. Let $typeIR$ be $typeIR_{object}$ with typedefs unrolled.
9. Check that $typeIR$ has type `externObjectTypeIR`.
10. Let $EXTERN typeId_{extern} < typeIR_{arg}^* > externMethodTypeDefEnv_{extern}$ be $typeIR$.
11. Let TC_1 be TC_0 where "this" to `EMPTY typeIR_{object} CTK` · is added to the LOCAL layer.
12. Let $typeFrame_{init}$ and $externMethodTypeDefEnv_{init}$ of the object declaration block and $objectDeclarationIR^*$ be
 - the result of typing `objectDeclarationList` under cursor `p` and typing context TC_1 .
13. Let $externMethodTypeDefEnv_{init_subst}$ be substitution of abstract methods in $externMethodTypeDefEnv_{extern}$ by implementations in $externMethodTypeDefEnv_{init}$.
14. Let $typeIR_{object_init}$ be $EXTERN typeId_{extern} < typeIR_{arg}^* > externMethodTypeDefEnv_{init_subst}$.
15. Check that $typeIR_{object_init}$ is an extern object type without abstract methods.
16. Let $nameIR$ be the name of `name`.
17. Let TC_2 be TC_0 where `nameIR` to `EMPTY typeIR_{object\_init} CTK` · is added to the `p` layer.
18. Let $constructorTargetIR_{inferred}$ be $prefixedNameIR < typeArgumentIR_{inferred}^* >$.

19. Let `instantiationIR` be `annotationList typeIRobject_init constructorTargetIRinferred ( argumentIRcast* ) nameIR = { objectDeclarationIR* } ;`.
20. Result in the updated typing context `TC2` and `instantiationIR`.

11.4.3. Compile-time evaluation

Instantiations are evaluated at compile-time with the relation:

`InstDecl_inst`:

- Loading `instantiationIR`, under cursor `cursor`, instantiation context `instContext`, and store `store`:
- Results in
 - the updated instantiation context `instContext`, store `store`,
 - and the reference to the object `constantDeclarationIR`.

`Loading annotationList typeIR constructorTargetIR ( argumentListIR ) nameIR objectInitializerOptIR ;`, under cursor `p`, instantiation context `IC0`, and store `STO0`:

1. Let `prefixedNameIR < typeArgumentListIR >` be `constructorTargetIR`.
2. Let the constructor `constructorDef < typeArgumentListIRinst >` with defaults `iddefault*` and optionals `idoptional*` be
 - the result of `finding constructor for` `prefixedNameIR < typeArgumentListIR > ( argumentListIR )` under cursor `p`, and instantiation context `IC0`.
3. Let `ICinner` be `IC0` with `nameIR` added to the path.
4. Let the updated store `STO1`, and the constructed object be
 - the result of `loading` `constructorDef < typeArgumentListIRinst > ( argumentListIR )` with defaults `iddefault*` and optionals `idoptional*`, under cursor `p`, instantiation context `ICinner`, and store `STO0`.
5. If let `EXTERN typeld < theta > { externState frame externMethodDefEnv }` be object:
 - a. Let `ICdecl` be an instantiation context same as `IC0` up to the GLOBAL layer.
 - b. Let `objectDeclarationListIR` be the list of object declarations in `objectInitializerOptIR`.
 - c. Let the updated instantiation context `ICdecl_post` and store `STO2` be
 - the result of `loading` `objectDeclarationListIR` under instantiation context `ICdecl` and store `STO1`.
 - d. Let `framemerged` be the merged frame of frame and `ICdecl_post.BLOCK.FRAME`.
 - e. Let `externMethodDefEnvmerged` be the merged extern method definition environment of `externMethodDefEnv` and `ICdecl_post.BLOCK.CALLABLE`.
 - f. Let `objectmerged` be `EXTERN typeld < theta > { externState framemerged externMethodDefEnvmerged }`.
 - g. Let `objectId` be `IC0.PATH` concatenated with `nameIR`.
 - h. Let `STO3` be `STO2` where `objectId` to `objectmerged` is added.
 - i. Let `IC1` be `IC0` where `nameIR` to REF `objectId` is added to the `p` layer.
 - j. Let `constantDeclarationIR` be ``EMPTY CONST typeIR nameIR = `VALUE REF objectId ;`.
 - k. Result in
 - the updated instantiation context `IC1`, store `STO3`,

- and the reference to the object `constantDeclarationIR`.

6. Else:

- Let `objectId` be `IC0.PATH` concatenated with `nameIR`.
- Let `STO2` be `STO1` where `objectId` to object is added.
- Let `IC1` be `IC0` where `nameIR` to `REF objectId` is added to the `p` layer.
- Let `constantDeclarationIR` be ``EMPTY CONST typeIR nameIR = `VALUE REF objectId ;`.
- Result in
 - the updated instantiation context `IC1`, store `STO2`,
 - and the reference to the object `constantDeclarationIR`.

When used as a top-level declaration, instantiations are evaluated with:

Loading `instantiationIR`, under instantiation context `IC0`, and store `STO`:

- Let the updated instantiation context `IC1`, store `STO1`, and the reference to the object `_` be
 - the result of loading `instantiationIR`, under cursor `GLOBAL`, instantiation context `IC0`, and store `STO`.
- Result in the updated instantiation context `IC1` and the store `STO1`.

11.5. Function declarations

Function declarations introduce user-defined functions into a P4 program. [Section 9.4.1](#) describes the semantics of user-defined functions in P4.

```
functionPrototype
: typeOrVoid name typeParameterListOpt `( parameterList )
;

functionDeclaration
: annotationList functionPrototype blockStatement
;
```

11.5.1. Type checking

Function declarations are type checked with the relation:

`FuncDecl_ok`:

- Typing `functionDeclaration`, under cursor `cursor` and typing context `typingContext`:
- Results in the updated typing context `typingContext` and `functionDeclarationIR`.

After type checking, function declarations are represented in `P4IR` as:

```
functionPrototypeIR
: typeIR nameIR `typeParameterListIR , typeParameterListIR > `( parameterListIR )
;
```

```

functionDeclarationIR
: annotationList functionPrototypeIR blockStatementIR
;

```

They are produced by the following rule:

Typing annotationList typeOrVoid name typeParameterListOpt (parameterList) blockStatement, **under cursor** p **and typing context** TC_0 :

1. Let the updated typing context TC_1 and type parameters $typed_{expl}^*$ be
 - the result of **typing** typeParameterListOpt, **under cursor** LOCAL **and typing context** TC_0 .
2. Let $typeIR_{ret}$ and a set of fresh type variables $typed^*$ be
 - the result of **typing** typeOrVoid, **under cursor** LOCAL **and typing context** TC_1 .
3. Check that $typed^*$ is an empty list.
4. Let the updated typing context TC_2 , $parameterIR^*$, and a set of fresh type variables $typed_{impl}^*$ be
 - the result of **typing** parameterList, **under cursor** LOCAL **and typing context** TC_1 .
5. Let TC_3 be TC_2 with LOCAL.KIND set to FUNCTION : $typeIR_{ret}$.
6. Let the updated typing context $_$, abstract control flow f , and blockStatementIR be
 - the result of **typing** blockStatement, **under typing context** TC_3 , **abstract control flow** CONT, **and loop context** NOLOOP.
7. Check that f is equal to RET or $typeIR_{ret}$ is equal to VOID.
8. Let callableId be the callable identifier for name (parameterList).
9. Let nameIR be the name of name.
10. Let definedFunctionTypeDefIR be FUNCTION nameIR < $typed_{expl}^*$, $typed_{impl}^*$ > ($parameterIR^*$) : $typeIR_{ret}$.
11. Let bound be bound type variables from the p layer of TC_0 .
12. Check that definedFunctionTypeDefIR is a well-formed callable type definition, with bound type variables bound.
13. Let TC_4 be TC_0 where callableId to definedFunctionTypeDefIR is added as an overloaded callable to the p layer.
14. Let functionDeclarationIR be annotationList $typeIR_{ret}$ nameIR < $typed_{expl}^*$, $typed_{impl}^*$ > ($parameterIR^*$) blockStatementIR.
15. Result in the updated typing context TC_4 and functionDeclarationIR.

When used as a top-level declaration, function declarations are typed with:

Typing functionDeclaration **under typing context** TC_0 :

1. Let the updated typing context TC_1 and functionDeclarationIR be
 - the result of **typing** functionDeclaration, **under cursor** GLOBAL **and typing context** TC_0 .
2. Result in the updated typing context TC_1 and functionDeclarationIR.

11.5.2. Compile-time evaluation

At compile-time, function declarations are loaded into the context with the relation:

FuncDecl_inst:

- Loading **functionDeclarationIR**, under cursor **cursor**, and instantiation context **instContext**:
- Results in the updated instantiation context **instContext**.

It is defined by the following rule:

Loading **annotationList** **typeIR** **nameIR** < **typeParameterListIR_{expl}** , **typeParameterListIR_{impl}** > (**parameterListIR**) **blockStatementIR**, under cursor **p**, and instantiation context **IC₀**:

1. Let **callableId** be the callable identifier for **nameIR** (**parameterListIR**).
2. Let **typeParameterListIR** be **typeParameterListIR_{expl}** concatenated with **typeParameterListIR_{impl}**.
3. Let **definedFunctionDef** be **FUNCTION** **nameIR** < **typeParameterListIR** > (**parameterListIR**) **blockStatementIR**.
4. Let **IC₁** be **IC₀** where **callableId** to **definedFunctionDef** is added as an overloaded callable to the **p** layer.
5. Result in the updated instantiation context **IC₁**.

When used as a top-level declaration, function declarations are loaded with:

Loading **functionDeclarationIR**, under instantiation context **IC₀**, and store **STO**:

1. Let the updated instantiation context **IC₁** be
 - the result of **loading** **functionDeclarationIR**, under cursor **GLOBAL**, and instantiation context **IC₀**.
2. Result in the updated instantiation context **IC₁** and the store **STO**.

11.6. Action declarations

action declarations introduce actions:

```
actionDeclaration
: annotationList ACTION name `( parameterList ) blockStatement
;
```

Section 9.3 describes the semantics of actions.

11.6.1. Type checking

Action declarations are type checked with the relation:

ActionDecl_ok:

- Typing **actionDeclaration**, under cursor **cursor** and typing context **typingContext**:
- Results in the updated typing context **typingContext** and **actionDeclarationIR**.

After type checking, action declarations are represented in $P4_{IR}$ as:

```
actionDeclarationIR
: annotationList ACTION nameIR `( parameterListIR ) blockStatementIR
;
```

These are produced by the following rule:

Typing `annotationList ACTION name ( parameterList ) blockStatement`, under cursor `p` and typing context TC_0 :

1. Let TC_1 be TC_0 with `LOCAL.KIND` set to `ACTION`.
2. Let the updated typing context TC_2 , `parameterIR*`, and a set of fresh type variables `typed*` be
 - the result of **typing** `parameterList`, under cursor `LOCAL` and typing context TC_1 .
3. Check that `typed*` is an empty list.
4. Let the updated typing context $_$, abstract control flow $_$, and `blockStatementIR` be
 - the result of **typing** `blockStatement`, under typing context TC_2 , abstract control flow `CONT`, and loop context `NOLOOP`.
5. Let `callableId` be the callable identifier for `name ( parameterList )`.
6. Let `nameIR` be the name of `name`.
7. Let `actionTypeDefIR` be `ACTION nameIR ( parameterIR* )`.
8. Let `bound` be bound type variables from the `p` layer of TC_0 .
9. Check that `actionTypeDefIR` is a well-formed callable type definition, with bound type variables `bound`.
10. Let TC_3 be TC_0 where `callableId` to `actionTypeDefIR` is added as a non-overloaded callable to the `p` layer.
11. Let `actionDeclarationIR` be `annotationList ACTION nameIR ( parameterIR* ) blockStatementIR`.
12. Result in the updated typing context TC_3 and `actionDeclarationIR`.

When used as a top-level declaration, action declarations are typed with:

Typing `actionDeclaration` under typing context TC_0 :

1. Let the updated typing context TC_1 and `actionDeclarationIR` be
 - the result of **typing** `actionDeclaration`, under cursor `GLOBAL` and typing context TC_0 .
2. Result in the updated typing context TC_1 and `actionDeclarationIR`.

11.6.2. Compile-time evaluation

At compile-time, action declarations are loaded into the context with the relation:

ActionDecl_inst:

- Loading `actionDeclarationIR`, under cursor `cursor`, and instantiation context `instContext`:
- Results in the updated instantiation context `instContext`.

It is defined by the following rule:

Loading `annotationList ACTION nameIR ( parameterListIR ) blockStatementIR`, under cursor `p`, and instantiation context `IC0`:

1. Let `callableId` be the callable identifier for `nameIR ( parameterListIR )`.
2. Let `actionDef` be `ACTION nameIR ( parameterListIR ) blockStatementIR`.
3. Let `IC1` be `IC0` where `callableId` to `actionDef` is added as a non-overloaded callable to the `p` layer.
4. Result in the updated instantiation context `IC1`.

When used as a top-level declaration, action declarations are loaded with:

Loading `actionDeclarationIR`, under instantiation context `IC0`, and store `STO`:

1. Let the updated instantiation context `IC1` be
 - the result of loading `actionDeclarationIR`, under cursor `GLOBAL`, and instantiation context `IC0`.
2. Result in the updated instantiation context `IC1` and the store `STO`.

11.7. Error declarations

```
errorDeclaration
: ERROR { nameList }
;
```

`error` declarations introduce error elements. See [Section 8.2.4](#) for more details on error elements.

11.7.1. Type checking

After type checking, an `error` declaration is represented as:

```
errorDeclarationIR
: ERROR { nameListIR }
;
```

Typing `ERROR { nameList }` under typing context `TC0`:

1. Let `name*` be the list of names in `nameList`.
2. Let `nameIR*` be the list of `nameIR`, obtained by repeating:
 - Let `nameIR` be the name of `name`.for each `name` in `name*`
3. Check that the elements of `nameIR*` are distinct.
4. Let `nameIRerror*` be the list of `nameIRerror`, obtained by repeating:

- Let $\text{nameIR}_{\text{error}}$ be "error." concatenated with nameIR .
- for each nameIR in nameIR^*
5. Let $\text{value}_{\text{error}}^*$ be the list of $\text{value}_{\text{error}}$, obtained by repeating:
 - Let $\text{value}_{\text{error}}$ be $\text{ERROR} . \text{nameIR}$.

for each nameIR in nameIR^*
 6. Let varTypeIR^* be the list of varTypeIR , obtained by repeating:
 - Let varTypeIR be $\text{EMPTY ERROR LCTK value}_{\text{error}}$.

for each $\text{value}_{\text{error}}$ in $\text{value}_{\text{error}}^*$
 7. Let TC_1 be TC_0 where each of $\text{nameIR}_{\text{error}}^*$ to each of varTypeIR^* are added to the GLOBAL layer.
 8. Result in the updated typing context TC_1 and $\text{ERROR} \{ \text{nameIR}^* \}$.

11.7.2. Compile-time evaluation

At compile-time, error declarations are loaded into the context by:

Loading $\text{ERROR} \{ \text{nameIR}^* \}$, under instantiation context IC_0 , and store STO :

1. Let $\text{nameIR}_{\text{error}}^*$ be the list of $\text{nameIR}_{\text{error}}$, obtained by repeating:
 - Let $\text{nameIR}_{\text{error}}$ be the concatenation of ["error.", nameIR].

for each nameIR in nameIR^*
2. Let $\text{value}_{\text{error}}^*$ be the list of $\text{value}_{\text{error}}$, obtained by repeating:
 - Let $\text{value}_{\text{error}}$ be $\text{ERROR} . \text{nameIR}$.

for each nameIR in nameIR^*
3. Let IC_1 be IC_0 where each of $\text{nameIR}_{\text{error}}^*$ to each of $\text{value}_{\text{error}}^*$ are added to the GLOBAL layer.
4. Result in the updated instantiation context IC_1 and the store STO .

11.8. Match kind declarations

```
matchKindDeclaration
: MATCH_KIND `{ nameList trailingCommaOpt }
;
```

Similar to `error` declarations, `match_kind` declarations introduce match kinds. See [Section 8.2.5](#) for more details on match kinds.

11.8.1. Type checking

After type checking, an `match_kind` declaration is represented as:

```
matchKindDeclarationIR
: MATCH_KIND `{ nameListIR }
;
```

Typing `MATCH_KIND { nameList trailingCommaOpt }` under typing context TC_0 :

1. Let $name^*$ be the list of names in `nameList`.
2. Let $nameIR^*$ be the list of `nameIR`, obtained by repeating:
 - Let `nameIR` be the name of `name`.for each `name` in $name^*$
3. Check that the elements of $nameIR^*$ are distinct.
4. Let $value_{match_kind}^*$ be the list of `value_{match\_kind}`, obtained by repeating:
 - Let `value_{match\_kind}` be `MATCH_KIND . nameIR`.for each `nameIR` in $nameIR^*$
5. Let $varTypeIR^*$ be the list of `varTypeIR`, obtained by repeating:
 - Let `varTypeIR` be `EMPTY MATCH_KIND LCTK value_{match\_kind}`.for each `value_{match\_kind}` in $value_{match_kind}^*$
6. Let TC_1 be TC_0 where each of $nameIR^*$ to each of $varTypeIR^*$ are added to the GLOBAL layer.
7. Result in the updated typing context TC_1 and `MATCH_KIND { nameIR^* }`.

11.8.2. Compile-time evaluation

At compile-time, match kind declarations are loaded into the context by:

Loading `MATCH_KIND { nameIR^* }`, under instantiation context IC_0 , and store `STO`:

1. Let $value_{match_kind}^*$ be the list of `value_{match\_kind}`, obtained by repeating:
 - Let `value_{match\_kind}` be `MATCH_KIND . nameIR`.for each `nameIR` in $nameIR^*$
2. Let IC_1 be IC_0 where each of $nameIR^*$ to each of $value_{match_kind}^*$ are added to the GLOBAL layer.
3. Result in the updated instantiation context IC_1 and the store `STO`.

11.9. Extern function declarations

```
externFunctionDeclaration
: annotationList EXTERN functionPrototype ;
;
```

An extern function declaration introduces an extern function. See [Section 9.4.2](#) for how extern functions are internally represented.

11.9.1. Type checking

After type checking, an extern function declaration is represented in $P4_{IR}$ as:

```
externFunctionDeclarationIR
: annotationList EXTERN functionPrototypeIR ;
;
```

Typing `annotationList EXTERN typeOrVoid name typeParameterListOpt ( parameterList ) ;` under typing context TC_0 :

1. Let the updated typing context TC_1 and type parameters $typeld_{expl}^*$ be
 - the result of **typing** `typeParameterListOpt`, under cursor `LOCAL` and typing context TC_0 .
2. Let the updated typing context TC_2 , $parameterIR^*$, and a set of fresh type variables $typeld_{impl}^*$ be
 - the result of **typing** `parameterList`, under cursor `LOCAL` and typing context TC_1 .
3. Let $typeIR_{ret}$ and a set of fresh type variables $typeld^*$ be
 - the result of **typing** `typeOrVoid`, under cursor `LOCAL` and typing context TC_1 .
4. Check that $typeld^*$ is an empty list.
5. Let `callableId` be the callable identifier for `name ( parameterList )`.
6. Let `nameIR` be the name of `name`.
7. Let `externFunctionTypeDefIR` be `EXTERN_FUNCTION nameIR < typeldexpl*, typeldimpl* > ( parameterIR* ) : typeIRret*`.
8. Let `bound` be bound type variables from the GLOBAL layer of TC_0 .
9. Check that `externFunctionTypeDefIR` is a well-formed callable type definition, with bound type variables `bound`.
10. Let TC_4 be TC_0 where `callableId` to `externFunctionTypeDefIR` is added as an overloaded callable to the GLOBAL layer.
11. Let `externFunctionDeclarationIR` be `annotationList EXTERN typeIRret nameIR < typeldexpl*, typeldimpl* > ( parameterIR* ) ;`.
12. Result in the updated typing context TC_4 and `externFunctionDeclarationIR`.

11.9.2. Compile-time evaluation

At compile-time, extern function declarations are loaded into the context by:

Loading `annotationList EXTERN typeIR nameIR < typeParameterListIRexpl, typeParameterListIRimpl > ( parameterListIR )`

;; under instantiation context IC_0 , and store STO :

1. Let `callableId` be the callable identifier for `nameIR ( parameterListIR )`.
2. Let `typeParameterListIR` be `typeParameterListIRexpl` concatenated with `typeParameterListIRimpl`.
3. Let `externFunctionDef` be `EXTERN_FUNCTION nameIR < typeParameterListIR > ( parameterListIR )`.
4. Let IC_1 be IC_0 where `callableId` to `externFunctionDef` is added as an overloaded callable to the GLOBAL layer.
5. Result in the updated instantiation context IC_1 and the store STO .

11.10. Extern object declarations

An extern object declaration introduces an extern object.

```
externObjectDeclaration
  : annotationList EXTERN nonTypeName typeParameterListOpt `{
  externConstructorOrMethodPrototypeList }
  ;

externConstructorOrMethodPrototype
  : externConstructorPrototype
  | externMethodPrototype
  ;

externConstructorOrMethodPrototypeList
  : /* empty */
  | externConstructorOrMethodPrototypeList externConstructorOrMethodPrototype
  ;
```

See [Section 10.3](#) for how extern objects are internally represented. An extern object declaration includes its constructors and methods, to be explained in the following sections.

11.10.1. Extern methods

Extern objects can have methods, which are declared using the following syntax:

```
externMethodPrototype
  : annotationList functionPrototype ;
  | annotationList ABSTRACT functionPrototype ;
  ;
```

See [Section 9.5.1](#) for details on how extern methods are represented internally.

11.10.1.1. Type checking

Extern methods are type checked using the following relation:

[ExternMethod_ok](#):

- Typing `externMethodPrototype`, under typing context `typingContext` for an extern object `typed`:
- Results in `externMethodPrototypeIR`.

After type checking, in $P4_{IR}$, an extern method declaration is represented as:

```
externMethodPrototypeIR
: annotationList functionPrototypeIR ;
| annotationList ABSTRACT functionPrototypeIR ;
;
```

These are produced by:

Typing `externMethodPrototype`, under typing context TC_0 for an extern object `typedextern`:

1. If let `annotationList functionPrototype ;` be `externMethodPrototype`:
 - a. Let `typeOrVoid name typeParameterListOpt ( parameterList )` be `functionPrototype`.
 - b. Let `nameIR` be the name of `name`.
 - c. Check that `nameIR` is not equal to `typedextern`.
 - d. Let the updated typing context TC_1 and type parameters `typedexpl*` be
 - the result of typing `typeParameterListOpt`, under cursor `LOCAL` and typing context TC_0 .
 - e. Let `typedret` and a set of fresh type variables `typed*` be
 - the result of typing `typeOrVoid`, under cursor `LOCAL` and typing context TC_1 .
 - f. Check that `typed*` is an empty list.
 - g. Let TC_2 be TC_1 with `LOCAL.KIND` set to `EXTERN_METHOD : typedret`.
 - h. Let the updated typing context TC_3 , `parameterIR*`, and a set of fresh type variables `typedimpl*` be
 - the result of typing `parameterList`, under cursor `LOCAL` and typing context TC_2 .
 - i. Let `externMethodTypeDefIR` be `EXTERN_METHOD nameIR < typedexpl*, typedimpl* > ( parameterIR* ) : typedret`.
 - j. Let `bound` be bound type variables from the `BLOCK` layer of TC_0 .
 - k. Check that `externMethodTypeDefIR` is a well-formed callable type definition, with bound type variables `bound`.
 - l. Let `externMethodPrototypeIR` be `annotationList typedret nameIR < typedexpl*, typedimpl* > ( parameterIR* ) ;`.
 - m. Result in `externMethodPrototypeIR`.
2. Else:
 - a. Let `annotationList ABSTRACT functionPrototype ;` be `externMethodPrototype`.
 - b. Let `typeOrVoid name typeParameterListOpt ( parameterList )` be `functionPrototype`.
 - c. Let `nameIR` be the name of `name`.
 - d. Check that `nameIR` is not equal to `typedextern`.
 - e. Let the updated typing context TC_1 and type parameters `typedexpl*` be
 - the result of typing `typeParameterListOpt`, under cursor `LOCAL` and typing context TC_0 .
 - f. Let `typedret` and a set of fresh type variables `typed*` be
 - the result of typing `typeOrVoid`, under cursor `LOCAL` and typing context TC_1 .
 - g. Check that `typed*` is an empty list.
 - h. Let TC_2 be TC_1 with `LOCAL.KIND` set to `EXTERN_METHOD ABSTRACT : typedret`.
 - i. Let the updated typing context TC_3 , `parameterIR*`, and a set of fresh type variables `typedimpl*` be

- the result of **typing** `parameterList`, under cursor `LOCAL` and typing context `TC2`.
- j. Let `externMethodTypeDefIR` be `EXTERN_METHOD ABSTRACT nameIR < typeIdexpl*, typeIdimpl* > ( parameterIR* ) : typeIRret*`.
- k. Let `bound` be **bound type variables from the BLOCK layer of TC₀**.
- l. Check that `externMethodTypeDefIR` is a well-formed callable type definition, with bound type variables `bound`.
- m. Let `externMethodPrototypeIR` be `annotationList ABSTRACT typeIRret nameIR < typeIdexpl*, typeIdimpl* > ( parameterIR* ) ;`.
- n. Result in `externMethodPrototypeIR`.

11.10.2. Extern constructors

Extern objects may have constructors, which are declared using the following syntax:

```
externConstructorPrototype
: annotationList typeIdIdentifier `( parameterList ) ;
;
```

11.10.2.1. Type checking

Extern constructors are type checked using the following relation:

ExternConstructor_ok:

- Typing `externConstructorPrototype`, under typing context `typingContext` for an extern object `typeId`:
- Results in `externConstructorPrototypeIR`.

After type checking, in `P4IR`, an extern constructor declaration is represented as:

```
externConstructorPrototypeIR
: annotationList nameIR `< , typeParameterListIR > `( parameterListIR ) ;
;
```

These are produced by:

Typing `annotationList typeIdIdentifier ( parameterList ) ;`, under typing context `TC0` for an extern object `typeIdextern`:

1. Let `nameIR` be **the name of** `typeIdIdentifier`.
2. Check that `nameIR` is equal to `typeIdextern`.
3. Let the updated typing context `TC1`, `constructorParameterIR*`, and a set of fresh type variables `typeIdimpl*` be
 - the result of **typing** `( parameterList )` under typing context `TC0`.
4. Let `typeDefIR'` be **! the type definition of** `typeIdextern` **from the GLOBAL layer of TC₀**.
5. Check that `typeDefIR'` has type `externObjectTypeDefIR`.
6. Let `externObjectTypeDefIR` be `typeDefIR'`.

7. Let $(\text{typed}_{\text{expl}}^*, \text{typeParameterIR}^*)$ be the type parameters of $\text{externObjectTypeDefIR}$.
8. Check that typeParameterIR^* is an empty list.
9. Let $\text{typeIR}_{\text{extern}}$ be the underlying type of $\text{externObjectTypeDefIR}$.
10. Let $\text{constructorTypeDefIR}$ be $\text{CONSTRUCTOR} < \text{typed}_{\text{expl}}^*, \text{typed}_{\text{impl}}^* > (\text{constructorParameterIR}^*) : \text{typeIR}_{\text{extern}}$.
11. Let bound be bound type variables from the BLOCK layer of TC_0 .
12. Check that $\text{constructorTypeDefIR}$ is a well-formed constructor type definition, with bound type variables bound .
13. Let $\text{externConstructorPrototypeIR}$ be $\text{annotationList nameIR} < , \text{typed}_{\text{impl}}^* > (\text{constructorParameterIR}^*) ;$.
14. Result in $\text{externConstructorPrototypeIR}$.

11.10.3. Type checking

After type checking, an extern object declaration is represented in P4_{IR} as:

```
externObjectDeclarationIR
: annotationList EXTERN nameIR `< typeParameterListIR , > `{
externConstructorPrototypeListIR externMethodPrototypeListIR }
;

externConstructorPrototypeIR
: annotationList nameIR `< , typeParameterListIR > `( parameterListIR ) ;
;

externMethodPrototypeIR
: annotationList functionPrototypeIR ;
| annotationList ABSTRACT functionPrototypeIR ;
;
```

Typing $\text{annotationList EXTERN nonTypeName typeParameterListOpt} \{ \text{externConstructorOrMethodPrototypeList} \}$ under typing context TC_0 :

1. Let $(\text{externConstructorPrototype}^*, \text{externMethodPrototype}^*)$ be $\text{externConstructorOrMethodPrototypeList}$ split into constructors and methods.
2. Let TC_1 be TC_0 with BLOCK.KIND set to EXTERN .
3. Let the updated typing context TC_2 and type parameters $\text{typed}_{\text{expl}}^*$ be
 - the result of typing $\text{typeParameterListOpt}$, under cursor BLOCK and typing context TC_1 .
4. Let nameIR be the name of nonTypeName .
5. Let $\text{externMethodPrototypeIR}^*$ be the list of $\text{externMethodPrototypeIR}$, obtained by repeating:
 - Let $\text{externMethodPrototypeIR}$ be
 - the result of typing $\text{externMethodPrototype}$, under typing context TC_2 for an extern object nameIR .

for each $\text{externMethodPrototype}$ in $\text{externMethodPrototype}^*$
6. Let $\text{callabled}_{\text{method}}^*$ be the list of $\text{callabled}_{\text{method}}$, obtained by repeating:
 - Let $\text{callabled}_{\text{method}}$ be the callable id of $\text{externMethodPrototype}$.

for each $\text{externMethodPrototype}$ in $\text{externMethodPrototype}^*$

7. Check that the elements of $\text{callableId}_{\text{method}}^*$ are distinct.

8. Let $\text{externMethodTypeDefIR}^*$ be the list of $\text{externMethodTypeDefIR}$, obtained by repeating:

- Let $\text{externMethodTypeDefIR}$ be the callable type definition of $\text{externMethodPrototypeIR}$.

for each $\text{externMethodPrototypeIR}$ in $\text{externMethodPrototypeIR}^*$

9. Let $\text{externMethodTypeDefEnv}$ be $\{ (\text{callableId}_{\text{method}} : \text{externMethodTypeDefIR})^* \}$.

10. Let $\text{typeDefIR}_{\text{extern}}$ be $\text{EXTERN nameIR} < \text{typeId}_{\text{expl}}^*, \cdot > \text{externMethodTypeDefEnv}$.

11. Let TC_3 be TC_0 where nameIR to $\text{typeDefIR}_{\text{extern}}$ is added to the GLOBAL layer.

12. Let TC_4 be TC_3 with BLOCK.KIND set to EXTERN .

13. Let TC_5 be type parameters $\text{typeId}_{\text{expl}}^*$ added to the BLOCK layer of TC_4 .

14. Let $\text{externConstructorPrototypeIR}^*$ be the list of $\text{externConstructorPrototypeIR}$, obtained by repeating:

- Let $\text{externConstructorPrototypeIR}$ be
 - the result of typing $\text{externConstructorPrototype}$, under typing context TC_5 for an extern object nameIR .

for each $\text{externConstructorPrototype}$ in $\text{externConstructorPrototype}^*$

15. Let constructorId^* be the list of constructorId , obtained by repeating:

- Let constructorId be the constructor id of $\text{externConstructorPrototype}$.

for each $\text{externConstructorPrototype}$ in $\text{externConstructorPrototype}^*$

16. Let $\text{typeIR}_{\text{extern}}$ be the underlying type of $\text{typeDefIR}_{\text{extern}}$.

17. Let $\text{constructorTypeDefIR}^*$ be the list of $\text{constructorTypeDefIR}$, obtained by repeating:

- Let $\text{constructorTypeDefIR}$ be the constructor type definition of $\text{externConstructorPrototypeIR}$ with type parameters $\text{typeId}_{\text{expl}}^*$ and type $\text{typeIR}_{\text{extern}}$.

for each $\text{externConstructorPrototypeIR}$ in $\text{externConstructorPrototypeIR}^*$

18. Let TC_6 be where each of constructorId^* to each of $\text{constructorTypeDefIR}^*$ are added to TC_3 .

19. Let $\text{externObjectDeclarationIR}$ be $\text{annotationList EXTERN nameIR} < \text{typeId}_{\text{expl}}^*, > \{ \text{externConstructorPrototypeIR}^* \text{externMethodPrototypeIR}^* \}$.

20. Result in the updated typing context TC_6 and $\text{externObjectDeclarationIR}$.

11.10.4. Compile-time evaluation

At compile-time, extern object declarations are loaded into the context by:

Loading $\text{annotationList EXTERN nameIR} < \text{typeParameterListIR}, > \{ \text{externConstructorPrototypeIR}^* \text{externMethodPrototypeIR}^* \}$, under instantiation context IC_0 , and store STO :

1. If $\text{externConstructorPrototypeIR}^*$ is equal to $\cdot$:

- Let $\text{callableId}_{\text{method}}^*$ be the list of $\text{callableId}_{\text{method}}$, obtained by repeating:

- Let $\text{callableId}_{\text{method}}$ be the callable id of $\text{externMethodPrototypeIR}$.
- for each $\text{externMethodPrototypeIR}$ in $\text{externMethodPrototypeIR}^*$
- b. Let $\text{externMethodTypeDefIR}^*$ be the list of $\text{externMethodTypeDefIR}$, obtained by repeating:
 - Let $\text{externMethodTypeDefIR}$ be the callable type definition of $\text{externMethodPrototypeIR}$.

for each $\text{externMethodPrototypeIR}$ in $\text{externMethodPrototypeIR}^*$
 - c. Let $\text{externMethodTypeDefEnv}$ be $\{ (\text{callableId}_{\text{method}} : \text{externMethodTypeDefIR})^* \}$.
 - d. Let $\text{externObjectTypeDefIR}$ be $\text{EXTERN nameIR} < \text{typeParameterListIR}, \cdot > \text{externMethodTypeDefEnv}$.
 - e. Let IC_1 be IC_0 where nameIR to $\text{externObjectTypeDefIR}$ is added to the GLOBAL layer.
 - f. Let $\text{callableId}_{\text{constructor}}$ be the callable identifier for $\text{nameIR}(\cdot)$.
 - g. Let constructorDef be $\text{EXTERN nameIR} < \text{typeParameterListIR} > (\cdot) \{ \text{externMethodPrototypeIR}^* \}$.
 - h. Let IC_2 be where $\text{callableId}_{\text{constructor}}$ to constructorDef is added to IC_1 .
 - i. Result in the updated instantiation context IC_2 and the store STO .
2. Else:
- a. Let $\text{callableId}_{\text{method}}^*$ be the list of $\text{callableId}_{\text{method}}$, obtained by repeating:
 - Let $\text{callableId}_{\text{method}}$ be the callable id of $\text{externMethodPrototypeIR}$.

for each $\text{externMethodPrototypeIR}$ in $\text{externMethodPrototypeIR}^*$
 - b. Let $\text{externMethodTypeDefIR}^*$ be the list of $\text{externMethodTypeDefIR}$, obtained by repeating:
 - Let $\text{externMethodTypeDefIR}$ be the callable type definition of $\text{externMethodPrototypeIR}$.

for each $\text{externMethodPrototypeIR}$ in $\text{externMethodPrototypeIR}^*$
 - c. Let $\text{externMethodTypeDefEnv}$ be $\{ (\text{callableId}_{\text{method}} : \text{externMethodTypeDefIR})^* \}$.
 - d. Let $\text{externObjectTypeDefIR}$ be $\text{EXTERN nameIR} < \text{typeParameterListIR}, \cdot > \text{externMethodTypeDefEnv}$.
 - e. Let IC_1 be IC_0 where nameIR to $\text{externObjectTypeDefIR}$ is added to the GLOBAL layer.
 - f. Let $\text{callableId}_{\text{constructor}}^*$ be the list of $\text{callableId}_{\text{constructor}}$, obtained by repeating:
 - Let $\text{callableId}_{\text{constructor}}$ be the constructor id of $\text{externConstructorPrototypeIR}$.

for each $\text{externConstructorPrototypeIR}$ in $\text{externConstructorPrototypeIR}^*$
 - g. Let constructorDef^* be the list of constructorDef , obtained by repeating:
 - Let constructorDef be the constructor definition of $\text{externConstructorPrototypeIR}$ with type parameters $\text{typeParameterListIR}$ and methods $\text{externMethodPrototypeIR}^*$.

for each $\text{externConstructorPrototypeIR}$ in $\text{externConstructorPrototypeIR}^*$
 - h. Let IC_2 be where each of $\text{callableId}_{\text{constructor}}^*$ to each of constructorDef^* are added to IC_1 .
 - i. Result in the updated instantiation context IC_2 and the store STO .

11.11. Parser declarations

A **parser** declaration introduces a programmable parser:

```
parserDeclaration
  : annotationList PARSE name typeParameterListOpt `( parameterList )
  constructorParameterListOpt `{ parserLocalDeclarationList parserStateList }
  ;

parserLocalDeclaration
  : constantDeclaration
  | instantiation
  | variableDeclaration
  | valueSetDeclaration
  ;

parserState
  : annotationList STATE name `{ parserStatementList transitionStatement }
  ;
```

11.11.1. Type checking

After type checking, a parser declaration is represented in $P4_{IR}$ as:

```
parserDeclarationIR
  : annotationList PARSE nameIR `< typeParameterListIR , typeParameterListIR > `(
  parameterListIR ) `( constructorParameterListIR ) `{ parserLocalDeclarationListIR
  parserStateListIR }
  ;

parserLocalDeclarationIR
  : constantDeclarationIR
  | instantiationIR
  | variableDeclarationIR
  | valueSetDeclarationIR
  ;

parserStateIR
  : annotationList STATE nameIR `{ parserStatementListIR transitionStatementIR }
  ;
```

Typing annotationList PARSE name typeParameterListOpt (parameterList) constructorParameterListOpt { parserLocalDeclarationList parserStateList } **under typing context** TC_0 :

1. Let TC_1 be TC_0 with **BLOCK.KIND** set to **PARSER**.
2. Let the updated typing context TC_2 and type parameters $typeld_{expl}^*$ be
 - the result of **typing** typeParameterListOpt, **under cursor** **BLOCK** and **typing context** TC_1 .
3. Let the updated typing context TC_3 , $constructorParameterIR^*$, and a set of fresh type variables $typeld_{impl}^*$ be
 - the result of **typing** constructorParameterListOpt **under typing context** TC_2 .
4. Let the updated typing context TC_4 , $parameterIR^*$, and a set of fresh type variables $typeld^*$ be
 - the result of **typing** parameterList, **under cursor** **BLOCK** and **typing context** TC_3 .
5. Check that $typeld^*$ is an empty list.
6. Let the updated typing context TC_5 and the typed parser local declarations $parserLocalDeclarationIR^*$ be
 - the result of **typing** parserLocalDeclarationList, **under typing context** TC_4 .

7. Let TC_6 be TC_5 with `LOCAL.KIND` set to `PARSER_STATE`.
8. Let the typed parser states $parserStateIR^*$ be
 - the result of `typing parserStateList`, under typing context TC_6 .
9. Let `callableTypeDefIR` be `PARSER_APPLY ( parameterIR^* )`.
10. Let `bound` be `bound type variables from the BLOCK layer of  $TC_2$` .
11. Check that `callableTypeDefIR` is a well-formed callable type definition, with bound type variables `bound`.
12. Let $nameIR$ be `the name of name`.
13. Let `constructorId` be `the constructor identifier for name constructorParameterListOpt`.
14. Let $typeDefIR_{parser}$ be `PARSER  $nameIR < typeId_{expl}^*, typeId_{impl}^* > ( parameterIR^* )$` .
15. Let $typeIR_{parser}$ be `the underlying type of  $typeDefIR_{parser}$` .
16. Let `constructorTypeDefIR` be `CONSTRUCTOR  $< typeId_{expl}^*, typeId_{impl}^* > ( constructorParameterIR^* ) : typeIR_{parser}$` .
17. Let `bound` be `bound type variables from the GLOBAL layer of  $TC_0$` .
18. Check that `constructorTypeDefIR` is a well-formed constructor type definition, with bound type variables `bound`.
19. Let TC_7 be `where  $constructorId$  to  $constructorTypeDefIR$  is added to  $TC_0$` .
20. Let $parserDeclarationIR$ be `annotationList PARSER  $nameIR < typeId_{expl}^*, typeId_{impl}^* > ( parameterIR^* ) ( constructorParameterIR^* ) \{ parserLocalDeclarationIR^* parserStateIR^* \}$` .
21. Result in the updated typing context TC_7 and $parserDeclarationIR$.

11.11.2. Compile-time evaluation

At compile-time, parser declarations are loaded into the context as parser object constructors.

Loading `annotationList PARSER  $nameIR < typeParameterListIR_{expl}, typeParameterListIR_{impl} > ( parameterListIR ) ( constructorParameterListIR ) \{ parserLocalDeclarationListIR parserStateListIR \}$` , under instantiation context IC_0 , and store STO :

1. Let `callableId` be `the callable identifier for  $nameIR ( constructorParameterListIR )$` .
2. Let `typeParameterListIR` be `$typeParameterListIR_{expl}$  concatenated with  $typeParameterListIR_{impl}$` .
3. Let `constructorDef` be `PARSER  $< typeParameterListIR > ( parameterListIR ) ( constructorParameterListIR ) \{ parserLocalDeclarationListIR parserStateListIR \}$` .
4. Let IC_1 be `where  $callableId$  to  $constructorDef$  is added to  $IC_0$` .
5. Result in the updated instantiation context IC_1 and the store STO .

When instantiated, an instance of a parser declaration is created:

```
parserObject
  : PARSER `< theta > `( parameterListIR ) `{ frame parserLocalDeclarationListIR
stateEnv }
  ;
```

See [Section 10.4](#) for details about parser objects.

11.12. Control declarations

A **control** declaration introduces a control block:

```
controlDeclaration
  : annotationList CONTROL name typeParameterListOpt `( parameterList )
  constructorParameterListOpt `{ controlLocalDeclarationList APPLY controlBody }
  ;

controlLocalDeclaration
  : constantDeclaration
  | instantiation
  | variableDeclaration
  | actionDeclaration
  | tableDeclaration
  ;

controlBody = blockStatement
```

11.12.1. Type checking

After type checking, a control declaration is represented in $P4_{IR}$ as:

```
controlDeclarationIR
  : annotationList CONTROL nameIR `< typeParameterListIR , typeParameterListIR > `(
  parameterListIR ) `( constructorParameterListIR ) `{ controlLocalDeclarationListIR APPLY
  controlBodyIR }
  ;

controlLocalDeclarationIR
  : constantDeclarationIR
  | instantiationIR
  | variableDeclarationIR
  | actionDeclarationIR
  | tableDeclarationIR
  ;

controlBodyIR = blockStatementIR
```

Typing annotationList CONTROL name typeParameterListOpt (parameterList) constructorParameterListOpt { controlLocalDeclarationList APPLY controlBody } **under typing context** TC_0 :

1. Let TC_1 be TC_0 with **BLOCK.KIND** set to **CONTROL**.
2. Let the updated typing context TC_2 and type parameters $typed_{expl}^*$ be
 - the result of **typing** typeParameterListOpt, **under cursor** **BLOCK** **and typing context** TC_1 .
3. Let the updated typing context TC_3 , $constructorParameterIR^*$, and a set of fresh type variables $typed_{impl}^*$ be
 - the result of **typing** constructorParameterListOpt **under typing context** TC_2 .
4. Let the updated typing context TC_4 , $parameterIR^*$, and a set of fresh type variables $typed^*$ be
 - the result of **typing** parameterList, **under cursor** **BLOCK** **and typing context** TC_3 .
5. Check that $typed^*$ is an empty list.
6. Let the updated typing context TC_5 and the typed control local declarations $controlLocalDeclarationIR^*$ be

- the result of **typing** `controlLocalDeclarationList`, under typing context `TC4`.
- Let `TC6` be `TC5` with `LOCAL.KIND` set to `CONTROL_APPLY_METHOD`.
 - Let the updated typing context `_`, abstract control flow `_`, and `controlBodyIR` be
 - the result of **typing** `controlBody`, under typing context `TC6`, abstract control flow `CONT`, and loop context `NOLOOP`.
 - Let `callableTypeDefIR` be `CONTROL_APPLY ( parameterIR* )`.
 - Let `bound` be bound type variables from the `BLOCK` layer of `TC2`.
 - Check that `callableTypeDefIR` is a well-formed callable type definition, with bound type variables `bound`.
 - Let `nameIR` be the name of `name`.
 - Let `constructorId` be the constructor identifier for `name` `constructorParameterListOpt`.
 - Let `typeDefIRcontrol` be `CONTROL nameIR < typeIdexpl*, typeIdimpl* > ( parameterIR* )`.
 - Let `typeIRcontrol` be the underlying type of `typeDefIRcontrol`.
 - Let `constructorTypeDefIR` be `CONSTRUCTOR < typeIdexpl*, typeIdimpl* > ( constructorParameterIR* ) : typeIRcontrol`.
 - Let `bound` be bound type variables from the `GLOBAL` layer of `TC0`.
 - Check that `constructorTypeDefIR` is a well-formed constructor type definition, with bound type variables `bound`.
 - Let `TC7` be where `constructorId` to `constructorTypeDefIR` is added to `TC0`.
 - Let `controlDeclarationIR` be `annotationList CONTROL nameIR < typeIdexpl*, typeIdimpl* > ( parameterIR* ) ( constructorParameterIR* ) { controlLocalDeclarationIR* APPLY controlBodyIR }`.
 - Result in the updated typing context `TC7` and `controlDeclarationIR`.

11.12.2. Compile-time evaluation

At compile-time, control declarations are loaded into the context as control object constructors.

Loading `annotationList CONTROL nameIR < typeParameterListIRexpl, typeParameterListIRimpl > ( parameterListIR ) ( constructorParameterListIR ) { controlLocalDeclarationListIR APPLY controlBodyIR }`, under instantiation context `IC0`, and store `STO`:

- Let `callableId` be the callable identifier for `nameIR ( constructorParameterListIR )`.
- Let `typeParameterListIR` be `typeParameterListIRexpl` concatenated with `typeParameterListIRimpl`.
- Let `constructorDef` be `CONTROL < typeParameterListIR > ( parameterListIR ) ( constructorParameterListIR ) { controlLocalDeclarationListIR APPLY controlBodyIR }`.
- Let `IC1` be where `callableId` to `constructorDef` is added to `IC0`.
- Result in the updated instantiation context `IC1` and the store `STO`.

When instantiated, an instance of a control declaration is created:

```
controlObject
: CONTROL `< theta > `( parameterListIR ) `{ frame controlLocalDeclarationListIR
actionDefEnv controlBodyIR }
;
```

See [Section 10.5](#) for details about control objects.

11.13. Enum type declaration

An enumeration type is defined using the following syntax:

```
enumTypeDeclaration
: annotationList ENUM name `{ nameList trailingCommaOpt }
| annotationList ENUM type name `{ namedExpressionList trailingCommaOpt }
;

namedExpressionList
: namedExpression
| namedExpressionList , namedExpression
;

namedExpression
: name = expression
;
```

After type checking, enum type declarations are represented as follows:

```
enumTypeDeclarationIR
: annotationList ENUM nameIR `{ nameListIR }
| annotationList ENUM typeIR nameIR `{ namedValueListIR }
;

namedValueListIR = namedValueIR*

namedValueIR
: nameIR = value
;
```

See [Section 8.4.8](#) for more details on enums.

11.13.1. Enum type declarations without an underlying type

11.13.1.1. Type checking

Typing `annotationList ENUM name { nameListfield trailingCommaOpt }` under typing context TC_0 :

1. Let $nameIR$ be the name of $name$.
2. Let $name_{field}^*$ be the list of names in $nameList_{field}$.
3. Let $nameIR_{field}^*$ be the list of $nameIR_{field}$, obtained by repeating:
 - Let $nameIR_{field}$ be the name of $name_{field}$.for each $name_{field}$ in $name_{field}^*$
4. Let $enumTypeDefIR$ be `ENUM nameIR { nameIRfield* }`.
5. Let $bound$ be bound type variables from the GLOBAL layer of TC_0 .
6. Check that $enumTypeDefIR$ is a well-formed type definition, with bound type variables $bound$.
7. Let TC_1 be TC_0 where $nameIR$ to $enumTypeDefIR$ is added to the GLOBAL layer.

8. Let id_{field}^* be the list of id_{field} , obtained by repeating:
 - Let id_{field} be $nameIR$ concatenated with "." concatenated with $nameIR_{field}$.
 for each $nameIR_{field}$ in $nameIR_{field}^*$
9. Let $value_{field}^*$ be the list of $value_{field}$, obtained by repeating:
 - Let $value_{field}$ be $nameIR . nameIR_{field}$.
 for each $nameIR_{field}$ in $nameIR_{field}^*$
10. Let $varTypeIR^*$ be the list of $varTypeIR$, obtained by repeating:
 - Let $varTypeIR$ be `EMPTY enumTypeDefIR LCTK $value_{field}$.
 for each $value_{field}$ in $value_{field}^*$
11. Let TC_2 be TC_1 where each of id_{field}^* to each of $varTypeIR^*$ are added to the GLOBAL layer.
12. Let $enumTypeDeclarationIR$ be annotationList ENUM $nameIR \{ nameIR_{field}^* \}$.
13. Result in the updated typing context TC_2 and $enumTypeDeclarationIR$.

11.13.1.2. Compile-time evaluation

At compile-time, enum type declarations are loaded into the context by:

Loading annotationList ENUM $nameIR \{ nameIR_{field}^* \}$, under instantiation context IC_0 , and store STO :

1. Let $nameIR_{enum_field}^*$ be the list of $nameIR_{enum_field}$, obtained by repeating:
 - Let $nameIR_{enum_field}$ be $nameIR$ concatenated with "." concatenated with $nameIR_{field}$.
 for each $nameIR_{field}$ in $nameIR_{field}^*$
2. Let $value_{enum_field}^*$ be the list of $value_{enum_field}$, obtained by repeating:
 - Let $value_{enum_field}$ be $nameIR . nameIR_{field}$.
 for each $nameIR_{field}$ in $nameIR_{field}^*$
3. Let IC_1 be IC_0 where each of $nameIR_{enum_field}^*$ to each of $value_{enum_field}^*$ are added to the GLOBAL layer.
4. Let $enumTypeDefIR$ be ENUM $nameIR \{ nameIR_{field}^* \}$.
5. Let IC_2 be IC_1 where $nameIR$ to $enumTypeDefIR$ is added to the GLOBAL layer.
6. Result in the updated instantiation context IC_2 and the store STO .

11.13.2. Enum type declarations with an underlying type

11.13.2.1. Type checking

Typing annotationList ENUM type name { namedExpressionList_{field} trailingCommaOpt } under typing context TC_0 :

1. Let B be bound type variables from the GLOBAL layer of TC_0 .
2. Let $typeIR$ and a set of fresh type variables $typed^*$ be
 - the result of typing $type$, under cursor GLOBAL and typing context TC_0 .
3. Check that $typed^*$ is an empty list.
4. Check that $typeIR$ is a well-formed type, with bound type variables B .
5. Let $nameIR$ be the name of $name$.
6. Let the updated typing context TC_1 and field values $namedValueIR_{field}^*$ be
 - the result of typing $namedExpressionList_{field}$ under typing context TC_0 with enum name $nameIR$ and underlying $typeIR$.
7. Let $nameIR_{field}^*$ be the list of $nameIR_{field}$ and $value_{field}^*$ be the list of $value_{field}$, obtained by repeating:
 - Let $nameIR_{field} = value_{field}$ be $namedValueIR_{field}$.
 for each $namedValueIR_{field}$ in $namedValueIR_{field}^*$
8. Let id_{field}^* be the list of id_{field} , obtained by repeating:
 - Let id_{field} be $nameIR$ concatenated with "." concatenated with $nameIR_{field}$.
 for each $nameIR_{field}$ in $nameIR_{field}^*$
9. Let $enumTypeDefIR$ be $ENUM\ nameIR < typeIR > \{ (nameIR_{field} = value_{field} ;)^* \}$.
10. Let $varTypeIR^*$ be the list of $varTypeIR$, obtained by repeating:
 - Let $varTypeIR$ be $EMPTY\ enumTypeDefIR\ LCTK\ value_{field}$.
 for each $value_{field}$ in $value_{field}^*$
11. Let TC_2 be TC_0 where each of id_{field}^* to each of $varTypeIR^*$ are added to the GLOBAL layer.
12. Check that $enumTypeDefIR$ is a well-formed type definition, with bound type variables B .
13. Let TC_3 be TC_2 where $nameIR$ to $enumTypeDefIR$ is added to the GLOBAL layer.
14. Let $enumTypeDeclarationIR$ be $annotationList\ ENUM\ typeIR\ nameIR\ \{ namedValueIR_{field}^* \}$.
15. Result in the updated typing context TC_3 and $enumTypeDeclarationIR$.

An enum field is type-checked as follows:

Enum_serializable_field_ok:

- Typing $namedExpression$ under typing context $typingContext$ with enum name $nameIR$ and underlying $typeIR$:
- Results in the updated typing context $typingContext$, and field value $namedValueIR$.

Typing $name = expression$ under typing context TC_0 with enum name $nameIR_{enum}$ and underlying $typeIR$:

1. Let $typedExpressionIR$ be
 - the result of typing $expression$, under cursor BLOCK and typing context TC_0 .

2. Let $\text{typedExpressionIR}_{\text{cast}}$ be ! typedExpressionIR implicitly cast to typeIR .
3. Check that the compile-time known-ness of $\text{typedExpressionIR}_{\text{cast}}$ is LCTK.
4. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{cast}}$ under cursor BLOCK and typing context TC_0 .
5. Let nameIR be the name of name .
6. Let varTypeIR be `EMPTY typeIR LCTK value.
7. Let TC_1 be TC_0 where nameIR to varTypeIR is added to the BLOCK layer.
8. Result in the updated typing context TC_1 , and field value $\text{nameIR} = \text{value}$.

A list of enum fields is type-checked with the following relation:

Enum_serializable_fieldList_ok:

- Typing $\text{namedExpressionList}$ under typing context typingContext with enum name nameIR and underlying typeIR :
- Results in the updated typing context typingContext and field values namedValueIR^* .

11.13.2.2. Compile-time evaluation

At compile-time, enum type declarations are loaded into the context by:

Loading $\text{annotationList ENUM typeIR nameIR } \{ (\text{nameIR}_{\text{field}} = \text{value}_{\text{field}})^* \}$, under instantiation context IC_0 , and store STO :

1. Let $\text{nameIR}_{\text{serenum_field}}^*$ be the list of $\text{nameIR}_{\text{serenum_field}}$, obtained by repeating:
 - Let $\text{nameIR}_{\text{serenum_field}}$ be nameIR concatenated with "." concatenated with $\text{nameIR}_{\text{field}}$.
 for each $\text{nameIR}_{\text{field}}$ in $\text{nameIR}_{\text{field}}^*$
2. Let $\text{value}_{\text{serenum_field}}^*$ be the list of $\text{value}_{\text{serenum_field}}$, obtained by repeating:
 - Let $\text{value}_{\text{serenum_field}}$ be $\text{nameIR} . \text{nameIR}_{\text{field}} . \text{value}_{\text{field}}$.
 for each $\text{nameIR}_{\text{field}}$ in $\text{nameIR}_{\text{field}}^*$ and $\text{value}_{\text{field}}$ in $\text{value}_{\text{field}}^*$
3. Let IC_1 be IC_0 where each of $\text{nameIR}_{\text{serenum_field}}^*$ to each of $\text{value}_{\text{serenum_field}}^*$ are added to the GLOBAL layer.
4. Let enumTypeDefIR be $\text{ENUM nameIR} < \text{typeIR} > \{ (\text{nameIR}_{\text{field}} = \text{value}_{\text{field}} ;)^* \}$.
5. Let IC_2 be IC_1 where nameIR to enumTypeDefIR is added to the GLOBAL layer.
6. Result in the updated instantiation context IC_2 and the store STO .

11.14. Struct type declaration

A struct type is defined with the following syntax:

```

structTypeDeclaration
  : annotationList STRUCT name typeParameterListOpt `{ typeFieldList }
  ;

typeFieldList
  : /* empty */
  | typeFieldList typeField
  ;

typeField
  : annotationList type name ;
  ;

```

See [Section 8.4.5](#) for more information about struct types.

11.14.1. Type checking

After type checking, a struct type declaration is represented in $P4_{IR}$ as:

```

structTypeDeclarationIR
  : annotationList STRUCT nameIR `< typeParameterListIR > `{ typeFieldListIR }
  ;

typeFieldListIR = typeFieldIR*

typeFieldIR
  : annotationList typeIR nameIR ;
  ;

```

Typing annotationList STRUCT name typeParameterListOpt { typeFieldList } **under typing context** TC_0 :

1. Let the updated typing context TC_1 and type parameters $typed^*$ be
 - the result of **typing** typeParameterListOpt, **under cursor** BLOCK **and typing context** TC_0 .
2. Let $(annotationList_{field} type_{field} name_{field} ;)^*$ be the list of type fields in typeFieldList.
3. Let $typedR_{field}^*$ be the list of $typedR_{field}$ and $typed^{**}$ be the list of $typed^*$, obtained by repeating:
 - Let $typedR_{field}$ and a set of fresh type variables $typed^*$ be
 - the result of **typing** $type_{field}$, **under cursor** BLOCK **and typing context** TC_1 .

for each $type_{field}$ in $typed_{field}^*$
4. Check that $typed^*$ is an empty list, for all $typed^*$ in $typed^{**}$.
5. Let nameIR be the name of name.
6. Let $nameIR_{field}^*$ be the list of $nameIR_{field}$, obtained by repeating:
 - Let $nameIR_{field}$ be the name of $name_{field}$.

for each $name_{field}$ in $name_{field}^*$
7. Let structTypeDefIR be STRUCT nameIR < $typed^*$ > { $(annotationList_{field} typedR_{field} nameIR_{field} ;)^*$ }.
8. Let bound be bound type variables from the GLOBAL layer of TC_0 .
9. Check that structTypeDefIR is a well-formed type definition, with bound type variables bound.

10. Let TC_2 be TC_0 where $nameIR$ to $structTypeDefIR$ is added to the GLOBAL layer.
11. Let $structTypeDeclarationIR$ be $annotationList$ STRUCT $nameIR < typeId^* > \{ (annotationList_{field} typeIR_{field} nameIR_{field} ;)^* \}$.
12. Result in the updated typing context TC_2 and $structTypeDeclarationIR$.

11.14.2. Compile-time evaluation

At compile-time, struct type declarations are loaded into the context by:

Loading $annotationList$ STRUCT $nameIR < typeParameterListIR > \{ typeFieldIR^* \}$, under instantiation context IC_0 , and store STO :

1. Let $annotationList_{field}^*$ be the list of $annotationList_{field}$, $nameIR_{field}^*$ be the list of $nameIR_{field}$, and $typeIR_{field}^*$ be the list of $typeIR_{field}$, obtained by repeating:
 - Let $annotationList_{field} typeIR_{field} nameIR_{field} ;$ be $typeFieldIR$.
 for each $typeFieldIR$ in $typeFieldIR^*$
2. Let $structTypeDefIR$ be STRUCT $nameIR < typeParameterListIR > \{ (annotationList_{field} typeIR_{field} nameIR_{field} ;)^* \}$.
3. Let IC_1 be IC_0 where $nameIR$ to $structTypeDefIR$ is added to the GLOBAL layer.
4. Result in the updated instantiation context IC_1 and the store STO .

11.15. Header type declaration

A header type is defined with the following syntax:

```
headerTypeDeclaration
: annotationList HEADER name typeParameterListOpt `{ typeFieldList }
;

typeFieldList
: /* empty */
| typeFieldList typeField
;

typeField
: annotationList type name ;
;
```

See [Section 8.4.6](#) for more information about header types.

11.15.1. Type checking

After type checking, a header type declaration is represented in $P4_{IR}$ as:

```
headerTypeDeclarationIR
: annotationList HEADER nameIR `< typeParameterListIR > `{ typeFieldListIR }
;

typeFieldListIR = typeFieldIR*
```

```

typeFieldIR
: annotationList typeIR nameIR ;
;

```

Typing annotationList HEADER name typeParameterListOpt { typeFieldList } **under typing context** TC_0 :

1. Let the updated typing context TC_1 and type parameters $typed^*$ be
 - the result of **typing** typeParameterListOpt, **under cursor** BLOCK **and typing context** TC_0 .
2. Let (annotationList_{*f*} type_{*f*} name_{*f*} ;)^{*} be the list of type fields in typeFieldList.
3. Let $typeIR_f^*$ be the list of $typeIR_f$ and $typed^{**}$ be the list of $typed^*$, obtained by repeating:
 - Let $typeIR_f$ and a set of fresh type variables $typed^*$ be
 - the result of **typing** type_{*f*}, **under cursor** BLOCK **and typing context** TC_1 .
 for each type_{*f*} in type_{*f*}^{*}
4. Check that $typed^{**}$ is an empty list, for all $typed^*$ in $typed^{**}$.
5. Let nameIR be the name of name.
6. Let $nameIR_f^*$ be the list of $nameIR_f$, obtained by repeating:
 - Let $nameIR_f$ be the name of name_{*f*}.
 for each name_{*f*} in name_{*f*}^{*}
7. Let headerTypeDefIR be HEADER nameIR < typed^{*} > { (annotationList_{*f*} typeIR_{*f*} nameIR_{*f*} ;)^{*} }.
8. Let bound be bound type variables from the GLOBAL layer of TC_0 .
9. Check that headerTypeDefIR is a well-formed type definition, with bound type variables bound.
10. Let TC_2 be TC_0 where nameIR to headerTypeDefIR is added to the GLOBAL layer.
11. Let headerTypeDeclarationIR be annotationList HEADER nameIR < typed^{*} > { (annotationList_{*f*} typeIR_{*f*} nameIR_{*f*} ;)^{*} }.
12. Result in the updated typing context TC_2 and headerTypeDeclarationIR.

11.15.2. Compile-time evaluation

At compile-time, header type declarations are loaded into the context by:

Loading annotationList HEADER nameIR < typeParameterListIR > { typeFieldIR^{*} }, **under instantiation context** IC_0 , **and store** STO:

1. Let $annotationList_{field}^*$ be the list of $annotationList_{field}$, $nameIR_{field}^*$ be the list of $nameIR_{field}$, and $typeIR_{field}^*$ be the list of $typeIR_{field}$, obtained by repeating:
 - Let $annotationList_{field}$ typeIR_{*field*} nameIR_{*field*} ; be typeFieldIR.
 for each typeFieldIR in typeFieldIR^{*}
2. Let headerTypeDefIR be HEADER nameIR < typeParameterListIR > { (annotationList_{*field*} typeIR_{*field*} nameIR_{*field*} ;)^{*} }.
3. Let IC_1 be IC_0 where nameIR to headerTypeDefIR is added to the GLOBAL layer.

4. Result in the updated instantiation context IC_1 and the store STO .

11.16. Header union type declaration

A header union type is defined with the following syntax:

```
headerUnionTypeDeclaration
  : annotationList HEADER_UNION name typeParameterListOpt `{ typeFieldList }
  ;

typeFieldList
  : /* empty */
  | typeFieldList typeField
  ;

typeField
  : annotationList type name ;
  ;
```

See [Section 8.4.7](#) for more information about header union types.

11.16.1. Type checking

After type checking, a header union type declaration is represented in $P4_{IR}$ as:

```
headerUnionTypeDeclarationIR
  : annotationList HEADER_UNION nameIR `< typeParameterListIR > `{ typeFieldListIR }
  ;

typeFieldListIR = typeFieldIR*

typeFieldIR
  : annotationList typeIR nameIR ;
  ;
```

Typing `annotationList HEADER_UNION name typeParameterListOpt { typeFieldList }` under typing context TC_0 :

1. Let the updated typing context TC_1 and type parameters $typeld^*$ be
 - the result of **typing** `typeParameterListOpt`, under cursor `BLOCK` and typing context TC_0 .
2. Let `( annotationListi typei namei ; )*` be the list of type fields in `typeFieldList`.
3. Let $typeIR_i^*$ be the list of $typeIR_i$ and $typeld^{**}$ be the list of $typeld^*$, obtained by repeating:
 - Let $typeIR_i$ and a set of fresh type variables $typeld^*$ be
 - the result of **typing** `typei`, under cursor `BLOCK` and typing context TC_1 .for each $type_i$ in $type_i^*$
4. Check that $typeld^*$ is an empty list, for all $typeld^*$ in $typeld^{**}$.
5. Let $nameIR$ be the name of `name`.
6. Let $nameIR_i^*$ be the list of $nameIR_i$, obtained by repeating:
 - Let $nameIR_i$ be the name of `namei`.

for each name_f in name_f^*

7. Let $\text{headerUnionTypeDefIR}$ be $\text{HEADER_UNION nameIR} < \text{typeId}^* > \{ (\text{annotationList}_f \text{ typeIR}_f \text{ nameIR}_f ;)^* \}$.
8. Let bound be **bound type variables from the GLOBAL layer** of TC_0 .
9. Check that $\text{headerUnionTypeDefIR}$ is a well-formed type definition, with bound type variables bound .
10. Let TC_2 be TC_0 **where** nameIR **to** $\text{headerUnionTypeDefIR}$ **is added to the GLOBAL layer**.
11. Let $\text{headerUnionTypeDeclarationIR}$ be $\text{annotationList HEADER_UNION nameIR} < \text{typeId}^* > \{ (\text{annotationList}_f \text{ typeIR}_f \text{ nameIR}_f ;)^* \}$.
12. Result in the updated typing context TC_2 and $\text{headerUnionTypeDeclarationIR}$.

11.16.2. Compile-time evaluation

At compile-time, header union type declarations are loaded into the context by:

Loading $\text{annotationList HEADER_UNION nameIR} < \text{typeParameterListIR} > \{ \text{typeFieldIR}^* \}$, **under instantiation context** IC_0 , **and store** STO :

1. Let $\text{annotationList}_{\text{field}}^*$ be the list of $\text{annotationList}_{\text{field}}$, $\text{nameIR}_{\text{field}}^*$ be the list of $\text{nameIR}_{\text{field}}$, and $\text{typeIR}_{\text{field}}^*$ be the list of $\text{typeIR}_{\text{field}}$, obtained by repeating:
 - Let $\text{annotationList}_{\text{field}} \text{ typeIR}_{\text{field}} \text{ nameIR}_{\text{field}} ;$ be typeFieldIR .

for each typeFieldIR in typeFieldIR^*
2. Let $\text{headerUnionTypeDefIR}$ be $\text{HEADER_UNION nameIR} < \text{typeParameterListIR} > \{ (\text{annotationList}_{\text{field}} \text{ typeIR}_{\text{field}} \text{ nameIR}_{\text{field}} ;)^* \}$.
3. Let IC_1 be IC_0 **where** nameIR **to** $\text{headerUnionTypeDefIR}$ **is added to the GLOBAL layer**.
4. Result in the updated instantiation context IC_1 and the store STO .

11.17. Typedef declaration

A **typedef** or **type** declaration introduces an alias for an existing type.

```
typedef
: type
| derivedTypeDeclaration
;

typedefDeclaration
: annotationList TYPEDEF typedef name ;
| annotationList TYPE type name ;
;
```

In P4_{IR} , **typedef** and **type** declarations are represented as:

```
typedefIR
: typeIR
| derivedTypeDeclarationIR
;
```



```

typedefDeclarationIR
: annotationList TYPEDEF typedefIR nameIR ;
| annotationList TYPE typeIR nameIR ;
;

```

11.17.1. typedef declaration

See [Section 8.5.7](#) for more details about `typedef` aliases.

11.17.1.1. Type checking

Typing `annotationList TYPEDEF typedef name ;` under typing context TC_0 :

1. If let `type` be `typedef`:
 - a. Let B be bound type variables from the GLOBAL layer of TC_0 .
 - b. Let `typeIR` and a set of fresh type variables `typeld*` be
 - the result of typing `type`, under cursor GLOBAL and typing context TC_0 .
 - c. Check that `typeld*` is an empty list.
 - d. Check that `typeIR` is a well-formed type, with bound type variables B .
 - e. Let `nameIR` be the name of `name`.
 - f. Let `typeDefIRtypedef` be `TYPEDEF nameIR typeIR`.
 - g. Check that `typeDefIRtypedef` is a well-formed type definition, with bound type variables B .
 - h. Let TC_1 be TC_0 where `nameIR` to `typeDefIRtypedef` is added to the GLOBAL layer.
 - i. Let `typedefDeclarationIR` be `annotationList TYPEDEF typeIR nameIR ;`.
 - j. Result in the updated typing context TC_1 and `typedefDeclarationIR`.
2. Else:
 - a. Let `derivedTypeDeclaration` be `typedef`.
 - b. Let the updated typing context TC_1 and `declarationIR` be
 - the result of typing `derivedTypeDeclaration` under typing context TC_0 .
 - c. Check that `declarationIR` has type `derivedTypeDeclarationIR`.
 - d. Let `derivedTypeDeclarationIR` be `declarationIR`.
 - e. Let `set<typeld>` be the domain of the map $TC_1.GLOBAL.TYPE$.
 - f. Let `set<typeld>'` be the domain of the map $TC_0.GLOBAL.TYPE$.
 - g. Let `{ typeld* }` be the difference of the sets `set<typeld>` and `set<typeld>'`.
 - h. Check that `typeld*` is a list of length 1.
 - i. Let `typeld'` be `typeld*`.
 - j. Let `typeDefIR'` be ! the type definition of `typeld'` from the GLOBAL layer of TC_1 .
 - k. If `typeDefIR'` is monomorphic:
 - i. Let `typeIR` be the underlying type of `typeDefIR'`.
 - ii. Let `nameIR` be the name of `name`.
 - iii. Let `typedefTypeIR` be `TYPEDEF nameIR typeIR`.

- iv. Let **bound** be **bound type variables from the GLOBAL layer of TC_0** .
- v. Check that **typedefTypeIR** is a well-formed type definition, with bound type variables **bound**.
- vi. Let TC_2 be TC_0 **where nameIR to typedefTypeIR is added to the GLOBAL layer**.
- vii. Let **typedefDeclarationIR** be **annotationList TYPEDEF derivedTypeDeclarationIR nameIR ;**.
- viii. Result in the updated typing context TC_1 and **typedefDeclarationIR**.
- l. Else:
 - i. Check that $(\cdot, \cdot)$ is equal to **the type parameters of typeDefIR'**.
 - ii. Let **nameIR** be **the name of name**.
 - iii. Let **typeIR** be **typeDefIR' specialized by $\cdot$** .
 - iv. Let **typedefTypeIR** be **TYPEDEF nameIR typeIR**.
 - v. Let **bound** be **bound type variables from the GLOBAL layer of TC_0** .
 - vi. Check that **typedefTypeIR** is a well-formed type definition, with bound type variables **bound**.
 - vii. Let TC_2 be TC_0 **where nameIR to typedefTypeIR is added to the GLOBAL layer**.
 - viii. Let **typedefDeclarationIR** be **annotationList TYPEDEF derivedTypeDeclarationIR nameIR ;**.
 - ix. Result in the updated typing context TC_1 and **typedefDeclarationIR**.

11.17.1.2. Compile-time evaluation

At compile-time, **typedefs** are loaded into the context with:

Loading **annotationList TYPEDEF typedefIR nameIR ;**, under instantiation context IC_0 , and store STO :

1. If let **typeIR** be **typedefIR**:
 - a. Let **aliasTypeDefIR** be **TYPEDEF nameIR typeIR**.
 - b. Let IC_1 be IC_0 **where nameIR to aliasTypeDefIR is added to the GLOBAL layer**.
 - c. Result in the updated instantiation context IC_1 and the store STO .
2. Else:
 - a. Let **derivedTypeDeclarationIR** be **typedefIR**.
 - b. Let the updated instantiation context IC_{local} and the store $_$ be
 - the result of **loading** **derivedTypeDeclarationIR**, under instantiation context IC_0 , and store an **empty store**.
 - c. Let **set<typeId>** be **the domain of the map $IC_{local}.GLOBAL.TYPE$** .
 - d. Let **set<typeId>'** be **the domain of the map $IC_0.GLOBAL.TYPE$** .
 - e. Let **{ typeId* }** be **the difference of the sets set<typeId> and set<typeId>'**.
 - f. Check that **typeId*** is a list of length 1.
 - g. Let **typeId_{newtype}** be **typeId***.
 - h. Let **typeDefIR'** be **! the type definition of $\cdot$ typeId_{newtype} from the GLOBAL layer of IC_{local}** .
 - i. If **typeDefIR'** is **monomorphic**:

- i. Let typeIR be the underlying type of $\text{typeDefIR}'$.
- ii. Let aliasTypeDefIR be $\text{TYPEDEF nameIR typeIR}$.
- iii. Let IC_1 be IC_0 where nameIR to aliasTypeDefIR is added to the GLOBAL layer.
- iv. Result in the updated instantiation context IC_1 and the store STO .
- j. Else:
 - i. Check that $(\cdot, \cdot)$ is equal to the type parameters of $\text{typeDefIR}'$.
 - ii. Let typeIR be $\text{typeDefIR}'$ specialized by $\cdot$.
 - iii. Let typedefTypeIR be $\text{TYPEDEF nameIR typeIR}$.
 - iv. Let IC_1 be IC_0 where nameIR to typedefTypeIR is added to the GLOBAL layer.
 - v. Result in the updated instantiation context IC_1 and the store STO .

11.17.2. New type declaration

See [Section 8.5.8](#) for more details about new type aliases.

11.17.2.1. Type checking

Typing $\text{annotationList TYPE type name ;}$ under typing context TC_0 :

1. Let B be bound type variables from the GLOBAL layer of TC_0 .
2. Let typeIR and a set of fresh type variables typeID^* be
 - the result of typing type, under cursor GLOBAL and typing context TC_0 .
3. Check that typeID^* is an empty list.
4. Check that typeIR is a well-formed type, with bound type variables B .
5. Let nameIR be the name of name .
6. Let newTypeIR be $\text{TYPE nameIR typeIR}$.
7. Check that newTypeIR is a well-formed type definition, with bound type variables B .
8. Let TC_1 be TC_0 where nameIR to newTypeIR is added to the GLOBAL layer.
9. Let $\text{typedefDeclarationIR}$ be $\text{annotationList TYPE typeIR nameIR ;}$.
10. Result in the updated typing context TC_1 and $\text{typedefDeclarationIR}$.

11.17.2.2. Compile-time evaluation

At compile-time, types are loaded into the context with:

Loading $\text{annotationList TYPE typeIR nameIR ;}$ under instantiation context IC_0 , and store STO :

1. Let aliasTypeDefIR be $\text{TYPE nameIR typeIR}$.
2. Let IC_1 be IC_0 where nameIR to aliasTypeDefIR is added to the GLOBAL layer.
3. Result in the updated instantiation context IC_1 and the store STO .

11.18. Parser type declaration

A parser type declaration introduces a new parser type.

```
parserTypeDeclaration
  : annotationList PARSE name typeParameterListOpt `( parameterList ) ;
  ;
```

See [Section 8.5.2](#) for more details on parser object types.

11.18.1. Type checking

After type checking, a parser type declaration has the following form:

```
parserTypeDeclarationIR
  : annotationList PARSE nameIR `< typeParameterListIR , typeParameterListIR > `(
parameterListIR ) ;
  ;
```

Typing `annotationList PARSE name typeParameterListOpt ( parameterList ) ;` under typing context TC_0 :

1. Let TC_1 be TC_0 with `BLOCK.KIND` set to `PARSER`.
2. Let the updated typing context TC_2 and type parameters $typeId_{expl}^*$ be
 - the result of **typing** `typeParameterListOpt`, under cursor `BLOCK` and typing context TC_1 .
3. Let the updated typing context TC_3 , $parameterIR^*$, and a set of fresh type variables $typeId_{impl}^*$ be
 - the result of **typing** `parameterList`, under cursor `BLOCK` and typing context TC_2 .
4. Let `nameIR` be the name of `name`.
5. Let `parserObjectTypeDefIR` be `PARSE nameIR < typeIdexpl* , typeIdimpl* > ( parameterIR* )`.
6. Let `bound` be bound type variables from the `GLOBAL` layer of TC_0 .
7. Check that `parserObjectTypeDefIR` is a well-formed type definition, with bound type variables `bound`.
8. Let TC_4 be TC_0 where `nameIR` to `parserObjectTypeDefIR` is added to the `GLOBAL` layer.
9. Let `parserTypeDeclarationIR` be `annotationList PARSE nameIR < typeIdexpl* , typeIdimpl* > ( parameterIR* ) ;`.
10. Result in the updated typing context TC_4 and `parserTypeDeclarationIR`.

11.18.2. Compile-time evaluation

At compile-time, a parser type declaration is loaded into the context with:

Loading `annotationList PARSE nameIR < typeParameterListIR , typeParameterListIRinferred > ( parameterIR* ) ;` under instantiation context IC_0 , and store `STO`:

1. Let `parserObjectTypeDefIR` be `PARSE nameIR < typeParameterListIR , typeParameterListIRinferred > ( parameterIR* )`.
2. Let IC_1 be IC_0 where `nameIR` to `parserObjectTypeDefIR` is added to the `GLOBAL` layer.

3. Result in the updated instantiation context IC_1 and the store STO .

11.19. Control type declaration

A control type declaration introduces a new control type.

```
controlTypeDeclaration
  : annotationList CONTROL name typeParameterListOpt `( parameterList ) ;
  ;
```

See [Section 8.5.3](#) for more details on control object types.

11.19.1. Type checking

After type checking, a control type declaration has the following form:

```
controlTypeDeclarationIR
  : annotationList CONTROL nameIR `< typeParameterListIR , typeParameterListIR > `(
  parameterListIR ) ;
  ;
```

Typing `annotationList CONTROL name typeParameterListOpt ( parameterList ) ;` under typing context TC_0 :

1. Let TC_1 be TC_0 with `BLOCK.KIND` set to `CONTROL`.
2. Let the updated typing context TC_2 and type parameters $typed_{expl}^*$ be
 - the result of **typing** `typeParameterListOpt`, under cursor `BLOCK` and typing context TC_1 .
3. Let the updated typing context TC_3 , $parameterIR^*$, and a set of fresh type variables $typed_{impl}^*$ be
 - the result of **typing** `parameterList`, under cursor `BLOCK` and typing context TC_2 .
4. Let `nameIR` be the name of `name`.
5. Let `controlObjectTypeDefIR` be `CONTROL nameIR < typedexpl* , typedimpl* > ( parameterIR* )`.
6. Let `bound` be bound type variables from the GLOBAL layer of TC_0 .
7. Check that `controlObjectTypeDefIR` is a well-formed type definition, with bound type variables `bound`.
8. Let TC_4 be TC_0 where `nameIR` to `controlObjectTypeDefIR` is added to the GLOBAL layer.
9. Let `controlTypeDeclarationIR` be `annotationList CONTROL nameIR < typedexpl* , typedimpl* > ( parameterIR* ) ;`.
10. Result in the updated typing context TC_4 and `controlTypeDeclarationIR`.

11.19.2. Compile-time evaluation

At compile-time, a control type declaration is loaded into the context with:

Loading `annotationList CONTROL nameIR < typeParameterListIR , typeParameterListIRinferred > ( parameterIR* ) ;`, under instantiation context IC_0 , and store STO :

1. Let `controlObjectTypeDefIR` be `CONTROL nameIR < typeParameterListIR , typeParameterListIRinferred > (`

parameterIR^{*}).

2. Let IC_1 be IC_0 where nameIR to controlObjectTypeDefIR is added to the GLOBAL layer.
3. Result in the updated instantiation context IC_1 and the store STO .

11.20. Package type declaration

A package type declaration introduces a new package type.

```
packageTypeDeclaration
: annotationList PACKAGE name typeParameterListOpt `( parameterList ) ;
;
```

See [Section 8.5.4](#) for more details on package object types.

11.20.1. Type checking

After type checking, a package type declaration has the following form:

```
packageTypeDeclarationIR
: annotationList PACKAGE nameIR `< typeParameterListIR , typeParameterListIR > `(
parameterListIR ) ;
;
```

Typing annotationList PACKAGE name typeParameterListOpt (parameterList) ; under typing context TC_0 :

1. Let B be bound type variables from the GLOBAL layer of TC_0 .
2. Let TC_1 be TC_0 with BLOCK.KIND set to PACKAGE.
3. Let the updated typing context TC_2 and type parameters $typed_{expl}^*$ be
 - the result of typing typeParameterListOpt, under cursor BLOCK and typing context TC_1 .
4. Let the updated typing context TC_3 , constructorParameterIR^{*}, and a set of fresh type variables $typed_{impl}^*$ be
 - the result of typing (parameterList) under typing context TC_2 .
5. Let nameIR be the name of name.
6. Let $typeIR_{package_inner}^*$ be the list of $typeIR_{package_inner}$ obtained by repeating:
 - Let $typeIR_{package_inner}$ be the type of constructorParameterIR.for each constructorParameterIR in constructorParameterIR^{*}
7. Let packageObjectTypeDefIR be PACKAGE nameIR < $typed_{expl}^*$, $typed_{impl}^*$ > { $typeIR_{package_inner}^*$ }.
8. Check that packageObjectTypeDefIR is a well-formed type definition, with bound type variables B .
9. Let TC_4 be TC_0 where nameIR to packageObjectTypeDefIR is added to the GLOBAL layer.
10. Let constructorId be the constructor identifier for name (parameterList).
11. Let $typeIR_{package}$ be the underlying type of packageObjectTypeDefIR.
12. Let constructorTypeDefIR be CONSTRUCTOR < $typed_{expl}^*$, $typed_{impl}^*$ > (constructorParameterIR^{*}) : $typeIR_{package}$.

13. Check that `constructorTypeDefIR` is a well-formed constructor type definition, with bound type variables `B`.
14. Let TC_5 be where `constructorId` to `constructorTypeDefIR` is added to TC_4 .
15. Let `packageTypeDeclarationIR` be `annotationList PACKAGE nameIR < typeIdexpl* , typeIdimpl* > ( constructorParameterIR* ) ;`.
16. Result in the updated typing context TC_5 and `packageTypeDeclarationIR`.

11.20.2. Compile-time evaluation

At compile-time, a package type declaration is loaded into the context with:

Loading `annotationList PACKAGE nameIR < typeParameterListIR , typeParameterListIRinferred > ( parameterIR* ) ;`, under instantiation context IC_0 , and store STO :

1. Let `typeIR*` be the list of `typeIR`, obtained by repeating:
 - Let `typeIR` be the type of `parameterIR`.
 for each `parameterIR` in `parameterIR*`
2. Let `packageObjectTypeDefIR` be `PACKAGE nameIR < typeParameterListIR , typeParameterListIRinferred > { typeIR* }`.
3. Let IC_1 be IC_0 where `nameIR` to `packageObjectTypeDefIR` is added to the GLOBAL layer.
4. Let `callableId` be the callable identifier for `nameIR ( parameterIR* )`.
5. Let `constructorDef` be `PACKAGE < typeParameterListIR ++ typeParameterListIRinferred > ( parameterIR* )`.
6. Let IC_2 be where `callableId` to `constructorDef` is added to IC_1 .
7. Result in the updated instantiation context IC_2 and the store STO .

12. L-values

L-values are expressions that may appear on the left side of an assignment operation or as arguments corresponding to `out` and `inout` function parameters. An l-value represents a storage reference. The following expressions are legal l-values:

```
lvalue
: referenceExpression
| lvalue . member
| lvalue `[ expression ]
| lvalue `[ expression : expression ]
| `( lvalue )
;
```

- Identifiers of a base or derived type.
- Structure, header, and header union field member access operations (using the dot notation).
- References to elements within header stacks (see Section 8.4.3): indexing, and references to `last` and `next`.
- The result of a bit-slice operator `[m:l]`.

The following is a legal l-value: `headers.stack[4].field`. Note that method and function calls cannot return l-values.

12.1. Semantics of l-values

12.1.1. Type checking

After type checking, l-values are represented in $P4_{IR}$ as:

```
typedLvalueIR
  : lvalueIR # lvalueNoteIR
  ;

lvalueIR
  : referenceExpressionIR
  | typedLvalueIR . nameIR
  | typedLvalueIR `[ typedExpressionIR ]
  | typedLvalueIR `[ typedExpressionIR : typedExpressionIR ]
  | `( typedLvalueIR )
  ;

lvalueNoteIR
  : `( typeIR )
  ;
```

Notice that l-values in $P4_{IR}$ carry type information. L-values are type checked according to the relation:

Lvalue_ok:

- Typing `lvalue`, under cursor `cursor` and typing context `typingContext`:
- Results in `typedLvalueIR`.

12.1.2. Runtime evaluation

At runtime, l-values are represented as storage references:

```
storageReference
  : referenceExpressionIR
  | storageReference . nameIR
  | storageReference `[ value ]
  | storageReference `[ value : value ]
  ;
```

12.1.2.1. Creating storage references

The following relation evaluates an l-value to a storage reference:

Lvalue_eval:

- Evaluating `typedLvalueIR`, under cursor `cursor`, evaluation context `evalContext`, and store `store`:
- Results in the updated evaluation context `evalContext`, store `store`, and result `storageReferenceResult`.

The result of evaluating an l-value is:

```
storageReferenceResult
: continueResult<storageReference?>
| abortResult
;

continueResult<X>
: ` X
;

abortResult
: exitResult
| rejectTransitionResult
;
```

12.1.2.2. Reading from storage references

The following relation reads a value from a storage reference:

Lvalue_read:

- Reading from `storageReference`, under cursor `cursor`, evaluation context `evalContext`, and store `store`:
- Results in the value `value`.

12.1.2.3. Writing to storage references

The following relation writes a value to a storage reference:

Lvalue_write:

- Writing `value` to `storageReference`, under cursor `cursor`, evaluation context `evalContext`, and store `store`:
- Results in the updated evaluation context `evalContext`.

The subsequent sections describe each kind of l-value in detail.

12.2. Identifiers

12.2.1. Type checking

Typing `prefixedNonTypeName`, under cursor `p` and typing context `TC`:

1. Let `prefixedNameIR` be the prefixed name of `prefixedNonTypeName`.
2. Let `direction typeIR ctk value?` be ! the type of variable `prefixedNameIR` from the `p` layer of `TC`.
3. Check that `ctk` is `DYN`.
4. Check that `value?` is none.
5. Check that `direction` is equal to `OUT` or `direction` is equal to `INOUT`.

6. Result in `prefixedNameIR` with note `( typeIR )`.

12.2.2. Creating a storage reference

Evaluating `referenceExpressionIR` with note `lvalueNoteIR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Result in the updated evaluation context `EC`, store `STO`, and result `referenceExpressionIR`.

12.2.3. Reading from a storage reference

Reading from `prefixedNameIR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let `value` be the value of `prefixedNameIR` from the `p` layer of `EC`.
2. Result in the value `value`.

12.2.4. Writing to a storage reference

Writing `value` to `prefixedNameIR`, under cursor `p`, evaluation context `EC0`, and store `STO`:

1. Let `EC1` be `EC0` where `prefixedNameIR` is updated to value from the `p` layer.
2. Result in the updated evaluation context `EC1`.

12.3. Member accesses

12.3.1. Type checking

Typing `lvaluebase . member`, under cursor `p` and typing context `TC`:

1. Let `typedLvalueIRbase` be
 - the result of typing `lvaluebase`, under cursor `p` and typing context `TC`.
2. Let `( typeIRbase )` be the note of `typedLvalueIRbase`.
3. Let `typeIR` be `typeIRbase` with typedefs unrolled.
4. If let `typeIR' [ _ ]` be `typeIR`:
 - a. Let `nameIR` be the name of `member`.
 - b. Check that `nameIR` is equal to "next" or `nameIR` is equal to "last".
 - c. Check that `p` is equal to `BLOCK` and `TC.BLOCK.KIND` is equal to `PARSER` or `p` is equal to `LOCAL` and `TC.LOCAL.KIND` is equal to `PARSER_STATE`.
 - d. Let `typedLvalueIR` be `typedLvalueIRbase . nameIR` with note `( typeIR' )`.
 - e. Result in `typedLvalueIR`.
5. Else if let `STRUCT _ < _ > { ( _ typeIRfield nameIRfield ; ) * }` be `typeIR`:

- a. Let `nameIR` be **the name of member**.
 - b. Let `typeIR` be **! the value associated to `nameIR` in (`nameIRfield`, `typeIRfield`)^{*}**.
 - c. Let `typedLvalueIR` be `typedLvalueIRbase` . `nameIR` **with note (`typeIR`)**.
 - d. Result in `typedLvalueIR`.
6. Else if let `HEADER_ < _ * > { ( _ typeIRfield nameIRfield ; )* }` be `typeIR`:
 - a. Let `nameIR` be **the name of member**.
 - b. Let `typeIR` be **! the value associated to `nameIR` in (`nameIRfield`, `typeIRfield`)^{*}**.
 - c. Let `typedLvalueIR` be `typedLvalueIRbase` . `nameIR` **with note (`typeIR`)**.
 - d. Result in `typedLvalueIR`.
 7. Else if let `HEADER_UNION_ < _ * > { ( _ typeIRfield nameIRfield ; )* }` be `typeIR`:
 - a. Let `nameIR` be **the name of member**.
 - b. Let `typeIR` be **! the value associated to `nameIR` in (`nameIRfield`, `typeIRfield`)^{*}**.
 - c. Let `typedLvalueIR` be `typedLvalueIRbase` . `nameIR` **with note (`typeIR`)**.
 - d. Result in `typedLvalueIR`.

12.3.2. Creating a storage reference

Evaluating `typedLvalueIR` . `nameIR` **with note** `lvalueNoteIR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let the updated evaluation context `EC1`, store `STO1`, and result `storageReferenceResult` be
 - the result of **evaluating** `typedLvalueIR`, under cursor `p`, evaluation context `EC`, and store `STO`.
2. If let `abortResult` be `storageReferenceResult`:
 - a. Result in the updated evaluation context `EC1`, store `STO1`, and result `abortResult`.
3. Else:
 - a. Let `storageReference`[?] be `storageReferenceResult`.
 - b. Check that `storageReference`[?] is defined.
 - c. Let `storageReference'` be `storageReference`[?].
 - d. Let the value `value` be
 - the result of **reading from** `storageReference'`, under cursor `p`, evaluation context `EC1`, and store `STO1`.
 - e. If let `headerStackValue` be `value`:
 - i. Let `HEADER_STACK [ valueelement * ( nidx ; nsize ) ]` be `headerStackValue`.
 - ii. Check that `nameIR` is equal to "next".
 - iii. If `nidx` is greater than or equal to `nsize`:
 - A. Let `rejectTransitionResult` be `REJECT ERROR . "StackOutOfBounds"`.
 - B. Result in the updated evaluation context `EC1`, store `STO1`, and result `rejectTransitionResult`.
 - iv. If `nidx` is less than `nsize`:

- A. Let `storageReferenceResult'` be `storageReference' . nameIR`.
- B. Result in the updated evaluation context `EC1`, store `STO1`, and result `storageReferenceResult'`.
- f. Else:
 - i. Let `storageReferenceResult'` be `storageReference' . nameIR`.
 - ii. Result in the updated evaluation context `EC1`, store `STO1`, and result `storageReferenceResult'`.

12.3.3. Reading from a storage reference

Reading from `storageReference' . nameIR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let the value `valuebase` be
 - the result of `reading from storageReference', under cursor p, evaluation context EC, and store STO`.
2. If let `REF objectId` be `valuebase`:
 - a. Let `object` be `the object objectId in STO`.
 - b. Check that `object` has type `packageObject`.
 - c. Let `PACKAGE < _ > { frame }` be `object`.
 - d. Let `value'` be `! nameIR in map frame`.
 - e. Result in the value `value'`.
3. Else if let `HEADER_STACK [ valueelement * ( nidx ; nsize ) ]` be `valuebase`:
 - a. Check that `nameIR` is equal to "next".
 - b. Check that `nidx` is less than `nsize`.
 - c. Let `value` be `valueelement * [nidx]`.
 - d. Result in the value `value`.
4. Else if let `STRUCT _ { ( valuefield nameIRfield ; ) * }` be `valuebase`:
 - a. Let `value'` be `! the value associated to nameIR in ( nameIRfield, valuefield ) *`.
 - b. Result in the value `value'`.
5. Else if let `HEADER _ { _ ; ( valuefield nameIRfield ; ) * }` be `valuebase`:
 - a. Let `value'` be `! the value associated to nameIR in ( nameIRfield, valuefield ) *`.
 - b. Result in the value `value'`.
6. Else if let `HEADER_UNION _ { ( valuefield nameIRfield ; ) * }` be `valuebase`:
 - a. Let `value'` be `! the value associated to nameIR in ( nameIRfield, valuefield ) *`.
 - b. Result in the value `value'`.

12.3.4. Writing to a storage reference

Writing `value` to `storageReference' . nameIR`, under cursor `p`, evaluation context `EC0`, and store `STO`:

1. Let the value `valuebase` be

- the result of **reading from** `storageReference'`, **under cursor** `p`, **evaluation context** `EC0`, and **store** `STO`.
2. If let `HEADER_STACK [ valueelement* ( nidx ; nsize ) ]` be `valuebase`:
 - a. Check that `nameIR` is equal to "next".
 - b. Let `valueelement_update*` be `valueelement*` with `[nidx]` set to `value`.
 - c. Let `valuebase_update` be `HEADER_STACK [valueelement_update* (nidx `` ``1`` ``;+ nsize)]`.
 - d. Let the updated evaluation context `EC1` be
 - the result of **writing** `valuebase_update` **to** `storageReference'`, **under cursor** `p`, **evaluation context** `EC0`, and **store** `STO`.
 - e. Result in the updated evaluation context `EC1`.
 3. Else if let `STRUCT typed { fieldValue* }` be `valuebase`:
 - a. Let `fieldValueupdate*` be `$update_fieldValue(fieldValue*, nameIR, value)`.
 - b. Let `valuebase_update` be `STRUCT typed { fieldValueupdate* }`.
 - c. Let the updated evaluation context `EC1` be
 - the result of **writing** `valuebase_update` **to** `storageReference'`, **under cursor** `p`, **evaluation context** `EC0`, and **store** `STO`.
 - d. Result in the updated evaluation context `EC1`.
 4. Else if let `HEADER typed { bvalid ; fieldValue* }` be `valuebase`:
 - a. Let `fieldValueupdate*` be `$update_fieldValue(fieldValue*, nameIR, value)`.
 - b. Let `valuebase_update` be `HEADER typed { bvalid ; fieldValueupdate* }`.
 - c. Let the updated evaluation context `EC1` be
 - the result of **writing** `valuebase_update` **to** `storageReference'`, **under cursor** `p`, **evaluation context** `EC0`, and **store** `STO`.
 - d. Result in the updated evaluation context `EC1`.
 5. Else if let `HEADER_UNION typed { fieldValue* }` be `valuebase`:
 - a. Let `( valuefield nameIRfield ; )*` be `fieldValue*`.
 - b. Let `valueold` be **! the value associated to** `nameIR` **in** `( nameIRfield, valuefield )*`.
 - c. Check that `valueold` has type `headerValue`.
 - d. Let `HEADER _ { bvalid_old ; _* }` be `valueold`.
 - e. Let `fieldValueupdate*` be `$update_fieldValue(fieldValue*, nameIR, value)`.
 - f. Check that `value` has type `headerValue`.
 - g. Let `HEADER _ { bvalid_new ; _* }` be `value`.
 - h. Let `btoggled` be `bvalid_old~` and `bvalid_new` or `bvalid_old` and `bvalid_new~`.
 - i. If `btoggled~`:
 - i. Let `valuebase_update` be `HEADER_UNION typed { fieldValueupdate* }`.
 - ii. Let the updated evaluation context `EC1` be
 - the result of **writing** `valuebase_update` **to** `storageReference'`, **under cursor** `p`, **evaluation context** `EC0`, and **store** `STO`.
 - iii. Result in the updated evaluation context `EC1`.
 - j. Else:

- i. Let $\text{fieldValue}_{\text{update}}^*$ be $\text{\$update_headerUnion}(\text{fieldValue}_{\text{update}}^*, \text{nameIR})$.
- ii. Let $\text{value}_{\text{base_update}}$ be $\text{HEADER_UNION typed } \{ \text{fieldValue}_{\text{update}}^* \}$.
- iii. Let the updated evaluation context EC_1 be
 - the result of **writing** $\text{value}_{\text{base_update}}$ to storageReference , under cursor p , evaluation context EC_0 , and store STO .
- iv. Result in the updated evaluation context EC_1 .

12.4. Index accesses

12.4.1. Type checking

Typing $\text{lvalue}_{\text{base}} [\text{expression}_{\text{index}}]$, under cursor p and typing context TC :

1. Let $\text{typedLvalueIR}_{\text{base}}$ be
 - the result of **typing** $\text{lvalue}_{\text{base}}$ under cursor p and typing context TC .
2. Let $(\text{typeIR}_{\text{base}})$ be the note of $\text{typedLvalueIR}_{\text{base}}$.
3. Let typeIR be $\text{typeIR}_{\text{base}}$ with **typedefs** unrolled.
4. Check that typeIR has type headerStackTypeIR .
5. Let $\text{typeIR}' [n_{\text{size}}]$ be typeIR .
6. Let $\text{typedExpressionIR}_{\text{index}}$ be
 - the result of **typing** $\text{expression}_{\text{index}}$ under cursor p and typing context TC .
7. Let $\text{typeIR}_{\text{index}}$ and **compile-time known-ness** $\text{ctk}_{\text{index}}$ be the note of $\text{typedExpressionIR}_{\text{index}}$.
8. Let $\text{typedExpressionIR}_{\text{index_reduced}}$ be ! the result of **reducing serializable enums** in $\text{typedExpressionIR}_{\text{index}}$ until $\text{\$compat_array_index}$ is satisfied.
9. If $\text{ctk}_{\text{index}}$ is LCTK:
 - a. Let value be
 - the result of **local compile-time evaluation** of $\text{typedExpressionIR}_{\text{index_reduced}}$, under cursor p and typing context TC .
 - b. Check that value has type integerValue .
 - c. Let $\text{integerValue}_{\text{index}}$ be value .
 - d. Let n_{index} be ! the **natural number representation** of $\text{integerValue}_{\text{index}}$.
 - e. Check that n_{index} is less than n_{size} .
 - f. Let typedLvalueIR be $\text{typedLvalueIR}_{\text{base}} [\text{typedExpressionIR}_{\text{index_reduced}}]$ with **note** (typeIR') .
 - g. Result in typedLvalueIR .
10. Else:
 - a. Let typedLvalueIR be $\text{typedLvalueIR}_{\text{base}} [\text{typedExpressionIR}_{\text{index_reduced}}]$ with **note** (typeIR') .
 - b. Result in typedLvalueIR .

12.4.2. Creating a storage reference

Evaluating `typedLvalueIR [ typedExpressionIR ]` with note `lvalueNoteIR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let the updated evaluation context `EC1`, store `STO1`, and result `storageReferenceResult` be
 - the result of evaluating `typedLvalueIR`, under cursor `p`, evaluation context `EC`, and store `STO`.
2. If let `abortResult` be `storageReferenceResult`:
 - a. Result in the updated evaluation context `EC1`, store `STO1`, and result `abortResult`.
3. Else:
 - a. Let `storageReference'` be `storageReferenceResult`.
 - b. Check that `storageReference'` is defined.
 - c. Let `storageReference'` be `storageReference'`.
 - d. Let the updated evaluation context `EC2`, store `STO2` and `expressionResult` be
 - the result of evaluating `typedExpressionIR`, under cursor `p`, evaluation context `EC1`, and store `STO1`.
 - e. If let `abortResult` be `expressionResult`:
 - i. Result in the updated evaluation context `EC2`, store `STO2`, and result `abortResult`.
 - f. Else:
 - i. Let `value` be `expressionResult`.
 - ii. Check that `value` has type `integerValue`.
 - iii. Let `integerValueindex` be `value`.
 - iv. Let `storageReferenceResult'` be `storageReference' [ integerValueindex ]`.
 - v. Result in the updated evaluation context `EC2`, store `STO2`, and result `storageReferenceResult'`.

12.4.3. Reading from a storage reference

Reading from `storageReference' [ integerValueindex ]`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let the value `valuebase` be
 - the result of reading from `storageReference'`, under cursor `p`, evaluation context `EC`, and store `STO`.
2. Check that `valuebase` has type `headerStackValue`.
3. Let `HEADER_STACK [ valueelement* ( _ ; nsize ) ]` be `valuebase`.
4. Let `nidx` be ! the natural number representation of `integerValueindex`.
5. If `nidx` is less than `nsize`:
 - a. Let `value'` be `valueelement* [ nidx ]`.
 - b. Result in the value `value'`.
6. If `nidx` is greater than or equal to `nsize`:
 - a. Let `value'` be `valueelement* [ 0 ]`.

- b. Check that `value'` has type `headerValue`.
- c. Let `HEADER typeId { bvalid ; fieldValue* }` be `value'`.
- d. Let `value''` be `HEADER typeId { false ; fieldValue* }`.
- e. Result in the value `value''`.

12.4.4. Writing to a storage reference

Writing `value` to `storageReference'` [`integerValueindex`], under cursor `p`, evaluation context `EC0`, and store `STO`:

1. Let the value `valuebase` be
 - the result of `reading from storageReference'`, under cursor `p`, evaluation context `EC0`, and store `STO`.
2. Check that `valuebase` has type `headerStackValue`.
3. Let `HEADER_STACK [ valueelement* ( nbase_idx ; nsize ) ]` be `valuebase*`.
4. Let `nidx` be ! the natural number representation of `integerValueindex*`.
5. If `nidx` is less than `nsize`:
 - a. Let `valueelement_update*` be `valueelement*` with `[nidx]` set to `value`.
 - b. Let `valuebase_update` be `HEADER_STACK [ valueelement_update* ( nbase_idx ; nsize ) ]`.
 - c. Let the updated evaluation context `EC1` be
 - the result of `writing valuebase_update to storageReference'`, under cursor `p`, evaluation context `EC0`, and store `STO`.
 - d. Result in the updated evaluation context `EC1`.
6. If `nidx` is greater than or equal to `nsize`:
 - a. Result in the updated evaluation context `EC0`.

12.5. Bit-slice accesses

12.5.1. Type checking

Typing `lvaluebase [ expressionhi : expressionlo ]`, under cursor `p` and typing context `TC`:

1. Let `typedLvalueIRbase` be
 - the result of `typing lvaluebase*`, under cursor `p` and typing context `TC`.
2. Let `( typeIRbase )` be the note of `typedLvalueIRbase*`.
3. Check that `typeIRbase` can be used as a bitslice base.
4. Let `typedExpressionIRhi` be
 - the result of `typing expressionhi`, under cursor `p` and typing context `TC`.
5. Let `typedExpressionIRlo` be
 - the result of `typing expressionlo`, under cursor `p` and typing context `TC`.

6. Let $\text{typedExpressionIR}_{hi_reduced}$ be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{hi}$ until $\$compat_bitslice_index$ is satisfied.
7. Let $\text{typedExpressionIR}_{lo_reduced}$ be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{lo}$ until $\$compat_bitslice_index$ is satisfied.
8. Let $\text{typeIR}_{hi_reduced}$ and compile-time known-ness $\text{ctk}_{hi_reduced}$ be the note of $\text{typedExpressionIR}_{hi_reduced}$.
9. Let $\text{typeIR}_{lo_reduced}$ and compile-time known-ness $\text{ctk}_{lo_reduced}$ be the note of $\text{typedExpressionIR}_{lo_reduced}$.
10. Check that $\text{ctk}_{hi_reduced}$ is LCTK.
11. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{hi_reduced}$, under cursor p and typing context TC .
12. Check that value has type integerValue .
13. Let integerValue_{hi} be value .
14. Let n_{hi} be ! the natural number representation of integerValue_{hi} .
15. Check that $\text{ctk}_{lo_reduced}$ is LCTK.
16. Let value' be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{lo_reduced}$, under cursor p and typing context TC .
17. Check that value' has type integerValue .
18. Let integerValue_{lo} be value' .
19. Let n_{lo} be ! the natural number representation of integerValue_{lo} .
20. Check that $\text{typeIR}_{base} [n_{lo} : n_{hi}]$ is a valid bitslice.
21. Let int be $n_{hi} - n_{lo} + 1$.
22. Check that int has type nat .
23. Let n_{slice} be int .
24. Let typeIR be $\text{BIT} < n_{slice} >$.
25. Let typedLvalueIR be $\text{typedLvalueIR}_{base} [\text{typedExpressionIR}_{hi_reduced} : \text{typedExpressionIR}_{lo_reduced}]$ with note (typeIR).
26. Result in typedLvalueIR .

12.5.2. Creating a storage reference

Evaluating $\text{typedLvalueIR} [\text{typedExpressionIR}_{hi} : \text{typedExpressionIR}_{lo}]$ with note lvalueNoteIR , under cursor p , evaluation context EC , and store STO :

1. Let the updated evaluation context EC_1 , store STO_1 , and result $\text{storageReferenceResult}$ be
 - the result of evaluating typedLvalueIR , under cursor p , evaluation context EC , and store STO .
2. If let abortResult be $\text{storageReferenceResult}$:
 - a. Result in the updated evaluation context EC_1 , store STO_1 , and result abortResult .
3. Else:
 - a. Let $\text{storageReference}^?$ be $\text{storageReferenceResult}$.
 - b. Check that $\text{storageReference}^?$ is defined.

- c. Let `storageReference'` be `storageReference'`.
- d. Let the updated evaluation context EC_2 , store STO_2 and `expressionListResult` be
 - the result of `evaluating` [`typedExpressionIRhi`, `typedExpressionIRlo`], under cursor `p`, evaluation context EC_1 , and store STO_1 .
- e. If let `abortResult` be `expressionListResult`:
 - i. Result in the updated evaluation context EC_2 , store STO_2 , and result `abortResult`.
- f. Else:
 - i. Let `value*` be `expressionListResult`.
 - ii. Check that `value*` is a list of length 2.
 - iii. Let [`valuehi`, `valuelo`] be `value*`.
 - iv. Let `storageReferenceResult'` be `storageReference'` [`valuehi` : `valuelo`].
 - v. Result in the updated evaluation context EC_1 , store STO_1 , and result `storageReferenceResult'`.

12.5.3. Reading from a storage reference

`Reading from` `storageReference'` [`valuehi` : `valuelo`], under cursor `p`, evaluation context EC , and store STO :

1. Let the value `valuebase` be
 - the result of `reading from` `storageReference'`, under cursor `p`, evaluation context EC , and store STO .
2. Let `value` be `valuebase` [`valuehi` : `valuelo`].
3. Result in the value `value`.

12.5.4. Writing to a storage reference

`Writing value to` `storageReference'` [`valuehi` : `valuelo`], under cursor `p`, evaluation context EC_0 , and store STO :

1. Let the value `valuebase` be
 - the result of `reading from` `storageReference'`, under cursor `p`, evaluation context EC_0 , and store STO .
2. Let `valuebase_update` be `valuebase` [`valuehi` : `valuelo`] `replaced with` `value`.
3. Let the updated evaluation context EC_1 be
 - the result of `writing` `valuebase_update` `to` `storageReference'`, under cursor `p`, evaluation context EC_0 , and store STO .
4. Result in the updated evaluation context EC_1 .

12.6. Parenthesized l-values

12.6.1. Type checking

Typing ($lvalue_{base}$), under cursor p and typing context TC :

1. Let $typedLvalueIR_{base}$ be
 - the result of typing $lvalue_{base}$, under cursor p and typing context TC .
2. Let ($typeIR_{base}$) be the note of $typedLvalueIR_{base}$.
3. Let $typedLvalueIR$ be ($typedLvalueIR_{base}$) with note ($typeIR_{base}$).
4. Result in $typedLvalueIR$.

12.6.2. Creating a storage reference

Evaluating ($typedLvalueIR$) with note $lvalueNoteIR$, under cursor p , evaluation context EC , and store STO :

1. Let the updated evaluation context EC_1 , store STO_1 , and result $storageReferenceResult$ be
 - the result of evaluating $typedLvalueIR$, under cursor p , evaluation context EC , and store STO .
2. Result in the updated evaluation context EC_1 , store STO_1 , and result $storageReferenceResult$.

13. Statements

The syntax of statements is defined as follows:

```
statement
: emptyStatement
| assignmentStatement
| callStatement
| directApplicationStatement
| returnStatement
| exitStatement
| blockStatement
| conditionalStatement
| forStatement
| breakStatement
| continueStatement
| switchStatement
;
```

13.1. Semantics of statements

13.1.1. Type checking

Stmt_ok:

- Typing $statement$, under cursor $cursor$, typing context $typingContext$, abstract control flow $flow$, and

loop context `loopctxt`:

- Results in
 - the updated typing context `typingContext`, abstract control flow `flow`, and `statementIR`.

After type checking, statements are represented in $P4_{IR}$ as follows:

```
statementIR
: emptyStatementIR
| assignmentStatementIR
| callStatementIR
| directApplicationStatementIR
| returnStatementIR
| exitStatementIR
| blockStatementIR
| conditionalStatementIR
| forStatementIR
| breakStatementIR
| continueStatementIR
| switchStatementIR
;
```

In order to statically determine the control flow of a P4 program (e.g., whether a function body always returns), the following indicator for abstract control flow is used:

```
flow
: CONT
| RET
;
```

As a marker of whether a statement is nested inside a loop, the following indicator is used:

```
loopctxt
: LOOP
| NOLOOP
;
```

13.1.2. Compile-time evaluation

`Stmt_inst`:

- Compile-time evaluation of `statementIR`, under cursor `cursor`, instantiation context `instContext`, and store `store`:
- Results in
 - the updated instantiation context `instContext`, store `store`, and `statementIR`.

Instantiations nested within statements are processed according to the above relation.

13.1.3. Runtime evaluation

`Stmt_eval`:

- Runtime evaluation of `statementIR`, under cursor `cursor`, evaluation context `evalContext`, and store `store`:
- Results in
 - the updated evaluation context `evalContext`, store `store`, and `statementResult`.

The result of evaluating a statement is represented as follows:

```
statementResult
: continueEmptyResult
| callResult
| forResult
;

continueEmptyResult
: /* empty */
;

callResult
: abortResult
| returnResult
;

forResult
: forBreakResult
| forContinueResult
;

abortResult
: exitResult
| rejectTransitionResult
;

returnResult
: RETURN value?
;

exitResult
: EXIT
;

rejectTransitionResult
: REJECT errorValue
;
```

The subsequent sections describe each kind of statement in detail.

13.2. Empty statements

The empty statement, written `;` is a no-op.

```
emptyStatement
: ;
;
```

13.2.1. Type checking

After type checking, an empty statement is represented as:

```
emptyStatementIR = emptyStatement
```

Typing `;`, under cursor `p`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. Result in
 - the updated typing context `TC`, abstract control flow `f`, and `;`.

13.2.2. Compile-time evaluation

Compile-time evaluation of `emptyStatementIR`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. Result in
 - the updated instantiation context `IC`, store `STO`, and `emptyStatementIR`.

13.2.3. Runtime evaluation

Runtime evaluation of `;`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Result in
 - the updated evaluation context `EC`, store `STO`, and `EMPTY`.

13.3. Assignment statements

```
assignop
: =
| +=
| -=
| |=
| -=
| *=
| /=
| %=
| <<=
| >>=
| &=
| ^=
| |=
;

assignmentStatement
: lvalue assignop expression ;
;
```

An assignment, written with the `=` sign, first evaluates its left sub-expression to an l-value, then evaluates its right sub-expression to a value, and finally copies the value into the l-value. Derived types (e.g. `structs`) are copied recursively, and all components of `headers` are copied, including validity bits. Assignment is not defined for `extern` values.

An assignment may also be written with a binary arithmetic or bit manipulation operator immediately

before the = sign. This performs the binary operator on the old value of the left sub-expression and the right sub-expression and assigns the result to the l-value. Thus an assignment like $A += B$ is equivalent to $A = A + B$, except that A is only evaluated once. This means that any side-effects within this operand (eg, a function call inside an array index or slice expression) only occur once. This is not valid for comparison operators, logical operators, concat, range, or mask operators.

13.3.1. Type checking

After type checking, an assignment statement has the form:

```
assignmentStatementIR
: typedLvalueIR assignop typedExpressionIR ;
;
```

Typing $lvalue$ $assignop$ $expression$;, under cursor p , typing context TC , abstract control flow f , and loop context l :

1. If $assignop$ is $=$:
 - a. Let $typedLvalueIR$ be
 - the result of **typing** $lvalue$, under cursor p and typing context TC .
 - b. Let $typedExpressionIR$ be
 - the result of **typing** $expression$, under cursor p and typing context TC .
 - c. Let $typeIR_l$ be the type of $lvalue$ $typedLvalueIR$.
 - d. Let $typeIR_e$ be the type of $typedExpressionIR$.
 - e. Let $typedExpressionIR_{cast}$ be $! typedExpressionIR$ implicitly cast to $typeIR_l$.
 - f. Result in
 - the updated typing context TC , abstract control flow f , and $typedLvalueIR = typedExpressionIR_{cast}$.
2. Else:
 - a. Let $binop'$ be $! the$ binary operator corresponding to $assignop$.
 - b. Let $typedLvalueIR$ be
 - the result of **typing** $lvalue$, under cursor p and typing context TC .
 - c. Let $expression_{lvalue}$ be $\$lvalue_to_expression(lvalue)$.
 - d. Let $expression_{binary}$ be $expression_{lvalue} binop' expression$.
 - e. Let $typedExpressionIR_{binary}$ be
 - the result of **typing** $expression_{binary}$, under cursor p and typing context TC .
 - f. Let ($typeIR_{lvalue}$) be the note of $typedLvalueIR$.
 - g. Let $expressionIR \# _$ be $! typedExpressionIR_{binary}$ implicitly cast to $typeIR_{lvalue}$.
 - h. Check that $expressionIR$ has type $binaryExpressionIR$.
 - i. Let $_ binop'' typedExpressionIR_{cast}$ be $expressionIR$.
 - j. Check that $binop''$ is equal to $binop'$.
 - k. Result in
 - the updated typing context TC , abstract control flow f , and $typedLvalueIR assignop$

$\text{typedExpressionIR}_{\text{cast}} ;$.

13.3.2. Compile-time evaluation

Compile-time evaluation of $\text{assignmentStatementIR}$, under cursor p , instantiation context IC , and store STO :

1. Result in
 - the updated instantiation context IC , store STO , and $\text{assignmentStatementIR}$.

13.3.3. Runtime evaluation

Runtime evaluation of typedLvalueIR assignop typedExpressionIR ;, under cursor p , evaluation context EC , and store STO :

1. Let the updated evaluation context EC_1 , store STO_1 , and result $\text{storageReferenceResult}$ be
 - the result of evaluating typedLvalueIR , under cursor p , evaluation context EC , and store STO .
2. If let abortResult be $\text{storageReferenceResult}$:
 - a. Result in
 - the updated evaluation context EC_1 , store STO_1 , and abortResult .
3. Else:
 - a. Let $\text{storageReference}'$ be $\text{storageReferenceResult}$.
 - b. Check that $\text{storageReference}'$ is defined.
 - c. Let $\text{storageReference}'$ be $\text{storageReference}'$.
 - d. Check that assignop does not match pattern $=$.
 - e. Let the value value_l be
 - the result of reading from $\text{storageReference}'$, under cursor p , evaluation context EC_1 , and store STO_1 .
 - f. Let the updated evaluation context EC_2 , store STO_2 and expressionResult be
 - the result of evaluating typedExpressionIR , under cursor p , evaluation context EC_1 , and store STO_1 .
 - g. If let abortResult be expressionResult :
 - i. Result in
 - the updated evaluation context EC_2 , store STO_2 , and abortResult .
 - h. Else:
 - i. Let value_r be expressionResult .
 - ii. Let binop' be ! the binary operator corresponding to assignop .
 - iii. Let $\text{value}_{\text{update}}$ be $\text{value}_l \text{ binop}' \text{value}_r$.
 - iv. Let the updated evaluation context EC_3 be
 - the result of writing $\text{value}_{\text{update}}$ to $\text{storageReference}'$, under cursor p , evaluation context EC_2 , and store STO_2 .

v. Result in

- the updated evaluation context EC_3 , store STO_2 , and ``EMPTY`.

4. If `assignop` is `=:`

a. Check that `storageReferenceResult` has type `continueResult<storageReference?>`.

b. Let `storageReference'` be `storageReferenceResult`.

c. Check that `storageReference'` is defined.

d. Let `storageReference'` be `storageReference'`.

e. Let the updated evaluation context EC_2 , store STO_2 and `expressionResult` be

- the result of `evaluating` `typedExpressionIR`, under cursor `p`, evaluation context EC_1 , and store STO_1 .

f. If let `abortResult` be `expressionResult`:

i. Result in

- the updated evaluation context EC_2 , store STO_2 , and `abortResult`.

g. Else:

i. Let `value` be `expressionResult`.

ii. Let the updated evaluation context EC_3 be

- the result of `writing value to storageReference'`, under cursor `p`, evaluation context EC_2 , and store STO_2 .

iii. Result in

- the updated evaluation context EC_3 , store STO_2 , and ``EMPTY`.

13.4. Call statements

Call statements are used to invoke actions, functions, and methods.

```
callStatement
: lvalue `( argumentList ) ;
| lvalue `< typeArgumentList > `( argumentList ) ;
;
```

Details of how calls are resolved and evaluated are described in [Chapter 18](#).

13.4.1. Type checking

After type checking, a call statement is represented as:

```
callStatementIR
: callableTargetIR `< typeArgumentListIR > `( argumentListIR ) ;
;

callableTargetIR
: referenceExpressionIR
| typedExpressionIR . nameIR
| TYPE prefixedNameIR . nameIR
| `( callableTargetIR )
;
```

Typing `callStatement`, under cursor `p`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. If `let lvaluecallable ( argumentList ) ; be callStatement`:
 - a. Let the typed callable target `callableTargetIR` be
 - the result of typing `lvaluecallable` as a callable target, under cursor `p` and typing context `TC`.
 - b. Let `argument*` be the list of arguments in `argumentList`.
 - c. Let `argumentIR*` be the list of `argumentIR`, obtained by repeating:
 - Let `argumentIR` be
 - the result of typing `argument`, under cursor `p` and typing context `TC`.
 - for each `argument` in `argument*`
 - d. Let the callable type `callableTypeIR` with fresh type variables `typeIdimpl*`, defaults `iddefault*` and optionals `idoptional*` be
 - the result of finding the type of callable call `callableTargetIR < · > ( argumentIR* )`, under cursor `p` and typing context `TC`.
 - e. Let the return type `typeIRret` with inferred type arguments `typeArgumentIRinferred*` and the casted argument list `argumentIRcast*` be
 - the result of typing the invocation `callableTypeIR < · > ( argumentIR* )` with omitted type arguments `typeIdimpl*`, defaults `iddefault*` and optionals `idoptional*`, under cursor `p` and typing context `TC`.
 - f. If `callableTypeIR` is a `static_assert` call:
 - i. Let `callExpressionIR` be `callableTargetIR < typeArgumentIRinferred* > ( argumentIRcast* )`.
 - ii. Let `expressionNoteIR` be `typeIRret` and compile-time known-ness LTK.
 - iii. Let `value` be
 - the result of local compile-time evaluation of `callExpressionIR` annotated with `expressionNoteIR`, under cursor `p` and typing context `TC`.
 - iv. Check that `value` is equal to ``B true`.
 - v. Let `emptyStatementIR` be `;`.
 - vi. Result in
 - the updated typing context `TC`, abstract control flow `f`, and `emptyStatementIR`.
 - g. Let `argumentIR*` be
 - the result of typing `argumentList`, under cursor `p` and typing context `TC`.
 - h. If `~$is_static_assert_callableTypeIR(callableTypeIR)`:
 - i. Let `callStatementIR` be `callableTargetIR < typeArgumentIRinferred* > ( argumentIRcast* ) ;`.
 - ii. Result in
 - the updated typing context `TC`, abstract control flow `f`, and `callStatementIR`.
2. Else:
 - a. Let `lvaluecallable < typeArgumentList > ( argumentList ) ; be callStatement`.
 - b. Let the typed callable target `callableTargetIR` be
 - the result of typing `lvaluecallable` as a callable target, under cursor `p` and typing context `TC`.

- c. Let typeArgumentIR^* and a set of fresh type variables $\text{typeId}_{\text{impl}}^*$ be
 - the result of **typing** typeArgumentList , **under cursor** p **and typing context** TC .
- d. Let argumentIR^* be
 - the result of **typing** argumentList , **under cursor** p **and typing context** TC .
- e. Let the callable type callableTypeIR with fresh type variables $\text{typeId}_{\text{inserted}}^*$, defaults $\text{id}_{\text{default}}^*$ and optionals $\text{id}_{\text{optional}}^*$ be
 - the result of **finding the type of callable** $\text{call} \text{ callableTargetIR} < \text{typeArgumentIR}^* > (\text{argumentIR}^*)$, **under cursor** p **and typing context** TC .
- f. Let $\text{typeId}_{\text{infer}}^*$ be $\text{typeId}_{\text{impl}}^*$ concatenated with $\text{typeId}_{\text{inserted}}^*$.
- g. Let the return type $\text{typeIR}_{\text{ret}}$ with inferred type arguments $\text{typeArgumentIR}_{\text{inferred}}^*$ and the casted argument list $\text{argumentIR}_{\text{cast}}^*$ be
 - the result of **typing the invocation** $\text{callableTypeIR} < \text{typeArgumentIR}^* > (\text{argumentIR}^*)$ **with omitted type arguments** $\text{typeId}_{\text{infer}}^*$, **defaults** $\text{id}_{\text{default}}^*$ **and optionals** $\text{id}_{\text{optional}}^*$, **under cursor** p **and typing context** TC .
- h. Let callStatementIR be $\text{callableTargetIR} < \text{typeArgumentIR}_{\text{inferred}}^* > (\text{argumentIR}_{\text{cast}}^*) ;$.
- i. Result in
 - the updated typing context TC , abstract control flow f , and callStatementIR .

13.4.2. Compile-time evaluation

Compile-time evaluation of callStatementIR , **under cursor** p , **instantiation context** IC , **and store** STO :

1. Result in
 - the updated instantiation context IC , store STO , and callStatementIR .

13.4.3. Runtime evaluation

Runtime evaluation of $\text{callableTargetIR} < \text{typeArgumentIR}^* > (\text{argumentIR}^*) ;$, **under cursor** p , **evaluation context** EC , **and store** STO :

1. Let the updated evaluation context EC_1 , store STO_1 , and callee calleeResult be
 - the result of **finding the callee** $\text{callableTargetIR} (\text{argumentIR}^*)$, **under cursor** p , **evaluation context** EC , **and store** STO .
2. If let abortResult be calleeResult :
 - a. Result in
 - the updated evaluation context EC_1 , store STO_1 , and abortResult .
3. Else:
 - a. Let callee be calleeResult .
 - b. Let the updated evaluation context EC_2 , store STO_2 , and call result callResult be
 - the result of **calling** $\text{callee} < \text{typeArgumentIR}^* > (\text{argumentIR}^*)$, **under cursor** p , **evaluation context** EC_1 , **and store** STO_1 .
 - c. If let abortResult be callResult :

i. Result in

- the updated evaluation context EC_2 , store STO_2 , and `abortResult`.

d. Else:

i. Result in

- the updated evaluation context EC_2 , store STO_2 , and `EMPTY`.

13.5. Direct type invocation

Controls and parsers are often instantiated exactly once. As a light syntactic sugar, control and parser declarations with no constructor parameters may be applied directly, as if they were an instance. This has the effect of creating and applying a local instance of that type.

```
control Callee(/* parameters omitted */) { /* body omitted */ }

control Caller(/* parameters omitted */)(/* parameters omitted */) {
  apply {
    Callee.apply(/* arguments omitted */); // Callee is treated as an instance
  }
}
```

The definition of `Caller` is equivalent to the following.

```
control Caller(/* parameters omitted */)(/* parameters omitted */) {
  @name("Callee") Callee() Callee_inst; // local instance of Callee
  apply {
    Callee_inst.apply(/* arguments omitted */); // Callee_inst is applied
  }
}
```

```
directApplicationStatement
: namedType . APPLY `( argumentList ) ;
;
```

This feature is intended to streamline the common case where a type is instantiated exactly once. The grammar allows direct calls for generic controls or parsers:

```
control Callee<T>(/* parameters omitted */) { /* body omitted */ }

control Caller(/* parameters omitted */)(/* parameters omitted */) {
  apply {
    // Callee<bit<32>> is treated as an instance
    Callee<bit<32>>.apply(/* arguments omitted */);
  }
}
```

For completeness, the behavior of directly invoking the same type more than once is defined as follows.

- Direct type invocation in different scopes will result in different local instances with different fully-qualified control names.
- In the same scope, direct type invocation will result in a different local instance per invocation—however, instances of the same type will share the same global name, via the `@name` annotation. If the type contains controllable entities, then invoking it directly more than once in the same scope is

illegal, because it will produce multiple controllable entities with the same fully-qualified control name.

See [\[sec-name-annotations\]](#) for details of `@name` annotations.

No direct invocation is possible for controls or parsers that require constructor arguments. These need to be instantiated before they are invoked.

NOTE Specify the controllable name part carefully in P4-SpecTec.

13.5.1. Type checking

After type checking, a direct application statement is represented as:

```
directApplicationStatementIR
: constructorTargetIR . APPLY `( argumentListIR ) ;
;
```

Typing `namedType . APPLY ( argumentList ) ;`, under cursor `p`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. Let `expressionIR` annotated with type `typeIRobject` and compile-time known-ness `_` be
 - the result of `typing namedType ( `EMPTY)`, under cursor `p` and typing context `TC`.
2. Check that `expressionIR` has type `callExpressionIR`.
3. Let `callExpressionIR` be `expressionIR`.
4. Check that `callExpressionIR` matches pattern `% ( % )`.
5. Let `constructorTargetIR ( argumentIR* )` be `callExpressionIR`.
6. Check that `argumentIR*` is an empty list.
7. Let `prefixedNameIR < typeArgumentIR* >` be `constructorTargetIR`.
8. Let `typeIRobject_unroll` be `typeIRobject` with `typeddefs` unrolled.
9. Check that `typeIRobject_unroll` has type `parserObjectTypeIR` or `typeIRobject_unroll` has type `controlObjectTypeIR`.
10. Let `nameIRobject` be `"__direct_application"`.
11. Let `TC1` be `TC` where `nameIRobject` to ``EMPTY typeIRobject CTK` is added to the `p` layer.
12. Let `lvalue` be ``ID nameIRobject . `ID "apply"`.
13. Let the updated typing context `_`, abstract control flow `_`, and `statementIR` be
 - the result of `typing lvalue ( argumentList ) ;`, under cursor `p`, typing context `TC1`, abstract control flow `f`, and loop context `l`.
14. Check that `statementIR` has type `callStatementIR`.
15. Let `callStatementIR` be `statementIR`.
16. Let `callableTargetIR < typeArgumentIR* > ( argumentIRcast* )` be `callStatementIR`.
17. Check that `callableTargetIR` matches pattern `% . %`.
18. Let `typedExpressionIRbase . text` be `callableTargetIR`.
19. Check that `text` is equal to `"apply"`.
20. Check that `typeArgumentIR*` is an empty list.

21. Check that $\text{nameIR}_{\text{object}}$ annotated with type $\text{typeIR}_{\text{object}}$ and compile-time known-ness CTK is equal to $\text{typedExpressionIR}_{\text{base}}$.
22. Check that $\text{constructorTargetIR}$ is equal to $\text{prefixedNameIR} < \text{typeArgumentIR}^* >$.
23. Let $\text{directApplicationStatementIR}$ be $\text{constructorTargetIR} . \text{APPLY} (\text{argumentIR}_{\text{cast}}^*) ;$.
24. Result in
 - the updated typing context TC , abstract control flow f , and $\text{directApplicationStatementIR}$.

13.5.2. Compile-time evaluation

At compile-time, a direct application is instantiated, and then desugared into a method call on that instance with the relation:

DirectApplicationStmt_inst:

- Compile-time evaluation of $\text{directApplicationStatementIR}$, under cursor cursor , instantiation context instContext , and store store :
- Results in
 - the updated store store and desugared $\text{constantDeclarationIR}$ callStatementIR .

It is implemented as:

Compile-time evaluation of $\text{constructorTargetIR} . \text{APPLY} (\text{argumentListIR}) ;$, under cursor p , instantiation context IC , and store STO_0 :

1. Let $\text{prefixedNameIR} < \text{typeArgumentListIR} >$ be $\text{constructorTargetIR}$.
2. Let the constructor $\text{constructorDef} < \text{typeArgumentListIR}_{\text{inst}} >$ with defaults $\text{id}_{\text{default}}^*$ and optionals $\text{id}_{\text{optional}}^*$ be
 - the result of finding constructor for $\text{prefixedNameIR} < \text{typeArgumentListIR} > (\cdot)$ under cursor p , and instantiation context IC .
3. Let typeId be the string form of prefixedNameIR .
4. Let $\text{IC}_{\text{callee}}$ be IC with typeId added to the path.
5. Let the updated store STO_1 , and the constructed object be
 - the result of loading $\text{constructorDef} < \text{typeArgumentListIR}_{\text{inst}} > (\cdot)$ with defaults $\text{id}_{\text{default}}^*$ and optionals $\text{id}_{\text{optional}}^*$, under cursor p , instantiation context $\text{IC}_{\text{callee}}$, and store STO_0 .
6. Let $\text{typeId}_{\text{fresh}}$ be a fresh type variable.
7. Let typeIR be typeId .
8. Let nameIR be the concatenation of $[\text{typeId}, \text{"_"}, \text{typeId}_{\text{fresh}}]$.
9. Let objectId be IC.PATH concatenated with typeId .
10. Let STO_2 be STO_1 where objectId to object is added.
11. Let $\text{constantDeclarationIR}$ be `EMPTY CONST typeIR nameIR = VALUE REF objectId ;`.
12. Let $\text{typedExpressionIR}_{\text{base}}$ be nameIR annotated with type typeIR and compile-time known-ness CTK .
13. Let callableTargetIR be $\text{typedExpressionIR}_{\text{base}} . \text{"apply"}$.
14. Let callStatementIR be $\text{callableTargetIR} < \text{typeArgumentListIR}_{\text{inst}} > (\text{argumentListIR}) ;$.

15. Result in

- the updated store STO_2 and desugared `constantDeclarationIR callStatementIR`.

A direct application statement is instantiated as:

Compile-time evaluation of `directApplicationStatementIR`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. Let the updated store STO_1 and desugared `constantDeclarationIR callStatementIR` be
 - the result of compile-time evaluation of `directApplicationStatementIR`, under cursor `p`, instantiation context `IC`, and store `STO`.
2. Let `blockStatementIR` be `EMPTY { [ constantDeclarationIR, callStatementIR ] }`.
3. Result in
 - the updated instantiation context `IC`, store STO_1 , and `blockStatementIR`.

13.6. Return statements

The `return` statement immediately terminates the execution of the `action`, `function` or `control` containing it. `return` statements are not allowed within parsers. `return` statements followed by an expression are only allowed within functions that return values; in this case the type of the expression must match the return type of the function. Any copy-out behavior due to direction `out` or `inout` parameters of the enclosing `action`, `function`, or `control` are still performed after the execution of the `return` statement. See [Section 18.4](#) for details on copy-out behavior.

```
returnStatement
: RETURN ;
| RETURN expression ;
;
```

13.6.1. Type checking

After type checking, a `return` statement has the form:

```
returnStatementIR
: RETURN ;
| RETURN typedExpressionIR ;
;
```

Typing `returnStatement`, under cursor `LOCAL`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. If `returnStatement` is `RETURN ;`:
 - a. Check that `VOID` is equal to the expected return type in `TC`.
 - b. Result in
 - the updated typing context `TC`, abstract control flow `RET`, and `RETURN ;`.
2. Else:

- a. Let `RETURN expression ;` be `returnStatement`.
- b. Let `typedExpressionIR` be
 - the result of `typing expression`, under cursor `LOCAL` and typing context `TC`.
- c. Let `typeIR` be the type of `typedExpressionIR`.
- d. Let `typeIRret` be ! the expected return type in `TC`.
- e. Let `typedExpressionIRcast` be ! `typedExpressionIR` implicitly cast to `typeIRret`.
- f. Result in
 - the updated typing context `TC`, abstract control flow `RET`, and `RETURN typedExpressionIRcast ;`.

13.6.2. Compile-time evaluation

Compile-time evaluation of `returnStatementIR`, under cursor `LOCAL`, instantiation context `IC`, and store `STO`:

1. Result in
 - the updated instantiation context `IC`, store `STO`, and `returnStatementIR`.

13.6.3. Runtime evaluation

Runtime evaluation of `returnStatementIR`, under cursor `LOCAL`, evaluation context `EC`, and store `STO`:

1. If `returnStatementIR` is `RETURN ;`:
 - a. Result in
 - the updated evaluation context `EC`, store `STO`, and `RETURN` .
2. Else:
 - a. Let `RETURN typedExpressionIR ;` be `returnStatementIR`.
 - b. Let the updated evaluation context `EC1`, store `STO1` and `expressionResult` be
 - the result of `evaluating typedExpressionIR`, under cursor `LOCAL`, evaluation context `EC`, and store `STO`.
 - c. If let `abortResult` be `expressionResult`:
 - i. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `abortResult`.
 - d. Else:
 - i. Let `value` be `expressionResult`.
 - ii. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `RETURN value`.

13.7. Exit statements

The `exit` statement immediately terminates the execution of all the blocks currently executing: the current

action (if invoked within an **action**), the current **control**, and all its callers. **exit** statements are not allowed within parsers or functions.

Any copy-out behavior due to direction **out** or **inout** parameters of the enclosing **action** or **control**, and all of its callers, are still performed after the execution of the **exit** statement. See [Section 18.4](#) for details on copy-out behavior.

```
exitStatement
: EXIT ;
;
```

There are some expressions whose evaluation might cause an **exit** statement to be executed. Examples include:

- **table.apply().action_run**, which can only appear as the expression of a **switch** statement (see [Section 13.13](#)), and when it appears there, it must be the entire expression.
- Any expression containing **table.apply().hit** or **table.apply().miss** (see [\[sec-invoke-mau\]](#)), which can be part of arbitrarily complex expressions in many places of a P4 program, such as:
 - the expression in an **if** statement.
 - the expression **e1** in a conditional expression **e1 ? e2 : e3**.
 - in an assignment statement, in the left and/or right hand sides.
 - an argument passed to a function or method call.
 - an expression to calculate a table key (see [\[sec-mau-semantics\]](#)).

This list is not intended to be exhaustive.

If applying the table causes an action to be executed, which in turn causes an **exit** statement to be executed, then evaluation of the expression ends immediately, and the rest of the current expression or statement does not complete its execution. See [\[sec-expr-eval-order\]](#) for the order of evaluation of the parts of an expression. For the examples listed above, it also means the following behavior after the expression evaluation is interrupted.

- For a **switch** statement, if **table.apply()** exits, then none of the blocks in the **switch** statement are executed.
- If **table.apply().hit** or **table.apply().miss** cause an exit during the evaluation of an expression:
 - If it is the expression of an **if** statement, then neither the 'then' nor 'else' branches of the **if** statement are executed.
 - If it is the expression **e1** in a conditional expression **e1 ? e2 : e3**, then neither expression **e2** nor **e3** are evaluated.
 - If the expression is the right hand side of an assignment statement, or part of the calculation of the L-value on the left hand side (e.g. the index expression of a header stack reference), then no assignment occurs.
 - If the expression is an argument passed to a function or method call, then the function/method call does not occur.
 - If the expression is a table key, then the table is not applied.

13.7.1. Type checking

After type checking, an **exit** statement is represented in P4_{IR} as:

```
exitStatementIR = exitStatement
```

Typing `EXIT ;`, under cursor `p`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. Result in
 - the updated typing context `TC`, abstract control flow `f`, and `EXIT ;`.

13.7.2. Compile-time evaluation

Compile-time evaluation of `exitStatementIR`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. Result in
 - the updated instantiation context `IC`, store `STO`, and `exitStatementIR`.

13.7.3. Runtime evaluation

Runtime evaluation of `EXIT ;`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Result in
 - the updated evaluation context `EC`, store `STO`, and `EXIT`.

13.8. Block statements

A block statement is denoted by curly braces. It contains a sequence of statements and declarations, which are executed sequentially. The declarations (e.g., variables and constants) within a block statement are only visible within the block.

```
blockStatement
: annotationList `{ blockElementStatementList }
;

blockElementStatementList
: /* empty */
| blockElementStatementList blockElementStatement
;

blockElementStatement
: constantDeclaration
| variableDeclaration
| statement
;
```

13.8.1. Type checking

After type checking, block statements are represented in $P4_{IR}$ as:

```
blockStatementIR
: annotationList `{ blockElementStatementListIR }
;

blockElementStatementListIR = blockElementStatementIR*
```

```

blockElementStatementIR
: constantDeclarationIR
| variableDeclarationIR
| statementIR
;

```

Typing `blockStatement`, under cursor `LOCAL`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. Let TC_1 be `TC` with a new local frame.
2. Let the updated typing context TC_2 , abstract control flow f_1 , and `blockStatementIR` be
 - the result of typing `blockStatement`, under typing context TC_1 , abstract control flow `f`, and loop context `l`.
3. Let TC_3 be TC_2 with the innermost frame removed.
4. Result in
 - the updated typing context TC_3 , abstract control flow f_1 , and `blockStatementIR`.

A block is type checked by the relation:

`Block_ok`:

- Typing `blockStatement`, under typing context `typingContext`, abstract control flow `flow`, and loop context `loopctx`:
- Results in
 - the updated typing context `typingContext`, abstract control flow `flow`, and `blockStatementIR`.

A statement or declaration within a block statement is type checked by the relation:

`BlockElementStmt_ok`:

- Typing `blockElementStatement`, under typing context `typingContext`, abstract control flow `flow`, and loop context `loopctx`:
- Results in
 - the updated typing context `typingContext`, abstract control flow `flow`, and `blockElementStatementIR`.

It is implemented as:

Typing `blockElementStatement`, under typing context TC_0 , abstract control flow `f`, and loop context `l`:

1. If let `constantDeclaration` be `blockElementStatement`:
 - a. Let the updated typing context TC_1 and `constantDeclarationIR` be
 - the result of typing `constantDeclaration`, under cursor `LOCAL` and typing context TC_0 .
 - b. Result in
 - the updated typing context TC_1 , abstract control flow `f`, and `constantDeclarationIR`.

2. Else if let `variableDeclaration` be `blockElementStatement`:
 - a. Let the updated typing context TC_1 and `variableDeclarationIR` be
 - the result of typing `variableDeclaration`, under cursor `LOCAL` and typing context TC_0 .
 - b. Result in
 - the updated typing context TC_1 , abstract control flow f , and `variableDeclarationIR`.
3. Else:
 - a. Let `statement` be `blockElementStatement`.
 - b. Let the updated typing context TC_1 , abstract control flow f_{post} , and `statementIR` be
 - the result of typing `statement`, under cursor `LOCAL`, typing context TC_0 , abstract control flow f , and loop context l .
 - c. Result in
 - the updated typing context TC_1 , abstract control flow f_{post} , and `statementIR`.

A list of block elements is type checked by the relation:

`BlockElementStmtList_ok`:

- Typing `blockElementStatementList`, under typing context `typingContext`, abstract control flow `flow`, and loop context `loopctxt`:
- Results in
 - the updated typing context `typingContext`, abstract control flow `flow`, and `blockElementStatementIR*`.

13.8.2. Compile-time evaluation

`Compile-time evaluation of annotationList { blockElementStatementListIR }, under cursor LOCAL, instantiation context IC, and store STO`:

1. Let IC_1 be `IC` with a new local frame.
2. Let the updated instantiation context IC_2 , store STO_1 , and `_ { blockElementStatementListIRinst }` be
 - the result of `compile-time evaluation of annotationList { blockElementStatementListIR }, under instantiation context  $IC_1$ , and store  $STO$` .
3. Let IC_3 be IC_2 with the innermost frame removed.
4. Result in
 - the updated instantiation context IC_3 , store STO_1 , and `annotationList { blockElementStatementListIRinst }`.

A block is compile-time evaluated by the relation:

`Block_inst`:

- Compile-time evaluation of `blockStatementIR`, under instantiation context `instContext`, and store `store`:

- Results in
 - the updated instantiation context `instContext`, store `store`, and `blockStatementIR`.

A block element is compile-time evaluated by the relation:

`BlockElementStmt_inst`:

- Compile-time evaluation of `blockElementStatementIR`, under instantiation context `instContext`, and store `store`:
- Results in
 - the updated instantiation context `instContext`, store `store`, and `blockElementStatementIR`.

Compile-time evaluation of `blockElementStatementIR`, under instantiation context `instContext`, and store `store`:

1. If let `constantDeclarationIR` be `blockElementStatementIR`:
 - a. Let the updated instantiation context IC_1 be
 - the result of `loading` `constantDeclarationIR`, under cursor `LOCAL`, and instantiation context `instContext`.
 - b. Result in
 - the updated instantiation context IC_1 , store `store`, and `constantDeclarationIR`.
2. Else if let `variableDeclarationIR` be `blockElementStatementIR`:
 - a. Result in
 - the updated instantiation context `instContext`, store `store`, and `variableDeclarationIR`.
3. Else:
 - a. Let `statementIR` be `blockElementStatementIR`.
 - b. Let the updated instantiation context IC_1 , store STO_1 , and `statementIRinst` be
 - the result of `compile-time evaluation of` `statementIR`, under cursor `LOCAL`, instantiation context `instContext`, and store `store`.
 - c. Result in
 - the updated instantiation context IC_1 , store STO_1 , and `statementIRinst`.

A list of block elements is compile-time evaluated by the relation:

`BlockElementStmtList_inst`:

- Compile-time evaluation of `blockElementStatementListIR`, under instantiation context `instContext`, and store `store`:
- Results in
 - the updated instantiation context `instContext`, store `store`, and `blockElementStatementIR`*

13.8.3. Runtime evaluation

Runtime evaluation of `blockStatementIR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let `EC1` be `EC` with a new local frame.
2. Let the updated evaluation context `EC2`, store `STO1`, and `statementResult` be
 - the result of runtime evaluation of `blockStatementIR`, under evaluation context `EC1`, and store `STO`.
3. Let `EC3` be `EC2` with the innermost frame removed.
4. Result in
 - the updated evaluation context `EC3`, store `STO1`, and `statementResult`.

A block is runtime evaluated by the relation:

`Block_eval`:

- Runtime evaluation of `blockStatementIR`, under evaluation context `evalContext`, and store `store`:
- Results in
 - the updated evaluation context `evalContext`, store `store`, and `statementResult`.

A block element is runtime evaluated by the relation:

`BlockElementStmt_eval`:

- Runtime evaluation of `blockElementStatementIR`, under evaluation context `evalContext`, and store `store`:
- Results in
 - the updated evaluation context `evalContext`, store `store`, and `statementResult`.

Runtime evaluation of `blockElementStatementIR`, under evaluation context `EC0`, and store `store`:

1. If let `constantDeclarationIR` be `blockElementStatementIR`:
 - a. Let the updated evaluation context `EC1` be
 - the result of evaluating `constantDeclarationIR`, under cursor `LOCAL`, and evaluation context `EC0`.
 - b. Result in
 - the updated evaluation context `EC1`, store `store`, and `'EMPTY'`.
2. Else if let `variableDeclarationIR` be `blockElementStatementIR`:
 - a. Let the updated evaluation context `EC1`, store `STO1`, and the result `declarationResult` be
 - the result of evaluating `variableDeclarationIR`, under cursor `LOCAL`, evaluation context `EC0`, and store `store`.
 - b. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `declarationResult`.
3. Else:

- a. Let `statementIR` be `blockElementStatementIR`.
- b. Let the updated evaluation context `EC1`, store `STO1`, and `statementResult` be
 - the result of runtime evaluation of `statementIR`, under cursor `LOCAL`, evaluation context `EC0`, and store `store`.
- c. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `statementResult`.

A list of block elements is runtime evaluated by the relation:

`BlockElementStmtList_eval`:

- Runtime evaluation of `blockElementStatementListIR`, under evaluation context `evalContext`, and store `store`:
- Results in
 - the updated evaluation context `evalContext`, store `store`, and `statementResult`.

13.9. Conditional statements

The conditional statement uses standard syntax and semantics familiar from many programming languages.

However, the condition expression in P4 is required to be a Boolean (and not an integer).

```
conditionalStatement
: IF `( expression ) statement
| IF `( expression ) statement ELSE statement
;
```

When several `if` statements are nested, the `else` applies to the innermost `if` statement that does not have an `else` statement.

13.9.1. Type checking

After type checking, a conditional statement has the form:

```
conditionalStatementIR
: IF `( typedExpressionIR ) statementIR
| IF `( typedExpressionIR ) statementIR ELSE statementIR
;
```

Typing `conditionalStatement`, under cursor `p`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. If let `IF ( expressioncond ) statementthen` be `conditionalStatement`:
 - a. Let `typedExpressionIRcond` be
 - the result of typing `expressioncond`, under cursor `p` and typing context `TC`.
 - b. Let `typeIRcond` be the type of `typedExpressionIRcond`.

- c. Check that **BOOL** is equal to $\text{typeIR}_{\text{cond}}$ with typedefs unrolled.
 - d. Let the updated typing context TC_{then} , abstract control flow f_{then} , and $\text{statementIR}_{\text{then}}$ be
 - the result of typing $\text{statement}_{\text{then}}$, under cursor p , typing context TC , abstract control flow f , and loop context l .
 - e. Result in
 - the updated typing context TC , abstract control flow f , and $\text{IF} (\text{typedExpressionIR}_{\text{cond}}) \text{statementIR}_{\text{then}}$.
2. Else:
- a. Let $\text{IF} (\text{expression}_{\text{cond}}) \text{statement}_{\text{then}} \text{ELSE} \text{statement}_{\text{else}}$ be **conditionalStatement**.
 - b. Let $\text{typedExpressionIR}_{\text{cond}}$ be
 - the result of typing $\text{expression}_{\text{cond}}$, under cursor p and typing context TC .
 - c. Let $\text{typeIR}_{\text{cond}}$ be the type of $\text{typedExpressionIR}_{\text{cond}}$.
 - d. Check that **BOOL** is equal to $\text{typeIR}_{\text{cond}}$ with typedefs unrolled.
 - e. Let the updated typing context TC_{then} , abstract control flow f_{then} , and $\text{statementIR}_{\text{then}}$ be
 - the result of typing $\text{statement}_{\text{then}}$, under cursor p , typing context TC , abstract control flow f , and loop context l .
 - f. Let the updated typing context TC_{else} , abstract control flow f_{else} , and $\text{statementIR}_{\text{else}}$ be
 - the result of typing $\text{statement}_{\text{else}}$, under cursor p , typing context TC , abstract control flow f , and loop context l .
 - g. Let f_{post} be the merged control flow of f_{then} and f_{else} .
 - h. Result in
 - the updated typing context TC , abstract control flow f_{post} , and $\text{IF} (\text{typedExpressionIR}_{\text{cond}}) \text{statementIR}_{\text{then}} \text{ELSE} \text{statementIR}_{\text{else}}$.

13.9.2. Compile-time evaluation

Compile-time evaluation of $\text{conditionalStatementIR}$, under cursor p , instantiation context IC , and store STO :

1. If let $\text{IF} (\text{typedExpressionIR}_{\text{cond}}) \text{statementIR}_{\text{then}}$ be **conditionalStatementIR**:
 - a. Let the updated instantiation context IC_{then} , store STO_1 , and $\text{statementIR}_{\text{then_inst}}$ be
 - the result of compile-time evaluation of $\text{statementIR}_{\text{then}}$, under cursor p , instantiation context IC , and store STO .
 - b. Result in
 - the updated instantiation context IC , store STO_1 , and $\text{IF} (\text{typedExpressionIR}_{\text{cond}}) \text{statementIR}_{\text{then_inst}}$.
2. Else:
 - a. Let $\text{IF} (\text{typedExpressionIR}_{\text{cond}}) \text{statementIR}_{\text{then}} \text{ELSE} \text{statementIR}_{\text{else}}$ be **conditionalStatementIR**.
 - b. Let the updated instantiation context IC_{then} , store STO_1 , and $\text{statementIR}_{\text{then_inst}}$ be
 - the result of compile-time evaluation of $\text{statementIR}_{\text{then}}$, under cursor p , instantiation context IC , and store STO .
 - c. Let the updated instantiation context IC_{else} , store STO_2 , and $\text{statementIR}_{\text{else_inst}}$ be

- the result of **compile-time evaluation of** $\text{statementIR}_{\text{else}}$, **under cursor** p , **instantiation context** IC , and **store** STO_1 .

d. Result in

- the updated instantiation context IC , store STO_2 , and **IF** ($\text{typedExpressionIR}_{\text{cond}}$) $\text{statementIR}_{\text{then_inst}}$ **ELSE** $\text{statementIR}_{\text{else_inst}}$.

13.9.3. Runtime evaluation

Runtime evaluation of $\text{conditionalStatementIR}$, **under cursor** p , **evaluation context** EC , and **store** STO :

1. If let **IF** ($\text{typedExpressionIR}_{\text{cond}}$) $\text{statementIR}_{\text{then}}$ **be** $\text{conditionalStatementIR}$:
 - a. Let the updated evaluation context EC_{cond} , store STO_{cond} and **expressionResult** be
 - the result of **evaluating** $\text{typedExpressionIR}_{\text{cond}}$, **under cursor** p , **evaluation context** EC , and **store** STO .
 - b. If let **abortResult** be **expressionResult**:
 - i. Result in
 - the updated evaluation context EC_{cond} , store STO_{cond} , and **abortResult**.
 - c. If **expressionResult** is equal to **`B true`**:
 - i. Let the updated evaluation context EC_{then} , store STO_{then} , and **statementResult** be
 - the result of **runtime evaluation of** $\text{statementIR}_{\text{then}}$, **under cursor** p , **evaluation context** EC_{cond} , and **store** STO_{cond} .
 - ii. Result in
 - the updated evaluation context EC_{then} , store STO_{then} , and **statementResult**.
 - d. If **expressionResult** is equal to **`B false`**:
 - i. Result in
 - the updated evaluation context EC_{cond} , store STO_{cond} , and **`EMPTY`**.
2. Else:
 - a. Let **IF** ($\text{typedExpressionIR}_{\text{cond}}$) $\text{statementIR}_{\text{then}}$ **ELSE** $\text{statementIR}_{\text{else}}$ **be** $\text{conditionalStatementIR}$.
 - b. Let the updated evaluation context EC_{cond} , store STO_{cond} and **expressionResult** be
 - the result of **evaluating** $\text{typedExpressionIR}_{\text{cond}}$, **under cursor** p , **evaluation context** EC , and **store** STO .
 - c. If let **abortResult** be **expressionResult**:
 - i. Result in
 - the updated evaluation context EC_{cond} , store STO_{cond} , and **abortResult**.
 - d. If **expressionResult** is equal to **`B true`**:
 - i. Let the updated evaluation context EC_{then} , store STO_{then} , and **statementResult** be
 - the result of **runtime evaluation of** $\text{statementIR}_{\text{then}}$, **under cursor** p , **evaluation context** EC_{cond} , and **store** STO_{cond} .
 - ii. Result in
 - the updated evaluation context EC_{then} , store STO_{then} , and **statementResult**.
 - e. If **expressionResult** is equal to **`B false`**:

- i. Let the updated evaluation context EC_{else} , store STO_{else} , and `statementResult` be
 - the result of runtime evaluation of `statementIRelse`, under cursor `p`, evaluation context EC_{cond} , and store STO_{cond} .
- ii. Result in
 - the updated evaluation context EC_{else} , store STO_{else} , and `statementResult`.

13.10. For statements

A `for` statement executes a statement zero or more times, based on a condition or a range or collection.

```
forStatement
  : annotationList FOR `( forInitStatementList ; expression ; forUpdateStatementList )
  statement
  | annotationList FOR `( type name IN forCollectionExpression ) statement
  | annotationList FOR `( annotationListNonEmpty type name IN forCollectionExpression )
  statement
  ;

forInitStatementList
  : /* empty */
  | forInitStatementListNonEmpty
  ;

forInitStatementListNonEmpty
  : forInitStatement
  | forInitStatementListNonEmpty , forInitStatement
  ;

forInitStatement
  : annotationList type name initializerOpt
  | forUpdateStatement
  ;

forUpdateStatementList
  : /* empty */
  | forUpdateStatementListNonEmpty
  ;

forUpdateStatementListNonEmpty
  : forUpdateStatement
  | forUpdateStatementListNonEmpty , forUpdateStatement
  ;

forUpdateStatement
  : lvalue `( argumentList )
  | lvalue `( typeArgumentList > `( argumentList )
  | lvalue assignop expression
  ;

forCollectionExpression
  : expression
  | expression .. expression
  ;
```

There are two forms of `for` statements:

- Basic 3-clause `for` statements with initialization, condition, and update.
- `for-in` statements that iterate over a range, list, or header stack.

A **break**; statement may be executed in a loop body to immediately exit the loop, without executing the rest of the body or the update statements. **break**; statements are explained in detail in [Section 13.11](#).

A **continue**; statement may be executed in a loop body to skip the rest of the current loop body, executing the update statements and then evaluating the condition to either execute the loop body again, or terminate the loop. **continue**; statements are explained in detail in [Section 13.12](#).

The scope of any declaration in a **for** statement is limited to the **for** statement and its body.

After type checking, a **for** statement has the form:

```
forStatementIR
  : annotationList FOR `( forInitStatementListIR ; typedExpressionIR ;
forUpdateStatementListIR ) statementIR
  | annotationList FOR `( typeIR nameIR IN forCollectionExpressionIR ) statementIR
  | annotationList FOR `( annotationList typeIR nameIR IN forCollectionExpressionIR )
statementIR
  ;
```

13.10.1. 3-clause **for** statements

The basic 3-clause **for** statement is similar to a C **for** statement. The init statements will be executed prior to executing the loop. The condition will be evaluated before each iteration, to determine if the loop should exit. If the condition is false, the loop will exit without executing any statements from the loop body or update. The update statements will be executed after the loop body and before evaluating the condition again for the next iteration.

13.10.1.1. Type checking

After type checking, a 3-clause **for** statement has the form:

```
forStatementIR
  : annotationList FOR `( forInitStatementListIR ; typedExpressionIR ;
forUpdateStatementListIR ) statementIR
  | annotationList FOR `( typeIR nameIR IN forCollectionExpressionIR ) statementIR
  | annotationList FOR `( annotationList typeIR nameIR IN forCollectionExpressionIR )
statementIR
  ;

forInitStatementListIR = forInitStatementIR*

forInitStatementIR
  : annotationList typeIR nameIR initializerOptIR
  | forUpdateStatementIR
  ;

forUpdateStatementListIR = forUpdateStatementIR*

forUpdateStatementIR
  : callableTargetIR `< typeArgumentListIR > `( argumentListIR )
  | typedLvalueIR assignop typedExpressionIR
  ;
```

Typing `annotationList FOR ( forInitStatementList ; expression ; forUpdateStatementList ) statement'`, under cursor `LOCAL`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. Let `TC1` be `TC` with a new local frame.

2. Let the updated typing context TC_2 and $forInitStatementIR^*$ be
 - the result of **typing** $forInitStatementList$ **under typing context** TC_1 .
3. Let $typedExpressionIR$ be
 - the result of **typing** $expression$, **under cursor** $LOCAL$ **and typing context** TC_2 .
4. Let $typeIR$ be **the type of** $typedExpressionIR$.
5. Check that $BOOL$ is equal to $typeIR$ **with typedefs unrolled**.
6. Let $forUpdateStatementIR^*$ be
 - the result of **typing** $forUpdateStatementList$ **under typing context** TC_2 .
7. Let the updated typing context TC_3 , abstract control flow f_{post} , and $statementIR$ be
 - the result of **typing** $statement'$, **under cursor** $LOCAL$, **typing context** TC_2 , **abstract control flow** f , **and loop context** $LOOP$.
8. Let TC_4 be TC_3 **with the innermost frame removed**.
9. Let $forStatementIR$ be $annotationList$ **FOR** ($forInitStatementIR^*$; $typedExpressionIR$; $forUpdateStatementIR^*$) $statementIR$.
10. Result in
 - the updated typing context TC , abstract control flow f_{post} , and $forStatementIR$.

A list of initialization statements are type checked with the relation:

ForInitStmtList_ok:

- Typing $forInitStatementList$ under typing context $typingContext$:
- Results in the updated typing context $typingContext$ and $forInitStatementIR^*$.

This relation invokes the following relation for each initialization statement:

ForInitStmt_ok:

- Typing $forInitStatement$ under typing context $typingContext$:
- Results in the updated typing context $typingContext$ and $forInitStatementIR$.

Typing $forInitStatement$ **under typing context** $typingContext$:

1. If let $annotationList$ type name $initializerOpt$ be $forInitStatement$:
 - a. If $initializerOpt$ is `EMPTY`:
 - i. Let $typeIR$ and a set of fresh type variables $typeId^*$ be
 - the result of **typing** $type$, **under cursor** $LOCAL$ **and typing context** $typingContext$.
 - ii. Check that $typeId^*$ is an empty list.
 - iii. Let $bound$ be **bound type variables from the** $LOCAL$ **layer of** $typingContext$.
 - iv. Check that $typeIR$ is a well-formed type, with bound type variables $bound$.
 - v. Check that $typeIR$ supports assignment.

- vi. Let nameIR be the name of name .
 - vii. Let TC_1 be typingContext where nameIR to INOUT typeIR DYN · is added to the LOCAL layer.
 - viii. Result in the updated typing context TC_1 and annotationList typeIR nameIR ·.
- b. Else:
- i. Let $\text{= expression}_{\text{init}}$ be initializerOpt.
 - ii. Let typeIR and a set of fresh type variables typeId^* be
 - the result of typing type, under cursor LOCAL and typing context typingContext.
 - iii. Check that typeId^* is an empty list.
 - iv. Let bound be bound type variables from the LOCAL layer of typingContext.
 - v. Check that typeIR is a well-formed type, with bound type variables bound .
 - vi. Check that typeIR supports assignment.
 - vii. Let $\text{typedExpressionIR}_{\text{init}}$ be
 - the result of typing $\text{expression}_{\text{init}}$, under cursor LOCAL and typing context typingContext.
 - viii. Let $\text{typedExpressionIR}_{\text{init_cast}}$ be ! $\text{typedExpressionIR}_{\text{init}}$ implicitly cast to typeIR .
 - ix. Let nameIR be the name of name .
 - x. Let TC_1 be typingContext where nameIR to INOUT typeIR DYN · is added to the LOCAL layer.
 - xi. Result in the updated typing context TC_1 and annotationList $\text{typeIR nameIR} = \text{typedExpressionIR}_{\text{init_cast}}$.
2. Else:
- a. Let $\text{forUpdateStatement}$ be forInitStatement .
 - b. Let $\text{forUpdateStatementIR}$ be
 - the result of typing $\text{forUpdateStatement}$ under typing context typingContext.
 - c. Result in the updated typing context typingContext and $\text{forUpdateStatementIR}$.

A list of update statements are type checked with the relation:

ForUpdateStmtList_ok:

- Typing $\text{forUpdateStatementList}$ under typing context typingContext:
- Results in $\text{forUpdateStatementIR}^*$.

This relation invokes the following relation for each update statement:

ForUpdateStmt_ok:

- Typing $\text{forUpdateStatement}$ under typing context typingContext:
- Results in $\text{forUpdateStatementIR}$.

Typing $\text{forUpdateStatement}$ under typing context TC:

1. If let $lvalue_{callable} (argumentList)$ be $forUpdateStatement$:
 - a. Let the updated typing context $_$, abstract control flow $_$, and $statementIR$ be
 - the result of **typing** $lvalue_{callable} (argumentList)$;, **under cursor** $LOCAL$, **typing context** TC , **abstract control flow** $CONT$, and **loop context** $NOLOOP$.
 - b. Check that $statementIR$ has type $callStatementIR$.
 - c. Let $callableTargetIR < typeArgumentIR^* > (argumentIR^*)$; be $statementIR$.
 - d. Result in $callableTargetIR < typeArgumentIR^* > (argumentIR^*)$.
2. Else if let $lvalue_{callable} < typeArgumentList > (argumentList)$ be $forUpdateStatement$:
 - a. Let the updated typing context $_$, abstract control flow $_$, and $statementIR$ be
 - the result of **typing** $lvalue_{callable} < typeArgumentList > (argumentList)$;, **under cursor** $LOCAL$, **typing context** TC , **abstract control flow** $CONT$, and **loop context** $NOLOOP$.
 - b. Check that $statementIR$ has type $callStatementIR$.
 - c. Let $callableTargetIR < typeArgumentIR^* > (argumentIR^*)$; be $statementIR$.
 - d. Result in $callableTargetIR < typeArgumentIR^* > (argumentIR^*)$.
3. Else:
 - a. Let $lvalue$ **assignop** expression be $forUpdateStatement$.
 - b. Let the updated typing context $_$, abstract control flow $_$, and $statementIR$ be
 - the result of **typing** $lvalue$ **assignop** expression ;, **under cursor** $LOCAL$, **typing context** TC , **abstract control flow** $CONT$, and **loop context** $NOLOOP$.
 - c. Check that $statementIR$ has type $assignmentStatementIR$.
 - d. Let $typedLvalueIR$ **assignop'** $typedExpressionIR$; be $statementIR$.
 - e. Check that **assignop'** is equal to **assignop**.
 - f. Result in $typedLvalueIR$ **assignop** $typedExpressionIR$.

13.10.1.2. Compile-time evaluation

Compile-time evaluation of $annotationList$ **FOR** ($forInitStatementListIR$; $typedExpressionIR$; $forUpdateStatementListIR$) $statementIR'$, **under cursor** p , **instantiation context** IC , and **store** STO :

1. Let the updated instantiation context IC_{body} , store STO_1 , and $statementIR_{body}$ be
 - the result of **compile-time evaluation of** $statementIR'$, **under cursor** p , **instantiation context** IC , and **store** STO .
2. Let $forStatementIR'$ be $annotationList$ **FOR** ($forInitStatementListIR$; $typedExpressionIR$; $forUpdateStatementListIR$) $statementIR_{body}$.
3. Result in
 - the updated instantiation context IC , store STO_1 , and $forStatementIR'$.

13.10.1.3. Runtime evaluation

Runtime evaluation of $annotationList$ **FOR** ($forInitStatementListIR$; $typedExpressionIR$; $forUpdateStatementListIR$) $statementIR'$, **under cursor** $LOCAL$, **evaluation context** EC , and **store** STO :

1. Let EC_1 be EC with a new local frame.
2. Let the updated evaluation context EC_2 , store STO_1 , and $statementResult$ be
 - the result of runtime evaluation of $forInitStatementListIR$, under evaluation context EC_1 and store STO .
3. If let $abortResult$ be $statementResult$:
 - a. Let EC_3 be EC_2 with the innermost frame removed.
 - b. Result in
 - the updated evaluation context EC_3 , store STO_1 , and $abortResult$.
4. Else if let $continueEmptyResult$ be $statementResult$:
 - a. Let the updated evaluation context EC_3 , store STO_2 , and $statementResult'$ be
 - the result of looping $statementIR'$ with condition $typedExpressionIR$ and updates $forUpdateStatementListIR$, under evaluation context EC_2 and store STO_1 .
 - b. Let EC_4 be EC_3 with the innermost frame removed.
 - c. Result in
 - the updated evaluation context EC_4 , store STO_2 , and $statementResult'$.

A list of initialization statements is evaluated with the relation:

$ForInitStmts_eval$:

- Runtime evaluation of $forInitStatementListIR$, under evaluation context $evalContext$ and store $store$:
- Results in
 - the updated evaluation context $evalContext$, store $store$, and $statementResult$.

An initialization statement is evaluated with the relation:

$ForInitStmt_eval$:

- Runtime evaluation of $forInitStatementIR$, under evaluation context $evalContext$ and store $store$:
- Results in
 - the updated evaluation context $evalContext$, store $store$, and $statementResult$.

Runtime evaluation of $forInitStatementIR$, under evaluation context EC_0 and store $store$:

1. If let $annotationList$ $typeIR$ $nameIR$ $initializerIR^?$ be $forInitStatementIR$:
 - a. If $initializerIR^?$ is none:
 - i. Let $typeIR_{subst}$ be $typeIR$ substituted by bound type variables in EC_0 from the LOCAL layer.
 - ii. Let $value_{init}$ be the default value for type $typeIR_{subst}$.
 - iii. Let EC_1 be EC_0 where $nameIR$ to $value_{init}$ is added to the LOCAL layer.
 - iv. Result in
 - the updated evaluation context EC_1 , store $store$, and $EMPTY$.

- b. Let `annotationList` typeIR' nameIR' initializerOptIR be `forInitStatementIR`.
 - c. If let `= typedExpressionIRinit` be `initializerOptIR`:
 - i. Let the updated evaluation context `EC1`, store `STO1` and `expressionResult` be
 - the result of **evaluating** `typedExpressionIRinit`, **under cursor** `LOCAL`, **evaluation context** `EC0`, **and store** `store`.
 - ii. If let `abortResult` be `expressionResult`:
 - A. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `abortResult`.
 - iii. Else:
 - A. Let `valueinit` be `expressionResult`.
 - B. Let `EC2` be `EC1` **where** `nameIR'` **to** `valueinit` **is added to the** `LOCAL` **layer**.
 - C. Result in
 - the updated evaluation context `EC2`, store `STO1`, and `EMPTY`.
2. Else:
 - a. Let `forUpdateStatementIR` be `forInitStatementIR`.
 - b. Let the updated evaluation context `EC1`, store `STO1`, and `statementResult` be
 - the result of **runtime evaluation of** `forUpdateStatementIR`, **under evaluation context** `EC0` **and store** `store`.
 - c. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `statementResult`.

A list of update statements is evaluated with the relation:

ForUpdateStmts_eval:

- Runtime evaluation of `forUpdateStatementListIR`, under evaluation context `evalContext` and store `store`:
- Results in
 - the updated evaluation context `evalContext`, store `store`, and `statementResult`.

An update statement is evaluated with the relation:

ForUpdateStmt_eval:

- Runtime evaluation of `forUpdateStatementIR`, under evaluation context `evalContext` and store `store`:
- Results in
 - the updated evaluation context `evalContext`, store `store`, and `statementResult`.

Runtime evaluation of `forUpdateStatementIR`, **under evaluation context** `EC0` **and store** `STO0`:

1. If let `callableTargetIR < typeArgumentListIR > ( argumentListIR )` be `forUpdateStatementIR`:

- a. Let the updated evaluation context EC_1 , store STO_1 , and $statementResult$ be
 - the result of runtime evaluation of $callableTargetIR < typeArgumentListIR > (argumentListIR)$;, under cursor $LOCAL$, evaluation context EC_0 , and store STO_0 .
- b. Result in
 - the updated evaluation context EC_1 , store STO_1 , and $statementResult$.
2. Else:
 - a. Let $typedLvalueIR$ assignop $typedExpressionIR$ be forUpdateStatementIR.
 - b. Let the updated evaluation context EC_1 , store STO_1 , and $statementResult$ be
 - the result of runtime evaluation of $typedLvalueIR$ assignop $typedExpressionIR$;, under cursor $LOCAL$, evaluation context EC_0 , and store STO_0 .
 - c. Result in
 - the updated evaluation context EC_1 , store STO_1 , and $statementResult$.

With these, the 3-clause *for* statement is evaluated with the relation:

ForThreeClauseStmt_eval:

- Looping $statementIR$ with condition $typedExpressionIR$ and updates $forUpdateStatementListIR$, under evaluation context $evalContext$ and store $store$:
- Results in
 - the updated evaluation context $evalContext$, store $store$, and $statementResult$.

Looping $statementIR$ with condition $typedExpressionIR$ and updates $forUpdateStatementListIR$, under evaluation context EC_0 and store STO_0 :

1. Let the updated evaluation context EC_1 , store STO_1 and $expressionResult$ be
 - the result of evaluating $typedExpressionIR$, under cursor $LOCAL$, evaluation context EC_0 , and store STO_0 .
2. If let $abortResult$ be $expressionResult$:
 - a. Result in
 - the updated evaluation context EC_1 , store STO_1 , and $abortResult$.
3. If $expressionResult$ is equal to `B false':
 - a. Result in
 - the updated evaluation context EC_1 , store STO_1 , and `EMPTY'.
4. If $expressionResult$ is equal to `B true':
 - a. Let the updated evaluation context EC_2 , store STO_2 , and $statementResult$ be
 - the result of runtime evaluation of $statementIR$, under cursor $LOCAL$, evaluation context EC_1 , and store STO_1 .
 - b. If let $abortResult$ be $statementResult$:
 - i. Result in
 - the updated evaluation context EC_2 , store STO_2 , and $abortResult$.

- c. Else if let `returnResult` be `statementResult`:
 - i. Result in
 - the updated evaluation context EC_2 , store STO_2 , and `returnResult`.
- d. Else if let `forBreakResult` be `statementResult`:
 - i. Result in
 - the updated evaluation context EC_2 , store STO_2 , and `'EMPTY'`.
- e. If `statementResult` is equal to `'EMPTY'` or `statementResult` is equal to `CONTINUE`:
 - i. Let the updated evaluation context EC_3 , store STO_3 , and `statementResult'` be
 - the result of runtime evaluation of `forUpdateStatementListIR`, under evaluation context EC_2 and store STO_2 .
 - ii. If let `exitResult` be `statementResult'`:
 - A. Result in
 - the updated evaluation context EC_3 , store STO_3 , and `EXIT`.
 - iii. Else if let `continueEmptyResult` be `statementResult'`:
 - A. Let the updated evaluation context EC_4 , store STO_4 , and `statementResult''` be
 - the result of looping `statementIR` with condition `typedExpressionIR` and updates `forUpdateStatementListIR`, under evaluation context EC_3 and store STO_3 .
 - B. Result in
 - the updated evaluation context EC_4 , store STO_4 , and `statementResult''`.

13.10.2. `for-in` statements

The `for-in` statement executes the body once for each value in a range or each element in a list expression or header stack. The list or range expression itself will only be evaluated once, before the first iteration of the loop. All side effects in the list or range expression will occur before the first iteration of the loop body.

13.10.2.1. Type checking

After type checking, a `for-in` statement has the form:

```
forStatementIR
: annotationList FOR `( forInitStatementListIR ; typedExpressionIR ;
forUpdateStatementListIR ) statementIR
| annotationList FOR `( typeIR nameIR IN forCollectionExpressionIR ) statementIR
| annotationList FOR `( annotationList typeIR nameIR IN forCollectionExpressionIR )
statementIR
;

forCollectionExpressionIR
: typedExpressionIR
| typedExpressionIR .. typedExpressionIR
;
```

Typing `forStatement`, under cursor `LOCAL`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. If let `annotationList FOR ( type name IN forCollectionExpression ) statement'` be `forStatement`:

- a. Let `typeIR` and a set of fresh type variables `typeId*` be
 - the result of `typing type`, under cursor `LOCAL` and typing context `TC`.
 - b. Check that `typeId*` is an empty list.
 - c. Let `bound` be `bound type variables from the LOCAL layer of TC`.
 - d. Check that `typeIR` is a well-formed type, with bound type variables `bound`.
 - e. Check that `typeIR` supports assignment.
 - f. Let `forCollectionExpressionIR` be
 - the result of `typing forCollectionExpression` under typing context `TC` with iterator type `typeIR`.
 - g. Let `TC1` be `TC with a new local frame`.
 - h. Let `nameIR` be `the name of name`.
 - i. Let `TC2` be `TC1 where nameIR to INOUT typeIR DYN · is added to the LOCAL layer`.
 - j. Let the updated typing context `TC3`, abstract control flow `fpost`, and `statementIR` be
 - the result of `typing statement'`, under cursor `LOCAL`, typing context `TC2`, abstract control flow `f`, and loop context `LOOP`.
 - k. Let `TC4` be `TC3 with the innermost frame removed`.
 - l. Let `forStatementIR` be `annotationList FOR ( typeIR nameIR IN forCollectionExpressionIR ) statementIR`.
 - m. Result in
 - the updated typing context `TC`, abstract control flow `fpost`, and `forStatementIR`.
2. Else if let `annotationList FOR ( annotationListNonEmpty type name IN forCollectionExpression ) statement'` be `forStatement`:
 - a. Let the updated typing context `TC1`, abstract control flow `fpost`, and `statementIR` be
 - the result of `typing annotationList FOR ( type name IN forCollectionExpression ) statement'`, under cursor `LOCAL`, typing context `TC`, abstract control flow `f`, and loop context `l`.
 - b. Check that `statementIR` has type `forStatementIR`.
 - c. Let `forStatementIR` be `statementIR`.
 - d. Result in
 - the updated typing context `TC`, abstract control flow `fpost`, and `forStatementIR`.

Collection expressions are type checked with the relation:

`ForCollectionExpr_ok`:

- Typing `forCollectionExpression` under typing context `typingContext` with iterator type `typeIR`:
- Results in `forCollectionExpressionIR`.

`Typing forCollectionExpression` under typing context `TC` with iterator type `typeIRvar`:

1. If let `expression` be `forCollectionExpression`:
 - a. Let `typedExpressionIR` be
 - the result of `typing expression`, under cursor `LOCAL` and typing context `TC`.

- b. Let **type** $\text{typeIR}_{\text{list}}$ and **compile-time known-ness** $_$ be the note of typedExpressionIR .
 - c. Let typeIR be $\text{typeIR}_{\text{list}}$ with **typedefs unrolled**.
 - d. If let $\text{LIST} < \text{typeIR}_e >$ be typeIR :
 - i. Check that typeIR_e can be implicitly cast to $\text{typeIR}_{\text{var}}$.
 - ii. Result in typedExpressionIR .
 - e. Else if let $\text{typeIR}_e [_]$ be typeIR :
 - i. Check that typeIR_e and $\text{typeIR}_{\text{var}}$ are the same type.
 - ii. Result in typedExpressionIR .
2. Else:
- a. Let $\text{expression}_l \dots \text{expression}_r$ be $\text{forCollectionExpression}$.
 - b. Let $\text{typedExpressionIR}_l$ be
 - the result of **typing** expression_l , under cursor **LOCAL** and **typing context TC**.
 - c. Let $\text{typedExpressionIR}_r$ be
 - the result of **typing** expression_r , under cursor **LOCAL** and **typing context TC**.
 - d. Let $\text{typedExpressionIR}_{l_castd}$ be **!** $\text{typedExpressionIR}_l$ implicitly cast to $\text{typeIR}_{\text{var}}$.
 - e. Let $\text{typedExpressionIR}_{r_castd}$ be **!** $\text{typedExpressionIR}_r$ implicitly cast to $\text{typeIR}_{\text{var}}$.
 - f. Let **type** typeIR_l and **compile-time known-ness** $_$ be the note of $\text{typedExpressionIR}_{l_castd}$.
 - g. Let **type** typeIR_r and **compile-time known-ness** $_$ be the note of $\text{typedExpressionIR}_{r_castd}$.
 - h. Check that $\$compat_range(\text{typeIR}_l, \text{typeIR}_r)$.
 - i. Result in $\text{typedExpressionIR}_{l_castd} \dots \text{typedExpressionIR}_{r_castd}$.

13.10.2.2. Compile-time evaluation

Compile-time evaluation of forStatementIR , under cursor p , instantiation context IC , and store STO :

1. If let **annotationList FOR ($\text{typeIR nameIR IN forCollectionExpressionIR}$) $\text{statementIR}'$** be forStatementIR :
 - a. Let the updated instantiation context IC_{body} , store STO_1 , and $\text{statementIR}_{\text{body}}$ be
 - the result of **compile-time evaluation of $\text{statementIR}'$, under cursor p , instantiation context IC , and store STO** .
 - b. Let $\text{forStatementIR}'$ be **annotationList FOR ($\text{typeIR nameIR IN forCollectionExpressionIR}$) $\text{statementIR}_{\text{body}}$** .
 - c. Result in
 - the updated instantiation context IC , store STO_1 , and $\text{forStatementIR}'$.
2. Else if let **annotationList FOR ($\text{annotationList}' \text{typeIR nameIR IN forCollectionExpressionIR}$) $\text{statementIR}'$** be forStatementIR :
 - a. Check that annotationList is equal to $\text{annotationList}'$.
 - b. Let the updated instantiation context IC_{body} , store STO_1 , and $\text{statementIR}_{\text{body}}$ be
 - the result of **compile-time evaluation of $\text{statementIR}'$, under cursor p , instantiation context IC , and store STO** .
 - c. Let $\text{forStatementIR}'$ be **annotationList FOR ($\text{annotationList}' \text{typeIR nameIR IN forCollectionExpressionIR}$) $\text{statementIR}_{\text{body}}$** .

d. Result in

- the updated instantiation context IC , store STO_1 , and $forStatementIR'$.

13.10.2.3. Runtime evaluation

Runtime evaluation of $forStatementIR$, under cursor $LOCAL$, evaluation context EC , and store STO :

1. If let $annotationList$ FOR ($typeIR$ $nameIR$ IN $forCollectionExpressionIR$) $statementIR'$ be $forStatementIR$:
 - a. Let EC_1 be EC with a new local frame.
 - b. Let the updated evaluation context EC_2 , store STO_1 , and $expressionListResult$ be
 - the result of runtime evaluation of $forCollectionExpressionIR$, under evaluation context EC_1 and store STO .
 - c. If let $abortResult$ be $expressionListResult$:
 - i. Let EC_3 be EC_2 with the innermost frame removed.
 - ii. Result in
 - the updated evaluation context EC_3 , store STO_1 , and $abortResult$.
 - d. Else:
 - i. Let $value^*$ be $expressionListResult$.
 - ii. Let $typeIR_{subst}$ be $typeIR$ substituted by bound type variables in EC from the $LOCAL$ layer.
 - iii. Let $value_{init}$ be the default value for type $typeIR_{subst}$.
 - iv. Let EC_3 be EC_2 where $nameIR$ to $value_{init}$ is added to the $LOCAL$ layer.
 - v. Let the updated evaluation context EC_4 , store STO_2 , and $statementResult$ be
 - the result of looping over $value^*$ with loop variable $nameIR$ and body $statementIR'$, under evaluation context EC_3 and store STO_1 .
 - vi. Let EC_5 be EC_4 with the innermost frame removed.
 - vii. Result in
 - the updated evaluation context EC_5 , store STO_2 , and $statementResult$.
2. Else if let $annotationList$ FOR ($annotationList_{in}$ $typeIR$ $nameIR$ IN $forCollectionExpressionIR$) $statementIR'$ be $forStatementIR$:
 - a. Let the updated evaluation context EC_1 , store STO_1 , and $statementResult$ be
 - the result of runtime evaluation of $annotationList$ FOR ($typeIR$ $nameIR$ IN $forCollectionExpressionIR$) $statementIR'$, under cursor $LOCAL$, evaluation context EC , and store STO .
 - b. Result in
 - the updated evaluation context EC_1 , store STO_1 , and $statementResult$.

Collection expressions are evaluated with the relation:

$ForCollectionExpr_eval$:

- Runtime evaluation of $forCollectionExpressionIR$, under evaluation context $evalContext$ and store $store$:

- Results in
 - the updated evaluation context `evalContext`, store `store`, and `expressionListResult`.

Runtime evaluation of `typedExpressionIR`, under evaluation context `EC0` and store `STO0`:

1. Let the updated evaluation context `EC1`, store `STO1` and `expressionResult` be
 - the result of evaluating `typedExpressionIR`, under cursor `LOCAL`, evaluation context `EC0`, and store `STO0`.
2. If let `abortResult` be `expressionResult`:
 - a. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `abortResult`.
3. Else:
 - a. Let `value` be `expressionResult`.
 - b. If let `listValue` be `value`:
 - i. Let `LIST [ value* ]` be `listValue`.
 - ii. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `value*`.
 - c. Else if let `headerStackValue` be `value`:
 - i. Let `HEADER_STACK [ value* ( _ ; _ ) ]` be `headerStackValue`.
 - ii. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `value*`.

With these, the `for-in` statement is evaluated with the relation:

`ForInStmt_eval`:

- Looping over `value*` with loop variable `nameIR` and body `statementIR`, under evaluation context `evalContext` and store `store`:
- Results in
 - the updated evaluation context `evalContext`, store `store`, and `statementResult`.

Looping over `value*` with loop variable `nameIR` and body `statementIR`, under evaluation context `evalContext` and store `store`:

1. If `value*` is equal to `:`:
 - a. Result in
 - the updated evaluation context `evalContext`, store `store`, and `EMPTY`.
2. If let `valuen :: valuet` be `value*`:
 - a. Let `EC1` be `evalContext` where `nameIR` is updated to `valuen` from the `LOCAL` layer.
 - b. Let the updated evaluation context `EC2`, store `STO1`, and `statementResult` be
 - the result of runtime evaluation of `statementIR`, under cursor `LOCAL`, evaluation context `EC`

, and store store.

c. If let `abortResult` be `statementResult`:

i. Result in

- the updated evaluation context EC_2 , store STO_1 , and `abortResult`.

d. Else if let `returnResult` be `statementResult`:

i. Result in

- the updated evaluation context EC_2 , store STO_1 , and `returnResult`.

e. Else if let `forBreakResult` be `statementResult`:

i. Result in

- the updated evaluation context EC_2 , store STO_1 , and `'EMPTY'`.

f. If `statementResult` is equal to `'EMPTY'` or `statementResult` is equal to `CONTINUE`:

i. Let the updated evaluation context EC_3 , store STO_2 , and `statementResult'` be

- the result of looping over $value_i^*$ with loop variable `nameIR` and body `statementIR`, under evaluation context EC_2 and store STO_1 .

ii. Result in

- the updated evaluation context EC_3 , store STO_2 , and `statementResult'`.

13.11. Break statements

A `break;` statement exists the enclosing `for` loop.

```
breakStatement
: BREAK ;
;
```

13.11.1. Type checking

After type checking, a `break;` statement has the form:

```
breakStatementIR = breakStatement
```

Typing `breakStatement`, under cursor `LOCAL`, typing context TC , abstract control flow f , and loop context `LOOP`:

1. Result in

- the updated typing context TC , abstract control flow f , and `breakStatement`.

13.11.2. Compile-time evaluation

Compile-time evaluation of `breakStatementIR`, under cursor p , instantiation context IC , and store STO :

1. Result in

- the updated instantiation context **IC**, store **STO**, and **breakStatementIR**.

13.11.3. Runtime evaluation

A **break**; statement yields the result:

```
forBreakResult
: BREAK
;
```

Runtime evaluation of **BREAK** ;, under cursor **LOCAL**, evaluation context **EC**, and store **STO**:

1. Result in
 - the updated evaluation context **EC**, store **STO**, and **BREAK**.

13.12. Continue statements

A **continue**; statement skips the current iteration of the enclosing **for** loop.

```
continueStatement
: CONTINUE ;
;
```

13.12.1. Type checking

After type checking, a **continue**; statement has the form:

```
continueStatementIR = continueStatement
```

Typing **continueStatement**, under cursor **LOCAL**, typing context **TC**, abstract control flow **f**, and loop context **LOOP**:

1. Result in
 - the updated typing context **TC**, abstract control flow **f**, and **continueStatement**.

13.12.2. Compile-time evaluation

Compile-time evaluation of **continueStatementIR**, under cursor **p**, instantiation context **IC**, and store **STO**:

1. Result in
 - the updated instantiation context **IC**, store **STO**, and **continueStatementIR**.

13.12.3. Runtime evaluation

A `continue`; statement yields the result:

```
forContinueResult
: CONTINUE
;
```

Runtime evaluation of `CONTINUE`; , under cursor `LOCAL`, evaluation context `EC`, and store `STO`:

1. Result in

- the updated evaluation context `EC`, store `STO`, and `CONTINUE`.

13.13. Switch statements

The `switch` statement can only be used within `control` blocks, `action` bodies, or function bodies.

```
switchStatement
: SWITCH `( expression ) `{ switchCaseList }
;

switchCaseList
: /* empty */
| switchCaseList switchCase
;

switchCase
: switchLabel : blockStatement
| switchLabel :
;

switchLabel
: DEFAULT
| expressionNonBrace
;
```

The `expressionNonBrace` is the same as `expression` as defined in [Chapter 14](#), except it does not include any cases that can begin with a left brace `{` character, to avoid syntactic ambiguity with a block statement.

There are two kinds of `switch` expressions allowed, described separately in the following two subsections.

13.13.1. Notes common to all switch statements

It is a compile-time error if two labels of a `switch` statement equal each other. The switch label values need not include all possible values of the switch expression. It is optional to have a `switch` case with the `default` label, but if one is present, it must be the last one in the `switch` statement.

If a switch label is not followed by a block statement, it falls through to the next label. However, if a block statement is present, it does not fall through. Note that this is different from C-style `switch` statements, where a `break` is needed to prevent fall-through. If the last switch label is not followed by a block statement, the behavior is the same as if the last switch label were followed by an empty block statement `{ }`.

When a `switch` statement is executed, first the switch expression is evaluated, and any side effects from evaluating this expression are visible to any `switch` case that is executed. Among switch labels that are not `default`, at most one of them can equal the value of the switch expression. If one is equal, that switch case is executed.

If no labels are equal to the `switch` expression, then:

- if there is a `default` label, the case with the `default` label is executed.
- if there is no `default` label, then no switch case is executed, and execution continues after the end of the `switch` statement, with no side effects (except any that were caused by evaluating the `switch` expression).

See "Implementing generalized P4_16 switch statements" for possible techniques that one might use to implement generalized switch statements.^[1]

13.13.2. Switch statement on match-action table result

This type of `switch` statement can only be used within control `apply` blocks. For this variant of `switch` statement, the expression must be of the form `t.apply().action_run`, where `t` is the name of a table (see [\[sec-invoke-mau\]](#)). All switch labels must be names of actions of the table `t`, or `default`.

```
switch (t.apply().action_run) {  
  action1: // fall-through to action2:  
  action2: { /* body omitted */ }  
  action3: { /* body omitted */ } // no fall-through from action2 to action3 labels  
  default: { /* body omitted */ }  
}
```

Note that the `default` label of the `switch` statement is used to match on the kind of action executed, no matter whether there was a table hit or miss. The `default` label does not indicate that the table missed and the `default_action` was executed.

13.13.2.1. Type checking

Typing `SWITCH ( expressionswitch ) { switchCaseList }`, under cursor `LOCAL`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. Let `typedExpressionIRswitch` be
 - the result of typing `expressionswitch`, under cursor `LOCAL` and typing context `TC`.
2. Let `typeIRswitch` be the type of `typedExpressionIRswitch`.
3. Let `typeIR` be `typeIRswitch` with `typedefs` unrolled.
4. Check that `typeIR` has type `tableMetadataEnumTypeIR`.
5. Let `TABLE_ENUM typedIdtable_enum { _ }` be `typeIR`.
6. Let `text` be the text `typedIdtable_enum` with the suffix `"")` removed.
7. Let `typedIdtable` be the text `text` with the prefix `"action_list("` removed.
8. Let the abstract control flow `fpost`, `switchCaseIR*`, and labels `switchLabel*` be
 - the result of typing `switchCaseList` for table `typedIdtable`, under typing context `TC` with abstract control flow `f` and loop context `l`.
9. Check that `default` label is only in the last position of `switchLabel*`.
10. Check that the elements of `switchLabel*` are distinct.
11. Let `switchStatementIR` be `SWITCH ( typedExpressionIRswitch ) { switchCaseIR* }`.
12. Result in
 - the updated typing context `TC`, abstract control flow `fpost`, and `switchStatementIR`.

Switch cases are type checked with the relation:

SwitchCaseList_table_ok:

- Typing `switchCaseList` for table `typeld`, under typing context `typingContext` with abstract control flow `flow` and loop context `loopctx`:
- Results in the abstract control flow `flow`, `switchCaseListIR`, and labels `switchLabel*`.

A switch case is type checked with:

SwitchCase_table_ok:

- Typing `switchCase` for table `typeld`, under typing context `typingContext` with abstract control flow `flow` and loop context `loopctx`:
- Results in the abstract control flow `flow`, and label `switchCaseIR`.

Typing `switchCase` for table `typeldtable`, under typing context `TC` with abstract control flow `f` and loop context `l`:

1. If let `switchLabel : blockStatement` be `switchCase`:
 - a. Let `switchLabelIR` be
 - the result of typing `switchLabel` for table `typeldtable`, under typing context `TC`.
 - b. Let the updated typing context `TCpost`, abstract control flow `fpost`, and `blockStatementIR` be
 - the result of typing `blockStatement`, under typing context `TC`, abstract control flow `f`, and loop context `l`.
 - c. Let `switchCaseIR` be `switchLabelIR : blockStatementIR`.
 - d. Result in the abstract control flow `fpost`, and label `switchCaseIR`.
2. Else:
 - a. Let `switchLabel :` be `switchCase`.
 - b. Let `switchLabelIR` be
 - the result of typing `switchLabel` for table `typeldtable`, under typing context `TC`.
 - c. Let `switchCaseIR` be `switchLabelIR :`.
 - d. Result in the abstract control flow `f`, and label `switchCaseIR`.

A switch label is type checked with:

SwitchLabel_table_ok:

- Typing `switchLabel` for table `typeld`, under typing context `typingContext`:
- Results in `switchLabelIR`.

Typing `switchLabel` for table `typeldtable`, under typing context `TC`:

1. If `switchLabel` is `DEFAULT`:
 - a. Result in `DEFAULT`.
2. Else if let `prefixedNonTypeName` be `switchLabel`:
 - a. Let `prefixedNameIR` be the prefixed name of `prefixedNonTypeName`.
 - b. Check that `prefixedNameIR` matches pattern `` %`.
 - c. Let `nameIRlabel` be `prefixedNameIR`.
 - d. Let `typeIdtable_enum` be "action_list(" concatenated with `typeIdtable` concatenated with `")"`.
 - e. Let `idlabel` be `typeIdtable_enum` concatenated with `."` concatenated with `nameIRlabel`.
 - f. Let `_typeIRlabel ctklabel value?` be ! the type of variable `idlabel` from the LOCAL layer of TC.
 - g. Check that `value?` is defined.
 - h. Let `valuelabel` be `value?`.
 - i. Check that `valuelabel` is equal to `TABLE_ENUM typeIdtable_enum . nameIRlabel`.
 - j. Let `typedExpressionIRlabel` be `nameIRlabel` annotated with type `typeIRlabel` and compile-time knownness `ctklabel`.
 - k. Result in `typedExpressionIRlabel`.

13.13.2.2. Compile-time evaluation

Compile-time evaluation of `SWITCH ( typedExpressionIR ) { switchCaseListIR }`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. Let the updated store `STO1` and `switchCaseListIRinst` be
 - the result of compile-time evaluation of switch cases `switchCaseListIR`, under cursor `p`, instantiation context `IC`, and store `STO`.
2. Result in
 - the updated instantiation context `IC`, store `STO1`, and `SWITCH ( typedExpressionIR ) { switchCaseListIRinst }`.

The compile-time evaluation of switch cases is defined with the relation:

`SwitchCases_inst`:

- Compile-time evaluation of switch cases `switchCaseIR*`, under cursor `cursor`, instantiation context `instContext`, and store `store`:
- Results in the updated store `store` and `switchCaseIR*`.

A switch case is compile-time evaluated with:

`SwitchCase_inst`:

- Compile-time evaluation of `switchCaseIR`, under cursor `cursor`, instantiation context `instContext`, and store `store`:
- Results in the updated store `store` and `switchCaseIR`.

Compile-time evaluation of `switchCaseIR`, under cursor `p`, instantiation context `IC`, and store `store`:

1. If let `switchLabelIR : blockStatementIR` be `switchCaseIR`:
 - a. Let the updated instantiation context `ICswitch`, store `STO1`, and `statementIR` be
 - the result of compile-time evaluation of `blockStatementIR`, under cursor `p`, instantiation context `IC`, and store `store`.
 - b. Check that `statementIR` has type `blockStatementIR`.
 - c. Let `blockStatementIRinst` be `statementIR`.
 - d. Result in the updated store `STO1` and `switchLabelIR : blockStatementIRinst`.
2. Else:
 - a. Let `switchLabelIR` be `switchCaseIR`.
 - b. Result in the updated store `store` and `switchLabelIR` .:

13.13.2.3. Runtime evaluation

Runtime evaluation of `SWITCH ( typedExpressionIR ) { switchCaseIR* }`, under cursor `LOCAL`, evaluation context `EC`, and store `STO`:

1. Let the updated evaluation context `EC1`, store `STO1` and `expressionResult` be
 - the result of evaluating `typedExpressionIR`, under cursor `LOCAL`, evaluation context `EC`, and store `STO`.
2. If let `abortResult` be `expressionResult`:
 - a. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `abortResult`.
3. Else:
 - a. Let `value` be `expressionResult`.
 - b. Check that `value` has type `tableMetadataEnumValue`.
 - c. Let `TABLE_ENUM _ . idenum` be `value`.
 - d. Let the matched block `blockStatementIR2` be
 - the result of runtime evaluation of `switchCaseIR*` for invocation of table `idenum`.
 - e. If let `blockStatementIR'` be `blockStatementIR2`:
 - i. Let the updated evaluation context `EC2`, store `STO2`, and `statementResult` be
 - the result of runtime evaluation of `blockStatementIR'`, under cursor `LOCAL`, evaluation context `EC1`, and store `STO1`.
 - ii. Result in
 - the updated evaluation context `EC2`, store `STO2`, and `statementResult`.
 - f. Else:
 - i. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `EMPTY`.

Switch cases are runtime evaluated with the relation:

SwitchCases_table_eval:

- Runtime evaluation of `switchCaseListIR` for invocation of table `id`:
- Results in the matched block `blockStatementIR?`.

Runtime evaluation of `switchCaseIR*` for invocation of table `id`:

1. If `switchCaseIR*` is equal to `·`:
 - a. Result in the matched block `·`.
2. If let `switchCaseIRn :: switchCaseIRt*` be `switchCaseIR*`:
 - a. If let `switchLabelIRn : blockStatementIRn` be `switchCaseIRn`:
 - i. If `id` matches `switchLabelIRn`:
 - A. Result in the matched block `blockStatementIRn`.
 - ii. Else:
 - A. Let the matched block `blockStatementIR?` be
 - the result of runtime evaluation of `switchCaseIRt*` for invocation of table `id`.
 - B. Result in the matched block `blockStatementIR?`.
 - b. Else:
 - i. Let `switchLabelIRn : be switchCaseIRn`.
 - ii. If `id` matches `switchLabelIRn`:
 - A. Let `bool*` be the list of `bool`, obtained by repeating:
 - Let `bool` be `switchCaseIRt` is not a fall-through case.
 - for each `switchCaseIRt` in `switchCaseIRt*`
 - B. Let `switchCaseIR*` be `switchCaseIRt*` filtered by `bool*`.
 - C. Check that `switchCaseIR*` is a non-empty list.
 - D. Let `switchCaseIR" :: _*` be `switchCaseIR*`.
 - E. Check that `switchCaseIR"` matches pattern `% : %`.
 - F. Let `_ : blockStatementIR` be `switchCaseIR"`.
 - G. Result in the matched block `blockStatementIR`.
 - iii. Else:
 - A. Let the matched block `blockStatementIR?` be
 - the result of runtime evaluation of `switchCaseIRt*` for invocation of table `id`.
 - B. Result in the matched block `blockStatementIR?`.

13.13.3. Switch statement on numeric or enumerated type

For this variant of `switch` statement, the expression must evaluate to a result with one of these types:

- `bit<W>`

- `int<W>`
- `enum`, either with or without an underlying representation specified
- `error`

All switch labels must be expressions with compile-time known values, and must have a type that can be implicitly cast to the type of the `switch` expression (see [\[sec-implicit-casts\]](#)). Switch labels must not begin with a left brace character `{`, to avoid ambiguity with a block statement.

```
// Assume the expression hdr.ethernet.etherType has type bit<16>.
switch (hdr.ethernet.etherType) {
  0x86dd: { /* body omitted */ }
  0x0800: // fall-through to the next body
  0x0802: { /* body omitted */ }
  0xcafe: { /* body omitted */ }
  default: { /* body omitted */ }
}
```

13.13.3.1. Type checking

Typing `SWITCH ( expressionswitch ) { switchCaseList }`, under cursor `LOCAL`, typing context `TC`, abstract control flow `f`, and loop context `l`:

1. Let `typedExpressionIRswitch` be
 - the result of typing `expressionswitch` under cursor `LOCAL` and typing context `TC`.
2. Let `typeIRswitch` be the type of `typedExpressionIRswitch`.
3. Check that `$compat_switch(typeIRswitch)`.
4. Let the abstract control flow `fpost` and labels `switchCaseIR*` be
 - the result of typing `switchCaseList` under typing context `TC` with abstract control flow `f`, loop context `l`, and switch key type `typeIRswitch`.
5. Check that default label is only in the last position of `switchLabel*`.
6. Check that the elements of `switchLabel*` are distinct.
7. Let `switchStatementIR` be `SWITCH ( typedExpressionIRswitch ) { switchCaseIR* }`.
8. Result in
 - the updated typing context `TC`, abstract control flow `fpost`, and `switchStatementIR`.

Switch cases are type checked with the relation:

`SwitchCaseList_general_ok`:

- Typing `switchCaseList` under typing context `typingContext` with abstract control flow `flow`, loop context `loopctxt`, and switch key type `typeIR`:
- Results in the abstract control flow `flow`, and labels `switchCaseListIR`.

A switch case is type checked with:

`SwitchCase_general_ok`:

- Typing `switchCase` under typing context `typingContext` with abstract control flow `flow`, loop context `loopctx`, and switch key type `typeIR`:
- Results in the abstract control flow `flow`, and label `switchCaseIR`.

Typing `switchCase` under typing context `TC` with abstract control flow `f`, loop context `l`, and switch key type `typeIRswitch`:

1. If let `switchLabel : blockStatement` be `switchCase`:
 - a. Let `switchLabelIR` be
 - the result of typing `switchLabel` under typing context `TC` with switch key type `typeIRswitch`.
 - b. Let the updated typing context `TCpost`, abstract control flow `fpost`, and `blockStatementIR` be
 - the result of typing `blockStatement`, under typing context `TC`, abstract control flow `f`, and loop context `l`.
 - c. Let `switchCaseIR` be `switchLabelIR : blockStatementIR`.
 - d. Result in the abstract control flow `fpost`, and label `switchCaseIR`.
2. Else:
 - a. Let `switchLabel :` be `switchCase`.
 - b. Let `switchLabelIR` be
 - the result of typing `switchLabel` under typing context `TC` with switch key type `typeIRswitch`.
 - c. Let `switchCaseIR` be `switchLabelIR :`.
 - d. Result in the abstract control flow `f`, and label `switchCaseIR`.

A switch label is type checked with:

`SwitchLabel_general_ok`:

- Typing `switchLabel` under typing context `typingContext` with switch key type `typeIR`:
- Results in `switchLabelIR`.

Typing `switchLabel` under typing context `TC` with switch key type `typeIR`:

1. If `switchLabel` is `DEFAULT`:
 - a. Result in `DEFAULT`.
2. Else:
 - a. Let `expressionNonBracelabel` be `switchLabel`.
 - b. Let `expressionlabel` be the expression corresponding to `expressionNonBracelabel`.
 - c. Let `typedExpressionIRlabel` be
 - the result of typing `expressionlabel`, under cursor `LOCAL` and typing context `TC`.
 - d. Let `typedExpressionIRlabel_cast` be `! typedExpressionIRlabel implicitly cast to typeIR`.
 - e. Let `type _ and compile-time known-ness ctk` be the note of `typedExpressionIRlabel_cast`.

- f. Check that `ctx` is `LCTK`.
- g. Result in `typedExpressionIRlabel_cast`.

13.13.3.2. Compile-time evaluation

See [Section 13.13.2.2](#).

13.13.3.3. Runtime evaluation

Runtime evaluation of `SWITCH ( typedExpressionIR ) { switchCaseIR* }`, under cursor `LOCAL`, evaluation context `EC`, and store `STO`:

1. Let the updated evaluation context `EC1`, store `STO1` and `expressionResult` be
 - the result of evaluating `typedExpressionIR`, under cursor `LOCAL`, evaluation context `EC`, and store `STO`.
2. If let `abortResult` be `expressionResult`:
 - a. Result in
 - the updated evaluation context `EC1`, store `STO1`, and `abortResult`.
3. Else:
 - a. Let `value` be `expressionResult`.
 - b. Let the updated evaluation context `EC2`, store `STO2`, and `blockStatementIR'` be
 - the result of runtime evaluation of `switchCaseIR*`, under evaluation context `LOCAL` and store `EC1` with switch key `value`.
 - c. If let `blockStatementIR'` be `blockStatementIR'`:
 - i. Let the updated evaluation context `EC3`, store `STO3`, and `statementResult` be
 - the result of runtime evaluation of `blockStatementIR'`, under cursor `LOCAL`, evaluation context `EC2`, and store `STO2`.
 - ii. Result in
 - the updated evaluation context `EC3`, store `STO3`, and `statementResult`.
 - d. Else:
 - i. Result in
 - the updated evaluation context `EC2`, store `STO2`, and `EMPTY`.

Switch cases are runtime evaluated with the relation:

`SwitchCases_general_eval`:

- Runtime evaluation of `switchCaseListIR`, under evaluation context `cursor` and store `evalContext` with switch key `value`:
- Results in
 - the updated evaluation context `evalContext`, store `store`, and `blockStatementIR'`.

Runtime evaluation of switchCaseIR^* , under evaluation context LOCAL and store EC_0 with switch key value:

1. If switchCaseIR^* is equal to $\cdot$:
 - a. Result in
 - the updated evaluation context EC_0 , store STO_0 , and $\cdot$.
2. If let $\text{switchCaseIR}_h :: \text{switchCaseIR}_t^*$ be switchCaseIR^* :
 - a. If let $\text{switchLabelIR}_h : \text{blockStatementIR}_h$ be switchCaseIR_h :
 - i. Let the updated evaluation context EC_1 , store STO_1 , and expressionResult be
 - the result of runtime evaluation of STO_0 , under evaluation context LOCAL and store EC_0 .
 - ii. Check that expressionResult has type $\text{continueResult}\langle\text{value}\rangle$.
 - iii. Let $\text{value}_{\text{label}}$ be expressionResult .
 - iv. If value matches $\text{value}_{\text{label}}$:
 - A. Result in
 - the updated evaluation context EC_1 , store STO_1 , and $\text{blockStatementIR}_h$.
 - v. Else:
 - A. Let the updated evaluation context EC_2 , store STO_2 , and $\text{blockStatementIR}^2$ be
 - the result of runtime evaluation of switchCaseIR_t^* , under evaluation context LOCAL and store EC_1 with switch key value.
 - B. Result in
 - the updated evaluation context EC_2 , store STO_2 , and $\text{blockStatementIR}^2$.
 - b. Else:
 - i. Let switchLabelIR_h be switchCaseIR_h .
 - ii. Let the updated evaluation context EC_1 , store STO_1 , and expressionResult be
 - the result of runtime evaluation of STO_0 , under evaluation context LOCAL and store EC_0 .
 - iii. Check that expressionResult has type $\text{continueResult}\langle\text{value}\rangle$.
 - iv. Let $\text{value}_{\text{label}}$ be expressionResult .
 - v. If value matches $\text{value}_{\text{label}}$:
 - A. Let bool^* be the list of bool , obtained by repeating:
 - Let bool be switchCaseIR_t is not a fall-through case.
 for each switchCaseIR_t in switchCaseIR_t^*
 - B. Let switchCaseIR^* be switchCaseIR_t^* filtered by bool^* .
 - C. Check that switchCaseIR^* is a non-empty list.
 - D. Let $\text{switchCaseIR}'' :: _*$ be switchCaseIR^* .
 - E. Check that $\text{switchCaseIR}''$ matches pattern $\% : \%$.
 - F. Let $_ : \text{blockStatementIR}$ be $\text{switchCaseIR}''$.
 - G. Result in

- the updated evaluation context EC_1 , store STO_1 , and $blockStatementIR$.
- vi. Else:
- A. Let the updated evaluation context EC_2 , store STO_2 , and $blockStatementIR$ [?] be
 - the result of runtime evaluation of $switchCaseIR_t^*$, under evaluation context LOCAL and store EC_1 with switch key value.
 - B. Result in
 - the updated evaluation context EC_2 , store STO_2 , and $blockStatementIR$ [?].

A switch label is runtime evaluated with:

SwitchLabel_general_eval:

- Runtime evaluation of $store$, under evaluation context $cursor$ and store $evalContext$:
- Results in
 - the updated evaluation context $evalContext$, store $store$, and $expressionResult$.

Runtime evaluation of STO_0 , under evaluation context LOCAL and store EC_0 :

1. If $switchLabelIR$ is DEFAULT:
 - a. Result in
 - the updated evaluation context EC_0 , store STO_0 , and DEFAULT.
2. Else:
 - a. Let $typedExpressionIR$ be $switchLabelIR$.
 - b. Let the updated evaluation context EC_1 , store STO_1 and $expressionResult$ be
 - the result of evaluating $typedExpressionIR$, under cursor LOCAL, evaluation context EC_0 , and store STO_0 .
 - c. Result in
 - the updated evaluation context EC_1 , store STO_1 , and $expressionResult$.

[1] <https://github.com/p4lang/p4-spec/blob/master/p4-16/spec/docs/implementing-generalized-switch-statements.md>

14. Expressions

The syntax of expressions is defined as follows:

```
expression
: literalExpression
| referenceExpression
| defaultExpression
| unaryExpression
| binaryExpression
| ternaryExpression
| castExpression
| dataExpression
| accessExpression
| callExpression
| parenthesizedExpression
;
```

14.1. Semantics of expressions

Given a compound expression, the order in which sub-expressions are evaluated is important when the sub-expressions have side-effects. P4 expressions are evaluated as follows:

- Boolean operators `&&` and `||` use short-circuit evaluation—i.e., the second operand is only evaluated if necessary.
- The conditional operator `e1 ? e2 : e3` evaluates `e1`, and then either evaluates `e2` or `e3`.
- All other expressions are evaluated left-to-right as they appear in the source program.
- Method and function calls are evaluated as described in [Section 18.4](#).

14.1.1. Type checking

`Expr_ok`:

- Typing `expression`, under cursor `cursor` and typing context `typingContext`:
- Results in `typedExpressionIR`.

After type checking, expressions are represented in P4_{IR} as follows:

```
typedExpressionIR
: expressionIR # expressionNoteIR
;

expressionIR
: literalExpressionIR
| referenceExpressionIR
| defaultExpressionIR
| unaryExpressionIR
| binaryExpressionIR
| ternaryExpressionIR
| castExpressionIR
| dataExpressionIR
| accessExpressionIR
| callExpressionIR
| parenthesizedExpressionIR
```

```

;
expressionNoteIR
: `( typeIR ctk )
;

```

Notice that after type checking, expressions in P4 are annotated with their types and their compile-time known-ness. See [Section 7.5](#) for more details of compile-time known and local compile-time known values.

14.1.2. Local compile-time evaluation

Expr_eval_lctk:

- Local compile-time evaluation of `typedExpressionIR`, under cursor `cursor` and typing context `typingContext`:
- Results in `value`.

Local compile-time known expressions are evaluated with the above relation.

14.1.3. Compile-time evaluation

Expr_inst:

- Compile-time evaluation of `typedExpressionIR`, under cursor `cursor`, instantiation context `instContext`, and store `store`:
- Results in the updated store `store` and `value`.

Compile-time known expressions are evaluated with the above relation. A sequence of expressions can be evaluated with the following relation:

Exprs_inst:

- Compile-time evaluation of `typedExpressionIR*`, under cursor `cursor`, instantiation context `instContext`, and store `store`:
- Results in the updated store `store` and `value*`.

14.1.4. Runtime evaluation

Expr_eval:

- Evaluating `typedExpressionIR`, under cursor `cursor`, evaluation context `evalContext`, and store `store`:
- Results in the updated evaluation context `evalContext`, store `store` and `expressionResult`.

The result of evaluating an expression is represented as follows:

```
expressionResult
```

```

: continueResult<value>
| abortResult
;

continueResult<X>
: ` X
;

abortResult
: exitResult
| rejectTransitionResult
;

exitResult
: EXIT
;

rejectTransitionResult
: REJECT errorValue
;

```

See [Section 13.7](#) and [\[sec-packet\]](#) for more details on exit and reject results.

A sequence of expressions can be evaluated with the following relation:

[Exprs_eval](#):

- Evaluating `typedExpressionIR*`, under cursor `cursor`, evaluation context `evalContext`, and store `store`:
- Results in the updated evaluation context `evalContext`, store `store` and `expressionListResult`.

14.2. Literal expressions

Literal expressions denote fixed values of boolean, numeric, or string types:

```

literalExpression
: booleanLiteral
| integerLiteral
| stringLiteral
;

booleanLiteral
: TRUE
| FALSE
;

integerLiteral
: D int
| nat W int
| nat S int
;

stringLiteral
: " text "
;

```

14.2.1. Type checking

After type checking, literal expressions have the form:

literalExpressionIR = literalExpression

Typing expression, under cursor p and typing context TC:

1. If let booleanLiteral be expression:
 - a. Let expressionNoteIR be type BOOL and compile-time known-ness LCTK.
 - b. Result in booleanLiteral annotated with expressionNoteIR.
2. Else if let integerLiteral be expression:
 - a. If let D i be integerLiteral:
 - i. Let expressionNoteIR be type INT and compile-time known-ness LCTK.
 - ii. Result in D i annotated with expressionNoteIR.
 - b. Else if let n S i be integerLiteral:
 - i. Let expressionNoteIR be type INT < n > and compile-time known-ness LCTK.
 - ii. Result in n S i annotated with expressionNoteIR.
 - c. Else:
 - i. Let n W i be integerLiteral.
 - ii. Let expressionNoteIR be type BIT < n > and compile-time known-ness LCTK.
 - iii. Result in n W i annotated with expressionNoteIR.
3. Else if let stringLiteral be expression:
 - a. Let expressionNoteIR be type STRING and compile-time known-ness LCTK.
 - b. Result in stringLiteral annotated with expressionNoteIR.

14.2.2. Local compile-time evaluation

Local compile-time evaluation of expressionIR annotated with expressionNoteIR, under cursor p and typing context TC:

1. If let booleanLiteral be expressionIR:
 - a. Let boolValue be 'B true if booleanLiteral is equal to TRUE otherwise 'B false.
 - b. Result in boolValue.
2. Else if let integerLiteral be expressionIR:
 - a. Result in integerLiteral.
3. Else if let stringLiteral be expressionIR:
 - a. Result in stringLiteral.

14.2.3. Compile-time evaluation

Compile-time evaluation of expressionIR annotated with expressionNoteIR, under cursor p, instantiation context IC, and store STO:

1. If let `booleanLiteral` be `expressionIR`:
 - a. Let `boolValue` be ``B true if booleanLiteral is equal to TRUE otherwise `B false`.
 - b. Result in the updated store `STO` and `boolValue`.
2. Else if let `integerLiteral` be `expressionIR`:
 - a. Result in the updated store `STO` and `integerLiteral`.
3. Else if let `stringLiteral` be `expressionIR`:
 - a. Result in the updated store `STO` and `stringLiteral`.

14.2.4. Runtime evaluation

Evaluating `expressionIR` annotated with `expressionNodeIR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. If let `booleanLiteral` be `expressionIR`:
 - a. Let `boolValue` be ``B true if booleanLiteral is equal to TRUE otherwise `B false`.
 - b. Let `expressionResult` be `boolValue`.
 - c. Result in the updated evaluation context `EC`, store `STO` and `expressionResult`.
2. Else if let `integerLiteral` be `expressionIR`:
 - a. Let `expressionResult` be `integerLiteral`.
 - b. Result in the updated evaluation context `EC`, store `STO` and `expressionResult`.
3. Else if let `stringLiteral` be `expressionIR`:
 - a. Let `expressionResult` be `stringLiteral`.
 - b. Result in the updated evaluation context `EC`, store `STO` and `expressionResult`.

14.3. Reference expressions

A reference expression is an expression that refers to a named entity in a P4 program:

```
referenceExpression
: prefixedNonTypeName
| THIS
;

prefixedNonTypeName
: nonTypeName
| `ID . nonTypeName
;
```

References are resolved according to the scoping rules of P4 (see [Section 6.3](#)).

14.3.1. Type checking

After type checking, a reference expression is represented in $P4_{IR}$ as:


```
referenceExpressionIR = prefixedNameIR
```

Typing expression, under cursor p and typing context TC:

1. If let $\text{prefixedNonTypeName}$ be expression:
 - a. Let prefixedNameIR be the prefixed name of $\text{prefixedNonTypeName}$.
 - b. Let $_ \text{typeIR} \text{ ctk } _?$ be ! the type of variable prefixedNameIR from the p layer of TC.
 - c. Let expressionNoteIR be type typeIR and compile-time known-ness ctk .
 - d. Result in prefixedNameIR annotated with expressionNoteIR .
2. If expression is equal to THIS:
 - a. Let prefixedNameIR be "this".
 - b. Let $_ \text{typeIR} \text{ ctk } _?$ be ! the type of variable prefixedNameIR from the p layer of TC.
 - c. Let expressionNoteIR be type typeIR and compile-time known-ness ctk .
 - d. Result in prefixedNameIR annotated with expressionNoteIR .

A variable is looked up in the typing context with:

The type of variable prefixedNameIR from the p layer of TC

1. If let $\cdot \text{ id}$ be prefixedNameIR :
 - a. Let typeFrame be TC.GLOBAL.FRAME .
 - b. Return id in map typeFrame .
2. Else:
 - a. Let id be prefixedNameIR .
 - b. If p is GLOBAL:
 - i. Let typeFrame be TC.GLOBAL.FRAME .
 - ii. Return id in map typeFrame .
 - c. Else if p is BLOCK:
 - i. Let typeFrame be TC.BLOCK.FRAME .
 - ii. Let $\text{varTypeIR}^?$ be id in map typeFrame .
 - iii. If let $\text{varTypeIR}'$ be $\text{varTypeIR}^?$:
 - A. Return $\text{varTypeIR}'$.
 - iv. If $\cdot$ is equal to id in map typeFrame :
 - A. Return the type of variable id from the GLOBAL layer of TC.
 - d. Else:
 - i. Let typeFrame^* be TC.LOCAL.FRAMES .
 - ii. Let $\text{varTypeIR}^?$ be id in maps typeFrame^* .
 - iii. If let $\text{varTypeIR}'$ be $\text{varTypeIR}^?$:
 - A. Return $\text{varTypeIR}'$.

iv. If $\cdot$ is equal to $\text{id in maps typeFrame}^*$:

A. Return the type of variable id from the BLOCK layer of TC.

14.3.2. Local compile-time evaluation

Local compile-time evaluation of prefixedNameIR annotated with expressionNodeIR , under cursor p and typing context TC:

1. Let value be the value of variable prefixedNameIR from the p layer of TC.
2. Result in value .

14.3.3. Compile-time evaluation

Compile-time evaluation of prefixedNameIR annotated with expressionNodeIR , under cursor p , instantiation context IC, and store STO:

1. Let value be the value of prefixedNameIR from the p layer of IC.
2. Result in the updated store STO and value .

A variable is looked up in the instantiation context with:

The value of prefixedNameIR from the p layer of IC

1. If let $\cdot$ id be prefixedNameIR :
 - a. Let value' be $! \text{id in map IC.GLOBAL.FRAME}$.
 - b. Return value' .
2. Else:
 - a. Let id be prefixedNameIR .
 - b. If p is GLOBAL:
 - i. Let value' be $! \text{id in map IC.GLOBAL.FRAME}$.
 - ii. Return value' .
 - c. Else if p is BLOCK:
 - i. Let $\text{value}^?$ be $\text{id in map IC.BLOCK.FRAME}$.
 - ii. If let value' be $\text{value}^?$:
 - A. Return value' .
 - iii. If $\cdot$ is equal to $\text{id in map IC.BLOCK.FRAME}$:
 - A. Return the value of id from the GLOBAL layer of IC.
 - d. Else:
 - i. Let $\text{value}^?$ be $\text{id in maps IC.LOCAL.FRAMES}$.
 - ii. If let value' be $\text{value}^?$:
 - A. Return value' .

iii. If $\cdot$ is equal to `id in maps` `IC.LOCAL.FRAMES`:

A. Return the value of `id` from the `BLOCK` layer of `IC`.

14.3.4. Runtime evaluation

Evaluating `prefixedNameIR` annotated with `expressionNotelR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let `value` be the value of `prefixedNameIR` from the `p` layer of `EC`.
2. Let `expressionResult` be `value`.
3. Result in the updated evaluation context `EC`, store `STO` and `expressionResult`.

A variable is looked up in the runtime context with:

The value of `prefixedNameIR` from the `p` layer of `EC`

1. If let `id` be `prefixedNameIR`:
 - a. Let `value'` be `! id in map` `EC.GLOBAL.FRAME`.
 - b. Return `value'`.
2. Else:
 - a. Let `id` be `prefixedNameIR`.
 - b. If `p` is `GLOBAL`:
 - i. Let `value'` be `! id in map` `EC.GLOBAL.FRAME`.
 - ii. Return `value'`.
 - c. Else if `p` is `BLOCK`:
 - i. Let `value'` be `id in map` `EC.BLOCK.FRAME`.
 - ii. If let `value'` be `value'`:
 - A. Return `value'`.
 - iii. If $\cdot$ is equal to `id in map` `EC.BLOCK.FRAME`:
 - A. Return the value of `id` from the `GLOBAL` layer of `EC`.
 - d. Else:
 - i. Let `value'` be `id in maps` `EC.LOCAL.FRAMES`.
 - ii. If let `value'` be `value'`:
 - A. Return `value'`.
 - iii. If $\cdot$ is equal to `id in maps` `EC.LOCAL.FRAMES`:
 - A. Return the value of `id` from the `BLOCK` layer of `EC`.

14.4. Default expressions

Default expressions provide a way to specify a default value for a type:

```
defaultExpression
: ...
;
```

See [Section 8.6.1](#) for more details on default types. When default expressions are cast to a specific type, they evaluate to the default value for that type. See [Section 19.3](#) for more details on default values.

14.4.1. Type checking

After type checking, default expressions have the form:

```
defaultExpressionIR = defaultExpression
```

Typing ..., under cursor p and typing context TC :

1. Let $expressionNoteIR$ be **type** `DEFAULT` and **compile-time known-ness** `LCTK`.
2. Result in ... **annotated with** $expressionNoteIR$.

14.4.2. Local compile-time evaluation

Local compile-time evaluation of ... **annotated with** $expressionNoteIR$, under cursor p and typing context TC :

1. Result in `DEFAULT`.

14.4.3. Compile-time evaluation

Compile-time evaluation of ... **annotated with** $expressionNoteIR$, under cursor p , instantiation context IC , and store STO :

1. Result in the updated store STO and `DEFAULT`.

14.4.4. Runtime evaluation

Evaluating ... **annotated with** $expressionNoteIR$, under cursor p , evaluation context EC , and store STO :

1. Let $expressionResult$ be `DEFAULT`.
2. Result in the updated evaluation context EC , store STO and $expressionResult$.

14.5. Unary expressions

```
unaryExpression
: unop expression
;
```

```

unop
: !
| ~
| -
| +
;

```

See [Section 19.1](#) for details on how each unary operator is evaluated.

14.5.1. Type checking

After type checking, unary expressions have the form:

```

unaryExpressionIR
: unop typedExpressionIR
;

```

14.5.1.1. Negation

Typing `! expression'`, under cursor `p` and typing context `TC`:

1. Let `typedExpressionIR` be
 - the result of typing `expression'`, under cursor `p` and typing context `TC`.
2. Let `typedExpressionIRreduced` be ! the result of reducing serializable enums in `typedExpressionIR` until `$compat_inot` is satisfied.
3. Let `typeIRreduced` be the type of `typedExpressionIRreduced`.
4. Let `ctkreduced` be the compile-time known-ness of `typedExpressionIRreduced`.
5. Let `expressionNoteIR` be type `typeIRreduced` and compile-time known-ness `ctkreduced`.
6. Result in `! typedExpressionIRreduced` annotated with `expressionNoteIR`.

14.5.1.2. Bitwise complement

Typing `~ expression'`, under cursor `p` and typing context `TC`:

1. Let `typedExpressionIR` be
 - the result of typing `expression'`, under cursor `p` and typing context `TC`.
2. Let `typedExpressionIRreduced` be ! the result of reducing serializable enums in `typedExpressionIR` until `$compat_bnot` is satisfied.
3. Let `typeIRreduced` be the type of `typedExpressionIRreduced`.
4. Let `ctkreduced` be the compile-time known-ness of `typedExpressionIRreduced`.
5. Let `expressionNoteIR` be type `typeIRreduced` and compile-time known-ness `ctkreduced`.
6. Result in `~ typedExpressionIRreduced` annotated with `expressionNoteIR`.

14.5.1.3. Plus and minus

Typing unop expression' , under cursor p and typing context TC :

1. Check that unop is equal to $+$ or unop is equal to $-$.
2. Let typedExpressionIR be
 - the result of typing expression' , under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_{\text{reduced}}$ be ! the result of reducing serializable enums in typedExpressionIR until $\$compat_uplusminus$ is satisfied.
4. Let $\text{typeIR}_{\text{reduced}}$ be the type of $\text{typedExpressionIR}_{\text{reduced}}$.
5. Let $\text{ctk}_{\text{reduced}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{reduced}}$.
6. Let expressionNoteIR be type $\text{typeIR}_{\text{reduced}}$ and compile-time known-ness $\text{ctk}_{\text{reduced}}$.
7. Result in $\text{unop typedExpressionIR}_{\text{reduced}}$ annotated with expressionNoteIR .

14.5.2. Local compile-time evaluation

Local compile-time evaluation of $\text{unop typedExpressionIR}$ annotated with expressionNoteIR , under cursor p and typing context TC :

1. Let value be
 - the result of local compile-time evaluation of typedExpressionIR , under cursor p and typing context TC .
2. Result in unop value .

14.5.3. Compile-time evaluation

Compile-time evaluation of $\text{unop typedExpressionIR}$ annotated with expressionNoteIR , under cursor p , instantiation context IC , and store STO :

1. Let the updated store STO_1 and value be
 - the result of compile-time evaluation of typedExpressionIR , under cursor p , instantiation context IC , and store STO .
2. Result in the updated store STO_1 and unop value .

14.5.4. Runtime evaluation

Evaluating $\text{unop typedExpressionIR}$ annotated with expressionNoteIR , under cursor p , evaluation context EC , and store STO :

1. Let the updated evaluation context EC_1 , store STO_1 and expressionResult be
 - the result of evaluating typedExpressionIR , under cursor p , evaluation context EC , and store STO .
2. If let abortResult be expressionResult :

a. Result in the updated evaluation context EC_1 , store STO_1 and `abortResult`.

3. Else:

a. Let `value` be `expressionResult`.

b. Let `value'` be `unop value`.

c. Let `expressionResult'` be `value'`.

d. Result in the updated evaluation context EC_1 , store STO_1 and `expressionResult'`.

14.6. Binary expressions

```
binaryExpression
: expression binop expression
;

binop
: *
| /
| %
| +
| -
| | + |
| | - |
| <<
| >>
| <=
| >=
| <
| >
| !=
| ==
| &
| ^
| |
| ++
| &&
| ||
;
```

See [Section 19.2](#) for details on how each binary operator is evaluated. Note that logical operators (`&&`, `||`) use short-circuit evaluation.

14.6.1. Type checking

14.6.1.1. Plus, minus, and multiplication

Typing `expressioni`, `binop expressionr`, under cursor `p` and typing context `TC`:

1. Check that `binop` is in `[ +, -, * ]`.
2. Let `typedExpressionIRi` be
 - the result of typing `expressioni`, under cursor `p` and typing context `TC`.
3. Let `typedExpressionIRr` be
 - the result of typing `expressionr`, under cursor `p` and typing context `TC`.
4. Let (`typedExpressionIRi_cast`, `typedExpressionIRr_cast`) be ! `typedExpressionIRi` and `typedExpressionIRr` implicitly

cast to equal types.

5. Let ($\text{typedExpressionIR}_{l_reduced}, \text{typedExpressionIR}_{r_reduced}$) be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{l_cast}$ and $\text{typedExpressionIR}_{r_cast}$ until $\$compat_plusminusmult$ is satisfied.
6. Let $\text{typeIR}_{reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.
7. Let $\text{ctk}_{l_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{l_reduced}$.
8. Let $\text{ctk}_{r_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{r_reduced}$.
9. Let $\text{ctk}_{reduced}$ be the join of $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$.
10. Let expressionNoteIR be type $\text{typeIR}_{reduced}$ and compile-time known-ness $\text{ctk}_{reduced}$.
11. Result in $\text{typedExpressionIR}_{l_reduced} \text{ binop } \text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .

14.6.1.2. Saturating plus and minus

Typing $\text{expression}_l \text{ binop } \text{expression}_r$, under cursor p and typing context TC :

1. Check that binop is in $[|,|,``-|``|]+$.
2. Let $\text{typedExpressionIR}_l$ be
 - the result of typing expression_l , under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_r$ be
 - the result of typing expression_r , under cursor p and typing context TC .
4. Let ($\text{typedExpressionIR}_{l_cast}, \text{typedExpressionIR}_{r_cast}$) be ! $\text{typedExpressionIR}_l$ and $\text{typedExpressionIR}_r$ implicitly cast to equal types.
5. Let ($\text{typedExpressionIR}_{l_reduced}, \text{typedExpressionIR}_{r_reduced}$) be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{l_cast}$ and $\text{typedExpressionIR}_{r_cast}$ until $\$compat_satplusminus$ is satisfied.
6. Let $\text{typeIR}_{reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.
7. Let $\text{ctk}_{l_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{l_reduced}$.
8. Let $\text{ctk}_{r_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{r_reduced}$.
9. Let $\text{ctk}_{reduced}$ be the join of $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$.
10. Let expressionNoteIR be type $\text{typeIR}_{reduced}$ and compile-time known-ness $\text{ctk}_{reduced}$.
11. Result in $\text{typedExpressionIR}_{l_reduced} \text{ binop } \text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .

14.6.1.3. Division and modulo

Typing $\text{expression}_l \text{ binop } \text{expression}_r$, under cursor p and typing context TC :

1. Check that binop is in $[/, \%]$.
2. Let $\text{typedExpressionIR}_l$ be
 - the result of typing expression_l , under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_r$ be
 - the result of typing expression_r , under cursor p and typing context TC .
4. Let ($\text{typedExpressionIR}_{l_cast}, \text{typedExpressionIR}_{r_cast}$) be ! $\text{typedExpressionIR}_l$ and $\text{typedExpressionIR}_r$ implicitly cast to equal types.

5. Let ($\text{typedExpressionIR}_{l_reduced}, \text{typedExpressionIR}_{r_reduced}$) be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{l_cast}$ and $\text{typedExpressionIR}_{r_cast}$ until $\$compat_divmod$ is satisfied.
6. Let $\text{typeIR}_{r_reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.
7. Let $\text{ctk}_{l_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{l_reduced}$.
8. Let $\text{ctk}_{r_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{r_reduced}$.
9. If $\text{ctk}_{r_reduced}$ is LCTK:
 - a. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{r_reduced}$, under cursor p and typing context TC .
 - b. Check that value has type integerValue .
 - c. Let integerValue_r be value .
 - d. Let n_r be ! the natural number representation of integerValue_r .
 - e. Check that n_r is greater than 0.
 - f. Let $\text{ctk}_{reduced}$ be the join of $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$.
 - g. Let expressionNoteIR be type $\text{typeIR}_{reduced}$ and compile-time known-ness $\text{ctk}_{reduced}$.
 - h. Result in $\text{typedExpressionIR}_{l_reduced}$ binop $\text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .
10. Else:
 - a. Let $\text{ctk}_{reduced}$ be the join of $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$.
 - b. Let expressionNoteIR be type $\text{typeIR}_{reduced}$ and compile-time known-ness $\text{ctk}_{reduced}$.
 - c. Result in $\text{typedExpressionIR}_{l_reduced}$ binop $\text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .

14.6.1.4. Shift left and right

Typing expression_l binop expression_r , under cursor p and typing context TC :

1. Check that binop is in [<<, >>].
2. Let $\text{typedExpressionIR}_l$ be
 - the result of typing expression_l , under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_r$ be
 - the result of typing expression_r , under cursor p and typing context TC .
4. Let ($\text{typedExpressionIR}_{l_reduced}, \text{typedExpressionIR}_{r_reduced}$) be ! the result of reducing serializable enums in $\text{typedExpressionIR}_l$ and $\text{typedExpressionIR}_r$ until $\$compat_shift$ is satisfied.
5. Let $\text{typeIR}_{l_reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.
6. Let $\text{ctk}_{l_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{l_reduced}$.
7. Let $\text{typeIR}_{r_reduced}$ be the type of $\text{typedExpressionIR}_{r_reduced}$.
8. Let $\text{ctk}_{r_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{r_reduced}$.
9. If $\text{typeIR}_{r_reduced}$ has type fixedBitTypeIR :
 - a. Check that if $\text{typeIR}_{l_reduced}$ is equal to INT , then $\text{ctk}_{r_reduced}$ is equal to LCTK .
 - b. Let $\text{ctk}_{reduced}$ be the join of $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$.
 - c. Let expressionNoteIR be type $\text{typeIR}_{l_reduced}$ and compile-time known-ness $\text{ctk}_{reduced}$.

d. Result in $\text{typedExpressionIR}_{l_reduced}$ binop $\text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .

10. Else:

- a. Check that $\text{ctk}_{r_reduced}$ is LCTK.
- b. Let $\text{ctk}_{reduced}$ be the join of $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$.
- c. Let expressionNoteIR be type $\text{typeIR}_{l_reduced}$ and compile-time known-ness $\text{ctk}_{reduced}$.
- d. Result in $\text{typedExpressionIR}_{l_reduced}$ binop $\text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .

14.6.1.5. Equality and inequality

Typing expression_l binop expression_r , under cursor p and typing context TC :

1. Check that binop is in [==, !=].
2. Let $\text{typedExpressionIR}_l$ be
 - the result of typing expression_l , under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_r$ be
 - the result of typing expression_r , under cursor p and typing context TC .
4. Let ($\text{typedExpressionIR}_{l_cast}$, $\text{typedExpressionIR}_{r_cast}$) be ! $\text{typedExpressionIR}_l$ and $\text{typedExpressionIR}_r$ implicitly cast to equal types.
5. Let typeIR_{cast} be the type of $\text{typedExpressionIR}_{l_cast}$.
6. Let ctk_{l_cast} be the compile-time known-ness of $\text{typedExpressionIR}_{l_cast}$.
7. Let ctk_{r_cast} be the compile-time known-ness of $\text{typedExpressionIR}_{r_cast}$.
8. Check that typeIR_{cast} supports equality comparison.
9. Let ctk_{cast} be the join of ctk_{l_cast} and ctk_{r_cast} .
10. Let expressionNoteIR be type BOOL and compile-time known-ness ctk_{cast} .
11. Result in $\text{typedExpressionIR}_{l_cast}$ binop $\text{typedExpressionIR}_{r_cast}$ annotated with expressionNoteIR .

14.6.1.6. Comparison

Typing expression_l binop expression_r , under cursor p and typing context TC :

1. Check that binop is in [<=, >=, <, >].
2. Let $\text{typedExpressionIR}_l$ be
 - the result of typing expression_l , under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_r$ be
 - the result of typing expression_r , under cursor p and typing context TC .
4. Let ($\text{typedExpressionIR}_{l_cast}$, $\text{typedExpressionIR}_{r_cast}$) be ! $\text{typedExpressionIR}_l$ and $\text{typedExpressionIR}_r$ implicitly cast to equal types.
5. Let ($\text{typedExpressionIR}_{l_reduced}$, $\text{typedExpressionIR}_{r_reduced}$) be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{l_cast}$ and $\text{typedExpressionIR}_{r_cast}$ until \$compat_compare is satisfied.
6. Let $\text{typeIR}_{reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.

7. Let $ctk_{l_reduced}$ be the compile-time known-ness of $typedExpressionIR_{l_reduced}$.
8. Let $ctk_{r_reduced}$ be the compile-time known-ness of $typedExpressionIR_{r_reduced}$.
9. Let $ctk_{reduced}$ be the join of $ctk_{l_reduced}$ and $ctk_{r_reduced}$.
10. Let $expressionNoteIR$ be type `BOOL` and compile-time known-ness $ctk_{reduced}$.
11. Result in $typedExpressionIR_{l_reduced}$ binop $typedExpressionIR_{r_reduced}$ annotated with $expressionNoteIR$.

14.6.1.7. Bitwise and, xor, and or

Typing $expression_l$ binop $expression_r$, under cursor p and typing context TC :

1. Check that $binop$ is in `[ &, ^, | ]`.
2. Let $typedExpressionIR_l$ be
 - the result of typing $expression_l$, under cursor p and typing context TC .
3. Let $typedExpressionIR_r$ be
 - the result of typing $expression_r$, under cursor p and typing context TC .
4. Let $(typedExpressionIR_{l_cast}, typedExpressionIR_{r_cast})$ be ! $typedExpressionIR_l$ and $typedExpressionIR_r$ implicitly cast to equal types.
5. Let $(typedExpressionIR_{l_reduced}, typedExpressionIR_{r_reduced})$ be ! the result of reducing serializable enums in $typedExpressionIR_{l_cast}$ and $typedExpressionIR_{r_cast}$ until `$compat_bitwise` is satisfied.
6. Let $typeIR_{reduced}$ be the type of $typedExpressionIR_{l_reduced}$.
7. Let $ctk_{l_reduced}$ be the compile-time known-ness of $typedExpressionIR_{l_reduced}$.
8. Let $ctk_{r_reduced}$ be the compile-time known-ness of $typedExpressionIR_{r_reduced}$.
9. Let $ctk_{reduced}$ be the join of $ctk_{l_reduced}$ and $ctk_{r_reduced}$.
10. Let $expressionNoteIR$ be type $typeIR_{reduced}$ and compile-time known-ness $ctk_{reduced}$.
11. Result in $typedExpressionIR_{l_reduced}$ binop $typedExpressionIR_{r_reduced}$ annotated with $expressionNoteIR$.

14.6.1.8. Concatenation

Typing $expression_l$ $expression_r$, under cursor p and typing context TC :

1. Let $typedExpressionIR_l$ be
 - the result of typing $expression_l$, under cursor p and typing context TC .
2. Let $typedExpressionIR_r$ be
 - the result of typing $expression_r$, under cursor p and typing context TC .
3. Let $(typedExpressionIR_{l_reduced}, typedExpressionIR_{r_reduced})$ be ! the result of reducing serializable enums in $typedExpressionIR_l$ and $typedExpressionIR_r$ until `$compat_concat` is satisfied.
4. Let $typeIR_{l_reduced}$ be the type of $typedExpressionIR_{l_reduced}$.
5. Let $ctk_{l_reduced}$ be the compile-time known-ness of $typedExpressionIR_{l_reduced}$.
6. Let $typeIR_{r_reduced}$ be the type of $typedExpressionIR_{r_reduced}$.
7. Let $ctk_{r_reduced}$ be the compile-time known-ness of $typedExpressionIR_{r_reduced}$.

8. Check that if $\text{typeIR}_{l_reduced}$ is equal to **STRING** and $\text{typeIR}_{r_reduced}$ is equal to **STRING**, then $\text{ctk}_{l_reduced}$ is equal to **LCTK** and $\text{ctk}_{r_reduced}$ is equal to **LCTK**.
9. Let $\text{typeIR}_{reduced}$ be $\$result_concat(\text{typeIR}_{l_reduced}, \text{typeIR}_{r_reduced})$.
10. Let $\text{ctk}_{reduced}$ be the join of $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$.
11. Let expressionNoteIR be **type** $\text{typeIR}_{reduced}$ and **compile-time known-ness** $\text{ctk}_{reduced}$.
12. Result in $\text{typedExpressionIR}_{l_reduced} \text{ typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .

14.6.1.9. Logical and, and or

Typing expression_l binop expression_r , under cursor p and typing context TC :

1. Check that **binop** is in [**&&**, **||**].
2. Let $\text{typedExpressionIR}_l$ be
 - the result of typing expression_l , under cursor p and typing context TC .
3. Let $\text{typedExpressionIR}_r$ be
 - the result of typing expression_r , under cursor p and typing context TC .
4. Let ($\text{typedExpressionIR}_{l_cast}$, $\text{typedExpressionIR}_{r_cast}$) be ! $\text{typedExpressionIR}_l$ and $\text{typedExpressionIR}_r$ implicitly cast to equal types.
5. Let ($\text{typedExpressionIR}_{l_reduced}$, $\text{typedExpressionIR}_{r_reduced}$) be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{l_cast}$ and $\text{typedExpressionIR}_{r_cast}$ until $\$compat_logical$ is satisfied.
6. Let $\text{typeIR}_{reduced}$ be the type of $\text{typedExpressionIR}_{l_reduced}$.
7. Let $\text{ctk}_{l_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{l_reduced}$.
8. Let $\text{ctk}_{r_reduced}$ be the compile-time known-ness of $\text{typedExpressionIR}_{r_reduced}$.
9. Let $\text{ctk}_{reduced}$ be the join of $\text{ctk}_{l_reduced}$ and $\text{ctk}_{r_reduced}$.
10. Let expressionNoteIR be **type** $\text{typeIR}_{reduced}$ and **compile-time known-ness** $\text{ctk}_{reduced}$.
11. Result in $\text{typedExpressionIR}_{l_reduced} \text{ binop } \text{typedExpressionIR}_{r_reduced}$ annotated with expressionNoteIR .

14.6.2. Local compile-time evaluation

Local compile-time evaluation of $\text{typedExpressionIR}_l$ binop $\text{typedExpressionIR}_r$ annotated with expressionNoteIR , under cursor p and typing context TC :

1. If **binop** is not in [**&&**, **||**]:
 - a. Let value_l be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_l$, under cursor p and typing context TC .
 - b. Let value_r be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_r$, under cursor p and typing context TC .
 - c. Result in $\text{value}_l \text{ binop } \text{value}_r$.
2. If **binop** is **&&**:

- a. Let `value` be
 - the result of local compile-time evaluation of `typedExpressionIRi`, under cursor `p` and typing context `TC`.
 - b. If `value` is equal to ``B false`:
 - i. Result in ``B false`.
 - c. If `value` is equal to ``B true`:
 - i. Let `valuer` be
 - the result of local compile-time evaluation of `typedExpressionIRr`, under cursor `p` and typing context `TC`.
 - ii. Result in `valuer`.
3. Else if `binop` is `||`:
- a. Let `value` be
 - the result of local compile-time evaluation of `typedExpressionIRi`, under cursor `p` and typing context `TC`.
 - b. If `value` is equal to ``B true`:
 - i. Result in ``B true`.
 - c. If `value` is equal to ``B false`:
 - i. Let `valuer` be
 - the result of local compile-time evaluation of `typedExpressionIRr`, under cursor `p` and typing context `TC`.
 - ii. Result in `valuer`.

14.6.3. Compile-time evaluation

Compile-time evaluation of `typedExpressionIRi binop typedExpressionIRr` annotated with `expressionNotelR`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. If `binop` is not in `[ &&, || ]`:
 - a. Let the updated store `STO1` and `valuei` be
 - the result of compile-time evaluation of `typedExpressionIRi`, under cursor `p`, instantiation context `IC`, and store `STO`.
 - b. Let the updated store `STO2` and `valuer` be
 - the result of compile-time evaluation of `typedExpressionIRr`, under cursor `p`, instantiation context `IC`, and store `STO1`.
 - c. Result in the updated store `STO2` and `valuei binop valuer`.
2. If `binop` is `&&`:
 - a. Let the updated store `STO1` and `value` be
 - the result of compile-time evaluation of `typedExpressionIRi`, under cursor `p`, instantiation context `IC`, and store `STO`.
 - b. If `value` is equal to ``B false`:
 - i. Result in the updated store `STO1` and ``B false`.

- c. If **value** is equal to **`B true**:
 - i. Let the updated store STO_2 and **value_r** be
 - the result of **compile-time evaluation of** $typedExpressionIR_r$, **under cursor** p , **instantiation context** IC , and store STO_1 .
 - ii. Result in the updated store STO_2 and **value_r**.
3. Else if **binop** is **||**:
- a. Let the updated store STO_1 and **value** be
 - the result of **compile-time evaluation of** $typedExpressionIR_r$, **under cursor** p , **instantiation context** IC , and store STO .
 - b. If **value** is equal to **`B true**:
 - i. Result in the updated store STO_1 and **`B true**.
 - c. If **value** is equal to **`B false**:
 - i. Let the updated store STO_2 and **value_r** be
 - the result of **compile-time evaluation of** $typedExpressionIR_r$, **under cursor** p , **instantiation context** IC , and store STO_1 .
 - ii. Result in the updated store STO_2 and **value_r**.

14.6.4. Runtime evaluation

Evaluating $typedExpressionIR_i$ **binop** $typedExpressionIR_r$ **annotated with** $expressionNoteIR$, **under cursor** p , **evaluation context** EC , and store STO :

1. If **binop** is not in **[&&, ||]**:
 - a. Let the updated evaluation context EC_1 , store STO_1 and **expressionListResult** be
 - the result of **evaluating** $[typedExpressionIR_i, typedExpressionIR_r]$, **under cursor** p , **evaluation context** EC , and store STO .
 - b. If let **abortResult** be **expressionListResult**:
 - i. Result in the updated evaluation context EC_1 , store STO_1 and **abortResult**.
 - c. Else:
 - i. Let **value*** be **expressionListResult**.
 - ii. Check that **value*** is a list of length 2.
 - iii. Let $[value_r, value_i]$ be **value***.
 - iv. Let **value'** be $value_i$ **binop** $value_r$.
 - v. Let **expressionResult** be **value'**.
 - vi. Result in the updated evaluation context EC_1 , store STO_1 and **expressionResult**.
2. If **binop** is **&&**:
 - a. Let the updated evaluation context EC_1 , store STO_1 and **expressionResult** be
 - the result of **evaluating** $typedExpressionIR_i$, **under cursor** p , **evaluation context** EC , and store STO .
 - b. If let **abortResult** be **expressionResult**:
 - i. Result in the updated evaluation context EC_1 , store STO_1 and **abortResult**.

- c. If `expressionResult` is equal to ``B false``:
 - i. Let `expressionResult` be ``B false``.
 - ii. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult`'.
 - d. If `expressionResult` is equal to ``B true``:
 - i. Let the updated evaluation context `EC2`, store `STO2` and `expressionResult` be
 - the result of `evaluating` `typedExpressionIRi`, under cursor `p`, evaluation context `EC1`, and store `STO1`.
 - ii. If let `abortResult` be `expressionResult`':
 - A. Result in the updated evaluation context `EC2`, store `STO2` and `abortResult`.
 - iii. Result in the updated evaluation context `EC2`, store `STO2` and `expressionResult`'.
3. Else if `binop` is `||`:
- a. Let the updated evaluation context `EC1`, store `STO1` and `expressionResult` be
 - the result of `evaluating` `typedExpressionIRi`, under cursor `p`, evaluation context `EC`, and store `STO`.
 - b. If let `abortResult` be `expressionResult`:
 - i. Result in the updated evaluation context `EC1`, store `STO1` and `abortResult`.
 - c. If `expressionResult` is equal to ``B true``:
 - i. Let `expressionResult` be ``B true``.
 - ii. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult`'.
 - d. If `expressionResult` is equal to ``B false``:
 - i. Let the updated evaluation context `EC2`, store `STO2` and `expressionResult` be
 - the result of `evaluating` `typedExpressionIRi`, under cursor `p`, evaluation context `EC1`, and store `STO1`.
 - ii. If let `abortResult` be `expressionResult`':
 - A. Result in the updated evaluation context `EC2`, store `STO2` and `abortResult`.
 - iii. Result in the updated evaluation context `EC2`, store `STO2` and `expressionResult`'.

14.7. Ternary expressions

A conditional expression of the form `e1 ? e2 : e3` behaves the same as in languages like C. As described above, the expression `e1` is evaluated first, and second either `e2` or `e3` is evaluated depending on the result.

The first sub-expression `e1` must have Boolean type and the second and third sub-expressions must have the same type, which cannot both be arbitrary-precision integers unless the condition itself can be evaluated at compilation time. This restriction is designed to ensure that the width of the result of the conditional expression can be inferred statically at compile time.

```
ternaryExpression
: expression ? expression : expression
;
```

14.7.1. Type checking

After type checking, ternary expressions have the form:

```
ternaryExpressionIR
: typedExpressionIR ? typedExpressionIR : typedExpressionIR
;
```

Typing $\text{expression}_{\text{cond}} ? \text{expression}_{\text{true}} : \text{expression}_{\text{false}}$, under cursor p and typing context TC :

1. Let $\text{typedExpressionIR}_{\text{cond}}$ be
 - the result of typing $\text{expression}_{\text{cond}}$, under cursor p and typing context TC .
2. Let $\text{typeIR}_{\text{cond}}$ be the type of $\text{typedExpressionIR}_{\text{cond}}$.
3. Let ctk_{cond} be the compile-time known-ness of $\text{typedExpressionIR}_{\text{cond}}$.
4. Check that BOOL is equal to $\text{typeIR}_{\text{cond}}$ with typedefs unrolled.
5. Let $\text{typedExpressionIR}_{\text{true}}$ be
 - the result of typing $\text{expression}_{\text{true}}$, under cursor p and typing context TC .
6. Let $\text{typedExpressionIR}_{\text{false}}$ be
 - the result of typing $\text{expression}_{\text{false}}$, under cursor p and typing context TC .
7. Let ($\text{typedExpressionIR}_{\text{true_cast}}$ $\text{typedExpressionIR}_{\text{false_cast}}$) be ! $\text{typedExpressionIR}_{\text{true}}$ and $\text{typedExpressionIR}_{\text{false}}$ implicitly cast to equal types.
8. Let $\text{typeIR}_{\text{cast}}$ be the type of $\text{typedExpressionIR}_{\text{true_cast}}$.
9. Let $\text{ctk}_{\text{true_cast}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{true_cast}}$.
10. Let $\text{ctk}_{\text{false_cast}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{false_cast}}$.
11. Check that if $\text{typeIR}_{\text{cast}}$ is equal to INT , then ctk_{cond} is not equal to DYN .
12. Let ctk be the join of [ctk_{cond} , $\text{ctk}_{\text{true_cast}}$, $\text{ctk}_{\text{false_cast}}$].
13. Let expressionNoteIR be type $\text{typeIR}_{\text{cast}}$ and compile-time known-ness ctk .
14. Result in $\text{typedExpressionIR}_{\text{cond}} ? \text{typedExpressionIR}_{\text{true_cast}} : \text{typedExpressionIR}_{\text{false_cast}}$ annotated with expressionNoteIR .

14.7.2. Local compile-time evaluation

Local compile-time evaluation of $\text{typedExpressionIR}_{\text{cond}} ? \text{typedExpressionIR}_{\text{true}} : \text{typedExpressionIR}_{\text{false}}$ annotated with expressionNoteIR , under cursor p and typing context TC :

1. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{cond}}$, under cursor p and typing context TC .
2. If value is equal to 'B true' :
 - a. Let $\text{value}_{\text{true}}$ be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{true}}$, under cursor p and typing context TC .

- b. Result in $value_{true}$.
- 3. If $value$ is equal to `B false`:
 - a. Let $value_{false}$ be
 - the result of **local compile-time evaluation** of $typedExpressionIR_{false}$, under cursor p and typing context TC .
 - b. Result in $value_{false}$.

14.7.3. Compile-time evaluation

Compile-time evaluation of $typedExpressionIR_{cond} ? typedExpressionIR_{true} : typedExpressionIR_{false}$ annotated with $expressionNoteIR$, under cursor p , instantiation context IC , and store STO :

1. Let the updated store STO_1 and $value$ be
 - the result of **compile-time evaluation** of $typedExpressionIR_{cond}$, under cursor p , instantiation context IC , and store STO .
2. If $value$ is equal to `B true`:
 - a. Let the updated store STO_2 and $value_{true}$ be
 - the result of **compile-time evaluation** of $typedExpressionIR_{true}$, under cursor p , instantiation context IC , and store STO_1 .
 - b. Result in the updated store STO_2 and $value_{true}$.
3. If $value$ is equal to `B false`:
 - a. Let the updated store STO_2 and $value_{false}$ be
 - the result of **compile-time evaluation** of $typedExpressionIR_{false}$, under cursor p , instantiation context IC , and store STO_1 .
 - b. Result in the updated store STO_2 and $value_{false}$.

14.7.4. Runtime evaluation

Evaluating $typedExpressionIR_{cond} ? typedExpressionIR_{true} : typedExpressionIR_{false}$ annotated with $expressionNoteIR$, under cursor p , evaluation context EC , and store STO :

1. Let the updated evaluation context EC_1 , store STO_1 and $expressionResult$ be
 - the result of **evaluating** $typedExpressionIR_{cond}$, under cursor p , evaluation context EC , and store STO .
2. If let $abortResult$ be $expressionResult$:
 - a. Result in the updated evaluation context EC_1 , store STO_1 and $abortResult$.
3. If $expressionResult$ is equal to `B true`:
 - a. Let the updated evaluation context EC_2 , store STO_2 and $expressionResult_{true}$ be
 - the result of **evaluating** $typedExpressionIR_{true}$, under cursor p , evaluation context EC_1 , and store STO_1 .
 - b. Result in the updated evaluation context EC_2 , store STO_2 and $expressionResult_{true}$.
4. If $expressionResult$ is equal to `B false`:

- a. Let the updated evaluation context EC_2 , store STO_2 and $expressionResult_{false}$ be
 - the result of evaluating $typedExpressionIR_{false}$, under cursor p , evaluation context EC_1 , and store STO_1 .
- b. Result in the updated evaluation context EC_2 , store STO_2 and $expressionResult_{false}$.

14.8. Cast expressions

P4 provides a limited set of casts between types. A cast is written $(t) e$, where t is a type and e is an expression.

```
castExpression
: `( type ) expression
;
```

14.8.1. Type checking

Typing $(type_t) expression'$, under cursor p and typing context TC :

1. Let $typeIR_t$ and a set of fresh type variables $typed^*$ be
 - the result of typing $type_t$, under cursor p and typing context TC .
2. Check that $typed^*$ is an empty list.
3. Let $bound$ be bound type variables from the p layer of TC .
4. Check that $typeIR_t$ is a well-formed type, with bound type variables $bound$.
5. Let $typedExpressionIR$ be
 - the result of typing $expression'$, under cursor p and typing context TC .
6. Let $typeIR$ be the type of $typedExpressionIR$.
7. Let ctk be the compile-time known-ness of $typedExpressionIR$.
8. Check that $typeIR$ can be explicitly cast to $typeIR_t$.
9. Let $expressionNoteIR$ be type $typeIR_t$ and compile-time known-ness ctk .
10. Result in $(typeIR_t) typedExpressionIR$ annotated with $expressionNoteIR$.

14.8.2. Local compile-time evaluation

Local compile-time evaluation of $(typeIR) typedExpressionIR$ annotated with $expressionNoteIR$, under cursor p and typing context TC :

1. Let $value$ be
 - the result of local compile-time evaluation of $typedExpressionIR$, under cursor p and typing context TC .
2. Let $value_{cast}$ be $value$ casted to type $typeIR$.
3. Result in $value_{cast}$.

14.8.3. Compile-time evaluation

Compile-time evaluation of (typeIR) typedExpressionIR annotated with expressionNoteIR , under cursor p , instantiation context IC , and store STO :

1. Let the updated store STO_1 and value be
 - the result of compile-time evaluation of typedExpressionIR , under cursor p , instantiation context IC , and store STO .
2. Let $\text{value}_{\text{cast}}$ be value casted to type typeIR .
3. Result in the updated store STO_1 and $\text{value}_{\text{cast}}$.

14.8.4. Runtime evaluation

Evaluating (typeIR) typedExpressionIR annotated with expressionNoteIR , under cursor p , evaluation context EC , and store STO :

1. Let the updated evaluation context EC_1 , store STO_1 and expressionResult be
 - the result of evaluating typedExpressionIR , under cursor p , evaluation context EC , and store STO .
2. If let abortResult be expressionResult :
 - a. Result in the updated evaluation context EC_1 , store STO_1 and abortResult .
3. Else:
 - a. Let value be expressionResult .
 - b. Let $\text{value}_{\text{cast}}$ be value casted to type typeIR .
 - c. Result in the updated evaluation context EC_1 , store STO_1 and $\text{value}_{\text{cast}}$.

14.9. Invalid header expressions

$\{\#\}$ represents an invalid header expression. See [Section 8.6.2](#) for more details.

```
invalidHeaderExpression
: {#}
;
```

14.9.1. Type checking

After type checking, an invalid header expression is represented as:

```
invalidHeaderExpressionIR = invalidHeaderExpression
```

Typing $\{\#\}$, under cursor p and typing context TC :

1. Let expressionNoteIR be type HEADER_INVALID and compile-time known-ness LCTK .

2. Result in `{#}` annotated with `expressionNoteIR`.

14.9.2. Local compile-time evaluation

Local compile-time evaluation of `{#}` annotated with `expressionNoteIR`, under cursor `p` and typing context `TC`:

1. Result in `{#}`.

14.9.3. Compile-time evaluation

Compile-time evaluation of `{#}` annotated with `expressionNoteIR`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. Result in the updated store `STO` and `{#}`.

14.9.4. Runtime evaluation

Evaluating `{#}` annotated with `expressionNoteIR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let `expressionResult` be `{#}`.
2. Result in the updated evaluation context `EC`, store `STO` and `expressionResult`.

14.10. Sequence expressions

A sequence expression is a comma-separated list of expressions enclosed in curly braces `{ }`. See [Section 8.6.3](#) for details.

```
sequenceOrRecordExpression
  : `{ sequenceOrRecordElementExpression trailingCommaOpt }
  ;

sequenceElementExpression = expressionList

sequenceOrRecordElementExpression
  : sequenceElementExpression
  | recordElementExpression
  ;
```

14.10.1. Type checking

After type checking, a sequence expression is represented as:

```
sequenceExpressionIR
  : SEQ `{ typedExpressionListIR }
  | SEQ `{ typedExpressionListIR , ... }
  ;
```

Typing { expressionList trailingCommaOpt }, under cursor p and typing context TC:

1. Let expression_e^* be the list of expressions in expressionList.
2. If ... is not in expression_e^* :
 - a. Let $\text{typedExpressionIR}_e^*$ be the list of $\text{typedExpressionIR}_e$, obtained by repeating:
 - Let $\text{typedExpressionIR}_e$ be
 - the result of typing expression_e , under cursor p and typing context TC.for each expression_e in expression_e^*
 - b. Let typeIR_e^* be the list of typeIR_e , obtained by repeating:
 - Let typeIR_e be the type of $\text{typedExpressionIR}_e$.for each $\text{typedExpressionIR}_e$ in $\text{typedExpressionIR}_e^*$
 - c. Let ctk_e^* be the list of ctk_e , obtained by repeating:
 - Let ctk_e be the compile-time known-ness of $\text{typedExpressionIR}_e$.for each $\text{typedExpressionIR}_e$ in $\text{typedExpressionIR}_e^*$
 - d. Let typeIR be $\text{SEQ} < \text{typeIR}_e^* >$.
 - e. Let ctk be the join of ctk_e^* .
 - f. Let expressionNoteIR be type typeIR and compile-time known-ness ctk .
 - g. Result in $\text{SEQ} \{ \text{typedExpressionIR}_e^* \}$ annotated with expressionNoteIR .
3. Else:
 - a. Let expression^* be the reversal of expression_e^* .
 - b. Check that expression^* is a non-empty list.
 - c. Let $\text{expression}'' :: \text{expression}_{e_h_rev}^*$ be expression^* .
 - d. Check that $\text{expression}''$ has type defaultExpression .
 - e. Let defaultExpression be $\text{expression}''$.
 - f. Let $\text{expression}_{e_h}^*$ be the reversal of $\text{expression}_{e_h_rev}^*$.
 - g. Check that ... is not in $\text{expression}_{e_h}^*$.
 - h. Let $\text{typedExpressionIR}_{e_h}^*$ be the list of $\text{typedExpressionIR}_{e_h}$, obtained by repeating:
 - Let $\text{typedExpressionIR}_{e_h}$ be
 - the result of typing expression_{e_h} , under cursor p and typing context TC.for each expression_{e_h} in $\text{expression}_{e_h}^*$
 - i. Let $\text{typeIR}_{e_h}^*$ be the list of typeIR_{e_h} , obtained by repeating:
 - Let typeIR_{e_h} be the type of $\text{typedExpressionIR}_{e_h}$.for each $\text{typedExpressionIR}_{e_h}$ in $\text{typedExpressionIR}_{e_h}^*$

j. Let $\text{ctk}_{e_h}^*$ be the list of ctk_{e_h} , obtained by repeating:

- Let ctk_{e_h} be the compile-time known-ness of $\text{typedExpressionIR}_{e_h}$.

for each $\text{typedExpressionIR}_{e_h}$ in $\text{typedExpressionIR}_{e_h}^*$

k. Let typeIR be $\text{SEQ} < \text{typeIR}_{e_h}^*, \dots >$.

l. Let ctk be the join of $\text{ctk}_{e_h}^*$.

m. Let expressionNoteIR be type typeIR and compile-time known-ness ctk .

n. Result in $\text{SEQ} \{ \text{typedExpressionIR}_{e_h}^*, \dots \}$ annotated with expressionNoteIR .

14.10.2. Local compile-time evaluation

Local compile-time evaluation of $\text{sequenceExpressionIR}$ annotated with expressionNoteIR , under cursor p and typing context TC :

1. If let $\text{SEQ} \{ \text{typedExpressionIR}^* \}$ be $\text{sequenceExpressionIR}$:

a. Let value^* be the list of value , obtained by repeating:

- Let value be
 - the result of local compile-time evaluation of typedExpressionIR , under cursor p and typing context TC .

for each typedExpressionIR in $\text{typedExpressionIR}^*$

b. Result in $\text{SEQ} (\text{value}^*)$.

2. Else:

a. Let $\text{SEQ} \{ \text{typedExpressionIR}^*, \dots \}$ be $\text{sequenceExpressionIR}$.

b. Let value^* be the list of value , obtained by repeating:

- Let value be
 - the result of local compile-time evaluation of typedExpressionIR , under cursor p and typing context TC .

for each typedExpressionIR in $\text{typedExpressionIR}^*$

c. Result in $\text{SEQ} (\text{value}^*, \dots)$.

14.10.3. Compile-time evaluation

Compile-time evaluation of $\text{sequenceExpressionIR}$ annotated with expressionNoteIR , under cursor p , instantiation context IC , and store STO :

1. If let $\text{SEQ} \{ \text{typedExpressionListIR} \}$ be $\text{sequenceExpressionIR}$:

a. Let the updated store STO_1 and value^* be

- the result of compile-time evaluation of $\text{typedExpressionListIR}$, under cursor p , instantiation context IC , and store STO .

b. Result in the updated store STO_1 and $SEQ (value^*, \dots)$.

2. Else:

a. Let $SEQ \{ typedExpressionListIR, \dots \}$ be $sequenceExpressionIR$.

b. Let the updated store STO_1 and $value^*$ be

- the result of **compile-time evaluation** of $typedExpressionListIR$, under cursor p , **instantiation context** IC , and store STO .

c. Result in the updated store STO_1 and $SEQ (value^*, \dots)$.

14.10.4. Runtime evaluation

Evaluating $sequenceExpressionIR$ annotated with $expressionNotelR$, under cursor p , evaluation context EC , and store STO :

1. If let $SEQ \{ typedExpressionListIR \}$ be $sequenceExpressionIR$:

a. Let the updated evaluation context EC_1 , store STO_1 and $expressionListResult$ be

- the result of **evaluating** $typedExpressionListIR$, under cursor p , **evaluation context** EC , and **store** STO .

b. If let $abortResult$ be $expressionListResult$:

i. Result in the updated evaluation context EC_1 , store STO_1 and $abortResult$.

c. Else:

i. Let $value^*$ be $expressionListResult$.

ii. Let $expressionResult$ be $SEQ (value^*, \dots)$.

iii. Result in the updated evaluation context EC_1 , store STO_1 and $expressionResult$.

2. Else:

a. Let $SEQ \{ typedExpressionListIR, \dots \}$ be $sequenceExpressionIR$.

b. Let the updated evaluation context EC_1 , store STO_1 and $expressionListResult$ be

- the result of **evaluating** $typedExpressionListIR$, under cursor p , **evaluation context** EC , and **store** STO .

c. If let $abortResult$ be $expressionListResult$:

i. Result in the updated evaluation context EC_1 , store STO_1 and $abortResult$.

d. Else:

i. Let $value^*$ be $expressionListResult$.

ii. Let $expressionResult$ be $SEQ (value^*, \dots)$.

iii. Result in the updated evaluation context EC_1 , store STO_1 and $expressionResult$.

14.11. Record expressions

A record expression is a comma-separated list of named expressions enclosed in curly braces $\{ \}$. See [Section 8.6.4](#) for details.

`sequenceOrRecordExpression`

```

: `{ sequenceOrRecordElementExpression trailingCommaOpt }
;

recordElementExpression
: name = expression
| name = expression , ...
| name = expression , namedExpressionList
| name = expression , namedExpressionList , ...
;

sequenceOrRecordElementExpression
: sequenceElementExpression
| recordElementExpression
;

```

14.11.1. Type checking

After type checking, a record expression is represented as:

```

recordExpressionIR
: RECORD `{ namedExpressionListIR }
| RECORD `{ namedExpressionListIR , ... }
;

```

Typing { recordElementExpression trailingCommaOpt }, under cursor p and typing context TC :

1. If let $name_f = expression_f$ be recordElementExpression:
 - a. Let $nameIR_f$ be the name of $name_f$.
 - b. Let $typedExpressionIR_f$ be
 - the result of typing $expression_f$, under cursor p and typing context TC .
 - c. Let $typeIR_f$ be the type of $typedExpressionIR_f$.
 - d. Let ctk_f be the compile-time known-ness of $typedExpressionIR_f$.
 - e. Let $typeIR$ be $RECORD \{ \text{EMPTY } typeIR_f \text{ } nameIR_f ; \}$.
 - f. Let $expressionNoteIR$ be type $typeIR$ and compile-time known-ness ctk_f .
 - g. Result in $RECORD \{ nameIR_f = typedExpressionIR_f \}$ annotated with $expressionNoteIR$.
2. Else if let $name_f = expression_f , \dots$ be recordElementExpression:
 - a. Let $nameIR_f$ be the name of $name_f$.
 - b. Let $typedExpressionIR_f$ be
 - the result of typing $expression_f$, under cursor p and typing context TC .
 - c. Let $typeIR_f$ be the type of $typedExpressionIR_f$.
 - d. Let ctk_f be the compile-time known-ness of $typedExpressionIR_f$.
 - e. Let $typeIR$ be $RECORD \{ \text{EMPTY } typeIR_f \text{ } nameIR_f ; , \dots \}$.
 - f. Let $expressionNoteIR$ be type $typeIR$ and compile-time known-ness ctk_f .
 - g. Result in $RECORD \{ nameIR_f = typedExpressionIR_f , \dots \}$ annotated with $expressionNoteIR$.
3. Else if let $name_{f,h} = expression_{f,h} , namedExpressionList_t$ be recordElementExpression:
 - a. Let $(name_{f,t} = expression_{f,t})^*$ be the list of named expressions in $namedExpressionList_t$.
 - b. Let $name_f^*$ be $name_{f,h} :: name_{f,t}^*$.

- c. Let nameIR_f^* be the list of nameIR_f , obtained by repeating:
- Let nameIR_f be the name of name_f .
- for each name_f in name_f^*
- d. Let expression_f^* be $\text{expression}_{f_h} :: \text{expression}_{f_t}^*$.
- e. Let $\text{typedExpressionIR}_f^*$ be the list of $\text{typedExpressionIR}_f$, obtained by repeating:
- Let $\text{typedExpressionIR}_f$ be
 - the result of typing expression_f , under cursor p and typing context TC .
- for each expression_f in expression_f^*
- f. Let typeIR_f^* be the list of typeIR_f , obtained by repeating:
- Let typeIR_f be the type of $\text{typedExpressionIR}_f$.
- for each $\text{typedExpressionIR}_f$ in $\text{typedExpressionIR}_f^*$
- g. Let ctk_f^* be the list of ctk_f , obtained by repeating:
- Let ctk_f be the compile-time known-ness of $\text{typedExpressionIR}_f$.
- for each $\text{typedExpressionIR}_f$ in $\text{typedExpressionIR}_f^*$
- h. Let typeIR be $\text{RECORD } \{ (\text{EMPTY typeIR}_f \text{ nameIR}_f ;)^* \}$.
- i. Let ctk be the join of ctk_f^* .
- j. Let expressionNoteIR be type typeIR and compile-time known-ness ctk .
- k. Result in $\text{RECORD } \{ (\text{nameIR}_f = \text{typedExpressionIR}_f)^* \}$ annotated with expressionNoteIR .
4. Else:
- a. Let $\text{name}_{f_h} = \text{expression}_{f_h}$, $\text{namedExpressionList}_t, \dots$ be $\text{recordElementExpression}$.
- b. Let $(\text{name}_{f_t} = \text{expression}_{f_t})^*$ be the list of named expressions in $\text{namedExpressionList}_t$.
- c. Let name_f^* be $\text{name}_{f_h} :: \text{name}_{f_t}^*$.
- d. Let nameIR_f^* be the list of nameIR_f , obtained by repeating:
- Let nameIR_f be the name of name_f .
- for each name_f in name_f^*
- e. Let expression_f^* be $\text{expression}_{f_h} :: \text{expression}_{f_t}^*$.
- f. Let $\text{typedExpressionIR}_f^*$ be the list of $\text{typedExpressionIR}_f$, obtained by repeating:
- Let $\text{typedExpressionIR}_f$ be
 - the result of typing expression_f , under cursor p and typing context TC .
- for each expression_f in expression_f^*
- g. Let typeIR_f^* be the list of typeIR_f , obtained by repeating:
- Let typeIR_f be the type of $\text{typedExpressionIR}_f$.

for each $\text{typedExpressionIR}_f$ in $\text{typedExpressionIR}_f^*$

h. Let ctk_f^* be the list of ctk_f , obtained by repeating:

- Let ctk_f be the compile-time known-ness of $\text{typedExpressionIR}_f$.

for each $\text{typedExpressionIR}_f$ in $\text{typedExpressionIR}_f^*$

i. Let typeIR be $\text{RECORD } \{ (\text{EMPTY typeIR}_f \text{ nameIR}_f ;)^* , \dots \}$.

j. Let ctk be the join of ctk_f^* .

k. Let expressionNoteIR be type typeIR and compile-time known-ness ctk .

l. Result in $\text{RECORD } \{ (\text{nameIR}_f = \text{typedExpressionIR}_f)^* , \dots \}$ annotated with expressionNoteIR .

14.11.2. Local compile-time evaluation

Local compile-time evaluation of $\text{recordExpressionIR}$ annotated with expressionNoteIR , under cursor p and typing context TC :

1. If let $\text{RECORD } \{ (\text{nameIR} = \text{typedExpressionIR})^* \}$ be $\text{recordExpressionIR}$:

a. Let value^* be the list of value , obtained by repeating:

- Let value be
 - the result of local compile-time evaluation of typedExpressionIR , under cursor p and typing context TC .

for each typedExpressionIR in $\text{typedExpressionIR}^*$

b. Result in $\text{RECORD } \{ (\text{value nameIR} ;)^* \}$.

2. Else:

a. Let $\text{RECORD } \{ (\text{nameIR} = \text{typedExpressionIR})^* , \dots \}$ be $\text{recordExpressionIR}$.

b. Let value^* be the list of value , obtained by repeating:

- Let value be
 - the result of local compile-time evaluation of typedExpressionIR , under cursor p and typing context TC .

for each typedExpressionIR in $\text{typedExpressionIR}^*$

c. Result in $\text{RECORD } \{ (\text{value nameIR} ;)^* , \dots \}$.

14.11.3. Compile-time evaluation

Compile-time evaluation of $\text{recordExpressionIR}$ annotated with expressionNoteIR , under cursor p , instantiation context IC , and store STO :

1. If let $\text{RECORD } \{ (\text{nameIR} = \text{typedExpressionIR})^* \}$ be $\text{recordExpressionIR}$:

a. Let the updated store STO_1 and value^* be

- the result of **compile-time evaluation of** `typedExpressionIR*`, under cursor `p`, instantiation context `IC`, and store `STO`.
- b. Result in the updated store `STO1` and `RECORD { ( value nameIR ; )* }`.
2. Else:
- a. Let `RECORD { ( nameIR = typedExpressionIR )* , ... }` be `recordExpressionIR`.
- b. Let the updated store `STO1` and `value*` be
- the result of **compile-time evaluation of** `typedExpressionIR*`, under cursor `p`, instantiation context `IC`, and store `STO`.
- c. Result in the updated store `STO1` and `RECORD { ( value nameIR ; )* , ... }`.

14.11.4. Runtime evaluation

Evaluating `recordExpressionIR` annotated with `expressionNoteIR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. If let `RECORD { ( nameIR = typedExpressionIR )* }` be `recordExpressionIR`:
 - a. Let the updated evaluation context `EC1`, store `STO1` and `expressionListResult` be
 - the result of **evaluating** `typedExpressionIR*`, under cursor `p`, evaluation context `EC`, and store `STO`.
 - b. If let `abortResult` be `expressionListResult`:
 - i. Result in the updated evaluation context `EC1`, store `STO1` and `abortResult`.
 - c. Else:
 - i. Let `value*` be `expressionListResult`.
 - ii. Let `expressionResult` be `RECORD { ( value nameIR ; )* }`.
 - iii. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult`.
2. Else:
 - a. Let `RECORD { ( nameIR = typedExpressionIR )* , ... }` be `recordExpressionIR`.
 - b. Let the updated evaluation context `EC1`, store `STO1` and `expressionListResult` be
 - the result of **evaluating** `typedExpressionIR*`, under cursor `p`, evaluation context `EC`, and store `STO`.
 - c. If let `abortResult` be `expressionListResult`:
 - i. Result in the updated evaluation context `EC1`, store `STO1` and `abortResult`.
 - d. Else:
 - i. Let `value*` be `expressionListResult`.
 - ii. Let `expressionResult` be `RECORD { ( value nameIR ; )* , ... }`.
 - iii. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult`.

14.12. Error access expression

An error access expression accesses a symbolic name in the `error` namespace. See [Section 8.2.4](#) for details about error types and values.

```
errorAccessExpression
: ERROR . member
;
```

14.12.1. Type checking

After type checking, an error access expression is represented as:

```
errorAccessExpressionIR
: ERROR . nameIR
;
```

Typing `ERROR . member`, under cursor `p` and typing context `TC`:

1. Let `nameIR` be the name of `member`.
2. Let `nameIRerror` be "error." concatenated with `nameIR`.
3. Check that `ERROR . nameIR` is equal to the value of variable `nameIRerror` from the `p` layer of `TC`.
4. Let `expressionNoteIR` be type `ERROR` and compile-time known-ness `LCTK`.
5. Result in `ERROR . nameIR` annotated with `expressionNoteIR`.

14.12.2. Local compile-time evaluation

Local compile-time evaluation of `ERROR . nameIR` annotated with type `typeIR` and compile-time known-ness `ctk`, under cursor `p` and typing context `TC`:

1. Let `nameIRerror` be "error." concatenated with `nameIR`.
2. Let `valueerror` be the value of variable `nameIRerror` from the `p` layer of `TC`.
3. Result in `valueerror`.

14.12.3. Compile-time evaluation

Compile-time evaluation of `ERROR . nameIR` annotated with `expressionNoteIR`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. Let `nameIRerror` be "error." concatenated with `nameIR`.
2. Let `valueerror` be the value of `nameIRerror` from the `p` layer of `IC`.
3. Result in the updated store `STO` and `valueerror`.

14.12.4. Runtime evaluation

Evaluating `ERROR . nameIR` annotated with `expressionNoteIR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let $\text{nameIR}_{\text{error}}$ be "error." concatenated with nameIR .
2. Let $\text{value}_{\text{error}}$ be the value of $\text{nameIR}_{\text{error}}$ from the p layer of EC.
3. Let expressionResult be $\text{value}_{\text{error}}$.
4. Result in the updated evaluation context EC, store STO and expressionResult .

14.13. Member access expressions

A member access expression accesses a user-defined or built-in field in an aggregate value. Also, it may access an element defined in an enum declaration.

```
memberAccessExpression
  : memberAccessBase . member
  ;

memberAccessBase
  : prefixedTypeName
  | expression
  ;
```

14.13.1. Type checking

After type checking, a member access expression is represented in $P4_{\text{IR}}$ as:

```
memberAccessExpressionIR
  : memberAccessBaseIR . nameIR
  ;

memberAccessBaseIR
  : TYPE prefixedNameIR
  | typedExpressionIR
  ;
```

Accessing an enum member

Typing $\text{prefixedTypeName}_{\text{base}} . \text{member}$, under cursor p and typing context TC:

1. Let $\text{prefixedNameIR}_{\text{base}}$ be the prefixed name of $\text{prefixedTypeName}_{\text{base}}$.
2. Let $\text{typeDefIR}_{\text{base}}$ be ! the type definition of $\text{prefixedNameIR}_{\text{base}}$ from the p layer of TC.
3. Check that $\text{typeDefIR}_{\text{base}}$ is monomorphic.
4. Let $\text{typeIR}_{\text{base}}$ be the underlying type of $\text{typeDefIR}_{\text{base}}$.
5. Let typeIR be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
6. Check that typeIR has type enumTypeIR .
7. Let enumTypeIR be typeIR .
8. If let $\text{ENUM } _ \{ \text{nameIR}_{\text{field}}^* \}$ be enumTypeIR :
 - a. Let nameIR be the name of member.
 - b. Check that nameIR is in $\text{nameIR}_{\text{field}}^*$.
 - c. Let expressionNoteIR be type $\text{typeIR}_{\text{base}}$ and compile-time known-ness LCTK.

d. Result in $\text{TYPE prefixedNameIR}_{\text{base}} \cdot \text{nameIR}$ annotated with expressionNoteIR .

9. Else:

- a. Let $\text{ENUM } _ < _ > \{ (\text{nameIR}_{\text{field}} = _ ;)^* \}$ be enumTypeIR .
- b. Let nameIR be the name of member.
- c. Check that nameIR is in $\text{nameIR}_{\text{field}}^*$.
- d. Let expressionNoteIR be type $\text{typeIR}_{\text{base}}$ and compile-time known-ness LCTK.
- e. Result in $\text{TYPE prefixedNameIR}_{\text{base}} \cdot \text{nameIR}$ annotated with expressionNoteIR .

Accessing an aggregate member

Typing $\text{expression}_{\text{base}} \cdot \text{member}$, under cursor p and typing context TC :

1. Let $\text{typedExpressionIR}_{\text{base}}$ be
 - the result of typing $\text{expression}_{\text{base}}$, under cursor p and typing context TC .
2. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
3. Let typeIR be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
4. If let $\text{typeIR}' [n_{\text{size}}]$ be typeIR :
 - a. If "size" is equal to the name of member:
 - i. Let expressionNoteIR be type $\text{BIT} < 32 >$ and compile-time known-ness LCTK.
 - ii. Result in $\text{typedExpressionIR}_{\text{base}} \cdot \text{"size"}$ annotated with expressionNoteIR .
 - b. If "lastIndex" is equal to the name of member:
 - i. Check that p is equal to BLOCK and TC.BLOCK.KIND is equal to PARSER or p is equal to LOCAL and TC.LOCAL.KIND is equal to PARSER_STATE .
 - ii. Let expressionNoteIR be type $\text{BIT} < 32 >$ and compile-time known-ness DYN.
 - iii. Result in $\text{typedExpressionIR}_{\text{base}} \cdot \text{"lastIndex"}$ annotated with expressionNoteIR .
 - c. If "last" is equal to the name of member:
 - i. Check that p is equal to BLOCK and TC.BLOCK.KIND is equal to PARSER or p is equal to LOCAL and TC.LOCAL.KIND is equal to PARSER_STATE .
 - ii. Let expressionNoteIR be type typeIR' and compile-time known-ness DYN.
 - iii. Result in $\text{typedExpressionIR}_{\text{base}} \cdot \text{"last"}$ annotated with expressionNoteIR .
 - d. If "next" is equal to the name of member:
 - i. Check that p is equal to BLOCK and TC.BLOCK.KIND is equal to PARSER or p is equal to LOCAL and TC.LOCAL.KIND is equal to PARSER_STATE .
 - ii. Let expressionNoteIR be type typeIR' and compile-time known-ness DYN.
 - iii. Result in $\text{typedExpressionIR}_{\text{base}} \cdot \text{"next"}$ annotated with expressionNoteIR .
5. Let ctk_{base} be the compile-time known-ness of $\text{typedExpressionIR}_{\text{base}}$.
6. If let $\text{STRUCT } _ < _ > \{ (_ \text{typeIR}_f \text{ nameIR}_f ;)^* \}$ be typeIR :
 - a. Let nameIR be the name of member.
 - b. Let typeIR'' be ! the value associated to nameIR in $(\text{nameIR}_f, \text{typeIR}_f)^*$.
 - c. Let expressionNoteIR be type typeIR'' and compile-time known-ness ctk_{base} .

- d. Result in $\text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR .
- 7. Else if let $\text{HEADER_}_ < _ > \{ (_ \text{typeIR}_f \text{nameIR}_f ;)^* \}$ be typeIR :
 - a. Let nameIR be the name of member.
 - b. Let typeIR be ! the value associated to nameIR in $(\text{nameIR}_f, \text{typeIR}_f)^*$.
 - c. Let expressionNoteIR be type typeIR and compile-time known-ness ctk_{base} .
 - d. Result in $\text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR .
- 8. Else if let $\text{HEADER_UNION_}_ < _ > \{ (_ \text{typeIR}_f \text{nameIR}_f ;)^* \}$ be typeIR :
 - a. Let nameIR be the name of member.
 - b. Let typeIR be ! the value associated to nameIR in $(\text{nameIR}_f, \text{typeIR}_f)^*$.
 - c. Let expressionNoteIR be type typeIR and compile-time known-ness ctk_{base} .
 - d. Result in $\text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR .
- 9. Else if let $\text{TABLE_STRUCT_}_ \{ \text{HIT } \text{boolTypeIR} ; \text{MISS } \text{boolTypeIR}' ; \text{ACTION_RUN } \text{tableMetadataEnumTypeIR} ; \}$ be typeIR :
 - a. Let nameIR be the name of member.
 - b. If nameIR is equal to "hit":
 - i. Let expressionNoteIR be type boolTypeIR and compile-time known-ness DYN.
 - ii. Result in $\text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR .
 - c. Else if nameIR is equal to "miss":
 - i. Let expressionNoteIR be type $\text{boolTypeIR}'$ and compile-time known-ness DYN.
 - ii. Result in $\text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR .
 - d. Else if nameIR is equal to "action_run":
 - i. Let expressionNoteIR be type $\text{tableMetadataEnumTypeIR}$ and compile-time known-ness DYN.
 - ii. Result in $\text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR .

14.13.2. Local compile-time evaluation

Accessing an enum member

Local compile-time evaluation of $\text{TYPE } \text{prefixedNameIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR , under cursor p and typing context TC :

1. Let $\text{typeDefIR}_{\text{base}}$ be ! the type definition of $\text{prefixedNameIR}_{\text{base}}$ from the p layer of TC .
2. Check that $\text{typeDefIR}_{\text{base}}$ is monomorphic.
3. Let $\text{typeIR}_{\text{base}}$ be the underlying type of $\text{typeDefIR}_{\text{base}}$.
4. Let typeIR' be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
5. Check that typeIR' has type enumTypeIR .
6. Let enumTypeIR be typeIR' .
7. If let $\text{ENUM } \text{typeID} \{ \text{nameIR}_{\text{field}}^* \}$ be enumTypeIR :
 - a. Check that nameIR is in $\text{nameIR}_{\text{field}}^*$.
 - b. Result in $\text{typeID} . \text{nameIR}$.

8. Else:

- a. Let $\text{ENUM typeId} < \text{typeIR} > \{ (\text{nameIR}_{\text{field}} = \text{value}_{\text{field}} ;)^* \}$ be enumTypeIR .
- b. Let value' be ! the value associated to nameIR in $(\text{nameIR}_{\text{field}}, \text{value}_{\text{field}})^*$.
- c. Result in $\text{typeId} . \text{nameIR} . \text{value}'$.

Accessing an aggregate member

Local compile-time evaluation of $\text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR , under cursor p and typing context TC :

1. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
2. Let typeIR' be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
3. If let $_ [n_{\text{size}}]$ be typeIR' :
 - a. Check that nameIR is equal to "size".
 - b. Result in $D n_{\text{size}}$.
4. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{base}}$ under cursor p and typing context TC .
5. If let structValue be value :
 - a. Let $\text{STRUCT } _ \{ (\text{value}_{\text{field}} \text{nameIR}_{\text{field}} ;)^* \}$ be structValue .
 - b. Let value'' be ! the value associated to nameIR in $(\text{nameIR}_{\text{field}}, \text{value}_{\text{field}})^*$.
 - c. Result in value'' .
6. Else if let headerValue be value :
 - a. Let $\text{HEADER } _ \{ _ ; (\text{value}_{\text{field}} \text{nameIR}_{\text{field}} ;)^* \}$ be headerValue .
 - b. Let value'' be ! the value associated to nameIR in $(\text{nameIR}_{\text{field}}, \text{value}_{\text{field}})^*$.
 - c. Result in value'' .
7. Else if let headerUnionValue be value :
 - a. Let $\text{HEADER_UNION } _ \{ (\text{value}_{\text{field}} \text{nameIR}_{\text{field}} ;)^* \}$ be headerUnionValue .
 - b. Let value'' be ! the value associated to nameIR in $(\text{nameIR}_{\text{field}}, \text{value}_{\text{field}})^*$.
 - c. Result in value'' .

14.13.3. Compile-time evaluation

Accessing an enum member

Compile-time evaluation of $\text{TYPE prefixedNameIR} . \text{nameIR}$ annotated with expressionNoteIR , under cursor p , instantiation context IC , and store STO :

1. Let $\text{typeDefIR}'$ be ! the type definition of prefixedNameIR from the p layer of IC .
2. Check that $\text{typeDefIR}'$ has type enumTypeIR .
3. Let enumTypeIR be $\text{typeDefIR}'$.

4. If let **ENUM** $\text{typed} \{ _ \}^*$ be enumTypeIR :
 - a. Result in the updated store STO and $\text{typed} . \text{nameIR}$.
5. Else:
 - a. Let **ENUM** $\text{typed} < _ > \{ (\text{id}_{\text{member}} = \text{value}_{\text{member}} ;)^* \}$ be enumTypeIR .
 - b. Let value' be ! the value associated to nameIR in $(\text{id}_{\text{member}}, \text{value}_{\text{member}})^*$.
 - c. Result in the updated store STO and $\text{typed} . \text{nameIR} . \text{value}'$.

Accessing an aggregate member

Compile-time evaluation of $\text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ annotated with expressionNoteIR , under cursor p , instantiation context IC , and store STO :

1. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
2. Let typeIR be $\text{typeIR}_{\text{base}}$ with **typedefs unrolled**.
3. If let $_ [n_{\text{size}}]$ be typeIR :
 - a. Check that nameIR is equal to "size".
 - b. Result in the updated store STO and $D n_{\text{size}}$.
4. Let the updated store STO_1 and value be
 - the result of compile-time evaluation of $\text{typedExpressionIR}_{\text{base}}$, under cursor p , instantiation context IC , and store STO .
5. If let structValue be value :
 - a. Let **STRUCT** $_ \{ (\text{value}_{\text{field}} \text{id}_{\text{field}} ;)^* \}$ be structValue .
 - b. Let value'' be ! the value associated to nameIR in $(\text{id}_{\text{field}}, \text{value}_{\text{field}})^*$.
 - c. Result in the updated store STO_1 and value'' .
6. Else if let headerValue be value :
 - a. Let **HEADER** $_ \{ _ ; (\text{value}_{\text{field}} \text{id}_{\text{field}} ;)^* \}$ be headerValue .
 - b. Let value'' be ! the value associated to nameIR in $(\text{id}_{\text{field}}, \text{value}_{\text{field}})^*$.
 - c. Result in the updated store STO_1 and value'' .
7. Else if let headerUnionValue be value :
 - a. Let **HEADER_UNION** $_ \{ (\text{value}_{\text{field}} \text{id}_{\text{field}} ;)^* \}$ be headerUnionValue .
 - b. Let value'' be ! the value associated to nameIR in $(\text{id}_{\text{field}}, \text{value}_{\text{field}})^*$.
 - c. Result in the updated store STO_1 and value'' .

14.13.4. Runtime evaluation

Accessing an enum member

Evaluating **TYPE** $\text{prefixedNameIR} . \text{nameIR}$ annotated with expressionNoteIR , under cursor p , evaluation context EC , and store STO :

1. Let $\text{typeDefIR}'$ be ! $\$find_typeDef_e(p, \text{EC}, \text{prefixedNameIR})$.

2. Check that `typeDefIR'` has type `enumTypeIR`.
3. Let `enumTypeIR` be `typeDefIR'`.
4. If let `ENUM typeld { _ * }` be `enumTypeIR`:
 - a. Let `expressionResult` be `typeld . nameIR`.
 - b. Result in the updated evaluation context `EC`, store `STO` and `expressionResult`.
5. Else:
 - a. Let `ENUM typeld < _ > { ( nameIRmember = valuemember ; ) * }` be `enumTypeIR`.
 - b. Let `value'` be **! the value associated to `nameIR` in `( nameIRmember, valuemember )*`**.
 - c. Let `expressionResult` be `typeld . nameIR . value'`.
 - d. Result in the updated evaluation context `EC`, store `STO` and `expressionResult`.

Accessing an aggregate member

Evaluating `typedExpressionIRbase . nameIR` annotated with `expressionNoteIR`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let the updated evaluation context `EC1`, store `STO1` and `expressionResult` be
 - the result of evaluating `typedExpressionIRbase`, under cursor `p`, evaluation context `EC`, and store `STO`.
2. If let `abortResult` be `expressionResult`:
 - a. Result in the updated evaluation context `EC1`, store `STO1` and `abortResult`.
3. Else:
 - a. Let `valuebase` be `expressionResult`.
 - b. If let `HEADER_STACK [ value* ( nat ; nsize ) ]` be `valuebase*`:
 - i. If `nameIR` is equal to "size":
 - A. Let `expressionResult'` be `D nsize*`.
 - B. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult'`.
 - ii. Else if `nameIR` is equal to "last":
 - A. Let `bout_of_bounds` be `nat` is less than 1 or `nat` is greater than `nsize`.
 - B. If `bout_of_bounds`:
 - I. Let `rejectTransitionResult` be `REJECT ERROR . "StackOutOfBounds"`.
 - II. Result in the updated evaluation context `EC1`, store `STO1` and `rejectTransitionResult`.
 - C. Else:
 - I. Let `int` be `nat - 1`.
 - II. Check that `int` has type `nat`.
 - III. Let `nidx_last` be `int`.
 - IV. Let `valuelast` be `value*[nidx_last]`.
 - V. Let `expressionResult'` be `valuelast*`.
 - VI. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult'`.

- iii. Else if `nameIR` is equal to `"lastIndex"`:
 - A. Let `nat'` be `max( [ nat, 1 ] )`.
 - B. Let `int` be `nat' - 1`.
 - C. Check that `int` has type `nat`.
 - D. Let `nidx_last` be `int`.
 - E. Let `valueidx_last` be `32 W nidx_last`.
 - F. Let `expressionResult'` be `valueidx_last`.
 - G. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult'`.
- c. Else if let `STRUCT _ { ( valuefield idfield ; ) * }` be `valuebase`:
 - i. Let `valuemember` be `! the value associated to nameIR in ( idfield, valuefield ) *`.
 - ii. Let `expressionResult'` be `valuemember`.
 - iii. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult'`.
- d. Else if let `HEADER _ { _ ; ( valuefield idfield ; ) * }` be `valuebase`:
 - i. Let `valuemember` be `! the value associated to nameIR in ( idfield, valuefield ) *`.
 - ii. Let `expressionResult'` be `valuemember`.
 - iii. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult'`.
- e. Else if let `HEADER_UNION _ { ( valuefield idfield ; ) * }` be `valuebase`:
 - i. Let `valuemember` be `! the value associated to nameIR in ( idfield, valuefield ) *`.
 - ii. Let `expressionResult'` be `valuemember`.
 - iii. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult'`.
- f. Else if let `TABLE_STRUCT _ { HIT boolValuehit ; MISS boolValue ; ACTION_RUN tableMetadataEnumValue ; }` be `valuebase`:
 - i. If `nameIR` is equal to `"hit"`:
 - A. Let `expressionResult'` be `boolValuehit`.
 - B. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult'`.
 - ii. Else if `nameIR` is equal to `"miss"`:
 - A. Let `expressionResult'` be `boolValue`.
 - B. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult'`.
 - iii. Else if `nameIR` is equal to `"action_run"`:
 - A. Let `expressionResult'` be `tableMetadataEnumValue`.
 - B. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult'`.

14.14. Index access expressions

An index access expression accesses an element of a tuple or a header stack.

```
indexAccessExpression
: expression `[ expression ]
;
```

14.14.1. Type checking

After type checking, an index access expression is represented as:

```
indexAccessExpressionIR
: typedExpressionIR `[ typedExpressionIR ]
;
```

Typing $\text{expression}_{\text{base}} [\text{expression}_{\text{index}}]$, under cursor p and typing context TC :

1. Let $\text{typedExpressionIR}_{\text{base}}$ be
 - the result of typing $\text{expression}_{\text{base}}$, under cursor p and typing context TC .
2. Let $\text{typedExpressionIR}_{\text{index}}$ be
 - the result of typing $\text{expression}_{\text{index}}$, under cursor p and typing context TC .
3. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
4. Let ctk_{base} be the compile-time known-ness of $\text{typedExpressionIR}_{\text{base}}$.
5. Let $\text{typeIR}_{\text{index}}$ be the type of $\text{typedExpressionIR}_{\text{index}}$.
6. Let $\text{ctk}_{\text{index}}$ be the compile-time known-ness of $\text{typedExpressionIR}_{\text{index}}$.
7. Let $\text{typedExpressionIR}_{\text{index_reduced}}$ be ! the result of reducing serializable enums in $\text{typedExpressionIR}_{\text{index}}$ until $\$compat_array_index$ is satisfied.
8. Let typeIR be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
9. If let $\text{TUPLE} < \text{typeIR}_e^* >$ be typeIR :
 - a. Check that $\text{ctk}_{\text{index}}$ is LCTK.
 - b. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{index_reduced}}$, under cursor p and typing context TC .
 - c. Check that value has type integerValue .
 - d. Let $\text{integerValue}_{\text{index}}$ be value .
 - e. Let n_{index} be ! the natural number representation of $\text{integerValue}_{\text{index}}$.
 - f. Check that n_{index} is less than the length of typeIR_e^* .
 - g. Let expressionNoteIR be $\text{typeIR}_e^*[n_{\text{index}}]$ and compile-time known-ness ctk_{base} .
 - h. Result in $\text{typedExpressionIR}_{\text{base}} [\text{typedExpressionIR}_{\text{index_reduced}}]$ annotated with expressionNoteIR .
10. Else if let $\text{typeIR}' [n_{\text{size}}]$ be typeIR :
 - a. If $\text{ctk}_{\text{index}}$ is LCTK:
 - i. Let value be
 - the result of local compile-time evaluation of $\text{typedExpressionIR}_{\text{index_reduced}}$, under cursor p and typing context TC .
 - ii. Check that value has type integerValue .
 - iii. Let $\text{integerValue}_{\text{index}}$ be value .
 - iv. Let n_{index} be ! the natural number representation of $\text{integerValue}_{\text{index}}$.
 - v. Check that n_{index} is less than n_{size} .

- vi. Let expressionNoteIR be **type** typeIR' and **compile-time known-ness** ctk_{base} .
 - vii. Result in $\text{typedExpressionIR}_{\text{base}} [\text{typedExpressionIR}_{\text{index_reduced}}]$ **annotated with** expressionNoteIR .
- b. Else:
- i. Let ctk be **the join of** ctk_{base} and $\text{ctk}_{\text{index}}$.
 - ii. Let expressionNoteIR be **type** typeIR' and **compile-time known-ness** ctk .
 - iii. Result in $\text{typedExpressionIR}_{\text{base}} [\text{typedExpressionIR}_{\text{index_reduced}}]$ **annotated with** expressionNoteIR .

14.14.2. Local compile-time evaluation

Local compile-time evaluation of $\text{typedExpressionIR}_{\text{base}} [\text{typedExpressionIR}_{\text{index}}]$ **annotated with** expressionNoteIR , **under cursor** p **and typing context** TC :

1. Let $\text{value}_{\text{base}}$ be
 - the result of **local compile-time evaluation of** $\text{typedExpressionIR}_{\text{base}}$, **under cursor** p **and typing context** TC .
2. Let value be
 - the result of **local compile-time evaluation of** $\text{typedExpressionIR}_{\text{index}}$, **under cursor** p **and typing context** TC .
3. Check that value has **type** integerValue .
4. Let $\text{integerValue}_{\text{index}}$ be value .
5. If let $\text{TUPLE} (\text{value}_e^*)$ be $\text{value}_{\text{base}}$:
 - a. Let n_{index} be **! the natural number representation of** $\text{integerValue}_{\text{index}}$.
 - b. Check that n_{index} is less than the length of value_e^* .
 - c. Let value' be $\text{value}_e^*[n_{\text{index}}]$.
 - d. Result in value' .
6. Else if let $\text{HEADER_STACK} [\text{value}_e^* (_ ; _)]$ be $\text{value}_{\text{base}}$:
 - a. Let n_{index} be **! the natural number representation of** $\text{integerValue}_{\text{index}}$.
 - b. Check that n_{index} is less than the length of value_e^* .
 - c. Let value' be $\text{value}_e^*[n_{\text{index}}]$.
 - d. Result in value' .

14.14.3. Compile-time evaluation

Compile-time evaluation of $\text{typedExpressionIR}_{\text{base}} [\text{typedExpressionIR}_{\text{index}}]$ **annotated with** expressionNoteIR , **under cursor** p , **instantiation context** IC , **and store** STO :

1. Let the updated store STO_1 and $\text{value}_{\text{base}}$ be
 - the result of **compile-time evaluation of** $\text{typedExpressionIR}_{\text{base}}$, **under cursor** p , **instantiation context** IC , **and store** STO .
2. Let the updated store STO_2 and value be
 - the result of **compile-time evaluation of** $\text{typedExpressionIR}_{\text{index}}$, **under cursor** p , **instantiation**

context IC , and store STO_1 .

3. Check that $value$ has type $integerValue$.
4. Let $integerValue_{index}$ be $value$.
5. If let $TUPLE (value_e^*)$ be $value_{base}$:
 - a. Let n_{index} be ! the natural number representation of $integerValue_{index}$.
 - b. Check that n_{index} is less than the length of $value_e^*$.
 - c. Let $value'$ be $value_e^*[n_{index}]$.
 - d. Result in the updated store STO_2 and $value'$.
6. Else if let $HEADER_STACK [value_e^* (_ ; _)]$ be $value_{base}$:
 - a. Let n_{index} be ! the natural number representation of $integerValue_{index}$.
 - b. Check that n_{index} is less than the length of $value_e^*$.
 - c. Let $value'$ be $value_e^*[n_{index}]$.
 - d. Result in the updated store STO_2 and $value'$.

14.14.4. Runtime evaluation

Evaluating $typedExpressionIR_{base} [typedExpressionIR_{index}]$ annotated with $expressionNodeIR$, under cursor p , evaluation context EC , and store STO :

1. Let the updated evaluation context EC_1 , store STO_1 and $expressionListResult$ be
 - the result of evaluating $[typedExpressionIR_{base}, typedExpressionIR_{index}]$, under cursor p , evaluation context EC , and store STO .
2. If let $abortResult$ be $expressionListResult$:
 - a. Result in the updated evaluation context EC_1 , store STO_1 and $abortResult$.
3. Else:
 - a. Let $value^*$ be $expressionListResult$.
 - b. Check that $value^*$ is a list of length 2.
 - c. Let $[value_{base}, value']$ be $value^*$.
 - d. Check that $value'$ has type $integerValue$.
 - e. Let $integerValue_{index}$ be $value'$.
 - f. If let $TUPLE (value_{elem}^*)$ be $value_{base}$:
 - i. Let n_{index} be ! the natural number representation of $integerValue_{index}$.
 - ii. Let $expressionResult$ be $value_{elem}^*[n_{index}]$.
 - iii. Result in the updated evaluation context EC_1 , store STO_1 and $expressionResult$.
 - g. Else if let $HEADER_STACK [value_{elem}^* (_ ; n_{size})]$ be $value_{base}$:
 - i. Let n_{index} be ! the natural number representation of $integerValue_{index}$.
 - ii. Let int be n_{index} if n_{index} is less than n_{size} otherwise $n_{size} - 1$.
 - iii. Check that int has type nat .
 - iv. Let n_{idx} be int .

- v. Let `expressionResult` be `valueelem*[nidx]`.
- vi. Result in the updated evaluation context `EC1`, store `STO1` and `expressionResult`.

14.15. Bitslice access expressions

A bitslice access expression is used to access a contiguous range of bits within a numeric value. The syntax for a bitslice access expression is as follows:

```
sliceAccessExpression
: expression `[ expression : expression ]
;
```

14.15.1. Type checking

After type checking, a bitslice access expression is represented as:

```
sliceAccessExpressionIR
: typedExpressionIR `[ typedExpressionIR : typedExpressionIR ]
;
```

Typing `expressionbase [ expressionhi : expressionlo ]`, under cursor `p` and typing context `TC`:

1. Let `typedExpressionIRbase` be
 - the result of typing `expressionbase`, under cursor `p` and typing context `TC`.
2. Let `typedExpressionIRhi` be
 - the result of typing `expressionhi`, under cursor `p` and typing context `TC`.
3. Let `typedExpressionIRlo` be
 - the result of typing `expressionlo`, under cursor `p` and typing context `TC`.
4. Let `typedExpressionIRbase_reduced` be ! the result of reducing serializable enums in `typedExpressionIRbase` until `$compat_bitslice_base` is satisfied.
5. Let `typeIRbase_reduced` be the type of `typedExpressionIRbase_reduced`.
6. Let `ctkbase_reduced` be the compile-time known-ness of `typedExpressionIRbase_reduced`.
7. Let `typedExpressionIRhi_reduced` be ! the result of reducing serializable enums in `typedExpressionIRhi` until `$compat_bitslice_index` is satisfied.
8. Let `typedExpressionIRlo_reduced` be ! the result of reducing serializable enums in `typedExpressionIRlo` until `$compat_bitslice_index` is satisfied.
9. Let `typeIRhi_reduced` be the type of `typedExpressionIRhi_reduced`.
10. Let `ctkhi_reduced` be the compile-time known-ness of `typedExpressionIRhi_reduced`.
11. Let `typeIRlo_reduced` be the type of `typedExpressionIRlo_reduced`.
12. Let `ctklo_reduced` be the compile-time known-ness of `typedExpressionIRlo_reduced`.
13. Check that `ctkhi_reduced` is LCTK.
14. Let `value` be
 - the result of local compile-time evaluation of `typedExpressionIRhi_reduced`, under cursor `p` and

typing context TC .

15. Check that $value$ has type $integerValue$.
16. Let $integerValue_{hi}$ be $value$.
17. Let n_{hi} be ! the natural number representation of $integerValue_{hi}$.
18. Check that $ctk_{lo_reduced}$ is LCTK.
19. Let $value'$ be
 - the result of local compile-time evaluation of $typedExpressionIR_{lo_reduced}$, under cursor p and typing context TC .
20. Check that $value'$ has type $integerValue$.
21. Let $integerValue_{lo}$ be $value'$.
22. Let n_{lo} be ! the natural number representation of $integerValue_{lo}$.
23. Check that $typeIR_{base_reduced} [n_{lo} : n_{hi}]$ is a valid bitslice.
24. Let int be $n_{hi} - n_{lo} + 1$.
25. Check that int has type nat .
26. Let n_{slice} be int .
27. Let $typeIR$ be $BIT < n_{slice} >$.
28. Let $expressionNoteIR$ be type $typeIR$ and compile-time known-ness $ctk_{base_reduced}$.
29. Result in $typedExpressionIR_{base} [typedExpressionIR_{hi_reduced} : typedExpressionIR_{lo_reduced}]$ annotated with $expressionNoteIR$.

14.15.2. Local compile-time evaluation

Local compile-time evaluation of $typedExpressionIR_{base} [typedExpressionIR_{hi} : typedExpressionIR_{lo}]$ annotated with $expressionNoteIR$, under cursor p and typing context TC :

1. Let $value_{base}$ be
 - the result of local compile-time evaluation of $typedExpressionIR_{base}$, under cursor p and typing context TC .
2. Let $value_{hi}$ be
 - the result of local compile-time evaluation of $typedExpressionIR_{hi}$, under cursor p and typing context TC .
3. Let $value_{lo}$ be
 - the result of local compile-time evaluation of $typedExpressionIR_{lo}$, under cursor p and typing context TC .
4. Result in $value_{base} [value_{hi} : value_{lo}]$.

14.15.3. Compile-time evaluation

Compile-time evaluation of $typedExpressionIR_{base} [typedExpressionIR_{hi} : typedExpressionIR_{lo}]$ annotated with $expressionNoteIR$, under cursor p , instantiation context IC , and store STO :

1. Let the updated store STO_1 and $value_{base}$ be
 - the result of compile-time evaluation of $typedExpressionIR_{base}$, under cursor p , instantiation context IC , and store STO .
2. Let the updated store STO_2 and $value_{hi}$ be
 - the result of compile-time evaluation of $typedExpressionIR_{hi}$, under cursor p , instantiation context IC , and store STO_1 .
3. Let the updated store STO_3 and $value_{lo}$ be
 - the result of compile-time evaluation of $typedExpressionIR_{lo}$, under cursor p , instantiation context IC , and store STO_2 .
4. Result in the updated store STO_3 and $value_{base} [value_{hi} : value_{lo}]$.

14.15.4. Runtime evaluation

Evaluating $typedExpressionIR_{base} [typedExpressionIR_{hi} : typedExpressionIR_{lo}]$ annotated with $expressionNotelR$, under cursor p , evaluation context EC , and store STO :

1. Let the updated evaluation context EC_1 , store STO_1 and $expressionListResult$ be
 - the result of evaluating $[typedExpressionIR_{base}, typedExpressionIR_{hi}, typedExpressionIR_{lo}]$, under cursor p , evaluation context EC , and store STO .
2. If let $abortResult$ be $expressionListResult$:
 - a. Result in the updated evaluation context EC_1 , store STO_1 and $abortResult$.
3. Else:
 - a. Let $value^*$ be $expressionListResult$.
 - b. Check that $value^*$ is a list of length 3.
 - c. Let $[value_{base}, value_{hi}, value_{lo}]$ be $value^*$.
 - d. Let $value'$ be $value_{base} [value_{hi} : value_{lo}]$.
 - e. Let $expressionResult$ be $value'$.
 - f. Result in the updated evaluation context EC_1 , store STO_1 and $expressionResult$.

14.16. Call expressions

Call expressions are used to invoke callables or constructors.

```
callExpression
: callTarget `( argumentList )
| callableTarget `< realTypeArgumentList > `( argumentList )
;

callTarget
: callableTarget
| constructorTarget
;

callableTarget = expression
constructorTarget = namedType
```

Details of how calls are resolved and evaluated are described in [Chapter 18](#).

14.16.1. Type checking

After type checking, call expressions are represented as:

```
callExpressionIR
: constructorTargetIR `( argumentListIR )
| callableTargetIR `< typeArgumentListIR > `( argumentListIR )
;

callableTargetIR
: referenceExpressionIR
| typedExpressionIR . nameIR
| TYPE prefixedNameIR . nameIR
| `( callableTargetIR )
;

constructorTargetIR
: prefixedNameIR `< typeArgumentListIR >
;
```

Calling a callable

Typing `callExpression`, under cursor `p` and typing context `TC`:

1. If let `callTarget ( argumentList )` be `callExpression`:
 - a. Check that `callTarget` has type `callableTarget`.
 - b. Let `callableTarget` be `callTarget`.
 - c. Let the typed callable target `callableTargetIR` be
 - the result of typing `callableTarget` as a callable target, under cursor `p` and typing context `TC`.
 - d. Let `argument*` be the list of arguments in `argumentList`.
 - e. Let `argumentIR*` be the list of `argumentIR`, obtained by repeating:
 - Let `argumentIR` be
 - the result of typing `argument`, under cursor `p` and typing context `TC`.for each `argument` in `argument*`
 - f. Let the callable type `callableTypeIR` with fresh type variables `typeIdinfer*`, defaults `iddefault*` and optionals `idoptional*` be
 - the result of finding the type of callable call `callableTargetIR < · > ( argumentIR* )`, under cursor `p` and typing context `TC`.
 - g. Let the return type `typeIRret` with inferred type arguments `typeArgumentIRinferred*` and the casted argument list `argumentIRcast*` be
 - the result of typing the invocation `callableTypeIR < · > ( argumentIR* )` with omitted type arguments `typeIdinfer*`, defaults `iddefault*` and optionals `idoptional*`, under cursor `p` and typing context `TC`.
 - h. If `typeIRret` is not equal to `VOID`:
 - i. Let `ctk` be the compile-time known-ness of `callableTypeIR`.

- ii. Check that `callableTypeIR` is a `static_assert` call.
 - iii. Let `callExpressionIR` be `callableTargetIR < typeArgumentIRinferred* > ( argumentIRcast* )`.
 - iv. Let `expressionNoteIR` be `type typeIRret` and `compile-time known-ness ctk`.
 - v. Let `value` be
 - the result of `local compile-time evaluation` of `callExpressionIR` annotated with `expressionNoteIR`, under cursor `p` and typing context `TC`.
 - vi. Check that `value` is equal to ``B true`.
 - vii. Let `literalExpressionIR` be `TRUE`.
 - viii. Result in `literalExpressionIR` annotated with `expressionNoteIR`.
- i. Let `argumentIR*` be
- the result of `typing argumentList`, under cursor `p` and typing context `TC`.
- j. If `typeIRret` is not equal to `VOID`:
- i. Let `ctk` be `the compile-time known-ness of callableTypeIR`.
 - ii. Check that `~$is_static_assert_callableTypeIR(callableTypeIR)`.
 - iii. Let `callExpressionIR` be `callableTargetIR < typeArgumentIRinferred* > ( argumentIRcast* )`.
 - iv. Let `expressionNoteIR` be `type typeIRret` and `compile-time known-ness ctk`.
 - v. Result in `callExpressionIR` annotated with `expressionNoteIR`.
2. Else:
- a. Let `callableTarget < realTypeArgumentList > ( argumentList )` be `callExpression`.
 - b. Let the typed callable target `callableTargetIR` be
 - the result of `typing callableTarget` as a callable target, under cursor `p` and typing context `TC`.
 - c. Let `realTypeArgument*` be `the list of real type arguments in realTypeArgumentList`.
 - d. Let `typeArgumentIR*` and a set of fresh type variables `typeIdimpl*` be
 - the result of `typing realTypeArgument*`, under cursor `p` and typing context `TC`.
 - e. Let `argumentIR*` be
 - the result of `typing argumentList`, under cursor `p` and typing context `TC`.
 - f. Let the callable type `callableTypeIR` with fresh type variables `typeIdinserted*`, defaults `iddefault*` and optionals `idoptional*` be
 - the result of `finding the type of callable call callableTargetIR < typeArgumentIR* > ( argumentIR* )`, under cursor `p` and typing context `TC`.
 - g. Let `typeIdinfer*` be `typeIdimpl*` concatenated with `typeIdinserted*`.
 - h. Let the return type `typeIRret` with inferred type arguments `typeArgumentIRinferred*` and the casted argument list `argumentIRcast*` be
 - the result of `typing the invocation callableTypeIR < typeArgumentIR* > ( argumentIR* )` with omitted type arguments `typeIdinfer*`, defaults `iddefault*` and optionals `idoptional*`, under cursor `p` and typing context `TC`.
 - i. Check that `typeIRret` is not equal to `VOID`.
 - j. Let `ctk` be `the compile-time known-ness of callableTypeIR`.
 - k. Let `callExpressionIR` be `callableTargetIR < typeArgumentIRinferred* > ( argumentIRcast* )`.

- l. Let expressionNoteIR be $\text{type } \text{typeIR}_{\text{ret}}$ and compile-time known-ness ctk .
- m. Result in callExpressionIR annotated with expressionNoteIR .

Calling a constructor

Typing $\text{callTarget}(\text{argumentList})$, under cursor p and typing context TC :

1. If let prefixedTypeName be callTarget :
 - a. Let argumentIR^* be
 - the result of typing argumentList , under cursor p and typing context TC .
 - b. Let prefixedNameIR be the prefixed name of prefixedTypeName .
 - c. Let $\text{constructorTargetIR}$ be $\text{prefixedNameIR} < \cdot >$.
 - d. Let constructorTypeIR with fresh type variables $\text{typeId}_{\text{impl}}^*$, defaults $\text{id}_{\text{default}}^*$ and optionals $\text{id}_{\text{optional}}^*$ be
 - the result of finding the type of constructor call $\text{constructorTargetIR}(\text{argumentIR}^*)$, under cursor p and typing context TC .
 - e. Let constructed type $\text{typeIR}_{\text{object}}$ with inferred $\text{typeArgumentIR}_{\text{inferred}}^*$ and cast-inserted $\text{argumentIR}_{\text{cast}}^*$ be
 - the result of typing the instantiation $\text{constructorTypeIR} < \cdot >(\text{argumentIR}^*)$ with omitted type arguments $\text{typeId}_{\text{impl}}^*$, defaults $\text{id}_{\text{default}}^*$ and optionals $\text{id}_{\text{optional}}^*$, under cursor p , typing context TC , and ANON instantiation site context.
 - f. Check that $\text{typeIR}_{\text{object}}$ is an extern object type without abstract methods.
 - g. Let callExpressionIR be $\text{prefixedNameIR} < \text{typeArgumentIR}_{\text{inferred}}^* >(\text{argumentIR}_{\text{cast}}^*)$.
 - h. Let expressionNoteIR be $\text{type } \text{typeIR}_{\text{object}}$ and compile-time known-ness CTK .
 - i. Result in callExpressionIR annotated with expressionNoteIR .
2. Else if let $\text{prefixedTypeName} < \text{typeArgumentList} >$ be callTarget :
 - a. Let typeArgumentIR^* and a set of fresh type variables $\text{typeId}_{\text{impl}}^*$ be
 - the result of typing typeArgumentList , under cursor p and typing context TC .
 - b. Let argumentIR^* be
 - the result of typing argumentList , under cursor p and typing context TC .
 - c. Let prefixedNameIR be the prefixed name of prefixedTypeName .
 - d. Let $\text{constructorTargetIR}$ be $\text{prefixedNameIR} < \text{typeArgumentIR}^* >$.
 - e. Let constructorTypeIR with fresh type variables $\text{typeId}_{\text{inserted}}^*$, defaults $\text{id}_{\text{default}}^*$ and optionals $\text{id}_{\text{optional}}^*$ be
 - the result of finding the type of constructor call $\text{constructorTargetIR}(\text{argumentIR}^*)$, under cursor p and typing context TC .
 - f. Let $\text{typeId}_{\text{infer}}^*$ be $\text{typeId}_{\text{impl}}^*$ concatenated with $\text{typeId}_{\text{inserted}}^*$.
 - g. Let constructed type $\text{typeIR}_{\text{object}}$ with inferred $\text{typeArgumentIR}_{\text{inferred}}^*$ and cast-inserted $\text{argumentIR}_{\text{cast}}^*$ be
 - the result of typing the instantiation $\text{constructorTypeIR} < \text{typeArgumentIR}^* >(\text{argumentIR}^*)$ with omitted type arguments $\text{typeId}_{\text{infer}}^*$, defaults $\text{id}_{\text{default}}^*$ and optionals $\text{id}_{\text{optional}}^*$, under cursor p , typing context TC , and ANON instantiation site context.
 - h. Check that $\text{typeIR}_{\text{object}}$ is an extern object type without abstract methods.

- i. Let callExpressionIR be $\text{prefixedNameIR} < \text{typeArgumentIR}_{\text{inferred}}^* > (\text{argumentIR}_{\text{cast}}^*)$.
- j. Let expressionNoteIR be **type** $\text{typeIR}_{\text{object}}$ **and compile-time known-ness** CTK.
- k. Result in callExpressionIR **annotated with** expressionNoteIR .

14.16.2. Local compile-time evaluation

Certain call expressions can be evaluated during type checking:

Local compile-time evaluation of $\text{callableTargetIR} < \text{typeArgumentIR}^* > (\text{argumentIR}^*)$ **annotated with** expressionNoteIR , **under cursor** p **and typing context** TC :

1. If let prefixedNameIR be callableTargetIR :
 - a. Let the callable type callableTypeIR with fresh type variables $_*$, defaults $\text{id}_{\text{default}}^*$ and optionals $\text{id}_{\text{optional}}^*$ be
 - the result of **finding the type of callable call** $\text{prefixedNameIR} < \text{typeArgumentIR}^* > (\text{argumentIR}^*)$, **under cursor** p **and typing context** TC .
 - b. Check that callableTypeIR has type $\text{externFunctionTypeIR}$.
 - c. Let $\text{externFunctionTypeIR}$ be callableTypeIR .
 - d. Let $\text{EXTERN_FUNCTION text (parameterIR}^*) : \text{typeIR}'$ be $\text{externFunctionTypeIR}$.
 - e. Check that text is equal to "static_assert".
 - f. Check that typeIR' has type boolTypeIR .
 - g. Let boolTypeIR be typeIR' .
 - h. Let $\text{nameIR}_{\text{param}}^*$ be the list of $\text{nameIR}_{\text{param}}$, obtained by repeating:
 - Let $\text{nameIR}_{\text{param}}$ be the name of parameterIR .
 for each parameterIR in parameterIR^*
 - i. Let $\text{TC}_{\text{callee}_0}$ be **an empty typing context**.
 - j. Let $\text{GIVEN parameterIR}_{\text{aligned}}^* \text{ DEFAULT parameterIR}^*$ be **aligning** parameterIR^* **with** argumentIR^* **where defaults are** $\text{id}_{\text{default}}^*$ **and optionals are** $\text{id}_{\text{optional}}^*$.
 - k. Check that parameterIR^* is an empty list.
 - l. Let $\text{value}_{\text{argument}}^*$ be the list of $\text{value}_{\text{argument}}$, obtained by repeating:
 - Let $\text{value}_{\text{argument}}$ be
 - the result of **local compile-time evaluation of** argumentIR , **under cursor** p **and typing context** TC .
 for each argumentIR in argumentIR^*
 - m. Let $\text{direction}_{\text{param_aligned}}^*$ be the list of $\text{direction}_{\text{param_aligned}}$, $\text{nameIR}_{\text{param_aligned}}^*$ be the list of $\text{nameIR}_{\text{param_aligned}}$, and $\text{typeIR}_{\text{param_aligned}}^*$ be the list of $\text{typeIR}_{\text{param_aligned}}$, obtained by repeating:
 - Let $\text{direction}_{\text{param_aligned}}$, $\text{typeIR}_{\text{param_aligned}}$, and $\text{nameIR}_{\text{param_aligned}}$ be the direction, the type, and the name of $\text{parameterIR}_{\text{aligned}}$.
 for each $\text{parameterIR}_{\text{aligned}}$ in $\text{parameterIR}_{\text{aligned}}^*$

- n. Let varTypeIR^* be the list of varTypeIR , obtained by repeating:
- Let varTypeIR be $\text{direction}_{\text{param_aligned}} \text{typeIR}_{\text{param_aligned}} \text{LCTK value}_{\text{argument}}$.
- for each $\text{direction}_{\text{param_aligned}}$ in $\text{direction}_{\text{param_aligned}}^*$, $\text{typeIR}_{\text{param_aligned}}$ in $\text{typeIR}_{\text{param_aligned}}^*$, and $\text{value}_{\text{argument}}$ in $\text{value}_{\text{argument}}^*$
- o. Let $\text{TC}_{\text{callee}_1}$ be $\text{TC}_{\text{callee}_0}$ where each of $\text{nameIR}_{\text{param_aligned}}^*$ to each of varTypeIR^* are added to the LOCAL layer.
- p. Let value be
- the result of calling an extern function "static_assert"($\text{nameIR}_{\text{param}}^*$) under typing context $\text{TC}_{\text{callee}_1}$.
- q. Check that value has type boolValue .
- r. Let boolValue be value .
- s. Check that 'B true is equal to boolValue .
- t. Result in boolValue .
2. Else if let $\text{typedExpressionIR}_{\text{base}} . \text{nameIR}$ be callableTargetIR :
- Let $\text{typeIR}_{\text{base}}$ and compile-time known-ness $_$ be the note of $\text{typedExpressionIR}_{\text{base}}$.
 - Check that nameIR is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"].
 - Result in $\$sizeof(\text{typeIR}_{\text{base}}, \text{nameIR})$.
3. Else if let $\text{TYPE prefixedNameIR} . \text{nameIR}$ be callableTargetIR :
- Let $\text{typeDefIR}_{\text{base}}$ be ! the type definition of prefixedNameIR from the p layer of TC.
 - If $\text{typeDefIR}_{\text{base}}$ is monomorphic:
 - Let $\text{typeIR}_{\text{base}}$ be the underlying type of $\text{typeDefIR}_{\text{base}}$.
 - Check that nameIR is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"].
 - Result in $\$sizeof(\text{typeIR}_{\text{base}}, \text{nameIR})$.
 - Else:
 - Check that $(\cdot, \cdot)$ is equal to the type parameters of $\text{typeDefIR}_{\text{base}}$.
 - Let $\text{typeIR}_{\text{base}}$ be the underlying type of $\text{typeDefIR}_{\text{base}}$.
 - Check that nameIR is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"].
 - Result in $\$sizeof(\text{typeIR}_{\text{base}}, \text{nameIR})$.
4. Else:
- Let $(\text{callableTargetIR}')$ be callableTargetIR .
 - Let value be
 - the result of local compile-time evaluation of $\text{callableTargetIR}' < \text{typeArgumentIR}^* > (\text{argumentIR}^*)$ annotated with type typeIR and compile-time known-ness ctl , under cursor p and typing context TC.
 - Result in value .

14.16.3. Compile-time evaluation

Call expressions may also be evaluated at compile time when instantiation phase takes place. A constructor call evaluates to an instance of the constructed type, while a callable call evaluates to the

result of the call.

Calling a callable

Compile-time evaluation of `callableTargetIR < typeArgumentListIR > ( argumentListIR )` annotated with `expressionNoteIR`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. If let `typedExpressionIRbase . nameIR` be `callableTargetIR`:
 - a. Let `typeIRbase` and compile-time known-ness `_` be the note of `typedExpressionIRbase`.
 - b. Check that `nameIR` is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"].
 - c. Result in the updated store `STO` and `$sizeof(typeIRbase, nameIR)`.
2. Else if let `TYPE prefixedNameIR . nameIR` be `callableTargetIR`:
 - a. Let `typeDefIRbase` be ! the type definition of `prefixedNameIR` from the `p` layer of `IC`.
 - b. If `typeDefIRbase` is monomorphic:
 - i. Let `typeIRbase` be the underlying type of `typeDefIRbase`.
 - ii. Check that `nameIR` is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"].
 - iii. Result in the updated store `STO` and `$sizeof(typeIRbase, nameIR)`.
 - c. Else:
 - i. Check that `( ·, · )` is equal to the type parameters of `typeDefIRbase`.
 - ii. Let `typeIRbase` be the underlying type of `typeDefIRbase`.
 - iii. Check that `nameIR` is in ["minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes"].
 - iv. Result in the updated store `STO` and `$sizeof(typeIRbase, nameIR)`.

Calling a constructor

Compile-time evaluation of `prefixedNameIR < typeArgumentListIR > ( argumentListIR )` annotated with `expressionNoteIR`, under cursor `p`, instantiation context `IC`, and store `STO`:

1. Let the constructor `constructorDef < typeArgumentListIRinst >` with defaults `iddefault*` and optionals `idoptional*` be
 - the result of finding constructor for `prefixedNameIR < typeArgumentListIR > ( argumentListIR )` under cursor `p`, and instantiation context `IC`.
2. Let the updated store `STO1`, and the constructed object be
 - the result of loading `constructorDef < typeArgumentListIRinst > ( argumentListIR )` with defaults `iddefault*` and optionals `idoptional*`, under cursor `p`, instantiation context `IC`, and store `STO`.
3. Let `STO2` be `STO1` where `IC.PATH` to object is added.
4. Let `valueref` be `REF IC.PATH`.
5. Result in the updated store `STO2` and `valueref`.

Parenthesized call

Compile-time evaluation of `( callableTargetIR' ) < typeArgumentListIR > ( argumentListIR )` annotated with

expressionNotelR, under cursor p, instantiation context IC, and store STO:

1. Let the updated store STO_1 and value be
 - the result of compile-time evaluation of callableTargetIR' < typeArgumentListIR > (argumentListIR) annotated with expressionNotelR, under cursor p, instantiation context IC, and store STO.
2. Result in the updated store STO_1 and value.

14.16.4. Runtime evaluation

Evaluating callableTargetIR < typeArgumentIR* > (argumentIR*) annotated with expressionNotelR, under cursor p, evaluation context EC, and store STO:

1. Let the updated evaluation context EC_1 , store STO_1 , and callee calleeResult be
 - the result of finding the callee callableTargetIR (argumentIR*), under cursor p, evaluation context EC, and store STO.
2. If let abortResult be calleeResult:
 - a. Result in the updated evaluation context EC_1 , store STO_1 and abortResult.
3. Else:
 - a. Let callee be calleeResult.
 - b. Let the updated evaluation context EC_2 , store STO_2 , and call result callResult be
 - the result of calling callee < typeArgumentIR* > (argumentIR*), under cursor p, evaluation context EC_1 , and store STO_1 .
 - c. If let abortResult be callResult:
 - i. Result in the updated evaluation context EC_2 , store STO_2 and abortResult.
 - d. Else:
 - i. Let RETURN value[?] be callResult.
 - ii. Check that value[?] is defined.
 - iii. Let value' be value[?].
 - iv. Result in the updated evaluation context EC_2 , store STO_2 and value'.

14.17. Parenthesized expressions

Expressions may be enclosed in parentheses to explicitly specify the order of evaluation.

```
parenthesizedExpression
: `( expression )
;
```

14.17.1. Type checking

After type checking, parenthesized expressions have the form:

```
parenthesizedExpressionIR
: `( typedExpressionIR )
```


;

Typing (*expression'*), under cursor *p* and typing context *TC*:

1. Let *typedExpressionIR* be
 - the result of typing *expression'*, under cursor *p* and typing context *TC*.
2. Let *typeIR* be the type of *typedExpressionIR*.
3. Let *ctk* be the compile-time known-ness of *typedExpressionIR*.
4. Let *expressionNoteIR* be type *typeIR* and compile-time known-ness *ctk*.
5. Result in (*typedExpressionIR*) annotated with *expressionNoteIR*.

14.17.2. Local compile-time evaluation

Local compile-time evaluation of (*typedExpressionIR*) annotated with *expressionNoteIR*, under cursor *p* and typing context *TC*:

1. Let *value* be
 - the result of local compile-time evaluation of *typedExpressionIR*, under cursor *p* and typing context *TC*.
2. Result in *value*.

14.17.3. Compile-time evaluation

Compile-time evaluation of (*typedExpressionIR*) annotated with *expressionNoteIR*, under cursor *p*, instantiation context *IC*, and store *STO*:

1. Let the updated store *STO₁* and *value* be
 - the result of compile-time evaluation of *typedExpressionIR*, under cursor *p*, instantiation context *IC*, and store *STO*.
2. Result in the updated store *STO₁* and *value*.

14.17.4. Runtime evaluation

Evaluating (*typedExpressionIR*) annotated with *expressionNoteIR*, under cursor *p*, evaluation context *EC*, and store *STO*:

1. Let the updated evaluation context *EC₁*, store *STO₁*, and *expressionResult* be
 - the result of evaluating *typedExpressionIR*, under cursor *p*, evaluation context *EC*, and store *STO*.
2. Result in the updated evaluation context *EC₁*, store *STO₁* and *expressionResult*.

15. Casting

P4 provides a limited set of casts between types. While this design is arguably more onerous for programmers, it has several benefits:

- It makes user intent unambiguous.
- It makes the costs associated with converting numeric values explicit. Implementing certain casts involves sign extensions, and thus can require significant computational resources on some targets.
- It reduces the number of cases that have to be considered in the P4 specification. Some targets may not support all casts.

15.1. Explicit casts

15.2. Implicit casts

15.3. Performing casts

Cast operations are evaluated with:

`$cast_op`:

- `value` casted to type `typeIR`

`value` casted to type `typeIR`

1. If let `boolValue` be `value`:
 - a. Let `typeIRunroll` be `typeIR` with typedefs unrolled.
 - b. Return `boolValue` casted to type `typeIRunroll`.
2. Else if let `integerValue` be `value`:
 - a. Let `typeIRunroll` be `typeIR` with typedefs unrolled.
 - b. Return `integerValue` casted to type `typeIRunroll`.
3. Else if let `errorValue` be `value`:
 - a. Let `typeIRunroll` be `typeIR` with typedefs unrolled.
 - b. Return `errorValue` casted to type `typeIRunroll`.
4. Else if let `listValue` be `value`:
 - a. Let `typeIRunroll` be `typeIR` with typedefs unrolled.
 - b. Return `listValue` casted to type `typeIRunroll`.
5. Else if let `headerStackValue` be `value`:
 - a. Let `typeIRunroll` be `typeIR` with typedefs unrolled.
 - b. Return `headerStackValue` casted to type `typeIRunroll`.
6. Else if let `structValue` be `value`:

- a. Let `typeIRunroll` be `typeIR` with `typedefs` unrolled.
- b. Return `structValue` casted to type `typeIRunroll`.
- 7. Else if let `headerValue` be `value`:
 - a. Let `typeIRunroll` be `typeIR` with `typedefs` unrolled.
 - b. Return `headerValue` casted to type `typeIRunroll`.
- 8. Else if let `enumValue` be `value`:
 - a. Let `typeIRunroll` be `typeIR` with `typedefs` unrolled.
 - b. Return `enumValue` casted to type `typeIRunroll`.
- 9. Else if let `sequenceValue` be `value`:
 - a. Let `typeIRunroll` be `typeIR` with `typedefs` unrolled.
 - b. Return `sequenceValue` casted to type `typeIRunroll`.
- 10. Else if let `recordValue` be `value`:
 - a. Let `typeIRunroll` be `typeIR` with `typedefs` unrolled.
 - b. Return `recordValue` casted to type `typeIRunroll`.
- 11. Else if let `defaultValue` be `value`:
 - a. Let `typeIRunroll` be `typeIR` with `typedefs` unrolled.
 - b. Return `DEFAULT` casted to type `typeIRunroll`.
- 12. Else if `value` has type `invalidHeaderValue`:
 - a. Let `typeIRunroll` be `typeIR` with `typedefs` unrolled.
 - b. Return `{#}` casted to type `typeIRunroll`.
- 13. Else if let `setValue` be `value`:
 - a. Let `typeIRunroll` be `typeIR` with `typedefs` unrolled.
 - b. Return `setValue` casted to type `typeIRunroll`.

15.3.1. From a Boolean value

`boolValue` casted to type `typeIR`

- 1. Let ``B b` be `boolValue`.
- 2. If let `boolTypeIR` be `typeIR`:
 - a. Return ``B b`.
- 3. Else if let `BIT < w >` be `typeIR`:
 - a. If `boolValue` is equal to ``B true`:
 - i. Return `w W 1`.
 - b. If `boolValue` is equal to ``B false`:
 - i. Return `w W 0`.
- 4. Else if let `TYPE _ typeIR'` be `typeIR`:
 - a. Return `boolValue` casted to type `typeIR'`.

5. Else if let $SET < typeIR' >$ be $typeIR$:
 - a. Let $value$ be $boolValue$ casted to type $typeIR'$.
 - b. Return $SET \{ value \}$.

15.3.2. From an integer value

$integerValue$ casted to type $typeIR$

1. If let $boolTypeIR$ be $typeIR$:
 - a. Check that $integerValue$ has type $integerLiteral$.
 - b. Let $integerLiteral$ be $integerValue$.
 - c. If let $D \ i$ be $integerLiteral$:
 - i. Return $\neg B \ i \neq 0$.
 - d. Else if let $w \ W \ i$ be $integerLiteral$:
 - i. Return $\neg B \ i \neq 0$.
2. Else if let $intTypeIR$ be $typeIR$:
 - a. Check that $integerValue$ has type $integerLiteral$.
 - b. Let $integerLiteral$ be $integerValue$.
 - c. If let $D \ i$ be $integerLiteral$:
 - i. Return $D \ i$.
 - d. Else if let $_ \ W \ i$ be $integerLiteral$:
 - i. Return $D \ i$.
 - e. Else:
 - i. Let $w \ S \ i$ be $integerLiteral$.
 - ii. Let int be the two's complement representation of bitstring i of width w .
 - iii. Return $D \ int$.
3. Else if let $BIT < w_{to} >$ be $typeIR$:
 - a. Check that $integerValue$ has type $integerLiteral$.
 - b. Let $integerLiteral$ be $integerValue$.
 - c. If let $D \ i$ be $integerLiteral$:
 - i. Let i_{cast} be the bitstring of width w_{to} representing the integer i .
 - ii. Return $w_{to} \ W \ i_{cast}$.
 - d. Else if let $_ \ W \ i$ be $integerLiteral$:
 - i. Let i_{cast} be the bitstring of width w_{to} representing the integer i .
 - ii. Return $w_{to} \ W \ i_{cast}$.
 - e. Else:
 - i. Let $w_{from} \ S \ i$ be $integerLiteral$.
 - ii. Let i_{from} be the two's complement representation of bitstring i of width w_{from} .

- iii. Let i_{cast} be the bitstring of width w_{to} representing the integer i_{from} .
 - iv. Return $w_{\text{to}} \ W \ i_{\text{cast}}$.
- 4. Else if let $\text{INT} < w_{\text{to}} >$ be typeIR:
 - a. Check that integerValue has type integerLiteral.
 - b. Let integerLiteral be integerValue.
 - c. If let $D \ i$ be integerLiteral:
 - i. Let i_{cast} be the bitstring of width w_{to} representing the integer i .
 - ii. Return $w_{\text{to}} \ S \ i_{\text{cast}}$.
 - d. Else if let $_W \ i$ be integerLiteral:
 - i. Let i_{cast} be the bitstring of width w_{to} representing the integer i .
 - ii. Return $w_{\text{to}} \ S \ i_{\text{cast}}$.
 - e. Else:
 - i. Let $w_{\text{from}} \ S \ i$ be integerLiteral.
 - ii. Let i_{from} be the two's complement representation of bitstring i of width w_{from} .
 - iii. Let i_{cast} be the bitstring of width w_{to} representing the integer i_{from} .
 - iv. Return $w_{\text{to}} \ S \ i_{\text{cast}}$.
- 5. Else if let $\text{VARBIT} < w_{\text{max}} >$ be typeIR:
 - a. Check that integerValue matches pattern $\% . \% V \%$.
 - b. Let $w_{\text{max}}' . w \ V \ i$ be integerValue.
 - c. Check that w_{max} is equal to w_{max}' .
 - d. Return $w_{\text{max}}' . w \ V \ i$.
- 6. Else if let $\text{TYPE} _ \text{typeIR}'$ be typeIR:
 - a. Check that integerValue has type integerLiteral.
 - b. Let integerLiteral be integerValue.
 - c. If let $D \ i$ be integerLiteral:
 - i. Return $D \ i$ casted to type typeIR'.
 - d. Else if let $w \ W \ i$ be integerLiteral:
 - i. Return $w \ W \ i$ casted to type typeIR'.
 - e. Else:
 - i. Let $w \ S \ i$ be integerLiteral.
 - ii. Return $w \ S \ i$ casted to type typeIR'.
- 7. Else if let enumTypeIR be typeIR:
 - a. Check that integerValue has type integerLiteral.
 - b. Let integerLiteral be integerValue.
 - c. If let $w \ W \ i$ be integerLiteral:
 - i. Return $w \ W \ i$ casted to enum type enumTypeIR.
 - d. Else if let $w \ S \ i$ be integerLiteral:
 - i. Return $w \ S \ i$ casted to enum type enumTypeIR.

8. Else if let `SET < typeIR' >` be `typeIR`:
 - a. Check that `integerValue` has type `integerLiteral`.
 - b. Let `integerLiteral` be `integerValue`.
 - c. If let `D i` be `integerLiteral`:
 - i. Let `value` be `D i` **casted to type** `typeIR'`.
 - ii. Return `SET { value }`.
 - d. Else if let `w W i` be `integerLiteral`:
 - i. Let `value` be `w W i` **casted to type** `typeIR'`.
 - ii. Return `SET { value }`.
 - e. Else:
 - i. Let `w S i` be `integerLiteral`.
 - ii. Let `value` be `w S i` **casted to type** `typeIR'`.
 - iii. Return `SET { value }`.

15.3.3. From an error value

`errorValue` **casted to type** `typeIR`

1. Let `ERROR . nameIR` be `errorValue`.
2. If let `errorTypeIR` be `typeIR`:
 - a. Return `ERROR . nameIR`.
3. Else if let `SET < typeIR' >` be `typeIR`:
 - a. Let `value` be `errorValue` **casted to type** `typeIR'`.
 - b. Return `SET { value }`.

15.3.4. From a list value

`LIST [ value* ]` **casted to type** `typeIR`

1. Check that `typeIR` has type `listTypeIR`.
2. Let `LIST < typeIR' >` be `typeIR`.
3. Let `value*` be the list of `value'`, obtained by repeating:
 - Let `value'` be `value` **casted to type** `typeIR'`.
 for each `value` in `value*`
4. Return `LIST [ value* ]`.

15.3.5. From a header stack value

`HEADER_STACK [ value* ( nidx ; nsize ) ]` **casted to type** `typeIR`

1. Check that `typeIR` has type `headerStackTypeIR`.
2. Let `typeIR' [ nsize' ]` be `typeIR`.
3. Check that `nsize'` is equal to `nsize'`.
4. Return `HEADER_STACK [ value* ( nidx ; nsize' ) ]`.

15.3.6. From a struct value

`STRUCT typeld { ( valuefield nameIRfield ; )* }` **casted to type** `typeIR`

1. Check that `typeIR` has type `structTypeIR`.
2. Let `STRUCT typeld' < _* > { _* }` be `typeIR`.
3. Check that `typeld'` is equal to `typeld'`.
4. Return `STRUCT typeld' { ( valuefield nameIRfield ; )* }`.

15.3.7. From a header value

`HEADER typeld { b ; ( valuefield nameIRfield ; )* }` **casted to type** `typeIR`

1. Check that `typeIR` has type `headerTypeIR`.
2. Let `HEADER typeld' < _* > { _* }` be `typeIR`.
3. Check that `typeld'` is equal to `typeld'`.
4. Return `HEADER typeld' { b ; ( valuefield nameIRfield ; )* }`.

15.3.8. From an enum value

`enumValue` **casted to type** `typeIR`

1. If let `enumTypeIR` be `typeIR`:
 - a. Check that `enumTypeIR` matches pattern `ENUM % { % }`.
 - b. Let `ENUM typeld { _* }` be `enumTypeIR`.
 - c. Check that `enumValue` matches pattern `% . %`.
 - d. Let `typeld' . nameIR` be `enumValue`.
 - e. Check that `typeld` is equal to `typeld'`.
 - f. Return `typeld . nameIR`.
2. If let `_ . _ . value` be `enumValue`:
 - a. Return `value` **casted to type** `typeIR`.

15.3.9. From a default value

defaultValue **casted to type** typeIR

1. Return **the default value for type** typeIR.

15.3.10. From an invalid header value

{#} **casted to type** typeIR

1. If let headerTypeIR be typeIR:
 - a. Return **the default value for type** headerTypeIR.
2. Else if let headerUnionTypeIR be typeIR:
 - a. Return **the default value for type** headerUnionTypeIR.

15.3.11. From a sequence value

sequenceValue **casted to type** typeIR

1. If let LIST < typeIR' > be typeIR:
 - a. Check that sequenceValue matches pattern SEQ (%).
 - b. Let SEQ (value*) be sequenceValue.
 - c. Let value* be the list of value', obtained by repeating:
 - Let value' be value **casted to type** typeIR'.for each value in value*
 - d. Return LIST [value*].
2. Else if let TUPLE < typeIR'* > be typeIR:
 - a. Check that sequenceValue matches pattern SEQ (%).
 - b. Let SEQ (value*) be sequenceValue.
 - c. Let value* be the list of value' and value** be the list of value*, obtained by repeating:
 - Let value' be value **casted to type** typeIR'.for each typeIR' in typeIR'* and value in value*
 - d. Return TUPLE (value*).
3. Else if let typeIR' [n_{size}] be typeIR:
 - a. Check that sequenceValue matches pattern SEQ (%).
 - b. Let SEQ (value*) be sequenceValue.
 - c. Let value_{cast}* be the list of value_{cast} obtained by repeating:

- Let $value_{cast}$ be value casted to type $typeIR$.
- for each value in $value^*$
- d. Let n_{idx} be the length of $value^*$.
 - e. Return $HEADER_STACK [value_{cast}^* (n_{idx} ; n_{size})]$.
4. Else if let $STRUCT\ typeId < _ > \{ (_ typeIR_{field} nameIR_{field} ;)^* \}$ be $typeIR$:
 - a. Check that $sequenceValue$ matches pattern $SEQ (\%)$.
 - b. Let $SEQ (value^*)$ be $sequenceValue$.
 - c. Let $value_{cast}^*$ be the list of $value_{cast}$, obtained by repeating:
 - Let $value_{cast}$ be value casted to type $typeIR_{field}$.

for each $typeIR_{field}$ in $typeIR_{field}^*$ and value in $value^*$
 - d. Return $STRUCT\ typeId \{ (value_{cast} nameIR_{field} ;)^* \}$.
 5. Else if let $HEADER\ typeId < _ > \{ (_ typeIR_{field} nameIR_{field} ;)^* \}$ be $typeIR$:
 - a. Check that $sequenceValue$ matches pattern $SEQ (\%)$.
 - b. Let $SEQ (value^*)$ be $sequenceValue$.
 - c. Let $value_{cast}^*$ be the list of $value_{cast}$, obtained by repeating:
 - Let $value_{cast}$ be value casted to type $typeIR_{field}$.

for each $typeIR_{field}$ in $typeIR_{field}^*$ and value in $value^*$
 - d. Return $HEADER\ typeId \{ true ; (value_{cast} nameIR_{field} ;)^* \}$.

15.3.12. From a record value

$recordValue$ casted to type $typeIR$

1. If let $STRUCT\ typeId < _ > \{ (_ typeIR_{t_field} id_{t_field} ;)^* \}$ be $typeIR$:
 - a. Check that $recordValue$ matches pattern $RECORD \{ \% \}$.
 - b. Let $RECORD \{ (value_{field} nameIR_{field} ;)^* \}$ be $recordValue$.
 - c. Let $value^{?*$ be the list of $value^?$, obtained by repeating:
 - Let $value^?$ be id_{t_field} in map $\{ (nameIR_{field} : value_{field})^* \}$.

for each id_{t_field} in $id_{t_field}^*$
 - d. Check that $value^?$ is defined, for all $value^?$ in $value^{?*}$.
 - e. Let $value_{field'}^*$ be the list of $value_{field'}$, obtained by repeating:
 - Let $value_{field'}$ be $value^?$.

for each $value^?$ in $value^{?*}$
 - f. Let $value_{field_cast}^*$ be the list of $value_{field_cast}$, obtained by repeating:

- Let $value_{field_cast}$ be $value_{field'}$ **casted to type** $typeIR_{t_field}$.

for each $typeIR_{t_field}$ in $typeIR_{t_field}^*$ and $value_{field'}$ in $value_{field'}^*$

g. Return **STRUCT** $typeID \{ (value_{field_cast} nameIR_{field} ;)^* \}$.

2. Else if let **HEADER** $typeID < _ > \{ (_ typeIR_{t_field} id_{t_field} ;)^* \}$ be $typeIR$:

a. Check that $recordValue$ matches pattern **RECORD** $\{ \% \}$.

b. Let **RECORD** $\{ (value_{field} nameIR_{field} ;)^* \}$ be $recordValue$.

c. Let $value^{?*$ be the list of $value^?$, obtained by repeating:

- Let $value^?$ be id_{t_field} **in map** $\{ (nameIR_{field} : value_{field})^* \}$.

for each id_{t_field} in $id_{t_field}^*$

d. Check that $value^?$ is defined, for all $value^?$ in $value^{?*}$.

e. Let $value_{field'}^*$ be the list of $value_{field'}$, obtained by repeating:

- Let $value_{field'}$ be $value^?$.

for each $value^?$ in $value^{?*}$

f. Let $value_{field_cast}^*$ be the list of $value_{field_cast}$, obtained by repeating:

- Let $value_{field_cast}$ be $value_{field'}$ **casted to type** $typeIR_{t_field}$.

for each $typeIR_{t_field}$ in $typeIR_{t_field}^*$ and $value_{field'}$ in $value_{field'}^*$

g. Return **HEADER** $typeID \{ true ; (value_{field_cast} nameIR_{field} ;)^* \}$.

15.3.13. From a set value

$setValue$ **casted to type** $typeIR$

1. Check that $typeIR$ has type $setTypeIR$.

2. Let **SET** $< typeIR' >$ be $typeIR$.

3. If let **SET** $\{ value \}$ be $setValue$:

a. Let $value'$ be $value$ **casted to type** $typeIR'$.

b. Return **SET** $\{ value' \}$.

4. Else if let **SET** $\{ value_b \ \&\& \ value_m \}$ be $setValue$:

a. Let $value_{b_cast}$ be $value_b$ **casted to type** $typeIR'$.

b. Let $value_{m_cast}$ be $value_m$ **casted to type** $typeIR'$.

c. Return **SET** $\{ value_{b_cast} \ \&\& \ value_{m_cast} \}$.

5. Else if let **SET** $\{ value_l \ .. \ value_u \}$ be $setValue$:

a. Let $value_{l_cast}$ be $value_l$ **casted to type** $typeIR'$.

b. Let $value_{u_cast}$ be $value_u$ **casted to type** $typeIR'$.

c. Return **SET** $\{ value_{l_cast} \ .. \ value_{u_cast} \}$.

16. Abstract Parser Machine

16.1. Parser local declarations

```
parserLocalDeclaration
  : constantDeclaration
  | instantiation
  | variableDeclaration
  | valueSetDeclaration
  ;

parserLocalDeclarationIR
  : constantDeclarationIR
  | instantiationIR
  | variableDeclarationIR
  | valueSetDeclarationIR
  ;
```

16.1.1. Constant declarations

```
constantDeclaration
  : annotationList CONST type name initializer ;
  ;

constantDeclarationIR
  : annotationList CONST typeIR nameIR constantInitializerIR ;
  ;
```

16.1.2. Instantiations

```
instantiation
  : annotationList type `( argumentList ) name ;
  | annotationList type `( argumentList ) name objectInitializer ;
  ;

instantiationIR
  : annotationList typeIR constructorTargetIR `( argumentListIR ) nameIR
  objectInitializerOptIR ;
  ;
```

16.1.3. Variable declarations

```
variableDeclaration
  : annotationList type name initializerOpt ;
  ;

variableDeclarationIR
  : annotationList typeIR nameIR initializerOptIR ;
  ;
```

16.1.4. Parser value set declarations

```
valueSetType
  : baseType
```

```

| tupleType
| prefixedTypeName
;

valueSetDeclaration
: annotationList VALUE_SET `< valueSetType > `( expression ) name ;
;

valueSetDeclarationIR
: annotationList VALUE_SET `< typeIR > `( typedExpressionIR ) nameIR ;
;

```

16.2. Parser states

```

parserState
: annotationList STATE name `{ parserStatementList transitionStatement }
;

parserStateIR
: annotationList STATE nameIR `{ parserStatementListIR transitionStatementIR }
;

```

16.3. Parser statements

```

parserStatement
: constantDeclaration
| variableDeclaration
| emptyStatement
| assignmentStatement
| callStatement
| directApplicationStatement
| parserBlockStatement
| parserConditionalStatement
;

parserStatementIR
: constantDeclarationIR
| variableDeclarationIR
| emptyStatementIR
| assignmentStatementIR
| callStatementIR
| directApplicationStatementIR
| parserBlockStatementIR
| parserConditionalStatementIR
;

```

16.3.1. Constant declaration

```

constantDeclaration
: annotationList CONST type name initializer ;
;

constantDeclarationIR
: annotationList CONST typeIR nameIR constantInitializerIR ;
;

```

16.3.2. Variable declaration

```

variableDeclaration
  : annotationList type name initializerOpt ;
;

variableDeclarationIR
  : annotationList typeIR nameIR initializerOptIR ;
;

```

16.3.3. Empty statement

```

emptyStatement
  : ;
;

emptyStatementIR = emptyStatement

```

16.3.4. Assignment statement

```

assignmentStatement
  : lvalue assignop expression ;
;

assignmentStatementIR
  : typedLvalueIR assignop typedExpressionIR ;
;

```

16.3.5. Call statement

```

callStatement
  : lvalue `( argumentList ) ;
  | lvalue `< typeArgumentList > `( argumentList ) ;
;

callStatementIR
  : callableTargetIR `< typeArgumentListIR > `( argumentListIR ) ;
;

```

16.3.6. Direct application statement

```

directApplicationStatement
  : namedType . APPLY `( argumentList ) ;
;

directApplicationStatementIR
  : constructorTargetIR . APPLY `( argumentListIR ) ;
;

```

16.3.7. Block statement

```

blockStatement
  : annotationList `{ blockElementStatementList }
;

blockStatementIR

```

```
: annotationList `{ blockElementStatementListIR }  
;
```

16.3.8. Conditional statement

```
conditionalStatement  
: IF `( expression ) statement  
| IF `( expression ) statement ELSE statement  
;  
  
conditionalStatementIR  
: IF `( typedExpressionIR ) statementIR  
| IF `( typedExpressionIR ) statementIR ELSE statementIR  
;
```

16.4. Parser transition statements

```
stateExpression  
: name ;  
| selectExpression  
;  
  
transitionStatement  
: /* empty */  
| TRANSITION stateExpression  
;  
  
stateExpressionIR  
: nameIR ;  
| selectExpressionIR  
;  
  
transitionStatementIR  
: TRANSITION stateExpressionIR  
;
```

16.4.1. Parser select expression

```
keysetExpression  
: simpleKeysetExpression  
| tupleKeysetExpression  
;  
  
selectCase  
: keysetExpression : name ;  
;  
  
selectExpression  
: SELECT `( expressionList ) `{ selectCaseList }  
;  
  
keysetExpressionIR  
: simpleKeysetExpressionIR  
| tupleKeysetExpressionIR  
;  
  
selectCaseIR  
: keysetExpressionIR : nameIR ;  
;
```

```

selectExpressionIR
  : SELECT `( typedExpressionListIR ) `{ selectCaseListIR }
  ;

```

17. Abstract Control Machine

17.1. Control local declarations

```

controlLocalDeclaration
  : constantDeclaration
  | instantiation
  | variableDeclaration
  | actionDeclaration
  | tableDeclaration
  ;

controlLocalDeclarationIR
  : constantDeclarationIR
  | instantiationIR
  | variableDeclarationIR
  | actionDeclarationIR
  | tableDeclarationIR
  ;

```

17.1.1. Constant declarations

```

constantDeclaration
  : annotationList CONST type name initializer ;
  ;

constantDeclarationIR
  : annotationList CONST typeIR nameIR constantInitializerIR ;
  ;

```

17.1.2. Instantiations

```

instantiation
  : annotationList type `( argumentList ) name ;
  | annotationList type `( argumentList ) name objectInitializer ;
  ;

instantiationIR
  : annotationList typeIR constructorTargetIR `( argumentListIR ) nameIR
  objectInitializerOptIR ;
  ;

```

17.1.3. Variable declarations

```

variableDeclaration
  : annotationList type name initializerOpt ;
  ;

variableDeclarationIR
  : annotationList typeIR nameIR initializerOptIR ;
  ;

```

```
;
```

17.1.4. Action declarations

```
actionDeclaration
: annotationList ACTION name `( parameterList ) blockStatement
;

actionDeclarationIR
: annotationList ACTION nameIR `( parameterListIR ) blockStatementIR
;
```

17.1.5. Table declarations

```
tableDeclaration
: annotationList TABLE name `{ tablePropertyList }
;

tableDeclarationIR
: annotationList TABLE typeIR nameIR `{ tablePropertyListIR }
;
```

17.2. Match-action tables

```
tableProperty
: KEY = `{ tableKeyList }
| ACTIONS = `{ tableActionList }
| annotationList constOpt ENTRIES = `{ tableEntryList }
| annotationList constOpt tableCustomName initializer ;
;

tableDeclaration
: annotationList TABLE name `{ tablePropertyList }
;

tablePropertyIR
: tableKeysPropertyIR
| tableActionsPropertyIR
| tableDefaultActionPropertyIR
| tableEntriesPropertyIR
| tableCustomPropertyIR
;

tableDeclarationIR
: annotationList TABLE typeIR nameIR `{ tablePropertyListIR }
;
```

17.2.1. Table keys

```
tableKey
: expression : name annotationList ;
;

tableKeyIR
: typedExpressionIR : nameIR annotationList ;
;
```


17.2.2. Table actions

```
tableActionReference
  : prefixedNonTypeName
  | prefixedNonTypeName `( argumentList )
  ;

tableAction
  : annotationList tableActionReference ;
  ;

tableActionReferenceIR
  : prefixedNameIR `( argumentListIR )
  ;

tableActionIR
  : annotationList tableActionReferenceIR # `( parameterListIR , parameterListIR ) ;
  ;
```

17.2.3. Table default actions

17.2.4. Table entries

```
keysetExpression
  : simpleKeysetExpression
  | tupleKeysetExpression
  ;

tableEntryPriority
  : PRIORITY = integerLiteral :
  | PRIORITY = `( expression ) :
  ;

tableEntry
  : constOpt tableEntryPriority keysetExpression : tableActionReference annotationList ;
  | constOpt keysetExpression : tableActionReference annotationList ;
  ;

keysetExpressionIR
  : simpleKeysetExpressionIR
  | tupleKeysetExpressionIR
  ;

tableEntryPriorityIR
  : PRIORITY = integerLiteral :
  | PRIORITY = `( typedExpressionIR ) :
  ;

tableEntryIR
  : constOptIR tableEntryPriorityOptIR keysetExpressionIR : tableActionReferenceIR
  annotationList ;
  ;
```

17.2.5. Table custom properties

18. Calls

Calls can be made in several different contexts:

- From call expressions ([Section 14.16](#)), to invoke a callable or a constructor.
- From call statements ([Section 13.4](#)), to invoke a callable.
- From a direct type invocation ([Section 13.5](#)), which is a syntactic sugar for invoking a constructor and then immediately calling an apply method on the resulting instance.
- From an instantiation declaration ([Section 11.4](#)), to invoke a constructor.

A callable invocation can optionally specify for each argument the corresponding parameter name. It is illegal to use names only for some arguments: either all or no arguments must specify the parameter name. Callable arguments are evaluated in the order they appear, left to right, before the function invocation takes place.

```
extern void f(in bit<32> x, out bit<16> y);
bit<32> xa = 0;
bit<16> ya;
f(xa, ya); // match arguments by position
f(x = xa, y = ya); // match arguments by name
f(y = ya, x = xa); // match arguments by name in any order
//f(x = xa); -- error: enough arguments
//f(x = xa, x = ya); -- error: argument specified twice
//f(x = xa, ya); -- error: some arguments specified by name
//f(z = xa, w = yz); -- error: no parameter named z or w
//f(x = xa, y = 0); -- error: y must be a left-value
```

The calling convention is copy-in/copy-out ([Section 6.5](#)). For generic callables and constructors the type arguments can be explicitly specified in the call. The compiler only inserts implicit casts for direction **in** or directionless arguments to methods, functions, or constructors as described in [\[sec-casts\]](#). The types for all other arguments must match the parameter types exactly.

The result returned by a call is discarded when the function call is used as a statement.

The "don't care" identifier (**_**) can only be used for an **out** function/method argument, when the value of returned in that argument is ignored by subsequent computations. When used in generic callables, the compiler may reject the program if it is unable to infer a type for the don't care argument.

This section describes the semantics of calls in detail. In particular, it explains how the call target is found (also with overload resolution), how the call convention is checked, and how the call is evaluated at runtime.

18.1. Call targets

Calls are composed of three components: the call target, type arguments, and arguments. Call target identifies what is being called. It may be a callable (actions, functions, and methods) or a constructor.

When calls are made from an expression, the call target is as follows:

```
callExpression
: callTarget `( argumentList )
| callableTarget `< realTypeArgumentList > `( argumentList )
;

callTarget
: callableTarget
| constructorTarget
;

callableTarget = expression
```

```
constructorTarget = namedType
```

When calls are made from a statement, the call target is as follows:

```
lvalue
: referenceExpression
| lvalue . member
| lvalue `[ expression ]
| lvalue `[ expression : expression ]
| `( lvalue )
;

callStatement
: lvalue `( argumentList ) ;
| lvalue `< typeArgumentList > `( argumentList ) ;
;

directApplicationStatement
: namedType . APPLY `( argumentList ) ;
;
```

And when calls are made from instantiation declarations, the call target is as follows:

```
instantiation
: annotationList type `( argumentList ) name ;
| annotationList type `( argumentList ) name objectInitializer ;
;
```

After type checking, call targets are represented as follows:

```
callableTargetIR
: referenceExpressionIR
| typedExpressionIR . nameIR
| TYPE prefixedNameIR . nameIR
| `( callableTargetIR )
;

constructorTargetIR
: prefixedNameIR `< typeArgumentListIR >
;
```

Thus, calls are represented as:

```
callExpressionIR
: constructorTargetIR `( argumentListIR )
| callableTargetIR `< typeArgumentListIR > `( argumentListIR )
;

callStatementIR
: callableTargetIR `< typeArgumentListIR > `( argumentListIR ) ;
;

directApplicationStatementIR
: constructorTargetIR . APPLY `( argumentListIR ) ;
;

instantiationIR
: annotationList typeIR constructorTargetIR `( argumentListIR ) nameIR
objectInitializerOptIR ;
;
```

The following relation is used to check callable targets:

CallableTarget_ok:

- Typing `callableTarget` as a callable target, under cursor `cursor` and typing context `typingContext`:
- Results in the typed callable target `callableTargetIR`.

Typing `callableTarget` as a callable target, under cursor `p` and typing context `TC`:

1. If let `prefixedNonTypeName` be `callableTarget`:
 - a. Let `prefixedNameIR` be the prefixed name of `prefixedNonTypeName`.
 - b. Result in the typed callable target `prefixedNameIR`.
2. Else if let `memberAccessBase . member` be `callableTarget`:
 - a. If let `prefixedTypeName` be `memberAccessBase`:
 - i. Let `prefixedNameIR` be the prefixed name of `prefixedTypeName`.
 - ii. Let `nameIR` be the name of `member`.
 - iii. Result in the typed callable target `TYPE prefixedNameIR . nameIR`.
 - b. Else:
 - i. Let `expressionbase` be `memberAccessBase`.
 - ii. Let `nameIR` be the name of `member`.
 - iii. Let `typedExpressionIRbase` be
 - the result of typing `expressionbase`, under cursor `p` and typing context `TC`.
 - iv. Result in the typed callable target `typedExpressionIRbase . nameIR`.
3. Else if let `( expression )` be `callableTarget`:
 - a. Let the typed callable target `callableTargetIR` be
 - the result of typing `expression` as a callable target, under cursor `p` and typing context `TC`.
 - b. Result in the typed callable target `( callableTargetIR )`.
4. If `callableTarget` is equal to `THIS`:
 - a. Result in the typed callable target `"this"`.

When a l-value is used as a callable target, the following relation is used to check it:

CallableTarget_lvalue_ok:

- Typing `lvalue` as a callable target, under cursor `cursor` and typing context `typingContext`:
- Results in the typed callable target `callableTargetIR`.

Typing `lvalue` as a callable target, under cursor `p` and typing context `TC`:

1. Let `expression` be the expression corresponding to `lvalue`.

2. Let the typed callable target `callableTargetIR` be
 - the result of `typing expression as a callable target, under cursor p and typing context TC`.
3. Result in the typed callable target `callableTargetIR`.

18.2. Overload resolution

As explained in [Section 6.6](#), functions, methods, and constructors can be overloaded. Overload should be resolved at compile time based on the number of arguments or the names of the arguments. Additionally, arguments may be omitted when the corresponding parameters have default values or are declared as optional.

The following algorithm is used to resolve overloads and omitted arguments:

`$find_overloaded`:

- Finding the item in `{ ( callableId : V )* }` matching `nameIRi ( idarg? )` using parameter getter `$get_parameterListIR`

The result of this algorithm is a matched overload item with the default and optional parameter names, or an empty result if no match is found.

```
callResolution<V>
  : callableId : V # nameIR* # nameIR*
  ;
```

The algorithm takes as input the key for overload resolution, which is the call target name and the argument names (if specified).

```
callTargetKey
  : nameIR `( id?* )
  ;
```

Based on whether the arguments are positional or named, the key is refined to either:

```
namedCallTargetKey
  : nameIR `( id* )
  ;

unnamedCallTargetKey
  : nameIR `( nat )
  ;
```

For each item in the overloaded set, the key is compared to determine if it is a match:

```
callTargetMatch
  : MATCH id* id*
  | NOMATCH
  ;
```

The algorithm is as follows:

Finding the item in $\{ (\text{callebleId} : V)^* \}$ matching $\text{nameIR}_f (\text{id}_{\text{arg}}^{?*})$ using parameter getter $\$get_parameterListIR$

1. If the length of $\text{id}_{\text{arg}}^{?*}$ is greater than 0:
 - a. Check that $\text{id}_{\text{arg}}^?$ is defined, for all $\text{id}_{\text{arg}}^?$ in $\text{id}_{\text{arg}}^{?*}$.
 - b. Let $\text{id}_{\text{arg_inner}}^*$ be the list of $\text{id}_{\text{arg_inner}}$, obtained by repeating:
 - Let $\text{id}_{\text{arg_inner}}^?$ be $\text{id}_{\text{arg}}^?$.
 for each $\text{id}_{\text{arg}}^?$ in $\text{id}_{\text{arg}}^{?*}$
 - c. If $\cdot$ is equal to finding all items in $\{ (\text{callebleId} : V)^* \}$ matching named key $\text{nameIR}_f (\text{id}_{\text{arg_inner}}^*)$ using parameter getter $\$get_parameterListIR$:
 - i. Return $\cdot$.
 - d. Let $\text{callResolution}<V>^*$ be finding all items in $\{ (\text{callebleId} : V)^* \}$ matching named key $\text{nameIR}_f (\text{id}_{\text{arg_inner}}^*)$ using parameter getter $\$get_parameterListIR$.
 - e. If let $\text{callebleId}_{\text{found}} : V_{\text{found}} \# \text{id}_{\text{default}}^* \# \text{id}_{\text{optional}}^*$ be $\text{callResolution}<V>^*$:
 - i. Return $\text{callebleId}_{\text{found}} : V_{\text{found}} \# \text{id}_{\text{default}}^* \# \text{id}_{\text{optional}}^*$.
2. If $\text{id}_{\text{arg}}^?$ is none, for all $\text{id}_{\text{arg}}^?$ in $\text{id}_{\text{arg}}^{?*}$:
 - a. If $\cdot$ is equal to finding all items in $\{ (\text{callebleId} : V)^* \}$ matching unnamed key $\text{nameIR}_f (\text{the length of } \text{id}_{\text{arg}}^{?*})$ using parameter getter $\$get_parameterListIR$:
 - i. Return $\cdot$.
 - b. Let $\text{callResolution}<V>^*$ be finding all items in $\{ (\text{callebleId} : V)^* \}$ matching unnamed key $\text{nameIR}_f (\text{the length of } \text{id}_{\text{arg}}^{?*})$ using parameter getter $\$get_parameterListIR$.
 - c. If let $\text{callebleId}_{\text{found}} : V_{\text{found}} \# \text{id}_{\text{omitted}}^* \# \text{id}_{\text{optional}}^*$ be $\text{callResolution}<V>^*$:
 - i. Return $\text{callebleId}_{\text{found}} : V_{\text{found}} \# \text{id}_{\text{omitted}}^* \# \text{id}_{\text{optional}}^*$.

18.2.1. Positional arguments

The following algorithm is used when arguments are positional:

$\$find_overloaded_unnamed$:

- Finding all items in $\text{set}<\text{pair}<\text{callebleId}, V>>$ matching unnamed key $\text{nameIR}_f (\text{nat})$ using parameter getter $\$get_parameterListIR$

Finding all items in $\text{set}<\text{pair}<\text{callebleId}, V>>$ matching unnamed key $\text{nameIR}_f (\text{nat})$ using parameter getter $\$get_parameterListIR$

1. If $\text{set}<\text{pair}<\text{callebleId}, V>>$ is equal to $\{ \cdot \}$:
 - a. Return $\cdot$.
2. Let $\{ \text{pair}<\text{callebleId}, V>^* \}$ be $\text{set}<\text{pair}<\text{callebleId}, V>>$.
3. If let $\text{callTargetId}_h : V_h :: (\text{callTargetId}_t : V_t)^*$ be $\text{pair}<\text{callebleId}, V>^*$:
 - a. If matching $\text{callTargetId}_h : V_h$ against unnamed key $\text{nameIR}_f (\text{nat})$ using parameter getter

$\$get_parameterListIR$ is NOMATCH:

- i. Return finding all items in $\{ (callTargetId_t : V_t)^* \}$ matching unnamed key $nameIR_f (nat)$ using parameter getter $\$get_parameterListIR$.
- b. Let $callTargetMatch$ be matching $callTargetId_h : V_h$ against unnamed key $nameIR_f (nat)$ using parameter getter $\$get_parameterListIR$.
- c. If let $MATCH id_{default}^* id_{optional}^*$ be $callTargetMatch$:
 - i. Let $callResolution<V>^*$ be finding all items in $\{ (callTargetId_t : V_t)^* \}$ matching unnamed key $nameIR_f (nat)$ using parameter getter $\$get_parameterListIR$.
 - ii. Return $callTargetId_h : V_h \# id_{default}^* \# id_{optional}^* :: callResolution<V>^*$.

18.2.2. Named arguments

The following algorithm is used when arguments are named:

$\$find_overloaded_named$:

- Finding all items in $set<pair<callableId, V>>$ matching named key $nameIR_f (id_{arg}^*)$ using parameter getter $\$get_parameterListIR$

Finding all items in $set<pair<callableId, V>>$ matching named key $nameIR_f (id_{arg}^*)$ using parameter getter $\$get_parameterListIR$

1. If $set<pair<callableId, V>>$ is equal to $\{ \cdot \}$:
 - a. Return $\cdot$.
2. Let $\{ pair<callableId, V>^* \}$ be $set<pair<callableId, V>>$.
3. If let $callTargetId_h : V_h :: (callTargetId_t : V_t)^*$ be $pair<callableId, V>^*$:
 - a. If matching $callTargetId_h : V_h$ against named key $nameIR_f (id_{arg}^*)$ using parameter getter $\$get_parameterListIR$ is NOMATCH:
 - i. Return finding all items in $\{ (callTargetId_t : V_t)^* \}$ matching named key $nameIR_f (id_{arg}^*)$ using parameter getter $\$get_parameterListIR$.
 - b. Let $callTargetMatch$ be matching $callTargetId_h : V_h$ against named key $nameIR_f (id_{arg}^*)$ using parameter getter $\$get_parameterListIR$.
 - c. If let $MATCH id_{default}^* id_{optional}^*$ be $callTargetMatch$:
 - i. Let $callResolution<V>^*$ be finding all items in $\{ (callTargetId_t : V_t)^* \}$ matching named key $nameIR_f (id_{arg}^*)$ using parameter getter $\$get_parameterListIR$.
 - ii. Return $callTargetId_h : V_h \# id_{default}^* \# id_{optional}^* :: callResolution<V>^*$.

18.3. Callee resolution

Given a call target, type arguments, and arguments, the callee should be resolved. This is done both during type checking and runtime evaluation.

18.3.1. Resolving constructors

When type checking, constructors are resolved by:

ConstructorType_ok:

- Finding the type of constructor call `constructorTargetIR ( argumentIR* )`, under cursor `cursor` and typing context `typingContext`:
- Results in `constructorTypeIR` with fresh type variables `typeId*`, defaults `id*` and optionals `id*`.

Finding the type of constructor call `prefixedNameIR < typeArgumentIR* > ( argumentIR* )`, under cursor `p` and typing context `TC`:

1. If `·` is equal to the type definition of `prefixedNameIR` from the `p` layer of `TC`:
 - a. Let `callableId : constructorTypeDefIR # iddefault* # idoptional*` be ! the overloaded constructor definition of `prefixedNameIR ( argumentIR* )` in `TC`.
 - b. Let `( constructorTypeIR, typeIdimpl* )` be `constructorTypeDefIR` specialized by `typeArgumentIR*`.
 - c. Let `bound` be bound type variables from the `p` layer of `TC`.
 - d. Let `B` be the union of the sets `bound` and `{ typeIdimpl* }`.
 - e. Check that `constructorTypeIR` is a well-formed constructor type, with bound type variables `B`.
 - f. Result in `constructorTypeIR` with fresh type variables `typeIdimpl*`, defaults `iddefault*` and optionals `idoptional*`.
2. Let `typeDefIR?` be the type definition of `prefixedNameIR` from the `p` layer of `TC`.
3. If let `typeDefIR'` be `typeDefIR?`:
 - a. Check that `typeDefIR'` does not have type `typedefTypeIR`.
 - b. Let `callableId : constructorTypeDefIR # iddefault* # idoptional*` be ! the overloaded constructor definition of `prefixedNameIR ( argumentIR* )` in `TC`.
 - c. Let `( constructorTypeIR, typeIdimpl* )` be `constructorTypeDefIR` specialized by `typeArgumentIR*`.
 - d. Let `bound` be bound type variables from the `p` layer of `TC`.
 - e. Let `B` be the union of the sets `bound` and `{ typeIdimpl* }`.
 - f. Check that `constructorTypeIR` is a well-formed constructor type, with bound type variables `B`.
 - g. Result in `constructorTypeIR` with fresh type variables `typeIdimpl*`, defaults `iddefault*` and optionals `idoptional*`.
4. If `typeArgumentIR*` is equal to `·`:
 - a. Check that `typeDefIR?` is defined.
 - b. Let `typeDefIR'` be `typeDefIR?`.
 - c. Check that `typeDefIR'` has type `typedefTypeIR`.
 - d. Let `TYPEDEF nameIRalias typeIRalias` be `typeDefIR'`.
 - e. Let `( nameIR, typeArgumentIR* )` be type alias in `typeIRalias` resolved to aliased name and type arguments.
 - f. Let `callableId : constructorTypeDefIR # iddefault* # idoptional*` be ! the overloaded constructor definition of `nameIR ( argumentIR* )` in `TC`.

- g. Let (constructorTypeIR, typeId_{impl}^{*}) be constructorTypeDefIR specialized by typeArgumentIR^{*}.
- h. Let bound be bound type variables from the p layer of TC.
- i. Let B be the union of the sets bound and { typeId_{impl}^{*} }.
- j. Check that constructorTypeIR is a well-formed constructor type, with bound type variables B.
- k. Result in constructorTypeIR with fresh type variables typeId_{impl}^{*}, defaults id_{default}^{*} and optionals id_{optional}^{*}.

During compile-time evaluation, constructors are resolved by:

Constructor_inst:

- Finding constructor for prefixedNameIR < typeArgumentListIR > (argumentListIR) under cursor cursor, and instantiation context instContext:
- Results in the constructor constructorDef < typeArgumentListIR > with defaults id^{*} and optionals id^{*}.

Finding constructor for prefixedNameIR < typeArgumentIR^{*} > (argumentIR^{*}) under cursor p, and instantiation context IC:

1. If · is equal to the type definition of prefixedNameIR from the p layer of IC:
 - a. Let _ : constructorDef # id_{default}^{*} # id_{optional}^{*} be ! the overloaded constructor definition of prefixedNameIR (argumentIR^{*}) in IC.
 - b. Result in the constructor constructorDef < typeArgumentIR^{*} > with defaults id_{default}^{*} and optionals id_{optional}^{*}.
2. Let typeDefIR[?] be the type definition of prefixedNameIR from the p layer of IC.
3. If let typeDefIR' be typeDefIR[?]:
 - a. Check that typeDefIR' does not have type typedefTypeIR.
 - b. Let _ : constructorDef # id_{default}^{*} # id_{optional}^{*} be ! the overloaded constructor definition of prefixedNameIR (argumentIR^{*}) in IC.
 - c. Result in the constructor constructorDef < typeArgumentIR^{*} > with defaults id_{default}^{*} and optionals id_{optional}^{*}.
4. If typeArgumentIR^{*} is equal to ::
 - a. Check that typeDefIR[?] is defined.
 - b. Let typeDefIR' be typeDefIR[?].
 - c. Check that typeDefIR' has type typedefTypeIR.
 - d. Let TYPEDEF nameIR_{alias} typeIR_{alias} be typeDefIR'.
 - e. Let (nameIR, typeArgumentIR^{*}) be type alias in typeIR_{alias} resolved to aliased name and type arguments.
 - f. Let _ : constructorDef # id_{default}^{*} # id_{optional}^{*} be ! the overloaded constructor definition of nameIR (argumentIR^{*}) in IC.
 - g. Result in the constructor constructorDef < typeArgumentIR^{*} > with defaults id_{default}^{*} and optionals id_{optional}^{*}.

18.3.2. Resolving callables

When type checking, callables are resolved by:

CallableType_ok:

- Finding the type of callable call `callableTargetIR < typeArgumentListIR > ( argumentIR* )`, under cursor `cursor` and typing context `typingContext`:
- Results in the callable type `callableTypeIR` with fresh type variables `typeId*`, defaults `id*` and optionals `id*`.

At runtime, callables are resolved to callees by:

Callee_eval:

- Finding the callee `callableTargetIR ( argumentListIR )`, under cursor `cursor`, evaluation context `evalContext`, and store `store`:
- Results in the updated evaluation context `evalContext`, store `store`, and callee `calleeResult`.

While resolving callees, `exit` statements may be encountered. Thus, the result of callee resolution is:

```
calleeResult
: continueResult<callee>
| abortResult
;
```

The following subsections describe the two phases, for each kind of callable.

18.3.2.1. Actions

During type checking, actions are resolved by:

```
actionTypeIR
: ACTION nameIR `( parameterIR* )
;
```

Finding the type of callable call `prefixedNameIR < · > ( argumentIR* )`, under cursor `p` and typing context `TC`:

1. Let `callableId : callableTypeDefIR # iddefault* # idoptional*` be ! the overloaded callable definition of `prefixedNameIR( argumentIR* )` from the `p` layer of `TC`.
2. Check that `callableTypeDefIR` has type `actionTypeIR`.
3. Let `actionTypeIR` be `callableTypeDefIR`.
4. Let `bound` be bound type variables from the `p` layer of `TC`.
5. Check that `actionTypeIR` is a well-formed callable type, with bound type variables `bound`.
6. Result in the callable type `actionTypeIR` with fresh type variables `·`, defaults `iddefault*` and optionals `idoptional*`.

During runtime evaluation, actions are resolved by:

```
actionCallee
: ACTION cursor . nameIR `( parameterListIR # id* ) blockStatementIR
;
```

Finding the callee `referenceExpressionIR ( argumentListIR )`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let `_ : (cursor, callableDef) # iddefault* # _*` be ! the overloaded callable definition of `referenceExpressionIR( argumentListIR )` in the `p` layer of `EC`.
2. Check that `(cursor, callableDef)` has type `(cursor, actionDef)`.
3. Let `( pref, actionDef )` be `(cursor, callableDef)`.
4. Let `ACTION nameIR ( parameterListIR ) blockStatementIR` be `actionDef`.
5. Let `actionCallee` be `ACTION pref . nameIR ( parameterListIR # iddefault* ) blockStatementIR`.
6. Result in the updated evaluation context `EC`, store `STO`, and callee `actionCallee`.

18.3.2.2. Functions

During type checking, functions are resolved by:

```
functionTypeIR
: definedFunctionTypeIR
| externFunctionTypeIR
;
```

Finding the type of callable `call prefixedNameIR < typeArgumentIR* > ( argumentIR* )`, under cursor `p` and typing context `TC`:

1. Let `callableId : callableTypeDefIR # iddefault* # idoptional*` be ! the overloaded callable definition of `prefixedNameIR( argumentIR* )` from the `p` layer of `TC`.
2. Check that `callableTypeDefIR` has type `functionTypeDefIR`.
3. Let `functionTypeDefIR` be `callableTypeDefIR`.
4. Let `(callableTypeIR, typeId*)` be `functionTypeDefIR` specialized by `typeArgumentIR*`.
5. Check that `(callableTypeIR, typeId*)` has type `(functionTypeIR, typeId*)`.
6. Let `( functionTypeIR, typeIdimpl* )` be `(callableTypeIR, typeId*)`.
7. Let `bound'` be bound type variables from the `p` layer of `TC`.
8. Let `bound` be the union of the sets `bound'` and `{ typeIdimpl* }`.
9. Check that `functionTypeIR` is a well-formed callable type, with bound type variables `bound`.
10. Result in the callable type `functionTypeIR` with fresh type variables `typeIdimpl*`, defaults `iddefault*` and optionals `idoptional*`.

During runtime evaluation, functions are resolved by:

```
functionCallee
```

```

: definedFunctionCallee
| externFunctionCallee
;

```

For extern functions,

Finding the callee `referenceExpressionIR ( argumentListIR )`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let `_ : ( _ callableDef ) # iddefault* # idoptional* be ! the overloaded callable definition of referenceExpressionIR( argumentListIR ) in the p layer of EC.`
2. Check that `callableDef` has type `externFunctionDef`.
3. Let `externFunctionDef` be `callableDef`.
4. Let `EXTERN_FUNCTION nameIR < typeParameterListIR > ( parameterListIR ) be externFunctionDef`.
5. Let `externFunctionCallee` be `EXTERN_FUNCTION nameIR < typeParameterListIR > ( parameterListIR # iddefault* # idoptional* )`.
6. Result in the updated evaluation context `EC`, store `STO`, and callee `externFunctionCallee`.

For user-defined functions,

Finding the callee `referenceExpressionIR ( argumentListIR )`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let `_ : ( _ callableDef ) # iddefault* # _* be ! the overloaded callable definition of referenceExpressionIR( argumentListIR ) in the p layer of EC.`
2. Check that `callableDef` has type `definedFunctionDef`.
3. Let `definedFunctionDef` be `callableDef`.
4. Let `FUNCTION nameIR < typeParameterListIR > ( parameterListIR ) blockStatementIR be definedFunctionDef`.
5. Let `definedFunctionCallee` be `FUNCTION nameIR < typeParameterListIR > ( parameterListIR # iddefault* ) blockStatementIR`.
6. Result in the updated evaluation context `EC`, store `STO`, and callee `definedFunctionCallee`.

18.3.2.3. Built-in methods

During type checking, built-in methods are resolved by:

```

builtinMethodTypeIR
: BUILTIN_METHOD nameIR `( parameterIR* ) : typeIR
;

```

Finding the type of callable call `typedExpressionIRbase . nameIR < · > ( argumentIR* )`, under cursor `p` and typing context `TC`:

1. If `argumentIR*` is equal to `:`:
 - a. If `nameIR` is in `[ "minSizeInBits", "minSizeInBytes", "maxSizeInBits", "maxSizeInBytes" ]`:
 - i. Let `methodTypeIR` be `BUILTIN_METHOD nameIR ( · ) : INT`.

- ii. Result in the callable type `methodTypeIR` with fresh type variables `·`, defaults `·` and optionals `·`.
 - b. Else if `nameIR` is in "isValid":
 - i. Let `typeIRbase` be the type of `typedExpressionIRbase`.
 - ii. Let `typeIR` be `typeIRbase` with `typedefs` unrolled.
 - iii. If `typeIR` has type `headerTypeIR`:
 - A. Let `methodTypeIR` be `BUILTIN_METHOD nameIR ( · ) : BOOL`.
 - B. Result in the callable type `methodTypeIR` with fresh type variables `·`, defaults `·` and optionals `·`.
 - iv. Else if `typeIR` has type `headerUnionTypeIR`:
 - A. Let `methodTypeIR` be `BUILTIN_METHOD nameIR ( · ) : BOOL`.
 - B. Result in the callable type `methodTypeIR` with fresh type variables `·`, defaults `·` and optionals `·`.
 - c. Else if `nameIR` is in ["setValid", "setInvalid"]:
 - i. Let `typeIRbase` be the type of `typedExpressionIRbase`.
 - ii. Let `typeIR` be `typeIRbase` with `typedefs` unrolled.
 - iii. Check that `typeIR` has type `headerTypeIR`.
 - iv. Let `methodTypeIR` be `BUILTIN_METHOD nameIR ( · ) : VOID`.
 - v. Result in the callable type `methodTypeIR` with fresh type variables `·`, defaults `·` and optionals `·`.
2. If `argumentIR*` is a list of length 1:
- a. Check that `nameIR` is in ["push_front", "pop_front"].
 - b. Let `typeIRbase` be the type of `typedExpressionIRbase`.
 - c. Let `typeIR` be `typeIRbase` with `typedefs` unrolled.
 - d. Check that `typeIR` has type `headerStackTypeIR`.
 - e. Let `parameterIR` be ``EMPTY `EMPTY INT "count" ·`.
 - f. Let `methodTypeIR` be `BUILTIN_METHOD nameIR ( parameterIR ) : INT`.
 - g. Result in the callable type `methodTypeIR` with fresh type variables `·`, defaults `·` and optionals `·`.

During runtime evaluation, built-in methods are resolved by:

```
builtinMethodCallee
  : BUILTIN_METHOD storageReference . nameIR `< typeArgumentListIR > `( parameterListIR
)
;
```

Finding the callee `typedExpressionIR . nameIR ( argumentListIR )`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let `typedLvalueIR?` be the l-value form of `typedExpressionIR`.
2. If let `typedLvalueIR?` be `typedLvalueIR?`:
 - a. Let the updated evaluation context `EC1`, store `STO1`, and result `storageReferenceResult` be

- the result of **evaluating** `typedLvalueIR'`, under cursor `p`, evaluation context `EC`, and store `STO`.
 - b. Check that `storageReferenceResult` has type `abortResult`.
 - c. Let `abortResult` be `storageReferenceResult`.
 - d. Result in the updated evaluation context `EC1`, store `STO1`, and callee `abortResult`.
3. If `nameIR` is equal to `"push_front"`:
- a. Check that `argumentListIR` is a list of length 1.
 - b. Check that `typedLvalueIR?` is defined.
 - c. Let `typedLvalueIR'` be `typedLvalueIR?`.
 - d. Let the updated evaluation context `EC1`, store `STO1`, and result `storageReferenceResult` be
 - the result of **evaluating** `typedLvalueIR'`, under cursor `p`, evaluation context `EC`, and store `STO`.
 - e. Check that `storageReferenceResult` has type `continueResult<storageReference?>`.
 - f. Let `storageReference?` be `storageReferenceResult`.
 - g. Check that `storageReference?` is defined.
 - h. Let `storageReference'` be `storageReference?`.
 - i. Let `parameterIR` be ``EMPTY` `EMPTY INT "count" `.`
 - j. Let `builtinMethodCallee` be `BUILTIN_METHOD storageReference' . "push_front" < · > ( parameterIR )`.
 - k. Result in the updated evaluation context `EC1`, store `STO1`, and callee `builtinMethodCallee`.
4. Else if `nameIR` is equal to `"pop_front"`:
- a. Check that `argumentListIR` is a list of length 1.
 - b. Check that `typedLvalueIR?` is defined.
 - c. Let `typedLvalueIR'` be `typedLvalueIR?`.
 - d. Let the updated evaluation context `EC1`, store `STO1`, and result `storageReferenceResult` be
 - the result of **evaluating** `typedLvalueIR'`, under cursor `p`, evaluation context `EC`, and store `STO`.
 - e. Check that `storageReferenceResult` has type `continueResult<storageReference?>`.
 - f. Let `storageReference?` be `storageReferenceResult`.
 - g. Check that `storageReference?` is defined.
 - h. Let `storageReference'` be `storageReference?`.
 - i. Let `parameterIR` be ``EMPTY` `EMPTY INT "count" `.`
 - j. Let `builtinMethodCallee` be `BUILTIN_METHOD storageReference' . "pop_front" < · > ( parameterIR )`.
 - k. Result in the updated evaluation context `EC1`, store `STO1`, and callee `builtinMethodCallee`.
5. Else if `nameIR` is equal to `"isValid"`:
- a. Check that `argumentListIR` is equal to `·`.
 - b. Check that `typedLvalueIR?` is defined.
 - c. Let `typedLvalueIR'` be `typedLvalueIR?`.
 - d. Let the updated evaluation context `EC1`, store `STO1`, and result `storageReferenceResult` be
 - the result of **evaluating** `typedLvalueIR'`, under cursor `p`, evaluation context `EC`, and store

STO.

- e. Check that `storageReferenceResult` has type `continueResult<storageReference?>`.
 - f. Let `storageReference?` be `storageReferenceResult`.
 - g. Check that `storageReference?` is defined.
 - h. Let `storageReference'` be `storageReference?`.
 - i. Let `builtinMethodCallee` be `BUILTIN_METHOD storageReference' . "isValid" < · > ( · )`.
 - j. Result in the updated evaluation context EC_1 , store STO_1 , and callee `builtinMethodCallee`.
6. Else if `nameIR` is equal to `"setValid"`:
- a. Check that `argumentListIR` is equal to `·`.
 - b. Check that `typedLvalueIR?` is defined.
 - c. Let `typedLvalueIR'` be `typedLvalueIR?`.
 - d. Let the updated evaluation context EC_1 , store STO_1 , and result `storageReferenceResult` be
 - the result of evaluating `typedLvalueIR'`, under cursor `p`, evaluation context EC , and store STO .
 - e. Check that `storageReferenceResult` has type `continueResult<storageReference?>`.
 - f. Let `storageReference?` be `storageReferenceResult`.
 - g. Check that `storageReference?` is defined.
 - h. Let `storageReference'` be `storageReference?`.
 - i. Let `builtinMethodCallee` be `BUILTIN_METHOD storageReference' . "setValid" < · > ( · )`.
 - j. Result in the updated evaluation context EC_1 , store STO_1 , and callee `builtinMethodCallee`.
7. Else if `nameIR` is equal to `"setInvalid"`:
- a. Check that `argumentListIR` is equal to `·`.
 - b. Check that `typedLvalueIR?` is defined.
 - c. Let `typedLvalueIR'` be `typedLvalueIR?`.
 - d. Let the updated evaluation context EC_1 , store STO_1 , and result `storageReferenceResult` be
 - the result of evaluating `typedLvalueIR'`, under cursor `p`, evaluation context EC , and store STO .
 - e. Check that `storageReferenceResult` has type `continueResult<storageReference?>`.
 - f. Let `storageReference?` be `storageReferenceResult`.
 - g. Check that `storageReference?` is defined.
 - h. Let `storageReference'` be `storageReference?`.
 - i. Let `builtinMethodCallee` be `BUILTIN_METHOD storageReference' . "setInvalid" < · > ( · )`.
 - j. Result in the updated evaluation context EC_1 , store STO_1 , and callee `builtinMethodCallee`.

18.3.2.4. Extern methods

During type checking, extern methods are resolved by:

```
externMethodTypeIR
: EXTERN_METHOD nameIR `( parameterIR* ) : typeIR
| EXTERN_METHOD ABSTRACT nameIR `( parameterIR* ) : typeIR
```

;

Finding the type of callable call $\text{typedExpressionIR}_{\text{base}} . \text{nameIR} < \text{typeArgumentIR}^* > (\text{argumentIR}^*)$, under cursor p and typing context TC :

1. Let $\text{typeIR}_{\text{base}}$ be the type of $\text{typedExpressionIR}_{\text{base}}$.
2. Let typeIR be $\text{typeIR}_{\text{base}}$ with typedefs unrolled.
3. Check that typeIR has type $\text{externObjectTypeIR}$.
4. Let $\text{EXTERN typedId} < _ > \{ (\text{callableId}_{\text{extern}} : \text{externMethodTypeDefIR}_{\text{extern}})^* \}$ be typeIR .
5. Let callTargetKey be the call target key for $\text{nameIR} (\text{argumentIR}^*)$.
6. Let $_ : \text{externMethodTypeDefIR} \# \text{id}_{\text{default}}^* \# \text{id}_{\text{optional}}^* \text{ be ! finding the item in } \{ (\text{callableId}_{\text{extern}} : \text{externMethodTypeDefIR}_{\text{extern}})^* \}$ matching callTargetKey using parameter getter $\$parameterListIR_of_externMethodTypeDefIR$.
7. Let $(\text{callableTypeIR}, \text{typedId}^*)$ be $\text{externMethodTypeDefIR}$ specialized by typeArgumentIR^* .
8. Check that $(\text{callableTypeIR}, \text{typedId}^*)$ has type $(\text{externMethodTypeIR}, \text{typedId}^*)$.
9. Let $(\text{externMethodTypeIR}, \text{typedId}_{\text{impl}}^*)$ be $(\text{callableTypeIR}, \text{typedId}^*)$.
10. Let bound' be bound type variables from the p layer of TC .
11. Let bound be the union of the sets bound' and $\{ \text{typedId}_{\text{impl}}^* \}$.
12. Check that $\text{externMethodTypeIR}$ is a well-formed callable type, with bound type variables bound .
13. Result in the callable type $\text{externMethodTypeIR}$ with fresh type variables $\text{typedId}_{\text{impl}}^*$, defaults $\text{id}_{\text{default}}^*$ and optionals $\text{id}_{\text{optional}}^*$.

During runtime evaluation, extern methods are resolved by:

```
externMethodCallee
: EXTERN_METHOD objectId . nameIR `< typeParameterListIR > `( parameterListIR # id* #
id* ) `{ theta ; frame ; blockStatementIR? }
;
```

Finding the callee $\text{typedExpressionIR} . \text{nameIR} (\text{argumentListIR})$, under cursor p , evaluation context EC , and store STO :

1. Let $\text{typedLvalueIR}'$ be ! the l-value form of typedExpressionIR .
2. Let the updated evaluation context EC_1 , store STO_1 , and result $\text{storageReferenceResult}$ be
 - the result of evaluating $\text{typedLvalueIR}'$, under cursor p , evaluation context EC , and store STO .
3. If let abortResult be $\text{storageReferenceResult}$:
 - a. Result in the updated evaluation context EC_1 , store STO_1 , and callee abortResult .
4. Else:
 - a. Let $\text{storageReference}^?$ be $\text{storageReferenceResult}$.
 - b. Check that $\text{storageReference}^?$ is defined.
 - c. Let $\text{storageReference}'$ be $\text{storageReference}^?$.
 - d. Let the value value be

- the result of **reading from** `storageReference'`, under cursor `p`, evaluation context `EC1`, and store `STO1`.
- e. Check that `value` has type `objectReferenceValue`.
- f. Let `REF objectId` be `value`.
- g. Let `object` be **the object** `objectId` in `STO1`.
- h. Check that `object` has type `externObject`.
- i. Let `EXTERN _ < theta > { _ frame externMethodDefEnv }` be `object`.
- j. Let `callTargetKey` be **the call target key for** `nameIR ( argumentListIR )`.
- k. Let `_ : externMethodDef # iddefault * # idoptional *` be **! finding the item in** `externMethodDefEnv` **matching** `callTargetKey` **using parameter getter** `$parameterListIR_of_externMethodDef`.
- l. Check that `externMethodDef` matches pattern `EXTERN_METHOD % < % > ( % ) %`.
- m. Let `EXTERN_METHOD nameIR' < typeParameterListIR > ( parameterListIR ) blockStatementIR?` be `externMethodDef`.
- n. Check that `nameIR'` is equal to `nameIR`.
- o. Let `externMethodCallee` be `EXTERN_METHOD objectId . nameIR' < typeParameterListIR > ( parameterListIR # iddefault * # idoptional * ) { theta ; frame ; blockStatementIR? }`.
- p. Result in the updated evaluation context `EC1`, store `STO1`, and callee `externMethodCallee`.

18.3.2.5. Parser apply methods

During type checking, parser apply methods are resolved by:

```
parserApplyMethodTypeIR
: PARSER_APPLY `( parameterIR* )
;
```

Finding the type of callable call `typedExpressionIRbase . "apply" < · > ( argumentIR* )`, under cursor `p` and typing context `TC`:

1. Let `typeIRbase` be **the type of** `typedExpressionIRbase`.
2. Let `typeIR` be `typeIRbase` **with typedefs unrolled**.
3. Check that `typeIR` has type `parserObjectTypeIR`.
4. Let `PARSER _ < _ * > ( parameterIR* )` be `typeIR`.
5. Let `parserApplyMethodTypeIR` be `PARSER_APPLY ( parameterIR* )`.
6. Let `idparam *` be the list of `idparam`, obtained by repeating:
 - Let `idparam` be the name of `parameterIR`.
 for each `parameterIR` in `parameterIR*`
7. Let `callableId` be `"apply" ( idparam * )`.
8. Let `callTargetKey` be **the call target key for** `"apply" ( argumentIR* )`.
9. Let `_ : _ # iddefault * # idoptional *` be **! finding the item in** `{ callableId : parserApplyMethodTypeIR }` **matching** `callTargetKey` **using parameter getter** `$parameterListIR_of_parserApplyMethodTypeIR`.

10. Result in the callable type `parserApplyMethodTypeIR` with fresh type variables `.`, defaults `iddefault*` and optionals `idoptional*`.

During runtime evaluation, parser apply methods are resolved by:

```
parserApplyMethodCallee
: PARSER objectId . APPLY `( parameterListIR # id* ) `{ theta ; frame ;
parserLocalDeclarationListIR ; stateEnv }
;
```

Finding the callee `typedExpressionIR . "apply" ( argumentListIR ), under cursor p, evaluation context EC, and store STO:`

1. Let `typedLvalueIR'` be ! the l-value form of `typedExpressionIR`.
2. Let the updated evaluation context `EC1`, store `STO1`, and result `storageReferenceResult` be
 - the result of **evaluating** `typedLvalueIR'`, under cursor `p`, evaluation context `EC`, and store `STO`.
3. If let `abortResult` be `storageReferenceResult`:
 - a. Result in the updated evaluation context `EC1`, store `STO1`, and callee `abortResult`.
4. Else:
 - a. Let `storageReference?` be `storageReferenceResult`.
 - b. Check that `storageReference?` is defined.
 - c. Let `storageReference'` be `storageReference?`.
 - d. Let the value `value` be
 - the result of **reading from** `storageReference'`, under cursor `p`, evaluation context `EC1`, and store `STO1`.
 - e. Check that `value` has type `objectReferenceValue`.
 - f. Let `REF objectId` be `value`.
 - g. Let `object` be the object `objectId` in `STO1`.
 - h. Check that `object` has type `parserObject`.
 - i. Let `PARSER < theta > ( parameterListIR ) { frame parserLocalDeclarationListIR stateEnv }` be `object`.
 - j. Let `callableId` be the callable identifier for `"apply" ( parameterListIR )`.
 - k. Let `parserApplyMethodDef` be `PARSER_APPLY ( parameterListIR ) { parserLocalDeclarationListIR ; stateEnv }`.
 - l. Let `callTargetKey` be the call target key for `"apply" ( argumentListIR )`.
 - m. Let `_ : _ # iddefault* # _*` be ! **finding the item in** `{ callableId : parserApplyMethodDef }` **matching** `callTargetKey` **using parameter getter** `$parameterListIR_of_parserApplyMethodDef`.
 - n. Let `parserApplyMethodCallee` be `PARSER objectId . APPLY ( parameterListIR # iddefault* ) { theta ; frame ; parserLocalDeclarationListIR ; stateEnv }`.
 - o. Result in the updated evaluation context `EC1`, store `STO1`, and callee `parserApplyMethodCallee`.

18.3.2.6. Control apply methods

During type checking, control apply methods are resolved by:

```

controlApplyMethodTypeIR
: CONTROL_APPLY `( parameterIR* )
;

```

Finding the type of callable call `typedExpressionIRbase . "apply" < · > ( argumentIR* )`, under cursor `p` and typing context `TC`:

1. Let `typeIRbase` be the type of `typedExpressionIRbase`.
2. Let `typeIR` be `typeIRbase` with `typeddefs` unrolled.
3. Check that `typeIR` has type `controlObjectTypeIR`.
4. Let `CONTROL _ < _ > ( parameterIR* )` be `typeIR`.
5. Let `controlApplyMethodTypeIR` be `CONTROL_APPLY ( parameterIR* )`.
6. Let `idparam*` be the list of `idparam`, obtained by repeating:
 - Let `idparam` be the name of `parameterIR`.
 for each `parameterIR` in `parameterIR*`
7. Let `callableId` be `"apply" ( idparam* )`.
8. Let `callTargetKey` be the call target key for `"apply" ( argumentIR* )`.
9. Let `_ : _ # iddefault* # idoptional*` be ! finding the item in `{ callableId : controlApplyMethodTypeIR }` matching `callTargetKey` using parameter getter `$parameterListIR_of_controlApplyMethodTypeIR`.
10. Result in the callable type `controlApplyMethodTypeIR` with fresh type variables `·`, defaults `iddefault*` and optionals `idoptional*`.

During runtime evaluation, control apply methods are resolved by:

```

controlApplyMethodCallee
: CONTROL objectId . APPLY `( parameterListIR # id* ) `{ theta ; frame ;
controlLocalDeclarationListIR ; actionDefEnv ; controlBodyIR }
;

```

Finding the callee `typedExpressionIR . "apply" ( argumentListIR )`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let `typedLvalueIR'` be ! the l-value form of `typedExpressionIR`.
2. Let the updated evaluation context `EC1`, store `STO1`, and result `storageReferenceResult` be
 - the result of evaluating `typedLvalueIR'`, under cursor `p`, evaluation context `EC`, and store `STO`.
3. If let `abortResult` be `storageReferenceResult`:
 - a. Result in the updated evaluation context `EC1`, store `STO1`, and callee `abortResult`.
4. Else:
 - a. Let `storageReference?` be `storageReferenceResult`.
 - b. Check that `storageReference?` is defined.
 - c. Let `storageReference'` be `storageReference?`.
 - d. Let the value `value` be

- the result of **reading from** `storageReference`, under cursor `p`, evaluation context `EC1`, and store `STO1`.
- e. Check that `value` has type `objectReferenceValue`.
- f. Let `REF objectId` be `value`.
- g. Let `object` be **the object** `objectId` in `STO1`.
- h. Check that `object` has type `controlObject`.
- i. Let `CONTROL < theta > ( parameterListIR ) { frame controlLocalDeclarationListIR actionDefEnv controlBodyIR }` be `object`.
- j. Let `callableId` be **the callable identifier for** `"apply" ( parameterListIR )`.
- k. Let `controlApplyMethodDef` be `CONTROL_APPLY ( parameterListIR ) { controlLocalDeclarationListIR ; actionDefEnv ; controlBodyIR }`.
- l. Let `callTargetKey` be **the call target key for** `"apply" ( argumentListIR )`.
- m. Let `_ : _ # iddefault * # _ *` be **! finding the item in** `{ callableId : controlApplyMethodDef }` **matching** `callTargetKey` **using parameter getter** `$parameterListIR_of_controlApplyMethodDef`.
- n. Let `controlApplyMethodCallee` be `CONTROL objectId . APPLY ( parameterListIR # iddefault * ) { theta ; frame ; controlLocalDeclarationListIR ; actionDefEnv ; controlBodyIR }`.
- o. Result in the updated evaluation context `EC1`, store `STO1`, and callee `controlApplyMethodCallee`.

18.3.2.7. Table apply methods

During type checking, table apply methods are resolved by:

```
tableApplyMethodTypeIR
: TABLE_APPLY : tableMetadataStructTypeIR
;
```

Finding the type of callable `call typedExpressionIRbase . "apply" < · > ( · )`, under cursor `p` and typing context `TC`:

1. Let `typeIRbase` be **the type of** `typedExpressionIRbase`.
2. Let `typeIR` be `typeIRbase` **with typedefs unrolled**.
3. Check that `typeIR` has type `tableObjectTypeIR`.
4. Let `TABLE _ { tableMetadataStructTypeIR }` be `typeIR`.
5. Let `tableApplyMethodTypeIR` be `TABLE_APPLY : tableMetadataStructTypeIR`.
6. Result in the callable type `tableApplyMethodTypeIR` with fresh type variables `·`, defaults `·` and optionals `·`.

During runtime evaluation, table apply methods are resolved by:

```
tableApplyMethodCallee
: TABLE objectId . APPLY `{ frame ; tableObjectProperty }
;
```

Finding the callee `typedExpressionIR . "apply" ( · )`, under cursor `p`, evaluation context `EC`, and store `STO`:

1. Let `typedLvalueIR` be ! the l-value form of `typedExpressionIR`.
2. Let the updated evaluation context `EC1`, store `STO1`, and result `storageReferenceResult` be
 - the result of evaluating `typedLvalueIR`, under cursor `p`, evaluation context `EC`, and store `STO`.
3. If let `abortResult` be `storageReferenceResult`:
 - a. Result in the updated evaluation context `EC1`, store `STO1`, and callee `abortResult`.
4. Else:
 - a. Let `storageReference?` be `storageReferenceResult`.
 - b. Check that `storageReference?` is defined.
 - c. Let `storageReference'` be `storageReference?`.
 - d. Let the value `value` be
 - the result of reading from `storageReference'`, under cursor `p`, evaluation context `EC1`, and store `STO1`.
 - e. Check that `value` has type `objectReferenceValue`.
 - f. Let `REF objectId` be `value`.
 - g. Let `object` be the object `objectId` in `STO1`.
 - h. Check that `object` has type `tableObject`.
 - i. Let `TABLE typeId { frame TBL }` be `object`.
 - j. Let `tableApplyMethodCallee` be `TABLE objectId . APPLY { frame ; TBL }`.
 - k. Result in the updated evaluation context `EC1`, store `STO1`, and callee `tableApplyMethodCallee`.

18.4. Call convention

18.5. Runtime evaluation of calls

19. Operations

19.1. Unary operations

Unary operations are evaluated with:

`unop value`

1. If `unop` is `~`:
 - a. Return `~ value`.
2. Else if `unop` is `!`:
 - a. Return `! value`.
3. Else if `unop` is `+`:
 - a. Return `+ value`.
4. Else:

a. Return - value.

19.1.1. Negation

! value

1. Check that value has type boolValue.
2. Let `B b` be value.
3. Return `B ~b`.

19.1.2. Bitwise complement

~ value

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. Check that integerLiteral matches pattern % W %.
4. Let w W i be integerLiteral.
5. Let i' be the two's complement negation of i.
6. Let int be 2 to the power of w.
7. Let i'' be i' if i' is greater than or equal to 0 otherwise int + i'.
8. Return w W i''.

19.1.3. Plus

+ value

1. Check that value has type integerLiteral.
2. Let integerLiteral be value.
3. If let D i be integerLiteral:
 - a. Return D i.
4. Else if let w W i be integerLiteral:
 - a. Return w W i.
5. Else:
 - a. Let w S i be integerLiteral.
 - b. Return w S i.

19.1.4. Minus

- value

1. Check that `value` has type `integerLiteral`.
2. Let `integerLiteral` be `value`.
3. If let `D i` be `integerLiteral`:
 - a. Return `D -i`.
4. Else if let `w W i` be `integerLiteral`:
 - a. Let `int` be 2 to the power of `w`.
 - b. Let `i'` be `int - i`.
 - c. Return `w W i'`.
5. Else:
 - a. Let `w S i` be `integerLiteral`.
 - b. Let `int` be the two's complement representation of bitstring `-i` of width `w`.
 - c. Let `i'` be the bitstring of width `w` representing the integer `int`.
 - d. Return `w S i'`.

19.2. Binary operations

Binary operations are evaluated with:

`valuel binop valuer`

1. If `binop` is `+`:
 - a. Return `valuel + valuer`.
2. Else if `binop` is `|+|`:
 - a. Return `valuel || + valuer`.
3. Else if `binop` is `-`:
 - a. Return `valuel - valuer`.
4. Else if `binop` is `|-|`:
 - a. Return `valuel |-| valuer`.
5. Else if `binop` is `*`:
 - a. Return `valuel * valuer`.
6. Else if `binop` is `/`:
 - a. Return `valuel / valuer`.
7. Else if `binop` is `%`:
 - a. Return `valuel % valuer`.
8. Else if `binop` is `<<`:

- a. Return `valuel << valuer`.
- 9. Else if `binop` is `>>`:
 - a. Return `valuel >> valuer`.
- 10. Else if `binop` is `:`
 - a. Let `bool` be `valuel <= valuer`.
 - b. Return ``B bool`.
- 11. Else if `binop` is `>=`:
 - a. Let `bool` be `valuel >= valuer`.
 - b. Return ``B bool`.
- 12. Else if `binop` is `<`:
 - a. Let `bool` be `valuel < valuer`.
 - b. Return ``B bool`.
- 13. Else if `binop` is `>`:
 - a. Let `bool` be `valuel > valuer`.
 - b. Return ``B bool`.
- 14. Else if `binop` is `==`:
 - a. Let `bool` be `valuel == valuer`.
 - b. Return ``B bool`.
- 15. Else if `binop` is `!=`:
 - a. Let `bool` be `valuel != valuer`.
 - b. Return ``B bool`.
- 16. Else if `binop` is `&`:
 - a. Return `valuel & valuer`.
- 17. Else if `binop` is `^`:
 - a. Return `valuel ^ valuer`.
- 18. Else if `binop` is `|`:
 - a. Return `valuel | valuer`.
- 19. Else if `binop` is `++`:
 - a. Return `valuel valuer`.

19.2.1. Addition

`value + value'`

1. Check that `value` has type `integerLiteral`.
2. Let `integerLiteral` be `value`.
3. If let `D il` be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.

- b. Let `integerLiteral` be `value`.
 - c. Check that `integerLiteral` matches pattern `D %`.
 - d. Let `D ir` be `integerLiteral`.
 - e. Return `D il + ir`.
4. Else if let `w W il` be `integerLiteral`:
 - a. Check that `value` has type `integerLiteral`.
 - b. Let `integerLiteral` be `value`.
 - c. Check that `integerLiteral` matches pattern `% W %`.
 - d. Let `w' W ir` be `integerLiteral`.
 - e. Check that `w` is equal to `w'`.
 - f. Let `ir` be the two's complement representation of bitstring `il` of width `w`.
 - g. Let `ir` be the two's complement representation of bitstring `ir` of width `w`.
 - h. Let `i` be the bitstring of width `w` representing the integer `il + ir`.
 - i. Return `w W i`.
5. Else:
 - a. Let `w S il` be `integerLiteral`.
 - b. Check that `value` has type `integerLiteral`.
 - c. Let `integerLiteral` be `value`.
 - d. Check that `integerLiteral` matches pattern `% S %`.
 - e. Let `w' S ir` be `integerLiteral`.
 - f. Check that `w` is equal to `w'`.
 - g. Let `ir` be the two's complement representation of bitstring `il` of width `w`.
 - h. Let `ir` be the two's complement representation of bitstring `ir` of width `w`.
 - i. Let `i` be the bitstring of width `w` representing the integer `il - ir`.
 - j. Return `w S i`.
 - k. Let `i'` be the bitstring of width `w` representing the integer `il - ir`.
 - l. Return `w S i'`.

19.2.2. Saturated addition

`value | | + value'`

1. Check that `value` has type `integerLiteral`.
2. Let `integerLiteral` be `value`.
3. If let `w W il` be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral` be `value'`.
 - c. Check that `integerLiteral` matches pattern `% W %`.

- d. Let $w' \ W \ i_r$ be `integerLiteral`'.
 - e. Check that w is equal to w' .
 - f. Let i be $i_l + i_r$.
 - g. Let i_{max} be 2 to the power of w .
 - h. Let i' be i if i is less than i_{max} otherwise $i_{max} - 1$.
 - i. Return $w \ W \ i'$.
4. Else if let $w \ S \ i_l$ be `integerLiteral`:
- a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `% S %`.
 - d. Let $w' \ S \ i_r$ be `integerLiteral'`.
 - e. Check that w is equal to w' .
 - f. Let i_r be the two's complement representation of bitstring i_l of width w .
 - g. Let i_r be the two's complement representation of bitstring i_r of width w .
 - h. Let i be $i_l + i_r$.
 - i. If i is greater than 0:
 - i. Let int be $w - 1$.
 - ii. Check that int has type `nat`.
 - iii. Let w'' be int .
 - iv. Let i_{max} be 2 to the power of w'' .
 - v. Let i' be i if i is less than i_{max} otherwise $i_{max} - 1$.
 - vi. Let i'' be the bitstring of width w representing the integer i' .
 - vii. Return $w \ S \ i''$.
 - j. If i is less than or equal to 0:
 - i. Let int be $w - 1$.
 - ii. Check that int has type `nat`.
 - iii. Let w'' be int .
 - iv. Let int' be 2 to the power of w'' .
 - v. Let i_{min} be $-int'$.
 - vi. Let i' be i if i is greater than or equal to i_{min} otherwise i_{min} .
 - vii. Let i'' be the bitstring of width w representing the integer i' .
 - viii. Return $w \ S \ i''$.

19.2.3. Subtraction

`value - value'`

1. Check that `value` has type `integerLiteral`.

2. Let `integerLiteral` be `value`.
3. If let `D il` be `integerLiteral`:
 - a. Check that `value`' has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value`'.
 - c. Check that `integerLiteral'` matches pattern `D %`.
 - d. Let `D ir` be `integerLiteral'`.
 - e. Return `D il - ir`.
4. Else if let `w W il` be `integerLiteral`:
 - a. Check that `value`' has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value`'.
 - c. Check that `integerLiteral'` matches pattern `% W %`.
 - d. Let `w' W ir` be `integerLiteral'`.
 - e. Check that `w` is equal to `w'`.
 - f. Let `ir` be the two's complement representation of bitstring `il` of width `w`.
 - g. Let `ir` be the two's complement representation of bitstring `ir` of width `w`.
 - h. Let `i` be the bitstring of width `w` representing the integer `il - ir`.
 - i. Return `w W i`.

19.2.4. Saturated subtraction

`value |-| value'`

1. Check that `value` has type `integerLiteral`.
2. Let `integerLiteral` be `value`.
3. If let `w W il` be `integerLiteral`:
 - a. Check that `value`' has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value`'.
 - c. Check that `integerLiteral'` matches pattern `% W %`.
 - d. Let `w' W ir` be `integerLiteral'`.
 - e. Check that `w` is equal to `w'`.
 - f. Let `i` be `il - ir`.
 - g. Let `i'` be `i` if `i` is greater than or equal to 0 otherwise 0.
 - h. Return `w W i'`.
4. Else if let `w S il` be `integerLiteral`:
 - a. Check that `value`' has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value`'.
 - c. Check that `integerLiteral'` matches pattern `% S %`.
 - d. Let `w' S ir` be `integerLiteral'`.

- e. Check that w is equal to w' .
- f. Let i_l be the two's complement representation of bitstring i_l of width w .
- g. Let i_r be the two's complement representation of bitstring i_r of width w .
- h. Let i be $i_l - i_r$.
- i. If i is greater than 0:
 - i. Let int be $w - 1$.
 - ii. Check that int has type nat .
 - iii. Let w'' be int .
 - iv. Let i_{max} be 2 to the power of w'' .
 - v. Let i' be i if i is less than i_{max} otherwise $i_{max} - 1$.
 - vi. Let i'' be the bitstring of width w representing the integer i' .
 - vii. Return $w \ S \ i''$.
- j. If i is less than or equal to 0:
 - i. Let int be $w - 1$.
 - ii. Check that int has type nat .
 - iii. Let w'' be int .
 - iv. Let int' be 2 to the power of w'' .
 - v. Let i_{min} be $-int'$.
 - vi. Let i' be i if i is greater than or equal to i_{min} otherwise i_{min} .
 - vii. Let i'' be the bitstring of width w representing the integer i' .
 - viii. Return $w \ S \ i''$.

19.2.5. Multiplication

$value * value'$

1. Check that $value$ has type $integerLiteral$.
2. Let $integerLiteral$ be $value$.
3. If let $D \ i_l$ be $integerLiteral$:
 - a. Check that $value'$ has type $integerLiteral$.
 - b. Let $integerLiteral'$ be $value'$.
 - c. Check that $integerLiteral'$ matches pattern $D \ %$.
 - d. Let $D \ i_r$ be $integerLiteral'$.
 - e. Return $D \ i_l * i_r$.
4. Else if let $w \ W \ i_l$ be $integerLiteral$:
 - a. Check that $value'$ has type $integerLiteral$.
 - b. Let $integerLiteral'$ be $value'$.
 - c. Check that $integerLiteral'$ matches pattern $\% \ W \ %$.

- d. Let $w' \ W \ i_r$ be `integerLiteral`.
 - e. Check that w is equal to w' .
 - f. Let i_r be the two's complement representation of bitstring i_l of width w .
 - g. Let i_r be the two's complement representation of bitstring i_r of width w .
 - h. Let i be the bitstring of width w representing the integer $i_l * i_r$.
 - i. Return $w \ W \ i$.
5. Else:
- a. Let $w \ S \ i_l$ be `integerLiteral`.
 - b. Check that `value'` has type `integerLiteral`.
 - c. Let `integerLiteral'` be `value'`.
 - d. Check that `integerLiteral'` matches pattern `% S %`.
 - e. Let $w' \ S \ i_r$ be `integerLiteral'`.
 - f. Check that w is equal to w' .
 - g. Let i_r be the two's complement representation of bitstring i_l of width w .
 - h. Let i_r be the two's complement representation of bitstring i_r of width w .
 - i. Let i be the bitstring of width w representing the integer $i_l * i_r$.
 - j. Return $w \ S \ i$.

19.2.6. Division

`value / value'`

1. Check that `value` has type `integerLiteral`.
2. Let `integerLiteral` be `value`.
3. Check that `integerLiteral` matches pattern `D %`.
4. Let $D \ i_l$ be `integerLiteral`.
5. Check that `value'` has type `integerLiteral`.
6. Let `integerLiteral'` be `value'`.
7. Check that `integerLiteral'` matches pattern `D %`.
8. Let $D \ i_r$ be `integerLiteral'`.
9. Return $D \ i_l / i_r$.

19.2.7. Modulus

`value % value'`

1. Check that `value` has type `integerLiteral`.
2. Let `integerLiteral` be `value`.
3. Check that `integerLiteral` matches pattern `D %`.

4. Let $D\ i_l$ be `integerLiteral`.
5. Check that `value'` has type `integerLiteral`.
6. Let `integerLiteral'` be `value'`.
7. Check that `integerLiteral'` matches pattern `D %`.
8. Let $D\ i_r$ be `integerLiteral'`.
9. Return $D\ i_l \setminus i_r$.

19.2.8. Shift left

`value << value'`

1. Check that `value` has type `integerLiteral`.
2. Let `integerLiteral` be `value`.
3. If let $D\ i_l$ be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. If let $D\ i_r$ be `integerLiteral'`:
 - i. Let `int` be the left shift of i_l by i_r .
 - ii. Return $D\ int$.
 - d. Else if let $w_r\ W\ i_r$ be `integerLiteral'`:
 - i. Let `int` be the left shift of i_l by i_r .
 - ii. Return $D\ int$.
 - e. Else:
 - i. Let $w_r\ S\ i_r$ be `integerLiteral'`.
 - ii. Let i_r' be the two's complement representation of bitstring i_r of width w_r .
 - iii. Let `int` be the left shift of i_l by i_r .
 - iv. Return $D\ int$.
4. Else if let $w_l\ W\ i_l$ be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. If let $D\ i_r$ be `integerLiteral'`:
 - i. Let `int` be the two's complement representation of bitstring i_r of width w_l .
 - ii. Return $w_l\ W\ i_l << w_l\ W\ int$.
 - d. Else if let $w_r\ W\ i_r$ be `integerLiteral'`:
 - i. Let `int` be the left shift of i_l by i_r .
 - ii. Let i be the bitstring of width w_l representing the integer `int`.
 - iii. Return $w_l\ W\ i$.
 - e. Else:

- i. Let $w_r \text{ S } i_r$ be `integerLiteral`'.
 - ii. Let i_r be the two's complement representation of bitstring i_r of width w_r .
 - iii. Let `int` be the left shift of i_l by i_r .
 - iv. Let i be the bitstring of width w_l representing the integer `int`.
 - v. Return $w_l \text{ W } i$.
5. Else:
- a. Let $w_l \text{ S } i_l$ be `integerLiteral`.
 - b. Check that `value'` has type `integerLiteral`.
 - c. Let `integerLiteral'` be `value'`.
 - d. If let $D \text{ } i_r$ be `integerLiteral'`:
 - i. Let `int` be the two's complement representation of bitstring i_r of width w_l .
 - ii. Return $w_l \text{ S } i_l << w_l \text{ S } \text{int}$.
 - e. Else if let $w_r \text{ W } i_r$ be `integerLiteral'`:
 - i. Let i_r be the two's complement representation of bitstring i_l of width w_l .
 - ii. Let `int` be the left shift of i_l by i_r .
 - iii. Let i be the bitstring of width w_l representing the integer `int`.
 - iv. Return $w_l \text{ S } i$.
 - f. Else:
 - i. Let $w_r \text{ S } i_r$ be `integerLiteral'`.
 - ii. Let i_r be the two's complement representation of bitstring i_l of width w_l .
 - iii. Let i_r be the two's complement representation of bitstring i_r of width w_r .
 - iv. Let `int` be the left shift of i_l by i_r .
 - v. Let i be the bitstring of width w_l representing the integer `int`.
 - vi. Return $w_l \text{ S } i$.

19.2.9. Shift right

`value >> value'`

1. Check that `value` has type `integerLiteral`.
2. Let `integerLiteral` be `value`.
3. If let $D \text{ } i_l$ be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. If let $D \text{ } i_r$ be `integerLiteral'`:
 - i. Let `int'` be the logical right shift of i_l by i_r .
 - ii. Return $D \text{ int'}$.
 - d. Else if let $w_r \text{ W } i_r$ be `integerLiteral'`:

- i. Let int' be the logical right shift of i_l by i_r .
 - ii. Return $D \text{ int}'$.
- e. Else:
 - i. Let $w_r S i_r$ be `integerLiteral`'.
 - ii. Let i_r be the two's complement representation of bitstring i_r of width w_r .
 - iii. Let int' be the logical right shift of i_l by i_r .
 - iv. Return $D \text{ int}'$.
- 4. Else if let $w_l W i_l$ be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. If let $D i_r$ be `integerLiteral'`:
 - i. Let int' be the two's complement representation of bitstring i_r of width w_l .
 - ii. Return $w_l W i_l \gg w_l W \text{int}'$.
 - d. Else if let $w_r W i_r$ be `integerLiteral'`:
 - i. Let int' be the logical right shift of i_l by i_r .
 - ii. Let i be the bitstring of width w_l representing the integer int' .
 - iii. Return $w_l W i$.
 - e. Else:
 - i. Let $w_r S i_r$ be `integerLiteral'`.
 - ii. Let i_r be the two's complement representation of bitstring i_r of width w_r .
 - iii. Let int' be the logical right shift of i_l by i_r .
 - iv. Let i be the bitstring of width w_l representing the integer int' .
 - v. Return $w_l W i$.
- 5. Else:
 - a. Let $w_l S i_l$ be `integerLiteral`.
 - b. Check that `value'` has type `integerLiteral`.
 - c. Let `integerLiteral'` be `value'`.
 - d. If let $D i_r$ be `integerLiteral'`:
 - i. Let i_r be the two's complement representation of bitstring i_l of width w_l .
 - ii. If i_r is less than 0:
 - A. Let int be $w_l - 1$.
 - B. Check that int has type `nat`.
 - C. Let w_r be int .
 - D. Let int' be 2 to the power of w_l .
 - E. Let i be the arithmetic right shift of i_l by i_r with sign extension int' .
 - F. Let i' be the bitstring of width w_l representing the integer i .
 - G. Return $w_l S i'$.
 - iii. If i_r is greater than or equal to 0:

- A. Let i be the logical right shift of i_l by i_r .
 - B. Let i' be the bitstring of width w_l representing the integer i .
 - C. Return $w_l S i'$.
- e. Else if let $w_r W i_r$ be `integerLiteral`:
- i. Let i_r be the two's complement representation of bitstring i_l of width w_l .
 - ii. If i_r is less than 0:
 - A. Let `int` be $w_l - 1$.
 - B. Check that `int` has type `nat`.
 - C. Let w_r be `int`.
 - D. Let `int'` be 2 to the power of w_r .
 - E. Let i be the arithmetic right shift of i_l by i_r with sign extension `int'`.
 - F. Let i' be the bitstring of width w_l representing the integer i .
 - G. Return $w_l S i'$.
 - iii. If i_r is greater than or equal to 0:
 - A. Let i be the logical right shift of i_l by i_r .
 - B. Let i' be the bitstring of width w_l representing the integer i .
 - C. Return $w_l S i'$.
- f. Else:
- i. Let $w_r S i_r$ be `integerLiteral`.
 - ii. Let i_r be the two's complement representation of bitstring i_l of width w_l .
 - iii. Let i_r be the two's complement representation of bitstring i_r of width w_r .
 - iv. If i_r is less than 0:
 - A. Let `int` be $w_l - 1$.
 - B. Check that `int` has type `nat`.
 - C. Let w_r be `int`.
 - D. Let `int'` be 2 to the power of w_r .
 - E. Let i be the arithmetic right shift of i_l by i_r with sign extension `int'`.
 - F. Let i' be the bitstring of width w_l representing the integer i .
 - G. Return $w_l S i'$.
 - v. If i_r is greater than or equal to 0:
 - A. Let i be the logical right shift of i_l by i_r .
 - B. Let i' be the bitstring of width w_l representing the integer i .
 - C. Return $w_l S i'$.

19.2.10. Equality and inequality

`value == value'`

1. If let ``B ba` be value:
 - a. Check that `value'` has type `boolValue`.
 - b. Let ``B bb` be `value'`.
 - c. Return `ba` is equal to `bb`.
2. Else if let `ERROR . ida` be value:
 - a. Check that `value'` has type `errorValue`.
 - b. Let `ERROR . idb` be `value'`.
 - c. Return `ida` is equal to `idb`.
3. Else if let `MATCH_KIND . ida` be value:
 - a. Check that `value'` has type `matchKindValue`.
 - b. Let `MATCH_KIND . idb` be `value'`.
 - c. Return `ida` is equal to `idb`.
4. Else if let `stringValuea` be value:
 - a. Check that `value'` has type `stringValue`.
 - b. Let `stringValueb` be `value'`.
 - c. Return `stringValuea` is equal to `stringValueb`.
5. Else if let `integerLiteral` be value:
 - a. If let `D ia` be `integerLiteral`:
 - i. Check that `value'` has type `integerLiteral`.
 - ii. Let `integerLiteral'` be `value'`.
 - iii. Check that `integerLiteral'` matches pattern `D %`.
 - iv. Let `D ib` be `integerLiteral'`.
 - v. Return `ia` is equal to `ib`.
 - b. Else if let `wa W ia` be `integerLiteral`:
 - i. Check that `value'` has type `integerLiteral`.
 - ii. Let `integerLiteral'` be `value'`.
 - iii. Check that `integerLiteral'` matches pattern `% W %`.
 - iv. Let `wb W ib` be `integerLiteral'`.
 - v. Return `wa` is equal to `wb` and `ia` is equal to `ib`.
 - c. Else:
 - i. Let `wa S ia` be `integerLiteral`.
 - ii. Check that `value'` has type `integerLiteral`.
 - iii. Let `integerLiteral'` be `value'`.
 - iv. Check that `integerLiteral'` matches pattern `% S %`.
 - v. Let `wb S ib` be `integerLiteral'`.
 - vi. Return `wa` is equal to `wb` and `ia` is equal to `ib`.
6. Else if let `LIST [ valuea* ]` be value:
 - a. Check that `value'` has type `listValue`.

- b. Let $LIST [value_b^*]$ be $value'$.
 - c. Let $bool'^*$ be the list of $bool'$ and $bool'^{**}$ be the list of $bool'^*$, obtained by repeating:
 - Let $bool'$ be $value_a == value_b$.
 for each $value_a$ in $value_a^*$ and $value_b$ in $value_b^*$
 - d. Let $bool$ be all $bool'^*$ are true.
 - e. Return the length of $value_a^*$ is equal to the length of $value_b^*$ and $bool$.
7. Else if let $TUPLE (value_a^*)$ be $value'$:
- a. Check that $value'$ has type $tupleValue$.
 - b. Let $TUPLE (value_b^*)$ be $value'$.
 - c. Let $bool'^*$ be the list of $bool'$ and $bool'^{**}$ be the list of $bool'^*$, obtained by repeating:
 - Let $bool'$ be $value_a == value_b$.
 for each $value_a$ in $value_a^*$ and $value_b$ in $value_b^*$
 - d. Let $bool$ be all $bool'^*$ are true.
 - e. Return the length of $value_a^*$ is equal to the length of $value_b^*$ and $bool$.
8. Else if let $HEADER_STACK [value_a^* (_ ; n_{size_a})]$ be $value'$:
- a. Check that $value'$ has type $headerStackValue$.
 - b. Let $HEADER_STACK [value_b^* (_ ; n_{size_b})]$ be $value'$.
 - c. Let $bool'^*$ be the list of $bool'$ and $bool'^{**}$ be the list of $bool'^*$, obtained by repeating:
 - Let $bool'$ be $value_a == value_b$.
 for each $value_a$ in $value_a^*$ and $value_b$ in $value_b^*$
 - d. Let $bool$ be all $bool'^*$ are true.
 - e. Return the length of $value_a^*$ is equal to the length of $value_b^*$ and $bool$ and n_{size_a} is equal to n_{size_b} .
9. Else if let $STRUCT\ typeId_a \{ fieldValue_a^* \}$ be $value'$:
- a. Check that $value'$ has type $structValue$.
 - b. Let $STRUCT\ typeId_b \{ fieldValue_b^* \}$ be $value'$.
 - c. Let $nameIR_{field_a}^*$ be the list of $nameIR_{field_a}$ and $value_{field_a}^*$ be the list of $value_{field_a}$, obtained by repeating:
 - Let $value_{field_a}\ nameIR_{field_a} ;$ be $fieldValue_a$.
 for each $fieldValue_a$ in $fieldValue_a^*$
 - d. Let $nameIR_{field_b}^*$ be the list of $nameIR_{field_b}$ and $value_{field_b}^*$ be the list of $value_{field_b}$, obtained by repeating:
 - Let $value_{field_b}\ nameIR_{field_b} ;$ be $fieldValue_b$.
 for each $fieldValue_b$ in $fieldValue_b^*$
 - e. Let $bool'^*$ be the list of $bool'$ and $bool'^{**}$ be the list of $bool'^*$, obtained by repeating:

- Let $bool'$ be $value_{field_a} == value_{field_b}$.
- for each $value_{field_a}$ in $value_{field_a}^*$ and $value_{field_b}$ in $value_{field_b}^*$
- f. Let $bool$ be all $bool'$ are true.
 - g. Let $bool''$ be all ($nameIR_{field_a} = nameIR_{field_b}$) are true.
 - h. Return $typeId_a$ is equal to $typeId_b$ and the length of $fieldValue_a^*$ is equal to the length of $fieldValue_b^*$ and $bool$ and $bool''$.
10. Else if let $HEADER$ $typeId_a \{ _ ; fieldValue_a^* \}$ be value:
- a. Check that $value'$ has type $headerValue$.
 - b. Let $HEADER$ $typeId_b \{ _ ; fieldValue_b^* \}$ be $value'$.
 - c. Let $nameIR_{field_a}^*$ be the list of $nameIR_{field_a}$ and $value_{field_a}^*$ be the list of $value_{field_a}$, obtained by repeating:
 - Let $value_{field_a}$ $nameIR_{field_a}$; be $fieldValue_a$.

for each $fieldValue_a$ in $fieldValue_a^*$
 - d. Let $nameIR_{field_b}^*$ be the list of $nameIR_{field_b}$ and $value_{field_b}^*$ be the list of $value_{field_b}$, obtained by repeating:
 - Let $value_{field_b}$ $nameIR_{field_b}$; be $fieldValue_b$.

for each $fieldValue_b$ in $fieldValue_b^*$
 - e. Let $bool'^*$ be the list of $bool'$ and $bool'^{**}$ be the list of $bool'^*$, obtained by repeating:
 - Let $bool'$ be $value_{field_a} == value_{field_b}$.

for each $value_{field_a}$ in $value_{field_a}^*$ and $value_{field_b}$ in $value_{field_b}^*$
 - f. Let $bool$ be all $bool'^*$ are true.
 - g. Let $bool''$ be all ($nameIR_{field_a} = nameIR_{field_b}$) are true.
 - h. Return $typeId_a$ is equal to $typeId_b$ and the length of $fieldValue_a^*$ is equal to the length of $fieldValue_b^*$ and $bool$ and $bool''$.
11. Else if let $HEADER_UNION$ $typeId_a \{ fieldValue_a^* \}$ be value:
- a. Check that $value'$ has type $headerUnionValue$.
 - b. Let $HEADER_UNION$ $typeId_b \{ fieldValue_b^* \}$ be $value'$.
 - c. Let $nameIR_{field_a}^*$ be the list of $nameIR_{field_a}$ and $value_{field_a}^*$ be the list of $value_{field_a}$, obtained by repeating:
 - Let $value_{field_a}$ $nameIR_{field_a}$; be $fieldValue_a$.

for each $fieldValue_a$ in $fieldValue_a^*$
 - d. Let $nameIR_{field_b}^*$ be the list of $nameIR_{field_b}$ and $value_{field_b}^*$ be the list of $value_{field_b}$, obtained by repeating:
 - Let $value_{field_b}$ $nameIR_{field_b}$; be $fieldValue_b$.

for each $fieldValue_b$ in $fieldValue_b^*$

- e. Let bool^* be the list of bool' and bool^{**} be the list of bool^* , obtained by repeating:
- Let bool' be $\text{value}_{\text{field}_a} == \text{value}_{\text{field}_b}$.
- for each $\text{value}_{\text{field}_a}$ in $\text{value}_{\text{field}_a}^*$ and $\text{value}_{\text{field}_b}$ in $\text{value}_{\text{field}_b}^*$
- f. Let bool be all bool^* are true.
- g. Let bool'' be all $(\text{nameIR}_{\text{field}_a} = \text{nameIR}_{\text{field}_b})^*$ are true.
- h. Return typeId_a is equal to typeId_b and the length of fieldValue_a^* is equal to the length of fieldValue_b^* and bool and bool'' .
12. Else if let enumValue be value :
- a. If let typeId_a . $\text{nameIR}_{\text{field}_a}$ be enumValue :
- Check that value' has type enumValue .
 - Let $\text{enumValue}'$ be value' .
 - Check that $\text{enumValue}'$ matches pattern $\% . \%$.
 - Let typeId_b . $\text{nameIR}_{\text{field}_b}$ be $\text{enumValue}'$.
 - Return typeId_a is equal to typeId_b and $\text{nameIR}_{\text{field}_a}$ is equal to $\text{nameIR}_{\text{field}_b}$.
- b. Else:
- Let typeId_a . $_ . \text{value}_{\text{field}_a}$ be enumValue .
 - Check that value' has type enumValue .
 - Let $\text{enumValue}'$ be value' .
 - Check that $\text{enumValue}'$ matches pattern $\% . \% . \%$.
 - Let typeId_b . $_ . \text{value}_{\text{field}_b}$ be $\text{enumValue}'$.
 - Let bool be $\text{value}_{\text{field}_a} == \text{value}_{\text{field}_b}$.
 - Return typeId_a is equal to typeId_b and bool .
13. Else if let $\text{invalidHeaderValue}$ be value :
- Check that value' has type $\text{invalidHeaderValue}$.
 - Let $\text{invalidHeaderValue}'$ be value' .
 - Return true.
14. If let integerValue be value :
- Check that integerValue matches pattern $\% . \% V \%$.
 - Let $w_{\text{max}_a} . _ V i_a$ be integerValue .
 - Check that value' has type integerValue .
 - Let $\text{integerValue}'$ be value' .
 - Check that $\text{integerValue}'$ matches pattern $\% . \% V \%$.
 - Let $w_{\text{max}_b} . _ V i_b$ be $\text{integerValue}'$.
 - Return w_{max_a} is equal to w_{max_b} and i_a is equal to i_b .

$\text{value}_l \neq \text{value}_r$

- Let bool be $\text{value}_l == \text{value}_r$.

2. Return $\sim\text{bool}$.

19.2.11. Comparisons

$\text{value} < \text{value}'$

1. Check that value has type `integerLiteral`.
2. Let `integerLiteral` be value .
3. If let $D\ i_l$ be `integerLiteral`:
 - a. Check that value' has type `integerLiteral`.
 - b. Let `integerLiteral'` be value' .
 - c. Check that `integerLiteral'` matches pattern $D\ \%$.
 - d. Let $D\ i_r$ be `integerLiteral'`.
 - e. Return i_l is less than i_r .
4. Else if let $w\ W\ i_l$ be `integerLiteral`:
 - a. Check that value' has type `integerLiteral`.
 - b. Let `integerLiteral'` be value' .
 - c. Check that `integerLiteral'` matches pattern $\%\ W\ \%$.
 - d. Let $w'\ W\ i_r$ be `integerLiteral'`.
 - e. Check that w is equal to w' .
 - f. Return i_l is less than i_r .
5. Else:
 - a. Let $w\ S\ i_l$ be `integerLiteral`.
 - b. Check that value' has type `integerLiteral`.
 - c. Let `integerLiteral'` be value' .
 - d. Check that `integerLiteral'` matches pattern $\% \ S\ \%$.
 - e. Let $w'\ S\ i_r$ be `integerLiteral'`.
 - f. Check that w is equal to w' .
 - g. Let i_r be the two's complement representation of bitstring i_l of width w .
 - h. Let i_r be the two's complement representation of bitstring i_r of width w .
 - i. Return i_l is less than i_r .

$\text{value} \leq \text{value}'$

1. Check that value has type `integerLiteral`.
2. Let `integerLiteral` be value .
3. If let $D\ i_l$ be `integerLiteral`:
 - a. Check that value' has type `integerLiteral`.
 - b. Let `integerLiteral'` be value' .

- c. Check that `integerLiteral'` matches pattern `D %`.
 - d. Let `D ir` be `integerLiteral'`.
 - e. Return `ii` is less than or equal to `ir`.
4. Else if let `w W ii` be `integerLiteral`:
- a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `% W %`.
 - d. Let `w' W ir` be `integerLiteral'`.
 - e. Check that `w` is equal to `w'`.
 - f. Return `ii` is less than or equal to `ir`.
5. Else:
- a. Let `w S ii` be `integerLiteral`.
 - b. Check that `value'` has type `integerLiteral`.
 - c. Let `integerLiteral'` be `value'`.
 - d. Check that `integerLiteral'` matches pattern `% S %`.
 - e. Let `w' S ir` be `integerLiteral'`.
 - f. Check that `w` is equal to `w'`.
 - g. Let `ir` be the two's complement representation of bitstring `ii` of width `w`.
 - h. Let `ir` be the two's complement representation of bitstring `ir` of width `w`.
 - i. Return `ii` is less than or equal to `ir`.

`value > value'`

- 1. Check that `value` has type `integerLiteral`.
- 2. Let `integerLiteral` be `value`.
- 3. If let `D ii` be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `D %`.
 - d. Let `D ir` be `integerLiteral'`.
 - e. Return `ii` is greater than `ir`.
- 4. Else if let `w W ii` be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `% W %`.
 - d. Let `w' W ir` be `integerLiteral'`.
 - e. Check that `w` is equal to `w'`.
 - f. Return `ii` is greater than `ir`.

5. Else:

- a. Let $w \ S \ i_l$ be `integerLiteral`.
- b. Check that `value'` has type `integerLiteral`.
- c. Let `integerLiteral'` be `value'`.
- d. Check that `integerLiteral'` matches pattern `% S %`.
- e. Let $w' \ S \ i_r$ be `integerLiteral'`.
- f. Check that w is equal to w' .
- g. Let i_r be the two's complement representation of bitstring i_l of width w .
- h. Let i_r be the two's complement representation of bitstring i_r of width w .
- i. Return i_r is greater than i_r .

`value >= value'`

1. Check that `value` has type `integerLiteral`.
2. Let `integerLiteral` be `value`.
3. If let $D \ i_l$ be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `D %`.
 - d. Let $D \ i_r$ be `integerLiteral'`.
 - e. Return i_l is greater than or equal to i_r .
4. Else if let $w \ W \ i_l$ be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `% W %`.
 - d. Let $w' \ W \ i_r$ be `integerLiteral'`.
 - e. Check that w is equal to w' .
 - f. Return i_l is greater than or equal to i_r .

5. Else:

- a. Let $w \ S \ i_l$ be `integerLiteral`.
- b. Check that `value'` has type `integerLiteral`.
- c. Let `integerLiteral'` be `value'`.
- d. Check that `integerLiteral'` matches pattern `% S %`.
- e. Let $w' \ S \ i_r$ be `integerLiteral'`.
- f. Check that w is equal to w' .
- g. Let i_r be the two's complement representation of bitstring i_l of width w .
- h. Let i_r be the two's complement representation of bitstring i_r of width w .
- i. Return i_r is greater than or equal to i_r .

19.2.12. Bitwise

value & value'

1. Check that `value` has type `integerLiteral`.
2. Let `integerLiteral` be `value`.
3. If let `w W il` be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `% W %`.
 - d. Let `w' W ir` be `integerLiteral'`.
 - e. Check that `w` is equal to `w'`.
 - f. Let `int` be the bitwise and of `il` and `ir`.
 - g. Let `i` be the bitstring of width `w` representing the integer `int`.
 - h. Return `w W i`.
4. Else if let `w S il` be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `% S %`.
 - d. Let `w' S ir` be `integerLiteral'`.
 - e. Check that `w` is equal to `w'`.
 - f. Let `ir` be the two's complement representation of bitstring `il` of width `w`.
 - g. Let `ir` be the two's complement representation of bitstring `ir` of width `w`.
 - h. Let `int` be the bitwise and of `il` and `ir`.
 - i. Let `i` be the bitstring of width `w` representing the integer `int`.
 - j. Return `w S i`.

value | value'

1. Check that `value` has type `integerLiteral`.
2. Let `integerLiteral` be `value`.
3. If let `w W il` be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `% W %`.
 - d. Let `w' W ir` be `integerLiteral'`.
 - e. Check that `w` is equal to `w'`.
 - f. Let `int` be the bitwise or of `il` and `ir`.
 - g. Let `i` be the bitstring of width `w` representing the integer `int`.

- h. Return $w \ W \ i$.
- 4. Else if let $w \ S \ i_l$ be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `% S %`.
 - d. Let $w' \ S \ i_r$ be `integerLiteral'`.
 - e. Check that w is equal to w' .
 - f. Let i_l be the two's complement representation of bitstring i_l of width w .
 - g. Let i_r be the two's complement representation of bitstring i_r of width w .
 - h. Let `int` be the bitwise or of i_l and i_r .
 - i. Let i be the bitstring of width w representing the integer `int`.
 - j. Return $w \ S \ i$.

`value ^ value'`

- 1. Check that `value` has type `integerLiteral`.
- 2. Let `integerLiteral` be `value`.
- 3. If let $w \ W \ i_l$ be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `% W %`.
 - d. Let $w' \ W \ i_r$ be `integerLiteral'`.
 - e. Check that w is equal to w' .
 - f. Let `int` be the bitwise xor of i_l and i_r .
 - g. Let i be the bitstring of width w representing the integer `int`.
 - h. Return $w \ W \ i$.
- 4. Else if let $w \ S \ i_l$ be `integerLiteral`:
 - a. Check that `value'` has type `integerLiteral`.
 - b. Let `integerLiteral'` be `value'`.
 - c. Check that `integerLiteral'` matches pattern `% S %`.
 - d. Let $w' \ S \ i_r$ be `integerLiteral'`.
 - e. Check that w is equal to w' .
 - f. Let i_l be the two's complement representation of bitstring i_l of width w .
 - g. Let i_r be the two's complement representation of bitstring i_r of width w .
 - h. Let `int` be the bitwise xor of i_l and i_r .
 - i. Let i be the bitstring of width w representing the integer `int`.
 - j. Return $w \ S \ i$.

19.3. Default values

Some P4 types define a *default value*, which can be used to automatically initialize values of that type. The high-level description of default values are as follows:

- For `int`, `bit<N>` and `int<N>` types the default value is 0.
- For `bool` the default value is `false`.
- For `error` the default value is `error.NoError` (defined in `core.p4`)
- For `string` the default value is the empty string `""`
- For `varbit<N>` the default value is a string of zero bits (there is currently no P4 literal to represent such a value).
- For `enum` values with an underlying type the default value is 0, even if 0 is actually not one of the named values in the enum.
- For `enum` values without an underlying type the default value is the first value that appears in the `enum` type declaration.
- For `header` types the default value is `invalid`.
- For header stacks the default value is that all elements are invalid and the `nextIndex` is 0.
- For `header_union` values the default value is that all union elements are invalid.
- For `struct` types the default value is a `struct` where each field has the default value of the suitable field type—`if all such default values are defined`.
- For a `tuple` type the default value is a `tuple` where each field has the default value of the suitable type—`if all such default values are defined`.

Note that some types do not have default values, e.g., `match_kind`, set types, function types, extern types, parser types, control types, and package types.

Default values are derived from:

`$default:`

- The default value for type `typeIR`

The default value for type `typeIR`

1. If let `boolTypeIR` be `typeIR`:
 - a. Return ``B false`.
2. Else if let `errorTypeIR` be `typeIR`:
 - a. Return `ERROR . "NoError"`.
3. Else if let `stringTypeIR` be `typeIR`:
 - a. Let `textempty` be `""`.
 - b. Return `" textempty "`.
4. Else if let `intTypeIR` be `typeIR`:
 - a. Return `D 0`.
5. Else if let `BIT < w >` be `typeIR`:

- a. Return `w W 0`.
6. Else if let `INT < w >` be `typeIR`:
 - a. Return `w S 0`.
7. Else if let `VARBIT < w >` be `typeIR`:
 - a. Return `w . 0 V 0`.
8. Else if let `TYPEDEF _ typeIR'` be `typeIR`:
 - a. Return `the default value for type typeIR'`.
9. Else if let `TYPE _ typeIR'` be `typeIR`:
 - a. Return `the default value for type typeIR'`.
10. Else if let `TUPLE < typeIR* >` be `typeIR`:
 - a. Let `value*` be the list of `value'`, obtained by repeating:
 - Let `value'` be `the default value for type typeIR'`.
 for each `typeIR'` in `typeIR*`
 - b. Return `TUPLE ( value* )`.
11. Else if let `typeIR' [ ns ]` be `typeIR`:
 - a. Let `value'` be `the default value for type typeIR'`.
 - b. Let `value*` be `a list of ns copies of value'`.
 - c. Return `HEADER_STACK [ value* ( 0 ; ns ) ]`.
12. Else if let `STRUCT typeId < _ * > { ( _ typeIRf idf ; ) * }` be `typeIR`:
 - a. Let `value*` be the list of `value'`, obtained by repeating:
 - Let `value'` be `the default value for type typeIRf`.
 for each `typeIRf` in `typeIRf*`
 - b. Return `STRUCT typeId { ( value' idf ; ) * }`.
13. Else if let `HEADER typeId < _ * > { ( _ typeIRf idf ; ) * }` be `typeIR`:
 - a. Let `value*` be the list of `value'`, obtained by repeating:
 - Let `value'` be `the default value for type typeIRf`.
 for each `typeIRf` in `typeIRf*`
 - b. Return `HEADER typeId { false ; ( value' idf ; ) * }`.
14. Else if let `HEADER_UNION typeId < _ * > { ( _ typeIRf idf ; ) * }` be `typeIR`:
 - a. Let `value*` be the list of `value'`, obtained by repeating:
 - Let `value'` be `the default value for type typeIRf`.
 for each `typeIRf` in `typeIRf*`
 - b. Return `HEADER_UNION typeId { ( value' idf ; ) * }`.
15. Else if let `enumTypeIR` be `typeIR`:

- a. If let `ENUM typeld { id* }` be `enumTypeIR`:
 - i. Check that `id*` is a non-empty list.
 - ii. Let `idf_h :: *` be `id*`.
 - iii. Return `typeld . idf_h`.
- b. Else:
 - i. Let `ENUM typeld < typeIR' > { ( nameIRfield = valuefield ; )* }` be `enumTypeIR`.
 - ii. Let `valuezero` be `D 0` **casted to type** `typeIR'`.
 - iii. Let `id?` be **the value associated to** `valuezero` **in** `( valuefield, nameIRfield )*`.
 - iv. If let `idzero` be `id?`:
 - A. Return `typeld . idzero . valuezero`.
 - v. If `·` is equal to **the value associated to** `valuezero` **in** `( valuefield, nameIRfield )*`:
 - A. Let `idzero` be `"__UNSPECIFIED"`.
 - B. Return `typeld . idzero . valuezero`.

20. Annotations

```

annotationToken
: UNEXPECTED_TOKEN
| ABSTRACT
| ACTION
| ACTIONS
| APPLY
| BOOL
| BIT
| BREAK
| CONST
| CONTINUE
| CONTROL
| DEFAULT
| ELSE
| ENTRIES
| ENUM
| ERROR
| EXIT
| EXTERN
| FALSE
| FOR
| HEADER
| HEADER_UNION
| IF
| IN
| INOUT
| INT
| KEY
| MATCH_KIND
| TYPE
| OUT
| PARSER
| PACKAGE
| PRAGMA
| RETURN
| SELECT
| STATE
| STRING
| STRUCT

```

```

| SWITCH
| TABLE
| THIS
| TRANSITION
| TRUE
| TUPLE
| TYPEDEF
| VARBIT
| VALUE_SET
| LIST
| VOID
|
| _
| identifier
| typeIdentifier
| stringLiteral
| integerLiteral
| &&&
|
| ..
| <<
| &&
| ||
| ==
| !=
| >=
| <=
| ++
| +
| |+|
| -
| |-|
| *
| /
| %
| |
| &
| ^
| ~
| [
| ]
| {
| }
| <
| >
| !
| :
| ,
| ?
| .
| =
| ;
| @
|
;

```

```

annotationBody
: /* empty */
| annotationBody `( annotationBody )
| annotationBody annotationToken
;

```

```

structuredAnnotationBody
: sequenceOrRecordElementExpression trailingCommaOpt
;

```

```

annotation
: @ name
| @ name `( annotationBody )
| @ name `[ structuredAnnotationBody ]
| @ PRAGMA name annotationBody

```

Appendix A: Revision History

A.1. Summary of changes made in unreleased version

- Clarified that numeric priorities cannot be assigned to entries of a table that has `const entries` ([[sec-entries](#)]).
- Clarified that `switch` statements are allowed in action and function bodies, and that `switch` statements with `action_run` expressions are only allowed in control `apply` blocks ([[sec-stmts](#)] and [[sec-switch-stmt](#)]).
- Added standard branch prediction annotations ([[sec-branch-annotations](#)])
- Added support for compile-time string concatenation using `++` operator ([[sec-string-type](#)] and [[sec-string-ops](#)]).
- Added compound assignment statements ([[sec-assignment](#)])
- Update the specification grammar for `for` loops to match the grammar used in the `p4c` implementation ([[sec-loop-stmt](#)]).

A.2. Summary of changes made in version 1.2.5, released October 11, 2024

- Improved type nesting rules ([[sec-type-nesting](#)]).
- Clarified that directionless extern parameters are passed by reference.
- Introduced distinction between local compile-time known and compile-time known values ([Section 7.5](#)).

A.3. Summary of changes made in version 1.2.4, released May 15, 2023

- Added header stack expressions ([[sec-hs-init](#)]).
- Allow casts from a type to itself ([[sec-casts](#)]).
- Added an invalid header or header union expression `{#}` ([[sec-ops-on-hdrs](#)] and [[sec-expr-hu](#)]).
- Added a concept of numeric values ([[sec-numeric-values](#)]).
- Added a section on operations on extern objects ([[sec-extern-operations](#)]).
- Added note in sections operations on types for types that support compile-time size determination.
- Clarified that header stacks are arrays of headers or header unions.
- Added distinctness of fields for types that have fields including error, match kind, struct, header, and header union.
- Clarified types `bit<W>`, `int<W>`, and `varbit<W>` encompass the case where the width is a compile-time known expression evaluating to an appropriate integer ([[sec-unsigned-integers](#)], [[sec-signed-integers](#)], [[sec-dynamically-sized-integers](#)]).
- Clarified restrictions for parameters with default values ([[sec-calling-convention](#)]).
- Added optional trailing commas ([[sec-trailing-commas](#)]).

- Clarified the scope of parser namespaces ([sec-parser-decl]).
- Specified that algorithm for generating control-plane names for keys is optional ([sec-cp-keys]).
- Clarified types of expressions that may appear in `select` ([sec-select]).
- Added description of semantics of the core.p4 match kinds ([sec-table-keys]).
- Explicitly disallow overloading of parsers, controls, and packages ([sec-extern-objects]).
- Clarified implicit casts present in select expressions ([sec-select]).
- Clarified that slices can be applied to arbitrary-precision integers ([sec-varint-ops]).
- Clarified that direct invocation is not possible for objects that have constructor arguments ([sec-direct-invocation]).
- Added comparison for tuples as a legal operation ([sec-tuple-exprs]).
- Clarified the behavior of `lookahead` on header-typed values ([sec-packet-lookahead]).
- Added `static_assert` function ([sec-static-assert]).
- Clarified semantics of ranges where the start is bigger than the end ([sec-ranges]).
- Allow ranges to be specified by serializable enums ([sec-ranges]).
- Specified type produced by the `*sizeInB*` methods ([sec-minsizeinbits]).
- Added section with operations on `match_kind` values ([sec-match-kind-exprs]).
- Renamed infinite-precision integers to arbitrary-precision integers ([sec-arbitrary-precision-integers]).
- compiler-inserted `default_action` is not `const` ([sec-tables]).
- Clarified the restrictions on run time for tables with `const` entries ([sec-entries]).
- renamed list expressions to tuple expressions
- Added `list` type ([sec-list-types]).
- Defined `entries` table property without `const`, for entries installed when the P4 program is loaded, but the control plane can later change them or add to them ([sec-entries]).
- Clarified behavior of table with no `key` property, or if its list of keys is empty ([sec-table-keys]).

A.4. Summary of changes made in version 1.2.3, released July 11, 2022.

- Extended `minSizeInBits` and `minSizeInBytes` to apply to more expressions ([sec-minsizeinbits]).
- Added support for `maxSizeInBits` and `maxSizeInBytes` ([sec-minsizeinbits]).
- Added support for empty lists of `const` entries in tables ([sec-entries]).
- Added support for `switch` statements in actions ([sec-actions]).
- Added support for direct invocation of controls and parsers ([sec-parameterization]).
- Added parser `value_set` to list of control-plane visible names ([sec-cp-names]).
- Added `match_kind` as a base type ([sec-match-kind-type]).
- Removed structure initializers as they are subsumed by structure-valued expressions ([sec-structure-expressions]).
- Specified operations on values typed as type variables ([sec-type-variable-operations]).
- Clarified semantics of compile-time known values (Section 7.5).
- Clarified semantics of directionless parameters ([sec-calling-convention]).

- Clarified semantics of arbitrary precision integers ([sec-arbitrary-precision-integers]).
- Clarified semantics of bit slices, shifts, and concatenation ([sec-bit-ops]).
- Clarified semantics of optional parameters ([sec-optional-parameters]).
- Clarified restrictions on extern method and function invocation ([sec-calling-restrictions]).
- Clarified semantics of implicit casts ([sec-implicit-casts]).

A.5. Summary of changes made in version 1.2.2, released May 17, 2021

- Added support for accessing tuple fields ([sec-tuple-exprs]).
- Added support for generic structures ([sec-type-spec]).
- Added support for integers, `enums`, and `errors` in `switch` statements ([sec-switch-stmt]).
- Added support for additional enumeration types ([sec-enum-types]).
- Added support for abstract methods ([sec-abstract-methods]).
- Added support for conditional statements and empty statements in parsers ([sec-parser-state-stmt]).
- Added support for casts from `int` to `bool` ([sec-casts]).
- Added support for 0-width bitstrings and varbits ([sec-uninitialized-values-and-writing-invalid-headers]).
- Clarified that `default_action` is `NoAction` if otherwise unspecified ([sec-tables]).
- Clarified the types of expressions that may be used as indexes for header stacks ([sec-expr-hs]).
- Clarified representation of Booleans in headers ([sec-header-types]).
- Clarified representation of empty types ([sec-uninitialized-values-and-writing-invalid-headers]).
- Clarified that action data can be specified by the control plane, `default_action` table property, or `const entries` table property ([sec-actions]).
- Fixed several typos and inconsistencies in grammar (Appendix E).
- Eliminated annotations on `const` entries in grammar (Appendix E).

A.6. Summary of changes made in version 1.2.1, released June 11, 2020

- Added structure-value expressions ([sec-structure-expressions]).
- Added support for default values ([sec-default-values]).
- Added support for concatenating signed strings ([sec-concatenation]).
- Added key-value and list-structured annotations ([sec-annotations]).
- Added `@pure` and `@noSideEffects` annotations ([sec-extern-annotations]).
- Added `@noWarn` annotation ([sec-nowarn-anno]).
- Generalized typing for masks to allow serializable `enums` ([sec-cubes]).
- Restricted the right operands of bit shifts involving arbitrary-precision integers to be constant and positive ([sec-varint-ops]).
- Clarified copy-out behavior for `return` ([sec-return-stmt]) and `exit` ([sec-exit-stmt]) statements.
- Clarified semantics of invalid header stacks ([sec-uninitialized-values-and-writing-invalid-headers]).

- Clarified initialization semantics ([[sec-lvalues](#)] and [[sec-calling-convention](#)]), especially for headers and local variables.
- Clarified evaluation order for table keys ([[sec-mau-semantics](#)]).
- Fixed grammar to clarify parsing of right shift operator (`>>`), allow empty statements in parser ([[sec-parser-state-stmt](#)]), and eliminate annotations on const entries ([[sec-entries](#)]).

A.7. Summary of changes made in version 1.2.0, released October 14, 2019

- Added `table.apply().miss` ([[sec-invoke-mau](#)]).
- Added `string` type ([[sec-string-type](#)]).
- Added implicit casts from enum values ([[sec-enum-exprs](#)]).
- Allow 1-bit signed values
- Define the type of bit slices from signed and unsigned values to be unsigned.
- Constrain `default` label position for `switch` statements.
- Allow empty tuples.
- Added `@deprecated` annotation.
- Relaxed the structure of annotation bodies.
- Removed the `@pkginfo` annotation, which is now defined by the P4Runtime specification.
- Added `int` type ([[sec-arbitrary-precision-integers](#)]).
- Added error `ParserInvalidArgument` ([[sec-packet-extract-two](#)], [[sec-skip-bits](#)]).
- Clarified the significance of order of entries in `const entries` ([[sec-entries](#)]).
- Added methods to calculate header size ([[sec-ops-on-hdrs](#)]).

A.8. Summary of changes made in version 1.1.0, released November 26, 2017.

- Top-level functions ([[sec-functions](#)])
 - Functions may be declared at the top-level of a P4 program.
- Optional and named parameters ([[sec-calling-convention](#)])
 - Parameters may be specified by name, with a default value, or designated as optional.
- `enum` representations ([[sec-enum-exprs](#)])
 - `enum` values to be specified with a concrete representation.
- Parser values sets ([[sec-value-set](#)])
 - `value_set` objects for control-plane programmable `select` labels.
- Type definitions ([Section 8.5.8](#))
 - New types may be introduced in programs.
- Saturating arithmetic ([[sec-bit-ops](#)])
 - Saturating arithmetic is supported on some targets.
- Structured annotations ([[sec-annotations](#)])

- Annotations may be specified as lists of key-value pairs
- Globalname ([[sec-name-annotations](#)])
 - The reserved `globalname` annotation has been removed.
- Table `size` property ([[sec-size-table-property](#)])
 - Meaning of optional `size` property for tables has been defined.
- Invalid headers ([[sec-ops-on-hdrs](#)])
 - Clarified semantics of operations on invalid headers.
- Calling restrictions ([[sec-calling-restrictions](#)])
 - Added restrictions on kinds of values that may be passed as arguments to calls.
- Bitwise operator precedence ([Appendix E](#))
 - Modified precedence conventions so that bitwise operators `&` `|` and `^` have higher precedence than relation operators `<` `>` `<=` `>=`.
- Computed bitwidths ([[sec-base-types](#)])
 - Added support for specifying widths using expressions in `bit` and `varbit` types.

A.9. Initial version 1.0.0, released May 17, 2017

Appendix B: P4 reserved keywords

The following table shows all P4 reserved keywords. Some identifiers are treated as keywords only in specific contexts (e.g., the keyword `actions`).

<code>abstract</code>	<code>action</code>	<code>apply</code>	<code>bit</code>
<code>bool</code>	<code>const</code>	<code>control</code>	<code>default</code>
<code>else</code>	<code>enum</code>	<code>error</code>	<code>extern</code>
<code>exit</code>	<code>false</code>	<code>header</code>	<code>header_union</code>
<code>if</code>	<code>in</code>	<code>inout</code>	<code>int</code>
<code>list</code>	<code>match_kind</code>	<code>package</code>	<code>parser</code>
<code>out</code>	<code>return</code>	<code>select</code>	<code>state</code>
<code>string</code>	<code>struct</code>	<code>switch</code>	<code>table</code>
<code>this</code>	<code>transition</code>	<code>true</code>	<code>tuple</code>
<code>type</code>	<code>typedef</code>	<code>value_set</code>	<code>varbit</code>
<code>verify</code>	<code>void</code>		

Appendix C: P4 reserved annotations

The following table shows all P4 reserved annotations.

Annotation	Purpose	See Section
<code>atomic</code>	specify atomic execution	[sec-concurrency]
<code>defaultonly</code>	action can only appear in the default action	[sec-annotations]

Annotation	Purpose	See Section
<code>hidden</code>	hides a controllable entity from the control plane	[sec-name-annotations]
<code>match</code>	specify <code>match_kind</code> of a field in a <code>value_set</code>	[sec-value-set]
<code>name</code>	assign local control-plane name	[sec-name-annotations]
<code>optional</code>	parameter is optional	[sec-optional-parameters]
<code>tableonly</code>	action cannot be a <code>default_action</code>	[sec-annotations]
<code>deprecated</code>	Construct has been deprecated	[sec-deprecated-anno]
<code>pure</code>	pure function	[sec-extern-annotations]
<code>noSideEffects</code>	function with no side effects	[sec-extern-annotations]
<code>noWarn</code>	Has a string argument; inhibits compiler warnings	[sec-nowarn-anno]

Appendix D: P4 core library

The P4 core library contains declarations that are useful to most programs.

For example, the core library includes the declarations of the predefined `packet_in` and `packet_out` extern objects, used in parsers and deparsers to access packet data.

```

/// Standard error codes. New error codes can be declared by users.
error {
    NoError,           /// No error.
    PacketTooShort,    /// Not enough bits in packet for 'extract'.
    NoMatch,           /// 'select' expression has no matches.
    StackOutOfBounds,  /// Reference to invalid element of a header stack.
    HeaderTooShort,    /// Extracting too many bits into a varbit field.
    ParserTimeout,     /// Parser execution time limit exceeded.
    ParserInvalidArgument /// Parser operation was called with a value
                        /// not supported by the implementation.
}

extern packet_in {
    /// Read a header from the packet into a fixed-sized header @hdr
    /// and advance the cursor.
    /// May trigger error PacketTooShort or StackOutOfBounds.
    /// @T must be a fixed-size header type
    void extract<T>(out T hdr);
    /// Read bits from the packet into a variable-sized header @variableSizeHeader
    /// and advance the cursor.
    /// @T must be a header containing exactly 1 varbit field.
    /// May trigger errors PacketTooShort, StackOutOfBounds, or HeaderTooShort.
    void extract<T>(out T variableSizeHeader,
                    in bit<32> variableFieldSizeInBits);
    /// Read bits from the packet without advancing the cursor.
    /// @returns: the bits read from the packet.
    /// T may be an arbitrary fixed-size type.
    T lookahead<T>();
    /// Advance the packet cursor by the specified number of bits.
    void advance(in bit<32> sizeInBits);
    /// @return packet length in bytes. This method may be unavailable on
    /// some target architectures.
    bit<32> length();

```

```

}
extern packet_out {
    /// Write @data into the output packet, skipping invalid headers
    /// and advancing the cursor
    /// @T can be a header type, a header stack, a header_union, or a struct
    /// containing fields with such types.
    void emit<T>(in T data);
}
action NoAction() {}
/// Standard match kinds for table key fields.
/// Some architectures may not support all these match kinds.
/// Architectures can declare additional match kinds.
match_kind {
    /// Match bits exactly.
    exact,
    /// Ternary match, using a mask.
    ternary,
    /// Longest-prefix match.
    lpm
}

/// Static assert evaluates a boolean expression
/// at compilation time. If the expression evaluates to
/// false, compilation is stopped and the corresponding message is printed.
/// The function returns a boolean, so that it can be used
/// as a global constant value in a program, e.g.:
/// const version = static_assert(V1MODEL_VERSION > 20180000, "Expected a v1 model
version >= 20180000");
extern bool static_assert(bool check, string message);

/// Like the above but using a default message.
extern bool static_assert(bool check);

```

Appendix E: P4 grammar

This is the grammar of P4₁₆ written using the YACC/bison language. Absent from this grammar is the precedence of various operations.

The grammar is actually ambiguous, so the lexer and the parser must collaborate for parsing the language. In particular, the lexer must be able to distinguish two kinds of identifiers:

- Type names previously introduced (**TYPE_IDENTIFIER** tokens)
- Regular identifiers (**IDENTIFIER** token)

The parser has to use a symbol table to indicate to the lexer how to parse subsequent appearances of identifiers. For example, given the following program fragment:

```

typedef bit<4> t;
struct s { /* body omitted */}
t x;
parser p(bit<8> b) { /* body omitted */ }

```

The lexer has to return the following terminal kinds:

```

t - TYPE_IDENTIFIER
s - TYPE_IDENTIFIER
x - IDENTIFIER
p - TYPE_IDENTIFIER
b - IDENTIFIER

```

This grammar has been heavily influenced by limitations of the Bison parser generator tool.

The **STRING_LITERAL** token corresponds to a string literal enclosed within double quotes, as described in [Section 6.2.2.3](#).

All other terminals are uppercase spellings of the corresponding keywords. For example, **RETURN** is the terminal returned by the lexer when parsing the keyword return.

```
trailingCommaOpt
: /* empty */
| ,
;

booleanLiteral
: TRUE
| FALSE
;

integerLiteral
: D int
| nat W int
| nat S int
;

stringLiteral
: " text "
;

identifier
: `ID text
;

typeIdentifier
: `TID text
;

nonTypeName
: identifier
| APPLY
| KEY
| ACTIONS
| STATE
| ENTRIES
| TYPE
| PRIORITY
;

prefixedNonTypeName
: nonTypeName
| `ID . nonTypeName
;

typeName = typeIdentifier

prefixedTypeName
: typeName
| `TID . typeName
;

tableCustomName
: identifier
| typeIdentifier
| APPLY
| STATE
| TYPE
| PRIORITY
;
```

```

name
: nonTypeName
| typeName
| LIST
;

nameList
: name
| nameList , name
;

member = name

direction
: /* empty */
| IN
| OUT
| INOUT
;

baseType
: BOOL
| ERROR
| MATCH_KIND
| STRING
| INT
| INT `< int >
| INT `< `( expression ) >
| BIT
| BIT `< int >
| BIT `< `( expression ) >
| VARBIT `< int >
| VARBIT `< `( expression ) >
;

specializedType
: prefixedTypeName `< typeArgumentList >
;

namedType
: prefixedTypeName
| specializedType
;

headerStackType
: namedType `[ expression ]
;

listType
: LIST `< typeArgument >
;

tupleType
: TUPLE `< typeArgumentList >
;

type
: baseType
| namedType
| headerStackType
| listType
| tupleType
;

typeOrVoid
: type
| VOID
| identifier

```

```

;

typeParameter = name

typeParameterList
: typeParameter
| typeParameterList , typeParameter
;

typeParameterListOpt
: /* empty */
| `< typeParameterList >
;

parameter
: annotationList direction type name initializerOpt
;

nonEmptyParameterList
: parameter
| nonEmptyParameterList , parameter
;

parameterList
: /* empty */
| nonEmptyParameterList
;

constructorParameter = parameter

constructorParameterList = parameterList

constructorParameterListOpt
: /* empty */
| `( parameterList )
;

namedExpression
: name = expression
;

namedExpressionList
: namedExpression
| namedExpressionList , namedExpression
;

literalExpression
: booleanLiteral
| integerLiteral
| stringLiteral
;

referenceExpression
: prefixedNonTypeName
| THIS
;

defaultExpression
: ...
;

unop
: !
| ~
| -
| +
;

unaryExpression

```



```

: unop expression
;

binop
: *
| /
| %
| +
| -
| |+|
| |-|
| <<
| >>
| <=
| >=
| <
| >
| !=
| ==
| &
| ^
| |
| ++
| &&
| ||
;

binaryExpression
: expression binop expression
;

binaryExpressionNonBrace
: expressionNonBrace binop expression
;

ternaryExpression
: expression ? expression : expression
;

ternaryExpressionNonBrace
: expressionNonBrace ? expression : expression
;

castExpression
: `( type ) expression
;

dataExpression
: invalidHeaderExpression
| sequenceOrRecordExpression
;

errorAccessExpression
: ERROR . member
;

memberAccessExpression
: memberAccessBase . member
;

indexAccessExpression
: expression `[ expression ]
;

sliceAccessExpression
: expression `[ expression : expression ]
;

accessExpression

```

```

: errorAccessExpression
| memberAccessExpression
| indexAccessExpression
| sliceAccessExpression
;

memberAccessExpressionNonBrace
: memberAccessBaseNonBrace . member
;

indexAccessExpressionNonBrace
: expressionNonBrace `[ expression ]
;

sliceAccessExpressionNonBrace
: expressionNonBrace `[ expression : expression ]
;

accessExpressionNonBrace
: errorAccessExpression
| memberAccessExpressionNonBrace
| indexAccessExpressionNonBrace
| sliceAccessExpressionNonBrace
;

callExpression
: callTarget `( argumentList )
| callableTarget `< realTypeArgumentList > `( argumentList )
;

callExpressionNonBrace
: callTargetNonBrace `( argumentList )
| callableTargetNonBrace `< realTypeArgumentList > `( argumentList )
;

parenthesizedExpression
: `( expression )
;

expression
: literalExpression
| referenceExpression
| defaultExpression
| unaryExpression
| binaryExpression
| ternaryExpression
| castExpression
| dataExpression
| accessExpression
| callExpression
| parenthesizedExpression
;

expressionList
: /* empty */
| expression
| expressionList , expression
;

memberAccessBase
: prefixedTypeName
| expression
;

sequenceElementExpression = expressionList

recordElementExpression
: name = expression
| name = expression , ...

```

```

| name = expression , namedExpressionList
| name = expression , namedExpressionList , ...
;

sequenceOrRecordElementExpression
: sequenceElementExpression
| recordElementExpression
;

callableTarget = expression

constructorTarget = namedType

callTarget
: callableTarget
| constructorTarget
;

expressionNonBrace
: literalExpression
| referenceExpression
| unaryExpression
| binaryExpressionNonBrace
| ternaryExpressionNonBrace
| castExpression
| accessExpressionNonBrace
| callExpressionNonBrace
| parenthesizedExpression
;

memberAccessBaseNonBrace
: prefixedTypeName
| expressionNonBrace
;

callableTargetNonBrace = expressionNonBrace

callTargetNonBrace
: callableTargetNonBrace
| constructorTarget
;

simpleKeysetExpression
: expression
| expression &&& expression
| expression .. expression
| DEFAULT
| _
;

simpleKeysetExpressionList
: simpleKeysetExpression
| simpleKeysetExpressionList , simpleKeysetExpression
;

tupleKeysetExpression
: `( expression &&& expression )
| `( expression .. expression )
| `( DEFAULT )
| `( _ )
| `( simpleKeysetExpression , simpleKeysetExpressionList )
;

keysetExpression
: simpleKeysetExpression
| tupleKeysetExpression
;

realTypeArgument

```

```

: type
| VOID
| _
;

realTypeArgumentList
: realTypeArgument
| realTypeArgumentList , realTypeArgument
;

typeArgument
: realTypeArgument
| nonTypeName
;

typeArgumentList
: /* empty */
| typeArgument
| typeArgumentList , typeArgument
;

argument
: expression
| name = expression
| name = _
| _
;

argumentListNonEmpty
: argument
| argumentListNonEmpty , argument
;

argumentList
: /* empty */
| argumentListNonEmpty
;

lvalue
: referenceExpression
| lvalue . member
| lvalue `[ expression ]
| lvalue `[ expression : expression ]
| `( lvalue )
;

emptyStatement
: ;
;

assignmentStatement
: lvalue assignop expression ;
;

callStatement
: lvalue `( argumentList ) ;
| lvalue `< typeArgumentList > `( argumentList ) ;
;

directApplicationStatement
: namedType . APPLY `( argumentList ) ;
;

returnStatement
: RETURN ;
| RETURN expression ;
;

exitStatement

```

```

: EXIT ;
;

blockStatement
: annotationList `{ blockElementStatementList }
;

conditionalStatement
: IF `( expression ) statement
| IF `( expression ) statement ELSE statement
;

forUpdateStatement
: lvalue `( argumentList )
| lvalue `< typeArgumentList > `( argumentList )
| lvalue assignop expression
;

forUpdateStatementListNonEmpty
: forUpdateStatement
| forUpdateStatementListNonEmpty , forUpdateStatement
;

forUpdateStatementList
: /* empty */
| forUpdateStatementListNonEmpty
;

forInitStatement
: annotationList type name initializerOpt
| forUpdateStatement
;

forInitStatementListNonEmpty
: forInitStatement
| forInitStatementListNonEmpty , forInitStatement
;

forInitStatementList
: /* empty */
| forInitStatementListNonEmpty
;

forCollectionExpression
: expression
| expression .. expression
;

forStatement
: annotationList FOR `( forInitStatementList ; expression ; forUpdateStatementList )
statement
| annotationList FOR `( type name IN forCollectionExpression ) statement
| annotationList FOR `( annotationListNonEmpty type name IN forCollectionExpression )
statement
;

switchLabel
: DEFAULT
| expressionNonBrace
;

switchCase
: switchLabel : blockStatement
| switchLabel :
;

switchCaseList
: /* empty */
| switchCaseList switchCase

```

```

;

switchStatement
: SWITCH `( expression ) `{ switchCaseList }
;

breakStatement
: BREAK ;
;

continueStatement
: CONTINUE ;
;

statement
: emptyStatement
| assignmentStatement
| callStatement
| directApplicationStatement
| returnStatement
| exitStatement
| blockStatement
| conditionalStatement
| forStatement
| breakStatement
| continueStatement
| switchStatement
;

initializer
: = expression
;

constantDeclaration
: annotationList CONST type name initializer ;
;

initializerOpt
: /* empty */
| initializer
;

variableDeclaration
: annotationList type name initializerOpt ;
;

blockElementStatement
: constantDeclaration
| variableDeclaration
| statement
;

blockElementStatementList
: /* empty */
| blockElementStatementList blockElementStatement
;

functionPrototype
: typeOrVoid name typeParameterListOpt `( parameterList )
;

functionDeclaration
: annotationList functionPrototype blockStatement
;

actionDeclaration
: annotationList ACTION name `( parameterList ) blockStatement
;

```

```

objectInitializer
: = `{ objectDeclarationList }
;

instantiation
: annotationList type `( argumentList ) name ;
| annotationList type `( argumentList ) name objectInitializer ;
;

objectDeclaration
: functionDeclaration
| instantiation
;

objectDeclarationList
: /* empty */
| objectDeclarationList objectDeclaration
;

errorDeclaration
: ERROR `{ nameList }
;

matchKindDeclaration
: MATCH_KIND `{ nameList trailingCommaOpt }
;

enumTypeDeclaration
: annotationList ENUM name `{ nameList trailingCommaOpt }
| annotationList ENUM type name `{ namedExpressionList trailingCommaOpt }
;

typeField
: annotationList type name ;
;

typeFieldList
: /* empty */
| typeFieldList typeField
;

structTypeDeclaration
: annotationList STRUCT name typeParameterListOpt `{ typeFieldList }
;

headerTypeDeclaration
: annotationList HEADER name typeParameterListOpt `{ typeFieldList }
;

headerUnionTypeDeclaration
: annotationList HEADER_UNION name typeParameterListOpt `{ typeFieldList }
;

derivedTypeDeclaration
: enumTypeDeclaration
| structTypeDeclaration
| headerTypeDeclaration
| headerUnionTypeDeclaration
;

typedef
: type
| derivedTypeDeclaration
;

typedefDeclaration
: annotationList TYPEDEF typedef name ;
| annotationList TYPE type name ;
;

```

```

externFunctionDeclaration
    : annotationList EXTERN functionPrototype ;
    ;

externConstructorPrototype
    : annotationList typeIdentifier `( parameterList ) ;
    ;

externMethodPrototype
    : annotationList functionPrototype ;
    | annotationList ABSTRACT functionPrototype ;
    ;

externConstructorOrMethodPrototype
    : externConstructorPrototype
    | externMethodPrototype
    ;

externConstructorOrMethodPrototypeList
    : /* empty */
    | externConstructorOrMethodPrototypeList externConstructorOrMethodPrototype
    ;

externObjectDeclaration
    : annotationList EXTERN nonTypeName typeParameterListOpt `{
externConstructorOrMethodPrototypeList }
    ;

externDeclaration
    : externFunctionDeclaration
    | externObjectDeclaration
    ;

selectCase
    : keysetExpression : name ;
    ;

selectCaseList
    : /* empty */
    | selectCaseList selectCase
    ;

selectExpression
    : SELECT `( expressionList ) `{ selectCaseList }
    ;

stateExpression
    : name ;
    | selectExpression
    ;

transitionStatement
    : /* empty */
    | TRANSITION stateExpression
    ;

valueSetType
    : baseType
    | tupleType
    | prefixedTypeName
    ;

valueSetDeclaration
    : annotationList VALUE_SET `< valueSetType > `( expression ) name ;
    ;

parserTypeDeclaration
    : annotationList PARSER name typeParameterListOpt `( parameterList ) ;

```



```

;

parserBlockStatement
: annotationList `{ parserStatementList }
;

parserConditionalStatement
: IF `( expression ) parserStatement
| IF `( expression ) parserStatement ELSE parserStatement
;

parserStatement
: constantDeclaration
| variableDeclaration
| emptyStatement
| assignmentStatement
| callStatement
| directApplicationStatement
| parserBlockStatement
| parserConditionalStatement
;

parserStatementList
: /* empty */
| parserStatementList parserStatement
;

parserState
: annotationList STATE name `{ parserStatementList transitionStatement }
;

parserStateList
: parserState
| parserStateList parserState
;

parserLocalDeclaration
: constantDeclaration
| instantiation
| variableDeclaration
| valueSetDeclaration
;

parserLocalDeclarationList
: /* empty */
| parserLocalDeclarationList parserLocalDeclaration
;

parserDeclaration
: annotationList PARSER name typeParameterListOpt `( parameterList )
constructorParameterListOpt `{ parserLocalDeclarationList parserStateList }
;

const
: CONST
;

constOpt
: /* empty */
| const
;

tableKey
: expression : name annotationList ;
;

tableKeyList
: /* empty */
| tableKeyList tableKey

```

```

;

tableActionReference
: prefixedNonTypeName
| prefixedNonTypeName `( argumentList )
;

tableAction
: annotationList tableActionReference ;
;

tableActionList
: /* empty */
| tableActionList tableAction
;

tableEntryPriority
: PRIORITY = integerLiteral :
| PRIORITY = `( expression ) :
;

tableEntry
: constOpt tableEntryPriority keysetExpression : tableActionReference annotationList ;
| constOpt keysetExpression : tableActionReference annotationList ;
;

tableEntryList
: /* empty */
| tableEntryList tableEntry
;

tableProperty
: KEY = `{ tableKeyList }
| ACTIONS = `{ tableActionList }
| annotationList constOpt ENTRIES = `{ tableEntryList }
| annotationList constOpt tableCustomName initializer ;
;

tablePropertyList
: /* empty */
| tablePropertyList tableProperty
;

tableDeclaration
: annotationList TABLE name `{ tablePropertyList }
;

controlTypeDeclaration
: annotationList CONTROL name typeParameterListOpt `( parameterList ) ;
;

controlBody = blockStatement

controlLocalDeclaration
: constantDeclaration
| instantiation
| variableDeclaration
| actionDeclaration
| tableDeclaration
;

controlLocalDeclarationList
: /* empty */
| controlLocalDeclarationList controlLocalDeclaration
;

controlDeclaration
: annotationList CONTROL name typeParameterListOpt `( parameterList )
constructorParameterListOpt `{ controlLocalDeclarationList APPLY controlBody }

```

```

;

packageTypeDeclaration
: annotationList PACKAGE name typeParameterListOpt `( parameterList ) ;
;

typeDeclaration
: derivedTypeDeclaration
| typedefDeclaration
| parserTypeDeclaration
| controlTypeDeclaration
| packageTypeDeclaration
;

declaration
: constantDeclaration
| instantiation
| functionDeclaration
| actionDeclaration
| errorDeclaration
| matchKindDeclaration
| externDeclaration
| parserDeclaration
| controlDeclaration
| typeDeclaration
;

annotationToken
: UNEXPECTED_TOKEN
| ABSTRACT
| ACTION
| ACTIONS
| APPLY
| BOOL
| BIT
| BREAK
| CONST
| CONTINUE
| CONTROL
| DEFAULT
| ELSE
| ENTRIES
| ENUM
| ERROR
| EXIT
| EXTERN
| FALSE
| FOR
| HEADER
| HEADER_UNION
| IF
| IN
| INOUT
| INT
| KEY
| MATCH_KIND
| TYPE
| OUT
| PARSER
| PACKAGE
| PRAGMA
| RETURN
| SELECT
| STATE
| STRING
| STRUCT
| SWITCH
| TABLE
| THIS

```

```

| TRANSITION
| TRUE
| TUPLE
| TYPEDEF
| VARBIT
| VALUE_SET
| LIST
| VOID
|
| identifier
| typeIdentifier
| stringLiteral
| integerLiteral
| &&
| ..
| <<
| &&
| ||
| ==
| !=
| >=
| <=
| ++
| +
| |+|
| -
| |-|
| *
| /
| %
| |
| &
| ^
| ~
| [
| ]
| {
| }
| <
| >
| !
| :
| ,
| ?
| .
| =
| ;
| @
;

annotationBody
: /* empty */
| annotationBody `( annotationBody )
| annotationBody annotationToken
;

structuredAnnotationBody
: sequenceOrRecordElementExpression trailingCommaOpt
;

annotation
: @ name
| @ name `( annotationBody )
| @ name `[ structuredAnnotationBody ]
| @ PRAGMA name annotationBody
;

annotationListNonEmpty
: annotation

```

```
| annotationListNonEmpty annotation
;

annotationList
: /* empty */
| annotationListNonEmpty
;

p4program
: /* empty */
| p4program declaration
| p4program ;
;
```